AI 應用開發

Enos Chou

© 2026 Enos Chou

1. GCP AI API 整合應用
2. Gemini 整合應用
3. Azure OpenAI 整合應用
4. RAG & Vector DB 實作
5. AI Agent 實作
6. GCP AI 模型訓練 & 整合應用
7. Web & API 整合應用
8. 整合工具提示

© 2026 Enos Chou

AI 應用架構

© 2026 Enos Chou

模組化 vs API 化

模組化

程式層級的封裝與重複使用

API 化

系統服務層級的封裝與重複使用

© 2026 Enos Chou

模組化

ALL in ONE

main.py (linebot)

```
import linebot
import google.genai

def recognize_plates():
    ...

...
plates = recognize_plates()
...

linebot.loop()
```

MODUALIZED

main.py (linebot)

```
import linebot
import car

...
plates = car.recognize_plates()
...

linebot.loop()
```

car.py

```
import google.genai

def recognize_plates():
    ....
```

© 2026 Enos Chou

模組化

car.py

```
import google.genai

def recognize_plates():
    ....
```

main.py (linebot)

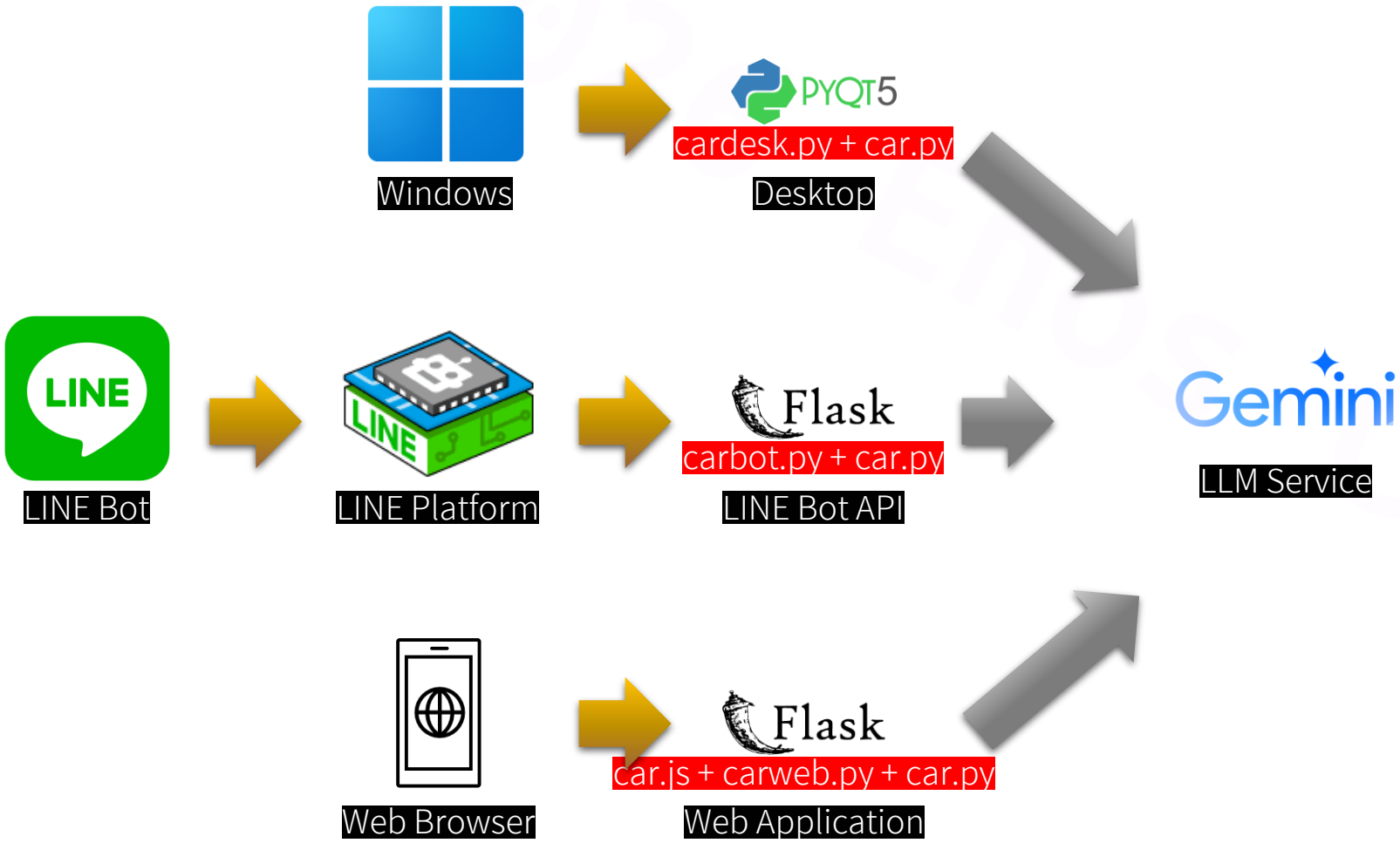
```
import linebot
import car

...
plates = car.recognize_plates()
...

linebot.loop()
```

main.py (desktop)

```
import tkinter
import car

...
plates = car.recognize_plates()
...

tkinter.loop()
```

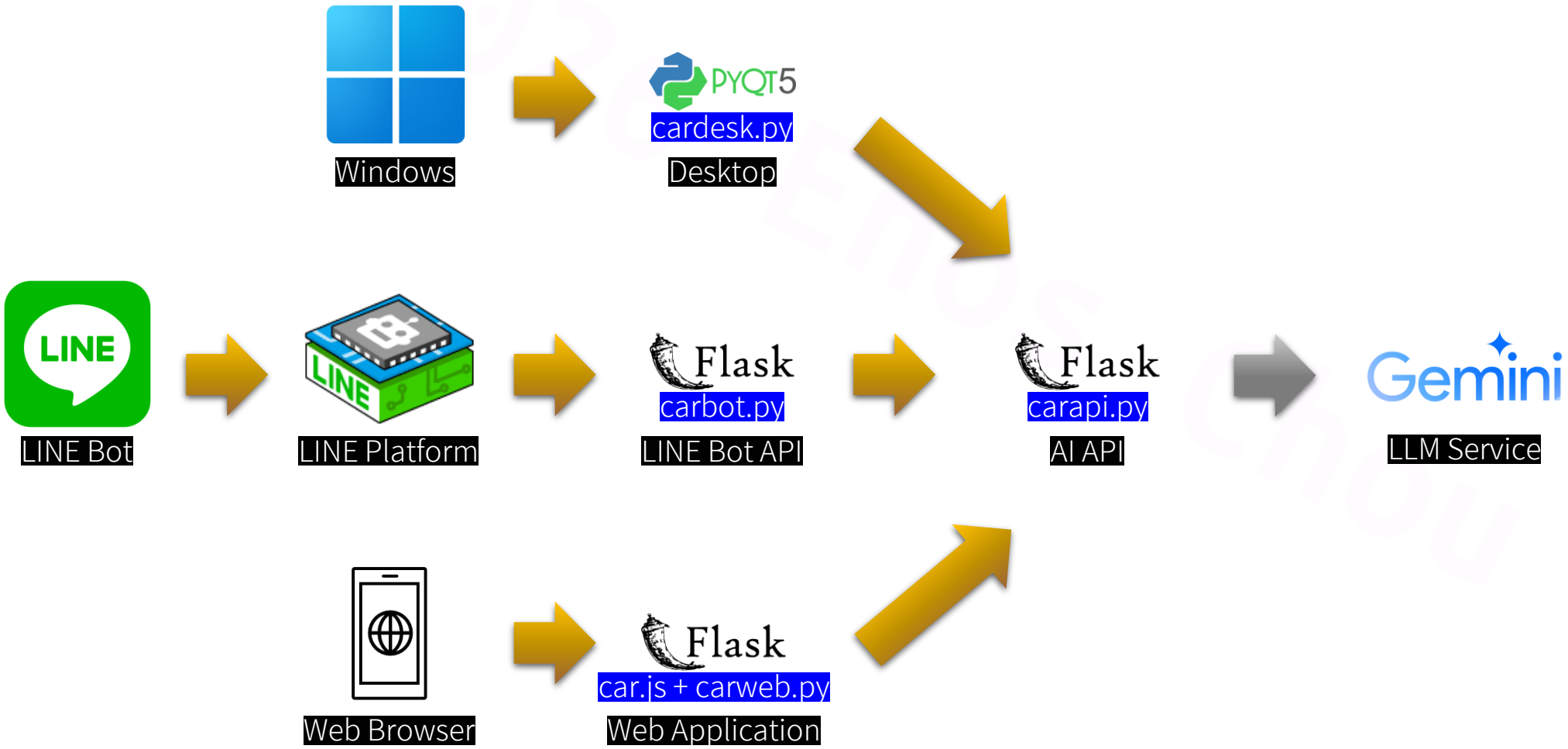
© 2026 Enos Chou

直覺的軟體架構

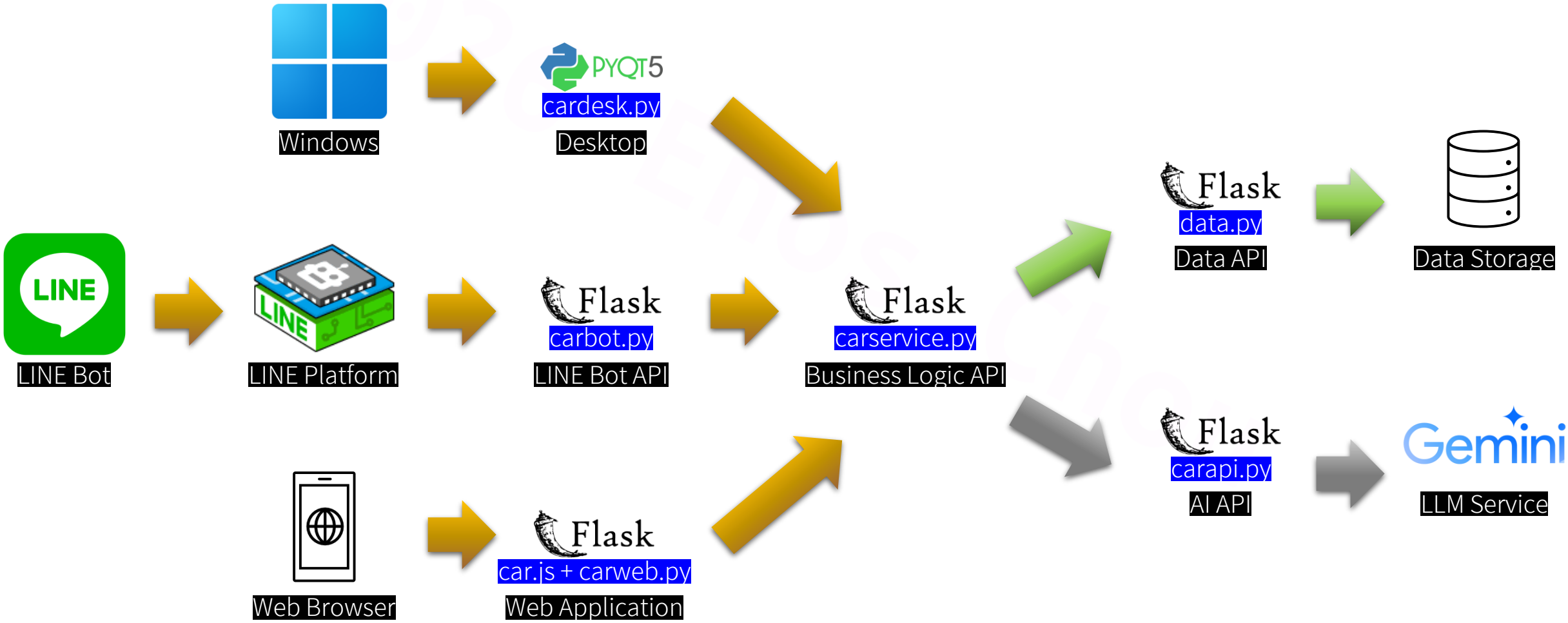


© 2026 Enos Chou

API 化的軟體架構


Windows
PYQT5
cardesk.py
Desktop
LINE Bot
LINE Platform
Flask
carbot.py
LINE Bot API
Flask
carapi.py
AI API
Gemini
LLM Service
Web Browser
Flask
car.js + carweb.py
Web Application

常見的軟體架構


Windows
PYQT5
cardesk.py
Desktop
LINE Bot
LINE Platform
Flask
carbot.py
LINE Bot API
Flask
carservice.py
Business Logic API
Flask
data.py
Data API
Data Storage
Web Browser
Flask
car.js + carweb.py
Web Application
Flask
carapi.py
AI API
Gemini
LLM Service

開發環境準備

© 2026 Enos Chou

開發環境準備

程式語言

Python 3.12

未安裝套件的虛擬環境

建立方式不限 (以 Python venv/ Anaconda/ Miniconda 建立皆可)

IDE

示範 IDE： Jupyter Notebook

學員 IDE：不限

© 2026 Enos Chou

Google Cloud 服務帳戶建立與權限管理

IAM 與管理

服務帳戶

© 2026 Enos Chou

服務帳戶概念

服務帳戶 (Service Account) 概念

a. 服務帳戶目的
   ① 提供外部應用程式驗證 Google Cloud 服務 (例如：Google Cloud Storage) 之帳號
   ② 提供外部應用程式綁定必要 Google Cloud 服務的權限

b. 服務帳戶建立與使用程序
   ① 建立服務帳戶搭配外部應用需求之適當權限
   ② 產生對應之金鑰 (JSON) 後下載 (此金鑰可視 Google Cloud 驗證外部應用程式之密碼)
   ③ 外部應用程式存取 Google Cloud 服務時搭配金鑰使用

© 2026 Enos Chou

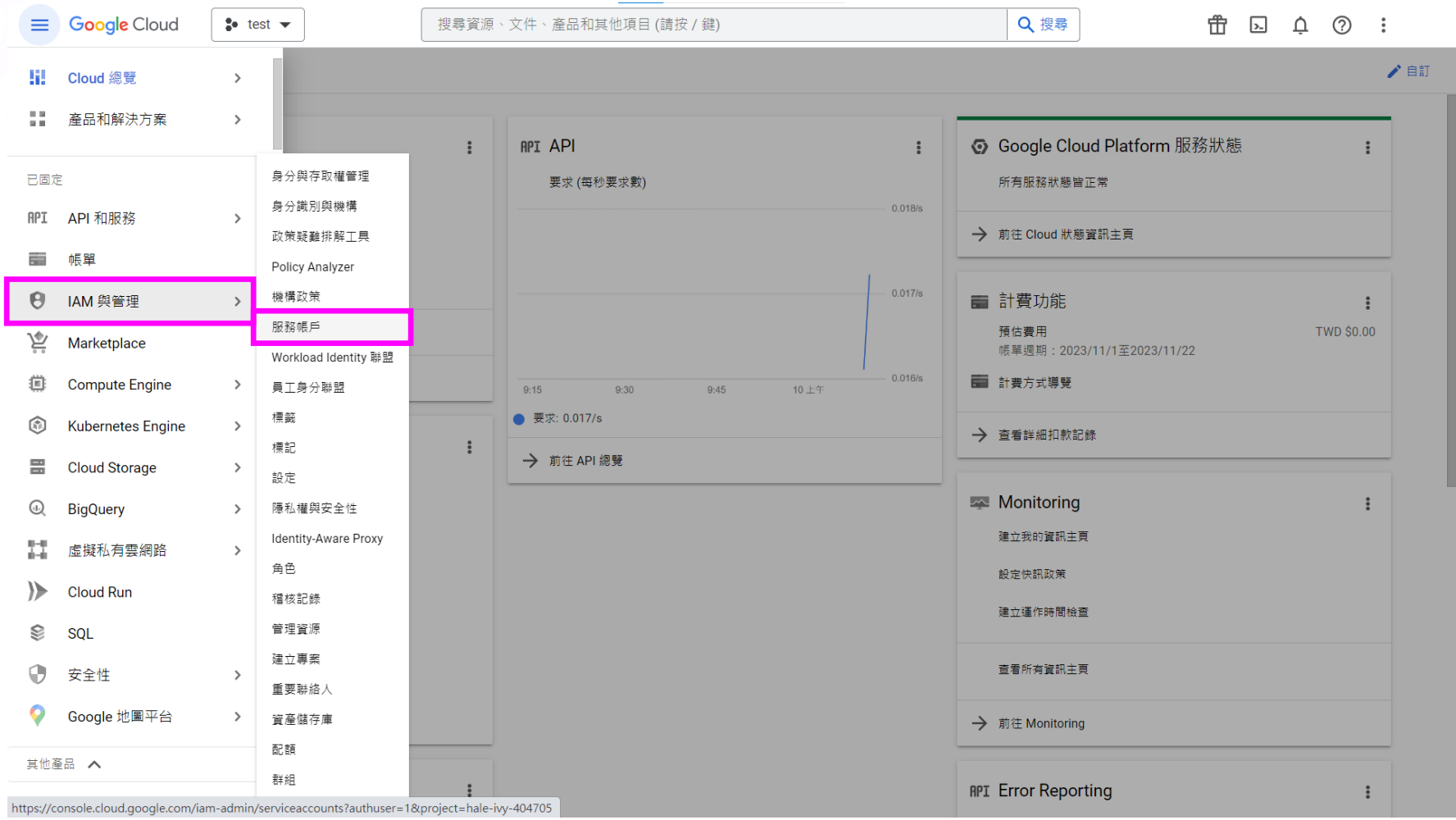
服務帳戶建立與調整

1. 建立服務帳戶
2. 下載金鑰
3. 調整權限
4. 調整權限案例

© 2026 Enos Chou

服務帳戶建立與調整

1. 建立服務帳戶



© 2026 Enos Chou

服務帳戶建立與調整

1. 建立服務帳戶

服務帳戶建立與調整

1. 建立服務帳戶

© 2026 Enos Chou

服務帳戶建立與調整

1. 建立服務帳戶

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

SYMPTOM
目前 Google Cloud 預設的組織政策不允許建立金鑰，直接下載會報錯。

ERROR
貴組織已強制執行組織政策，禁止建立金鑰

SOLUTION
欲下載金鑰須遵循以下程序調整組織政策

© 2026 Enos Chou

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

① 組織層級將自己的登入帳號加入 "組織政策管理員" 權限

"IAM 與管理" > "管理資源" >
勾選組織(名稱中後綴 "-org" 者即為組織)的核取方塊>
新增主體 >
新增主體: *自行輸入登入帳號*/ 選取角色: "組織政策管理員" >
儲存

© 2026 Enos Chou

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >

© 2026 Enos Chou

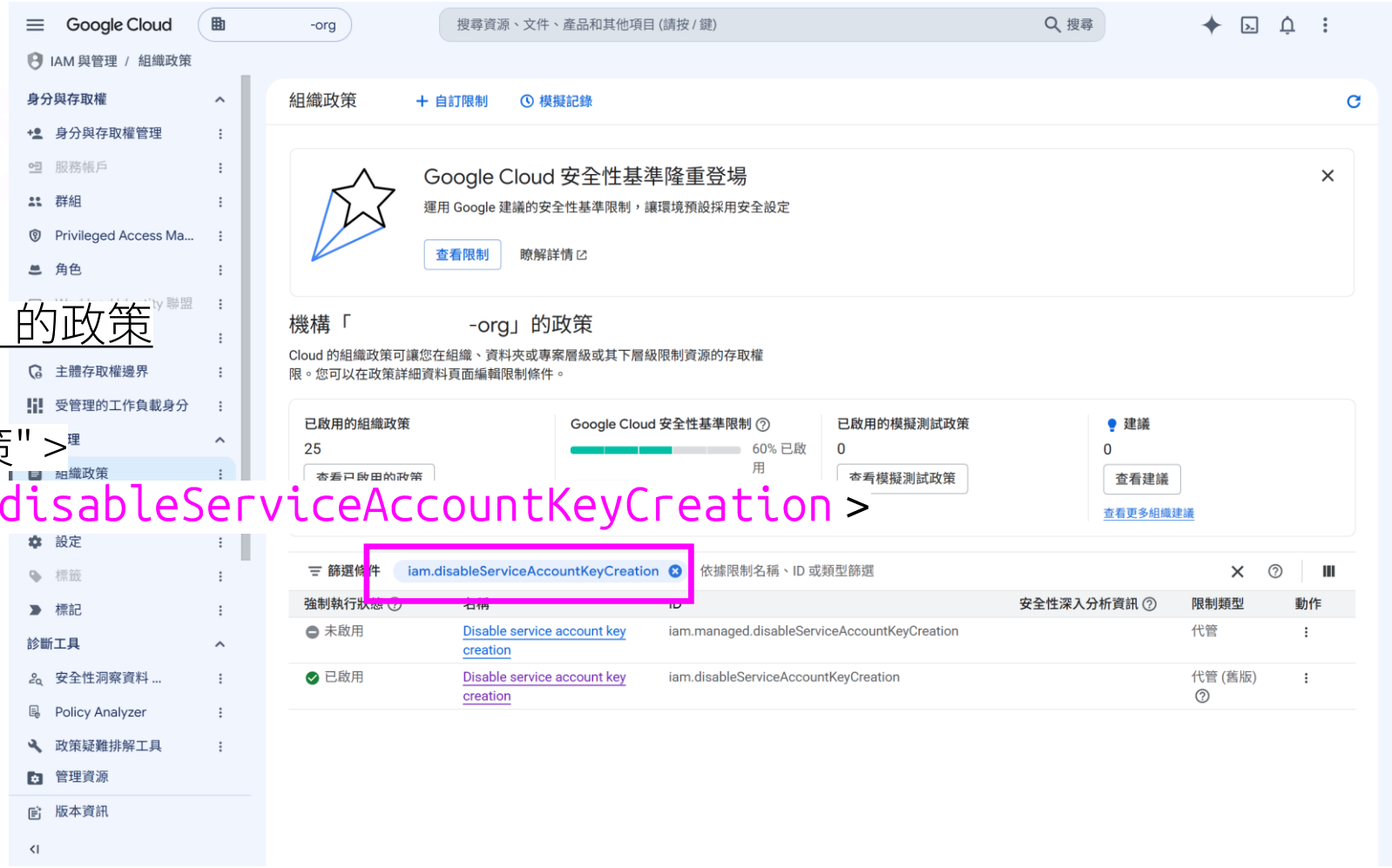
服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 iam.disableServiceAccountKeyCreation >



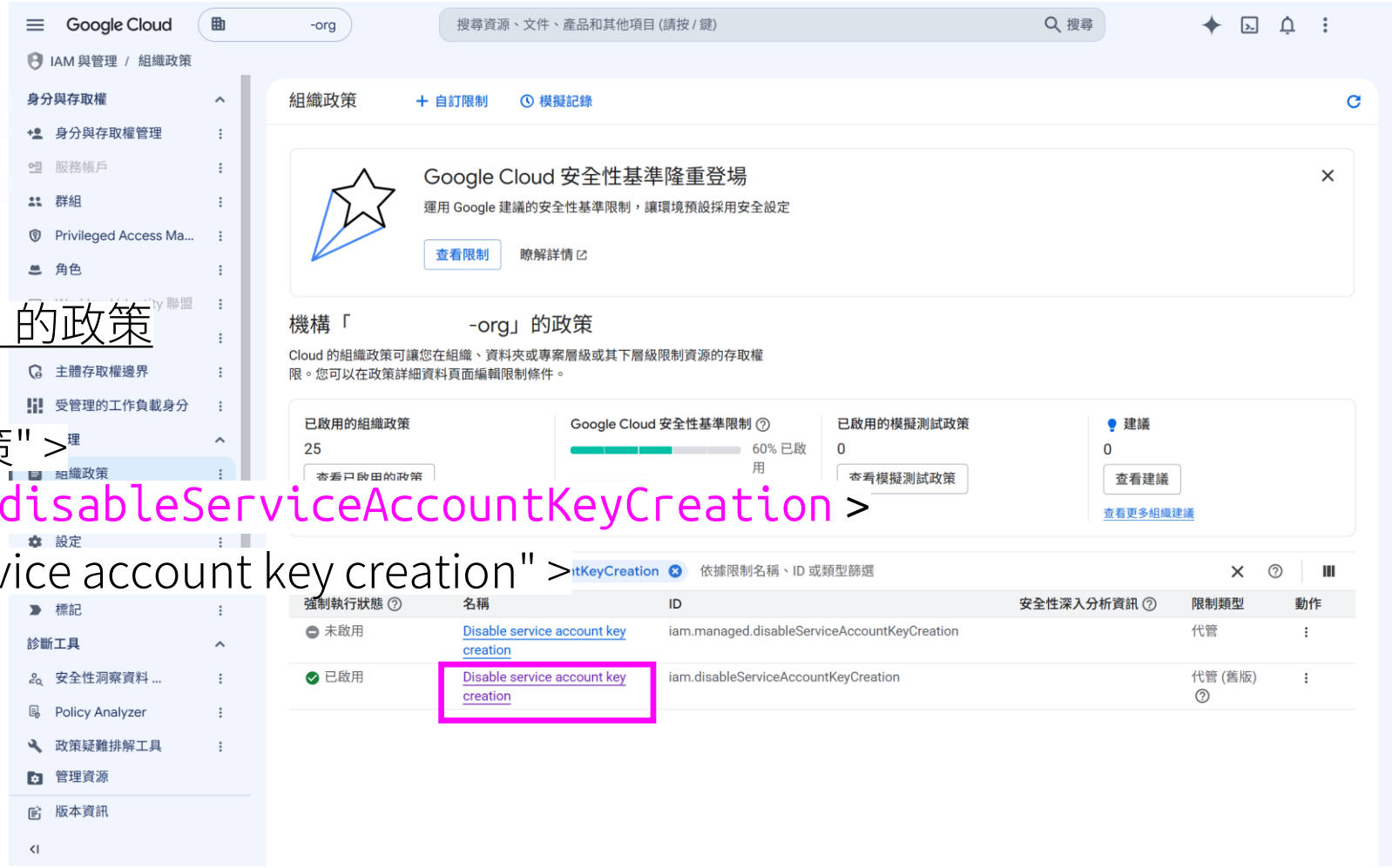
服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 iam.disableServiceAccountKeyCreation >
點擊對應的 "Disable service account key creation" >



服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 iam.disableServiceAccountKeyCreation >
點擊對應的 "Disable service account key creation" >

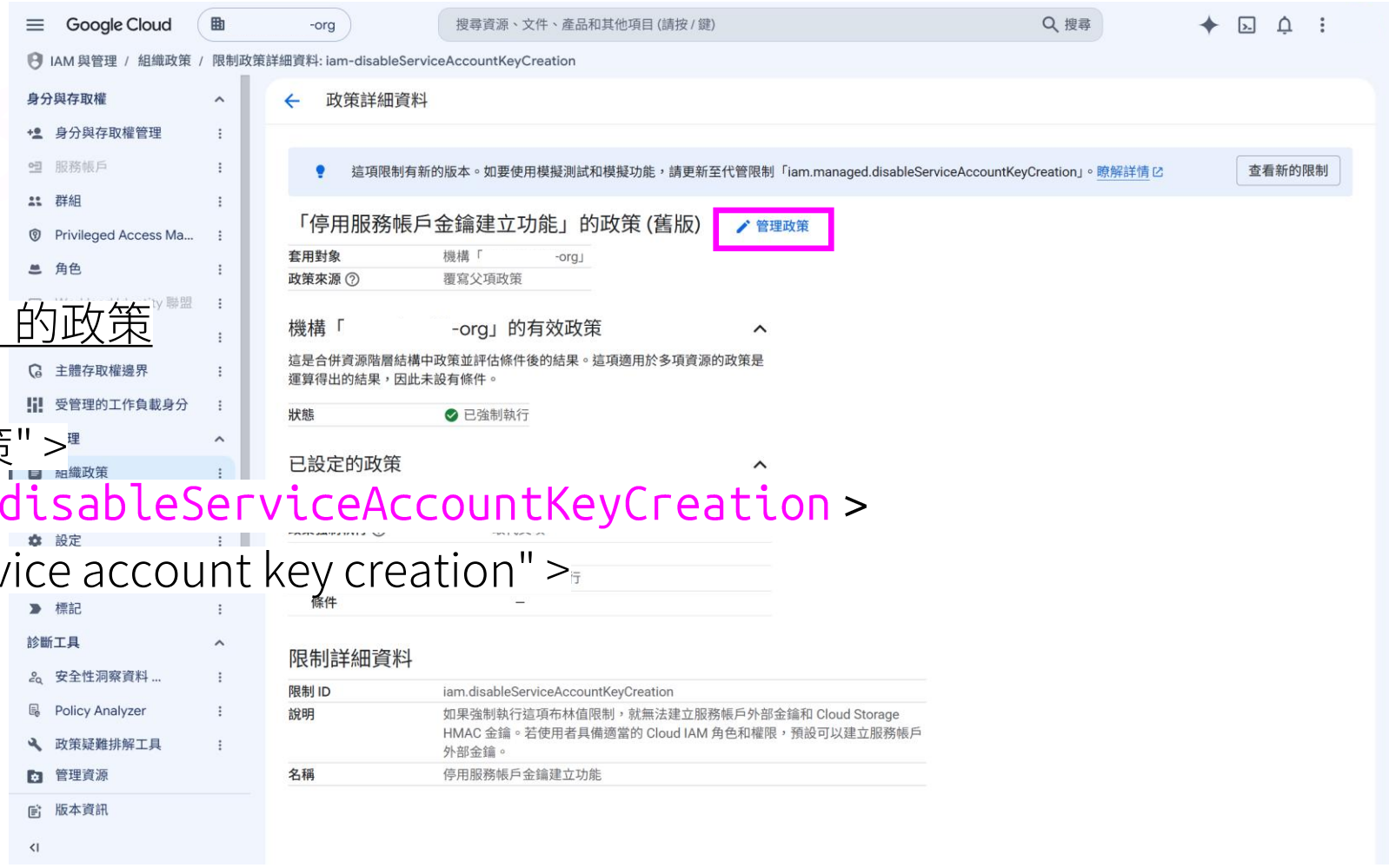
服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 iam.disableServiceAccountKeyCreation >
點擊對應的 "Disable service account key creation" >
管理政策 >



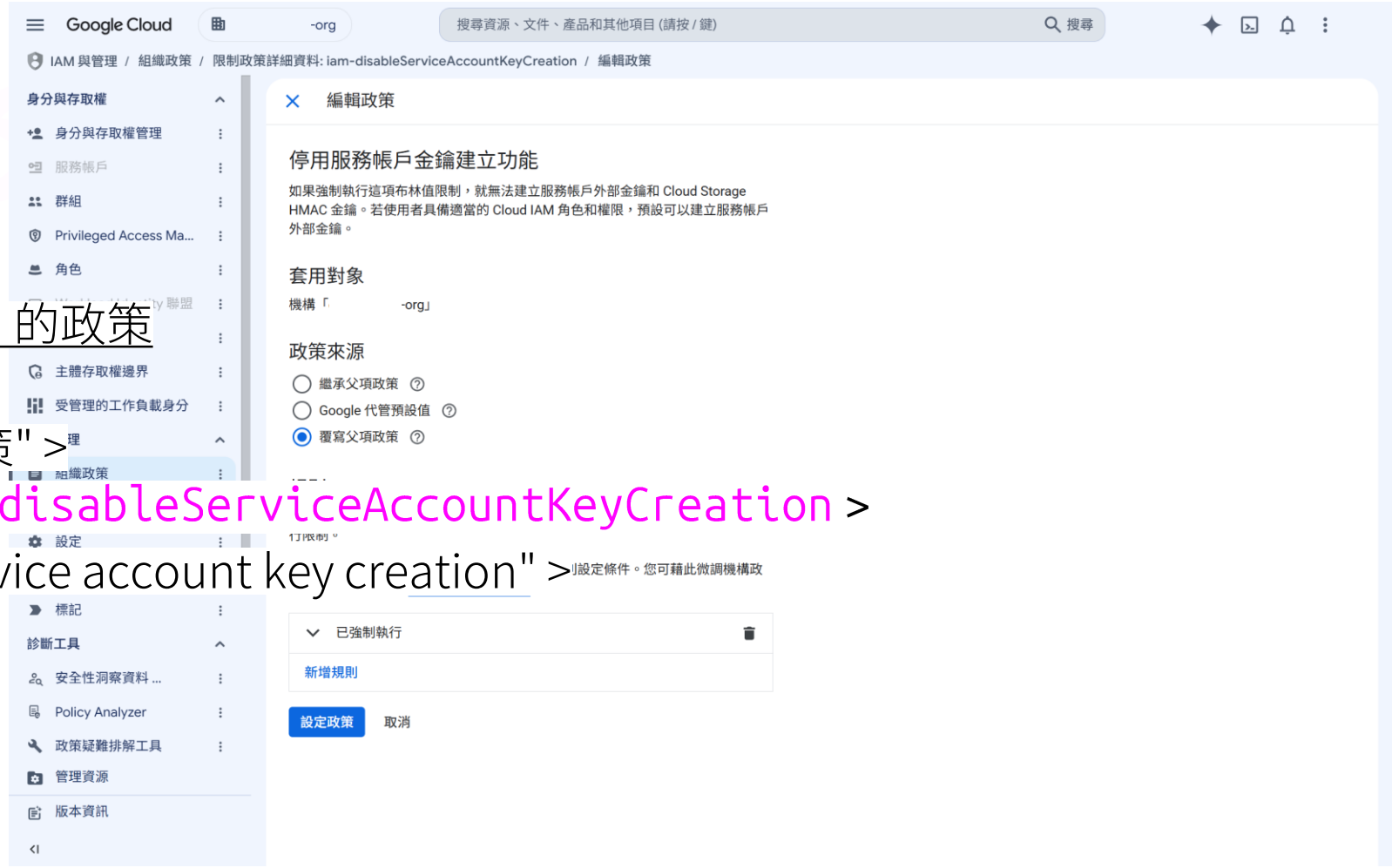
服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 iam.disableServiceAccountKeyCreation >
點擊對應的 "Disable service account key creation" >
管理政策 >



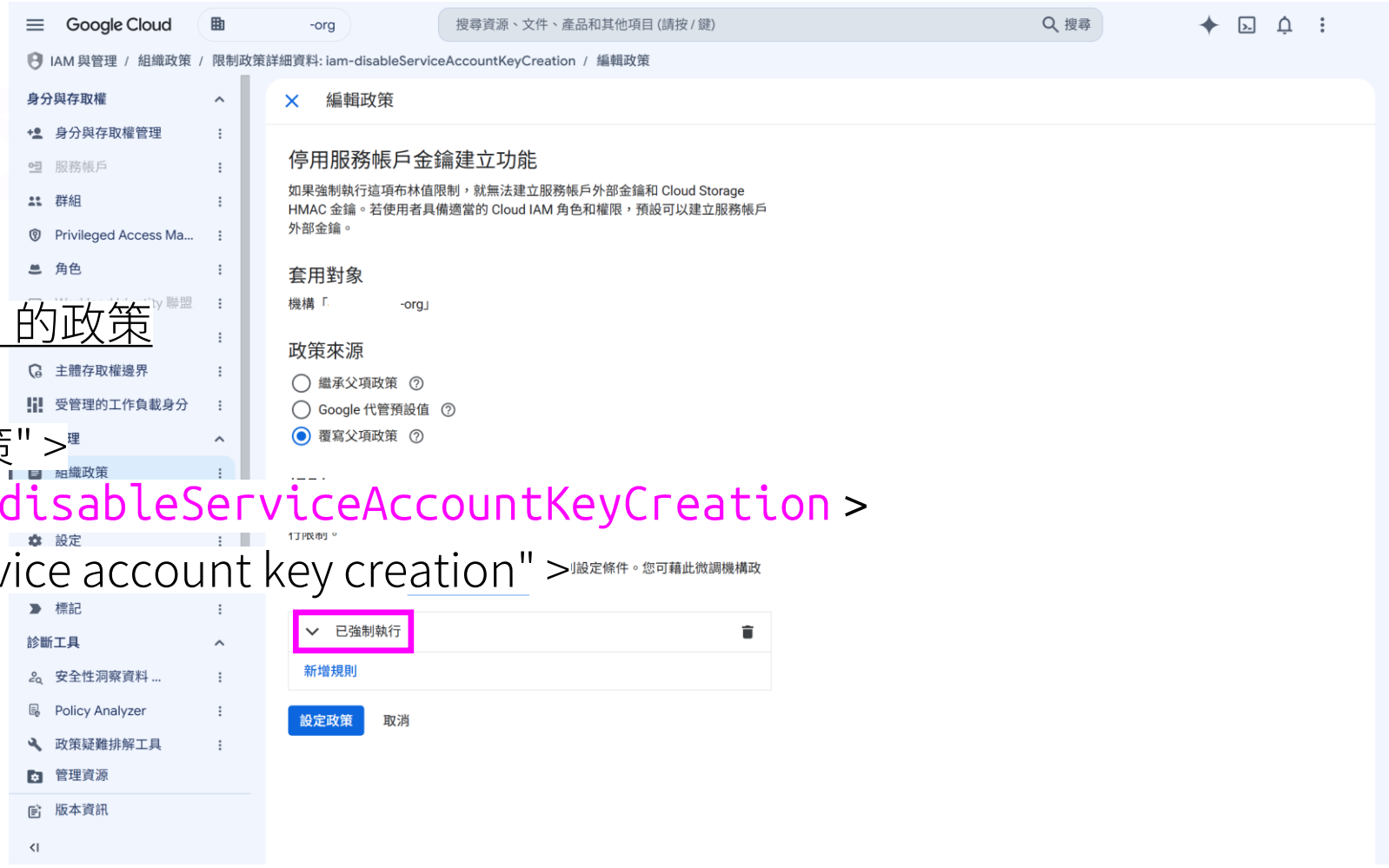
服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 iam.disableServiceAccountKeyCreation >
點擊對應的 "Disable service account key creation" >
管理政策 >
"已強制執行" >



服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 iam.disableServiceAccountKeyCreation >
點擊對應的 "Disable service account key creation" >
管理政策 >
"已強制執行" >

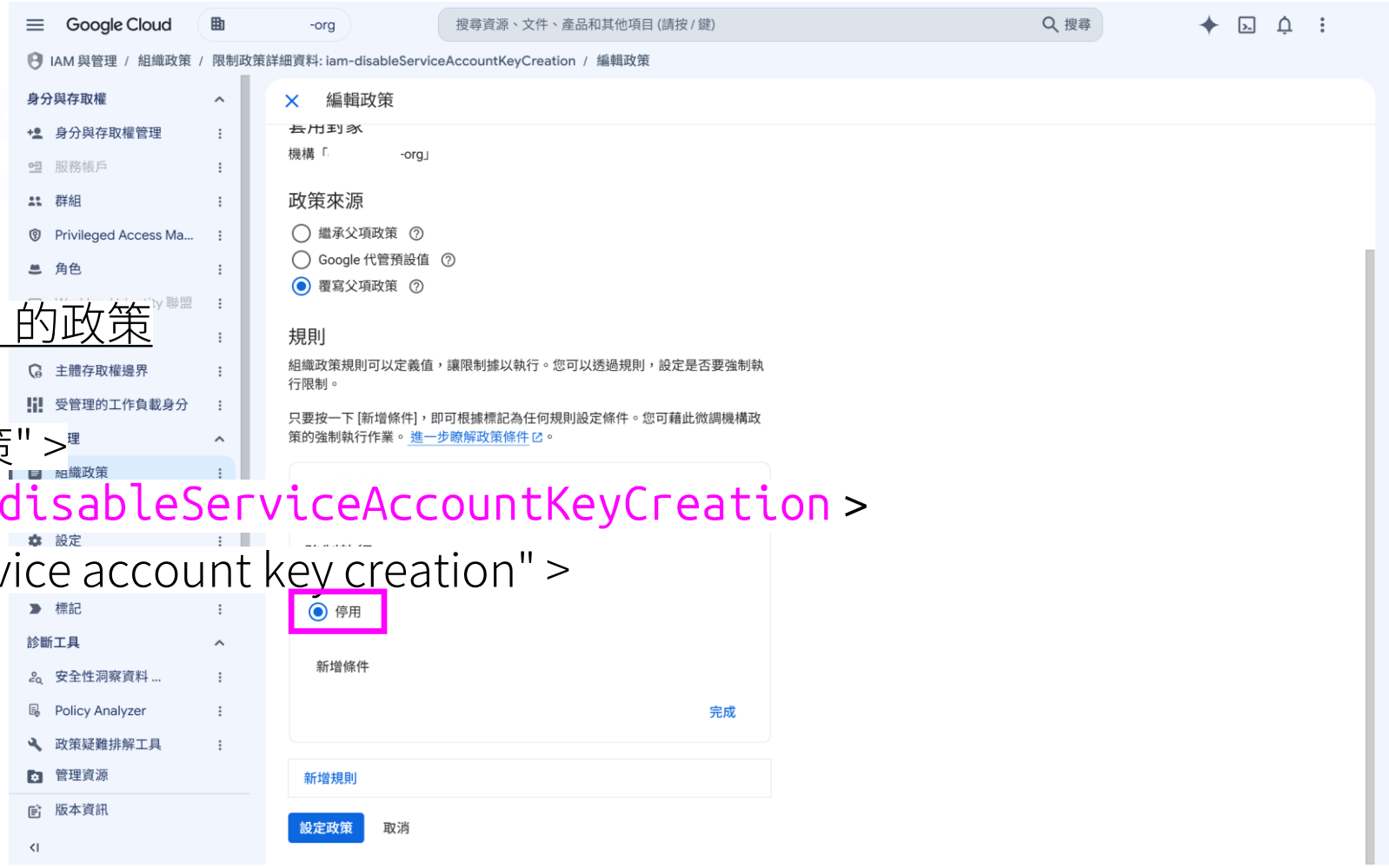
服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 iam.disableServiceAccountKeyCreation >
點擊對應的 "Disable service account key creation" >
管理政策 >
"已強制執行" >
點選 "停用" > 完成 >



服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 iam.disableServiceAccountKeyCreation >
點擊對應的 "Disable service account key creation" >
管理政策 >
"已強制執行" >
點選 "停用" > 完成 >

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 iam.disableServiceAccountKeyCreation >
點擊對應的 "Disable service account key creation" >
管理政策 >
"已強制執行" >
點選 "停用" > 完成 >
設定政策

© 2026 Enos Chou

服務帳戶建立與調整

2. 下載金鑰

a. (開啟下載金鑰權限)

② 停用 "不可建立金鑰" 的政策

"IAM 與管理" > "組織政策" >
於 "篩選條件" 篩選 iam.disableServiceAccountKeyCreation >
點擊對應的 "Disable service account key creation" >
管理政策 >
"已強制執行" >
點選 "停用" > 完成 >
設定政策

© 2026 Enos Chou

服務帳戶建立與調整

2. 下載金鑰

b. 建立並下載金鑰

管理金鑰

© 2026 Enos Chou

服務帳戶建立與調整

2. 下載金鑰

b. 建立並下載金鑰

新增金鑰

© 2026 Enos Chou

服務帳戶建立與調整

2. 下載金鑰

b. 建立並下載金鑰

建立新的金鑰

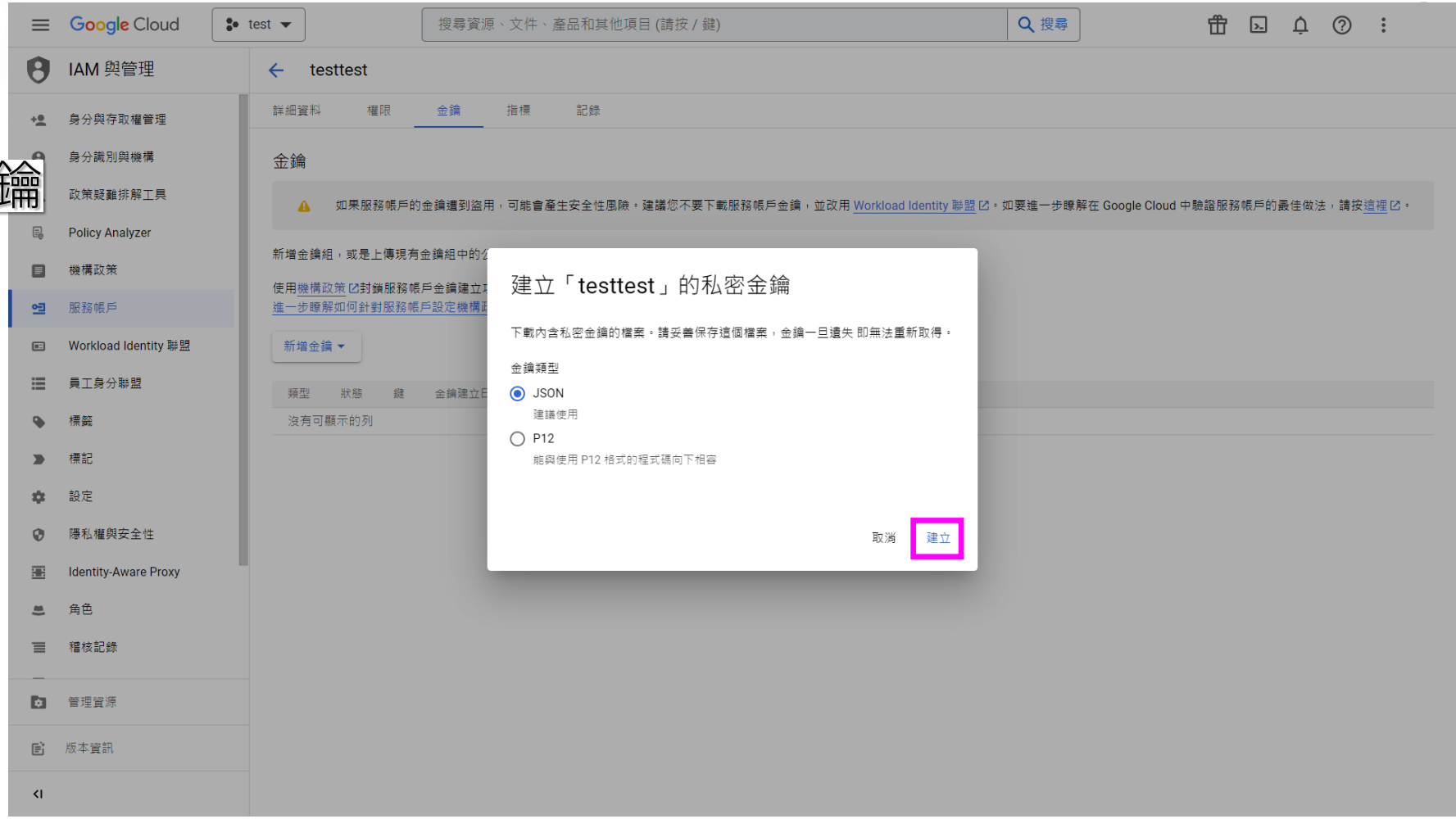
© 2026 Enos Chou

服務帳戶建立與調整

2. 下載金鑰

b. 建立並下載金鑰

建立 JSON 金鑰



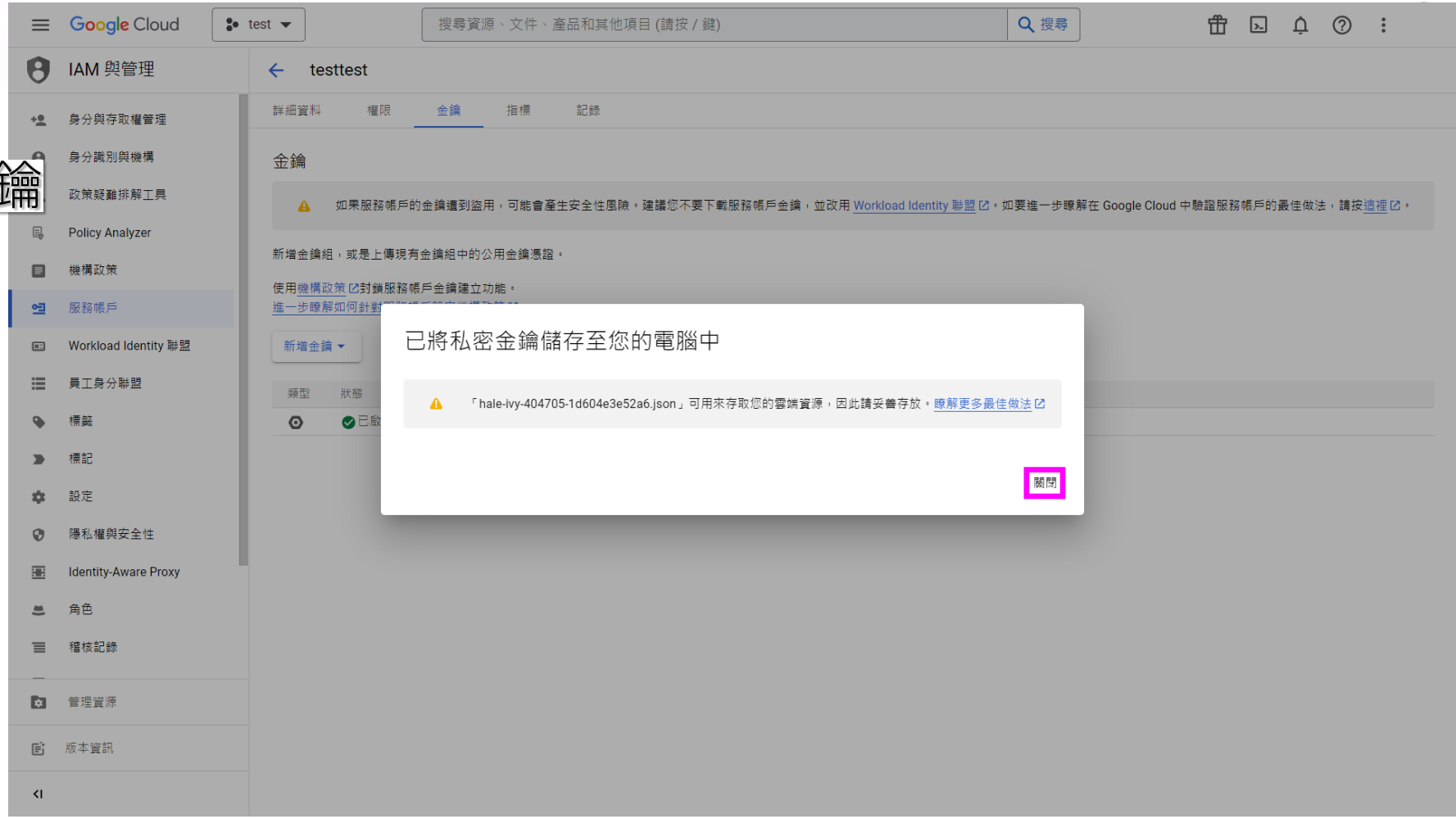
© 2026 Enos Chou

服務帳戶建立與調整

2. 下載金鑰

b. 建立並下載金鑰

妥善保管金鑰



服務帳戶建立與調整

3. 調整權限

a. 調整服務帳戶權限

ERROR

testtest-xxx@yyy.iam.gserviceaccount.com does not have storage.objects.create access to the Google Cloud Storage object. Permission 'storage.objects.create' denied on resource (or it may not exist).

SOLUTION

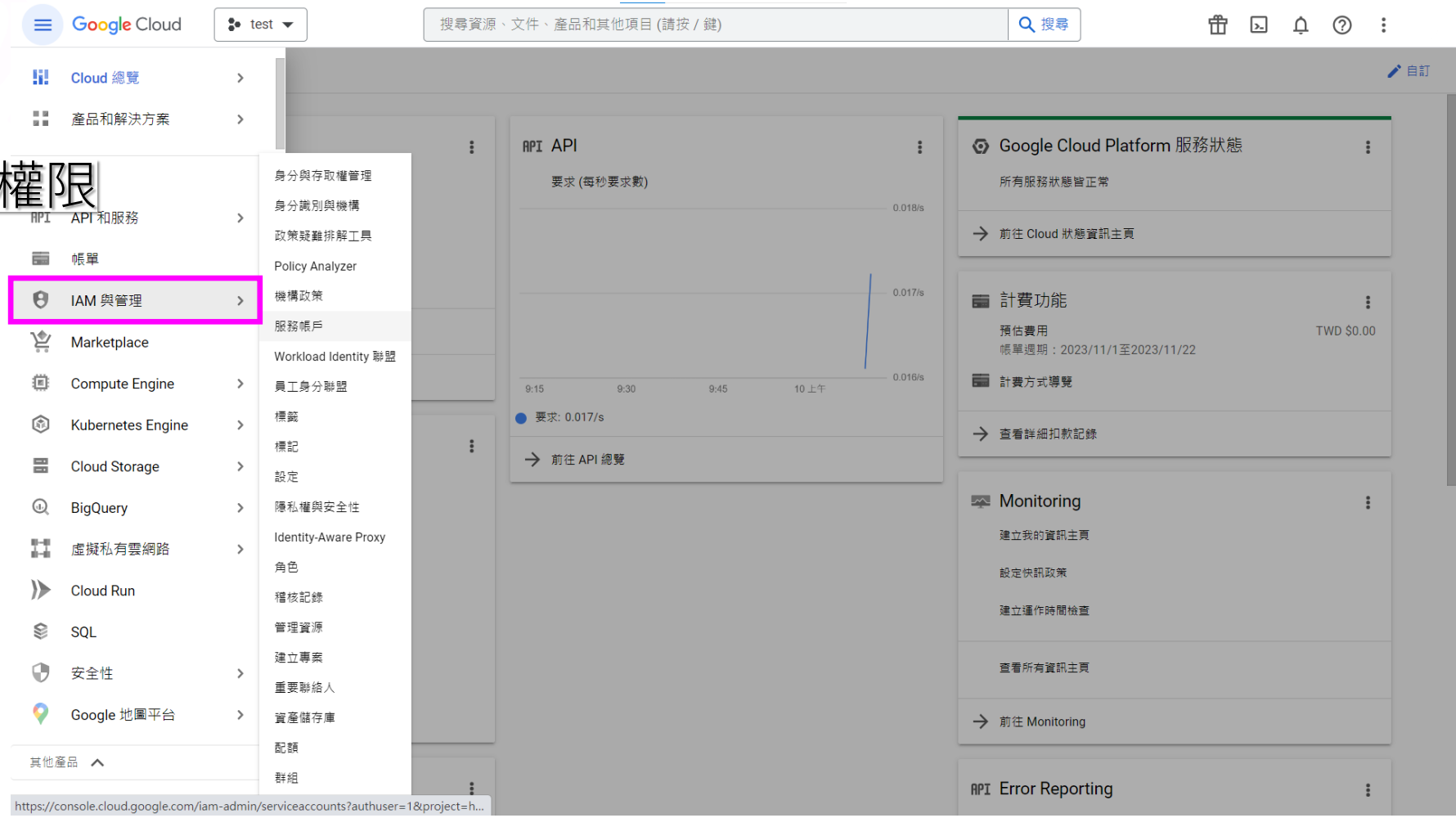
透過 "IAM 與管理" 針對服務帳戶增加 "Storage 物件使用者" 或 "Storage 管理員" 權限

© 2026 Enos Chou

服務帳戶建立與調整

3. 調整權限

a. 調整服務帳戶權限



© 2026 Enos Chou

服務帳戶建立與調整

3. 調整權限

a. 調整服務帳戶權限

授予存取權

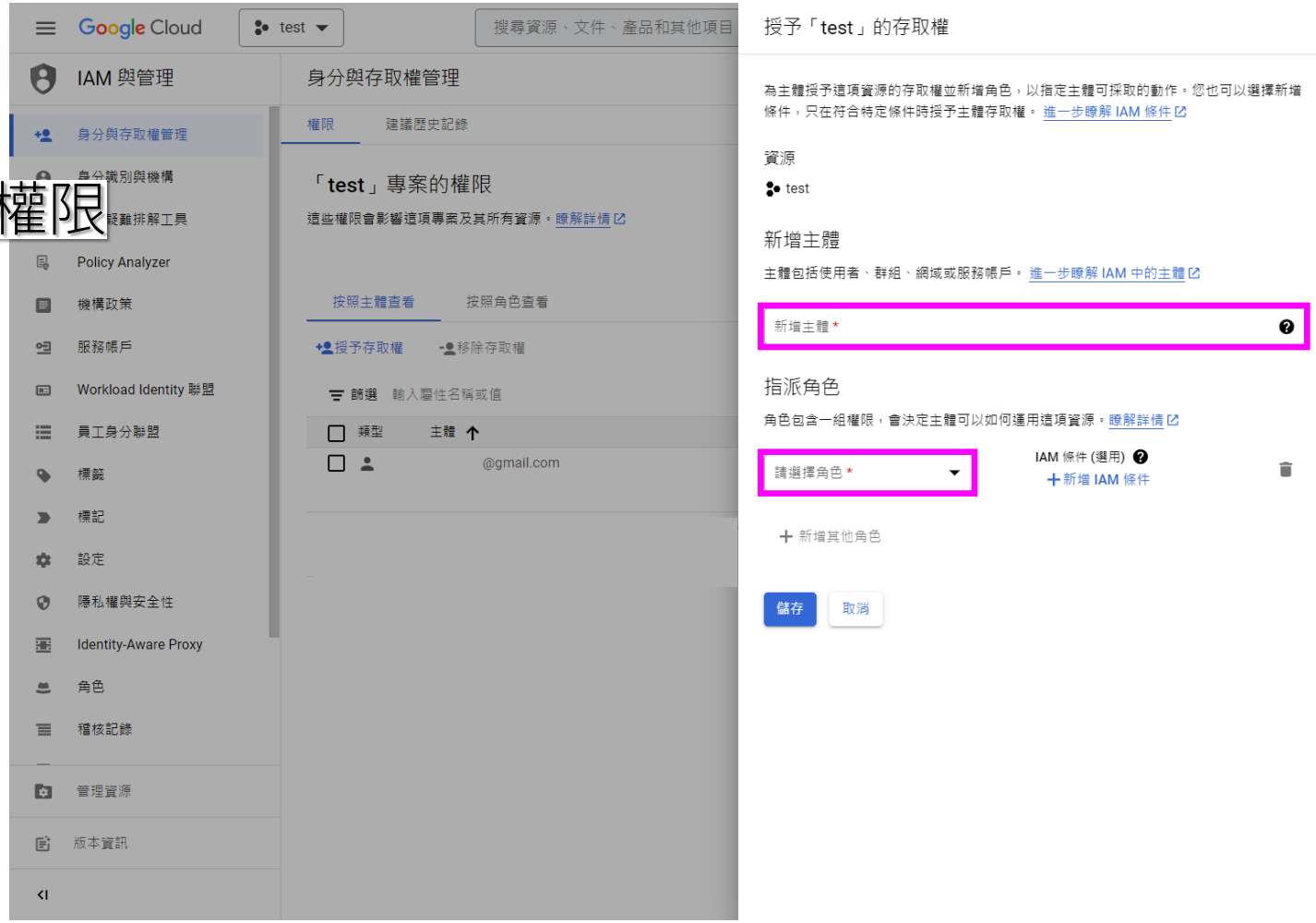
© 2026 Enos Chou

服務帳戶建立與調整

3. 調整權限

a. 調整服務帳戶權限

"新增主體" >
"請選擇角色"

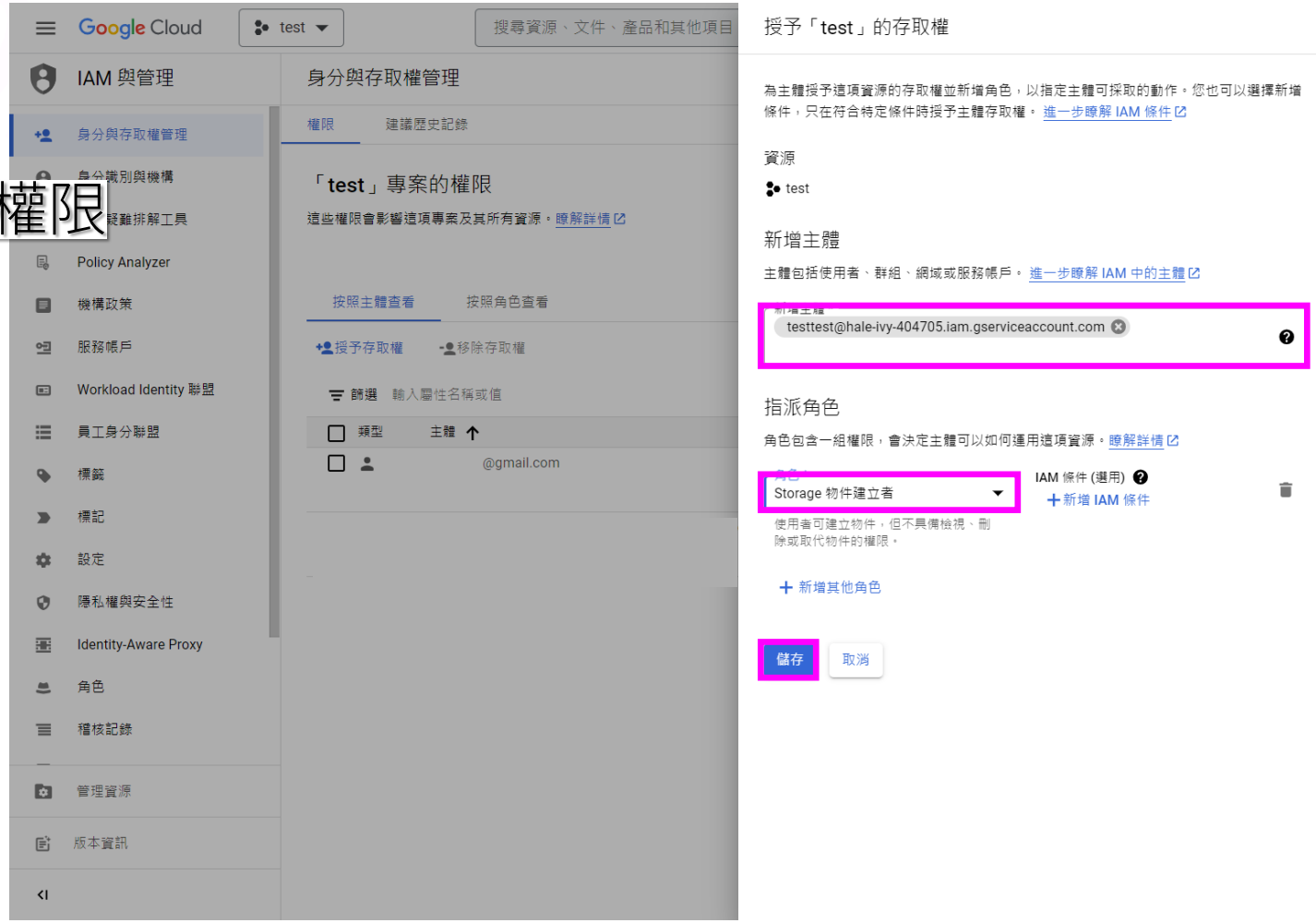


服務帳戶建立與調整

3. 調整權限

a. 調整服務帳戶權限

"新增主體" >
"請選擇角色" >
儲存



© 2026 Enos Chou

服務帳戶建立與調整

3. 調整權限

a. 調整服務帳戶權限

約 1 分鐘後生效

服務帳戶建立與調整

4. 調整權限案例

未啟用 Cloud Speech-toText API

ERROR

PermissionDenied: 403 Cloud Speech-to-Text API has not been used in project xxx before or it is disabled. Enable it by visiting https://console.developers.google.com/apis/api/speech.googleapis.com/overview?project=xxx then retry. If you enabled this API recently, wait a few minutes for the action to propagate to our systems and retry.

SOLUTION

啟用 Cloud Speech-to-Text API

© 2026 Enos Chou

服務帳戶建立與調整

4. 調整權限案例

未啟用 Cloud Vision API

ERROR

PermissionDenied: 403 Cloud Vision API has not been used in project xxx before or it is disabled. Enable it by visiting https://console.developers.google.com/apis/api/vision.googleapis.com/overview?project=xxx then retry. If you enabled this API recently, wait a few minutes for the action to propagate to our systems and retry.

SOLUTION

啟用 Cloud Vision API

© 2026 Enos Chou

服務帳戶建立與調整

4. 調整權限案例

無法使用 Vertex AI 產品，如 Gemini

ERROR

403 Permission 'aiplatform.endpoints.predict' denied on resource '//aiplatform.googleapis.com/projects/xxx/locations/global/publishers/google/models/gemini-3.5-flash' (or it may not exist).

SOLUTION

透過 "IAM 與管理" 針對服務帳戶增加 "Vertex AI: Agent Platform 使用者" 權限

© 2026 Enos Chou

服務帳戶建立與調整

4. 調整權限案例

其他常見服務帳戶權限需求

產品	角色類別	角色
Artifact Registry	Artifact Registry	Artifact Registry 推送建立寫入者
Cloud Firestore	Datastore	Cloud Datastore 使用者
Cloud Run Job	Cloud Run	Cloud Run 叫用者
Cloud Storage	Cloud Storage	Storage 物件使用者 Storage 檢視者
Gemini	Vertex AI	Agent Platform 使用者

© 2026 Enos Chou

服務帳戶使用

四種常見用戶端初始化服務帳戶範例 (以 Cloud Storage 為例)

case 1: 直接指定服務帳戶金鑰

```
from google.cloud import storage

storage_client = storage.Client.from_service_account_json(
    service_account_key_file_name_str
)
```

© 2026 Enos Chou

服務帳戶使用

四種常見用戶端初始化服務帳戶範例 (以 Cloud Storage 為例)

case 2: 以服務帳戶金鑰建立憑證

```
from google.cloud import storage
from google.oauth2 import service_account

credentials = service_account.Credentials.from_service_account_file(
    service_account_key_file_name_str,
    scopes=['https://www.googleapis.com/auth/cloud-platform'])
storage_client = storage.Client(credentials=credentials)
```

© 2026 Enos Chou

服務帳戶使用

四種常見用戶端初始化服務帳戶範例 (以 Cloud Storage 為例)

case 3: 以環境變數指定服務帳戶金鑰

```
import os

from google.cloud import storage

YOUR_SERVICE = service_account_key_file_name_str

os.environ['GOOGLE_APPLICATION_CREDENTIALS'] = YOUR_SERVICE
storage_client = storage.Client()
```

© 2026 Enos Chou

服務帳戶使用

四種常見用戶端初始化服務帳戶範例 (以 Cloud Storage 為例)

case 4: Google Cloud 指定服務帳戶金鑰

```
from google.cloud import storage

storage_client = storage.Client()
```

© 2026 Enos Chou

服務帳戶使用

快速驗證服務帳戶範例 (以 Cloud Storage 為例)

安裝套件

```
pip install google-cloud-storage
```

© 2026 Enos Chou

服務帳戶使用

快速驗證服務帳戶範例 (以 Cloud Storage 為例)

範例程式

```
from google.cloud import storage

storage_client = storage.Client.from_service_account_json(
    service_account_key_file_name_str
)
print(list(storage_client.list_buckets()))
```

© 2026 Enos Chou

GCP AI API 整合應用

Cloud Speech-to-Text API

Speech-to-Text Application

Cloud Vision API

Vision Application

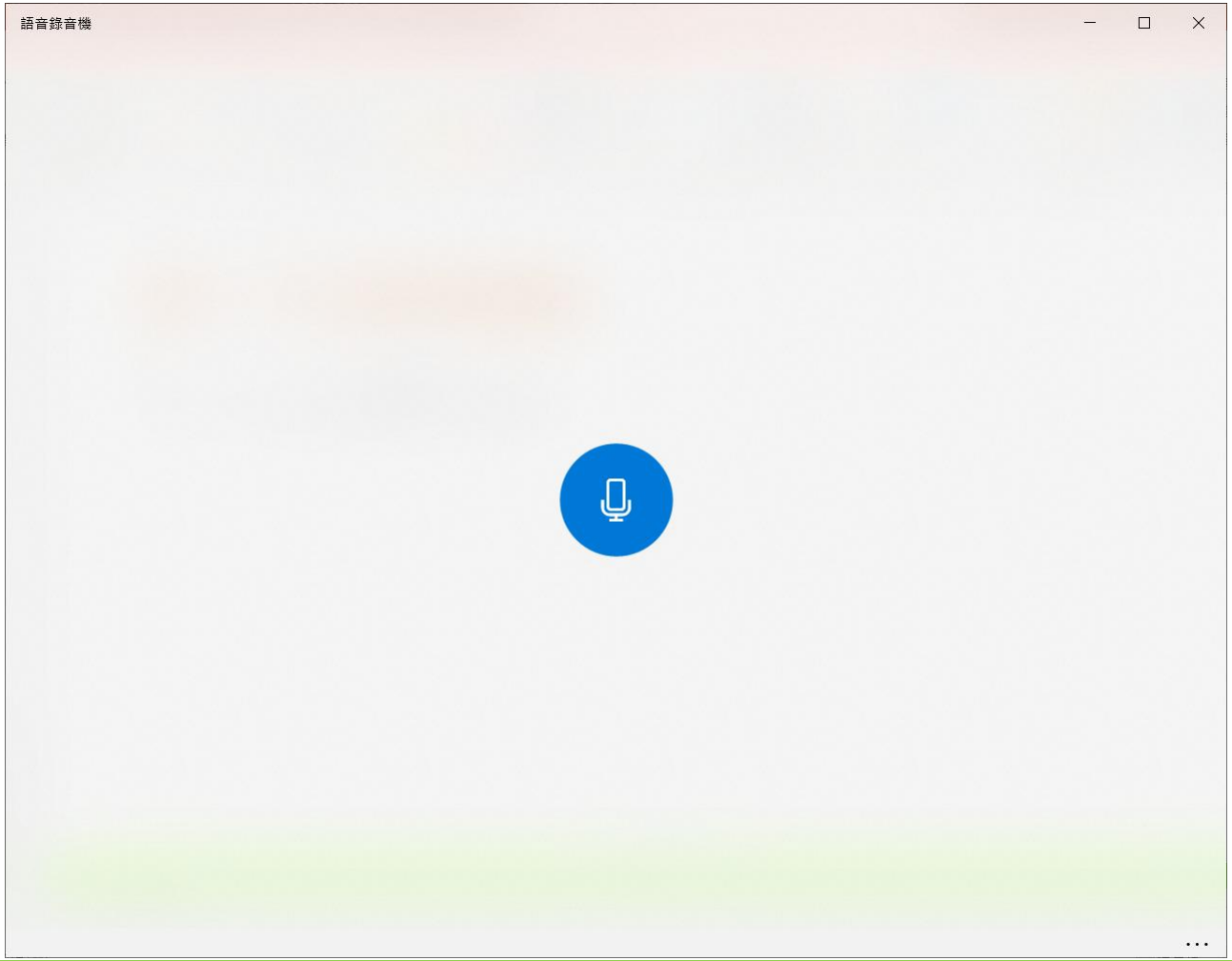
© 2026 Enos Chou

Cloud Speech-to-Text API

製作樣本

- Windows 10 語言錄音機
  Windows 鍵 ➔
  14筆 語音錄音機
- macOS 語音備忘錄 or 錄音機

自行錄製中文語音並儲存

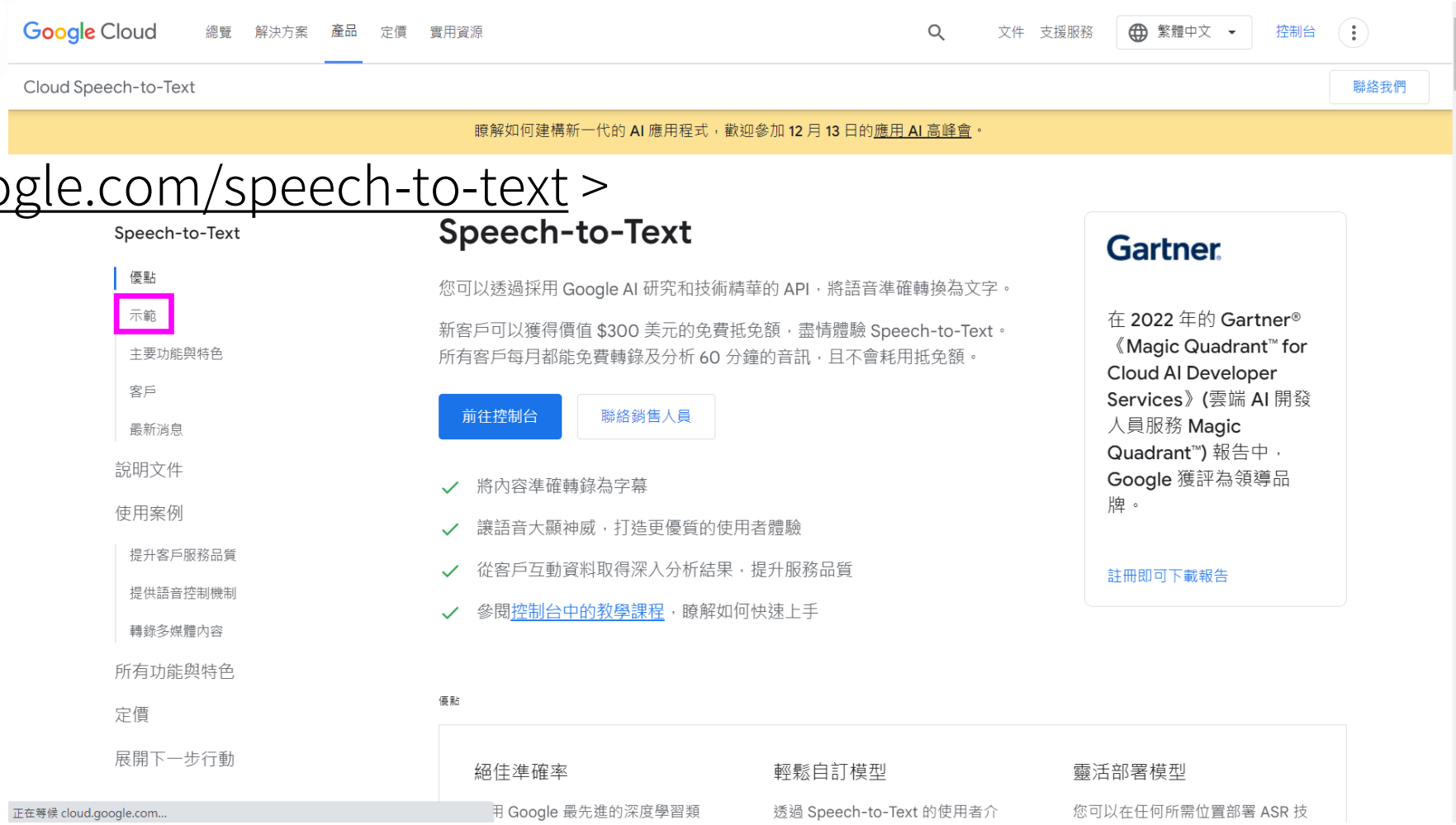


© 2026 Enos Chou

Cloud Speech-to-Text API

先試試

https://cloud.google.com/speech-to-text > 示範

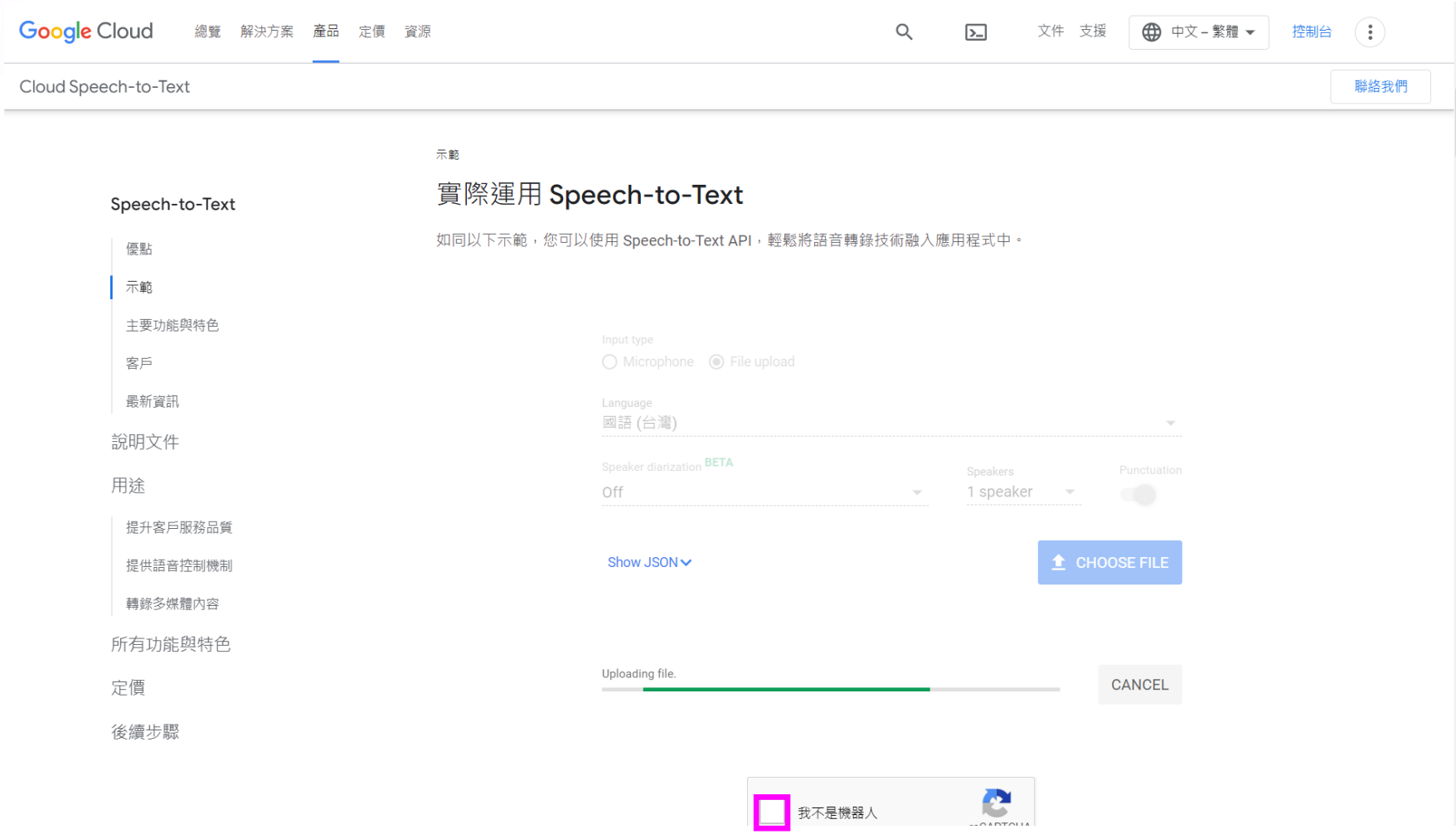


Cloud Speech-to-Text API

先試試

Cloud Speech-to-Text API

先試試



Cloud Speech-to-Text API

先試試

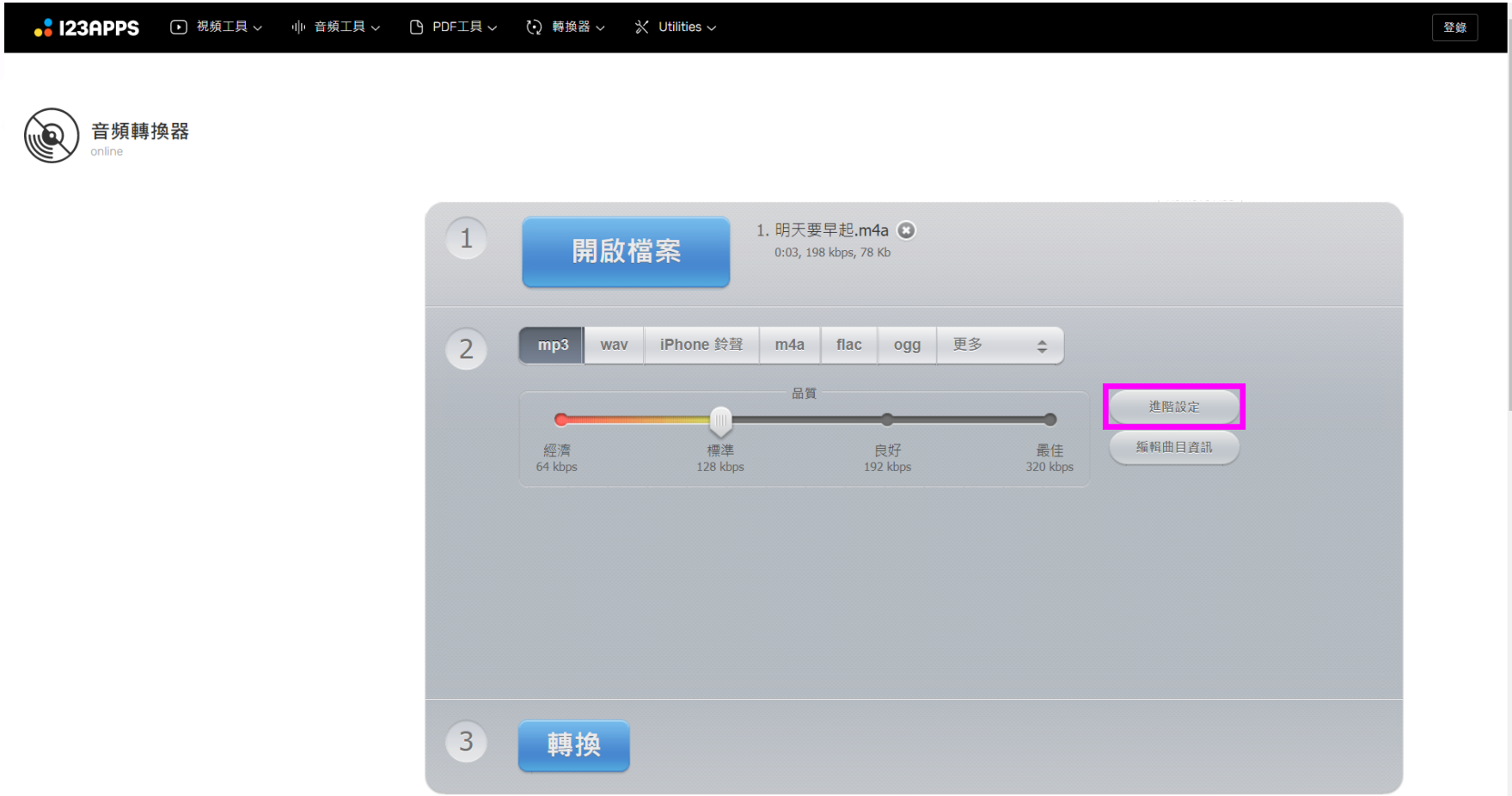
Cloud Speech-to-Text API

轉換語音編碼

https://online-audio-converter.com/tw/

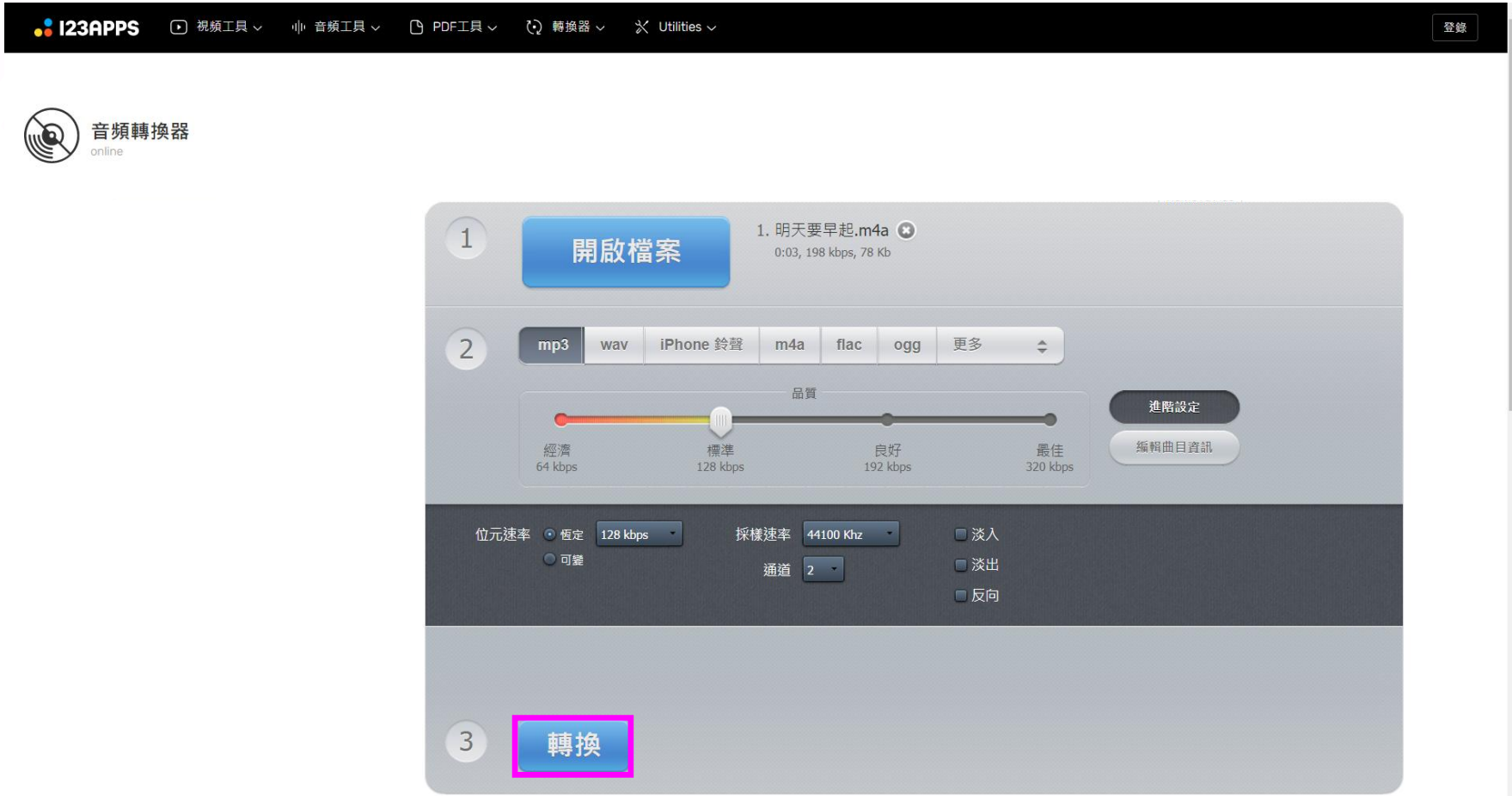
Cloud Speech-to-Text API

轉換語音編碼



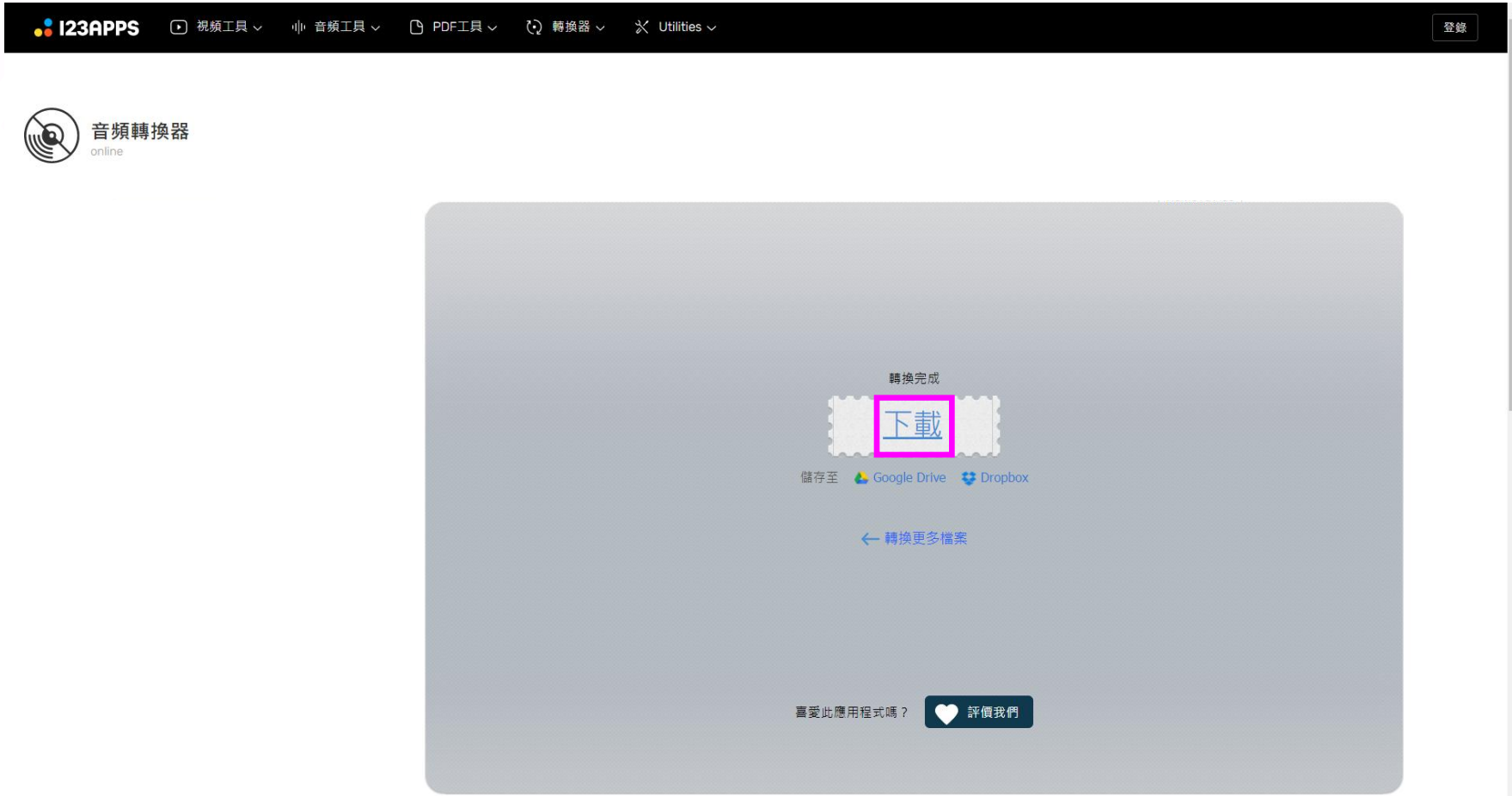
Cloud Speech-to-Text API

轉換語音編碼



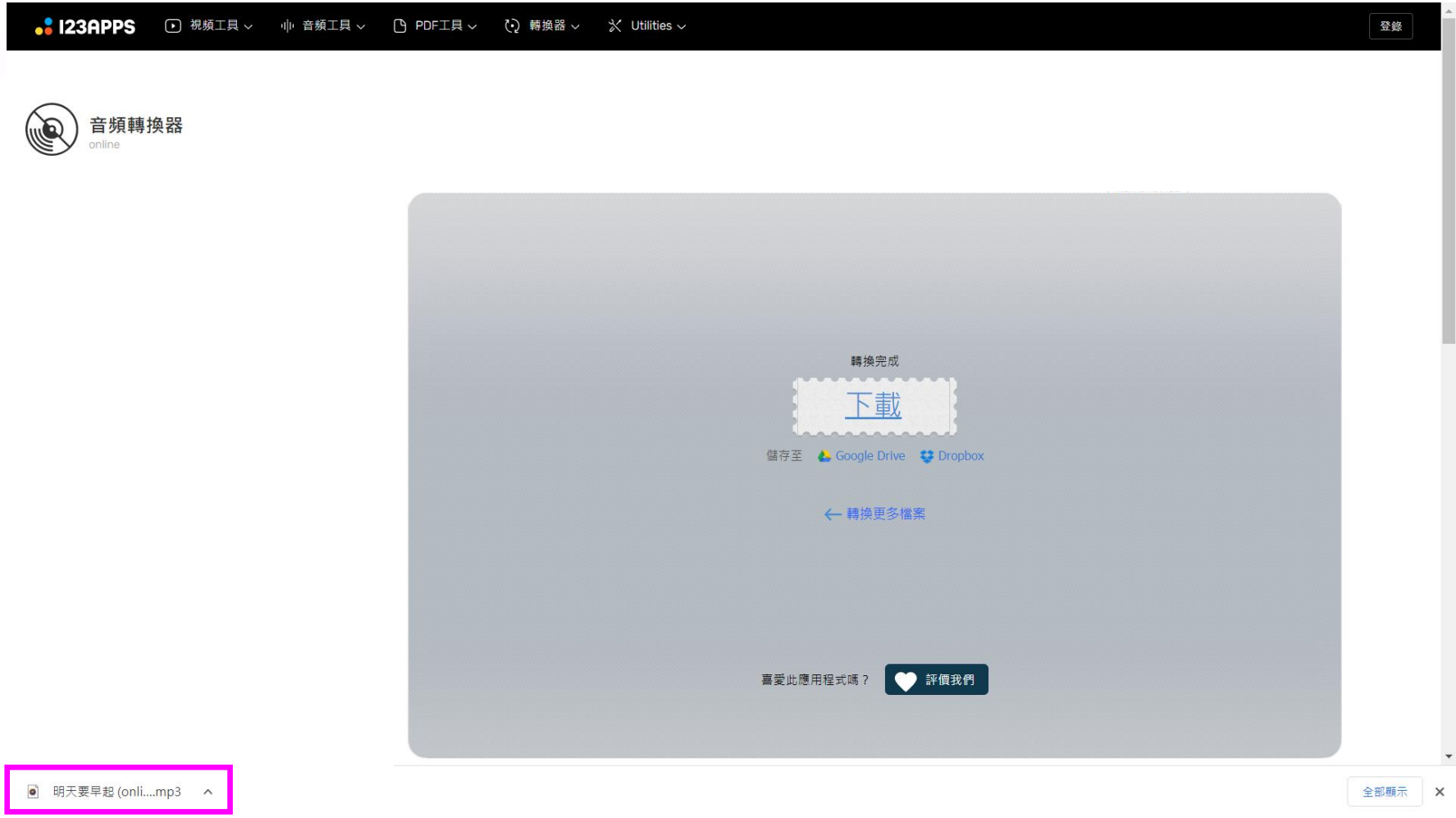
Cloud Speech-to-Text API

轉換語音編碼



Cloud Speech-to-Text API

轉換語音編碼



Cloud Speech-to-Text API

價格

https://cloud.google.com/speech-to-text >
定價 >
定價 >
查看價格詳細資料

© 2026 Enos Chou

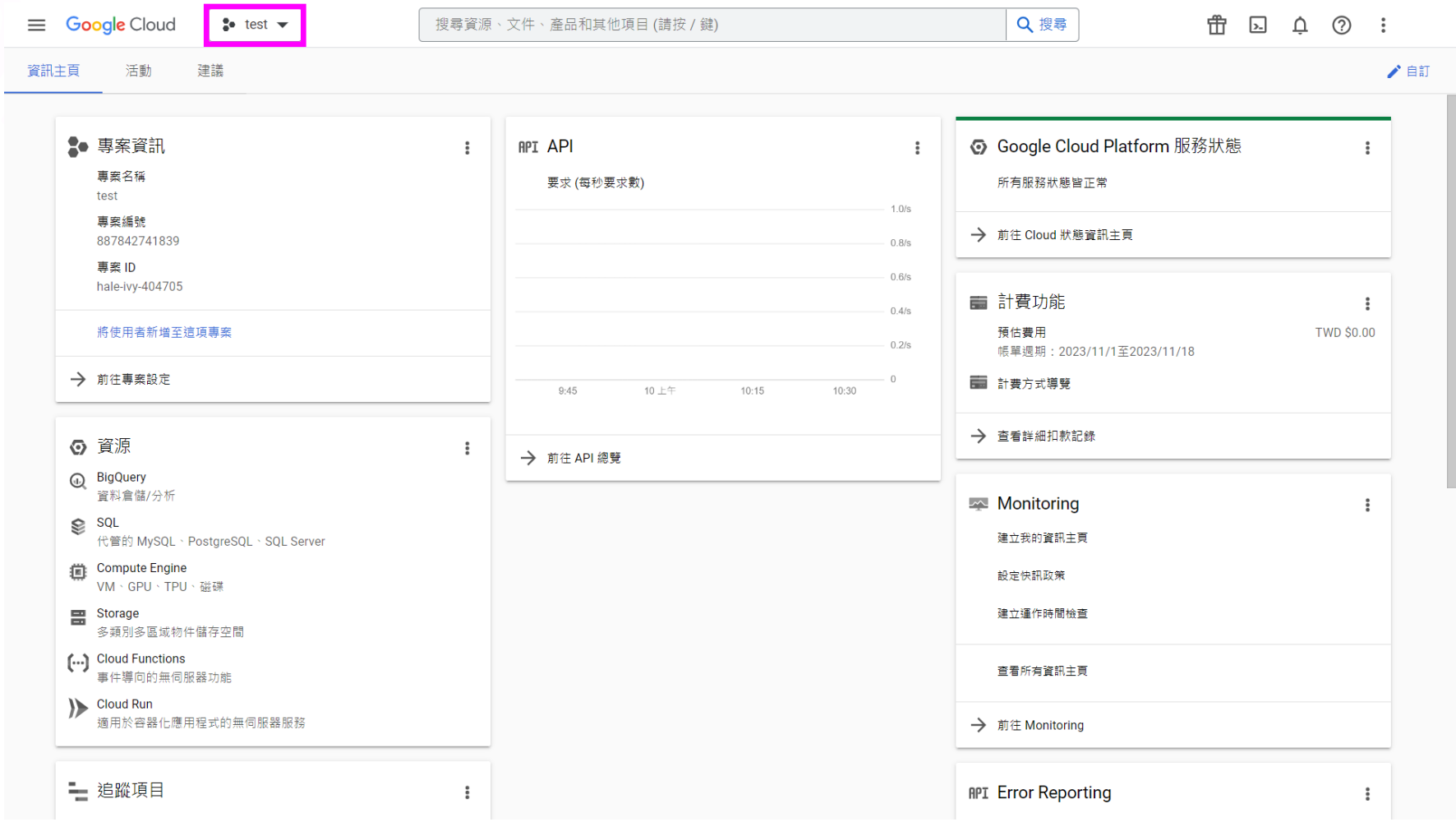
Cloud Speech-to-Text API

1. 啟用 API
2. 建立轉錄工作
3. 檢視轉錄結果

© 2026 Enos Chou

Cloud Speech-to-Text API

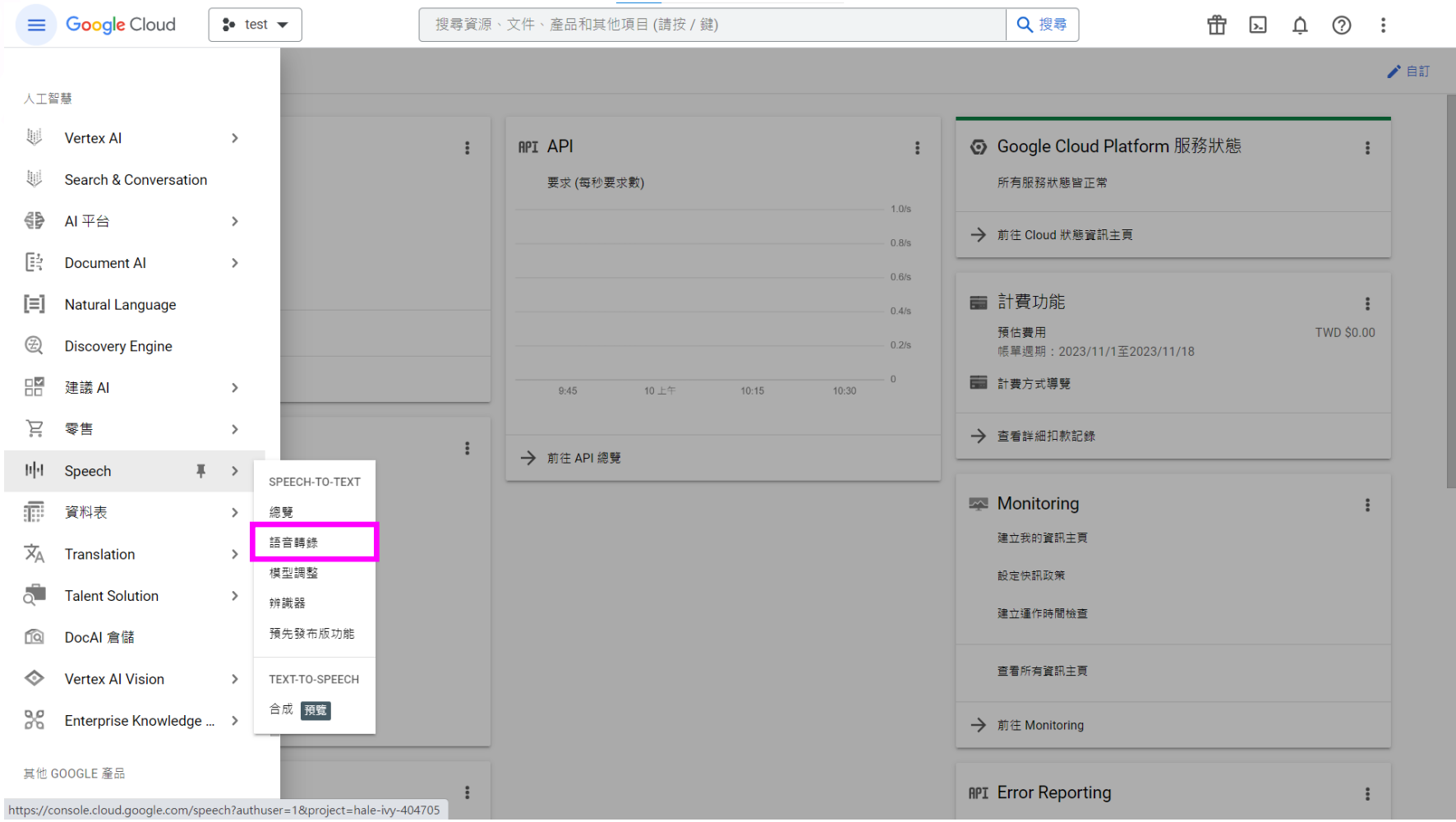
1. 啟用 API



© 2026 Enos Chou

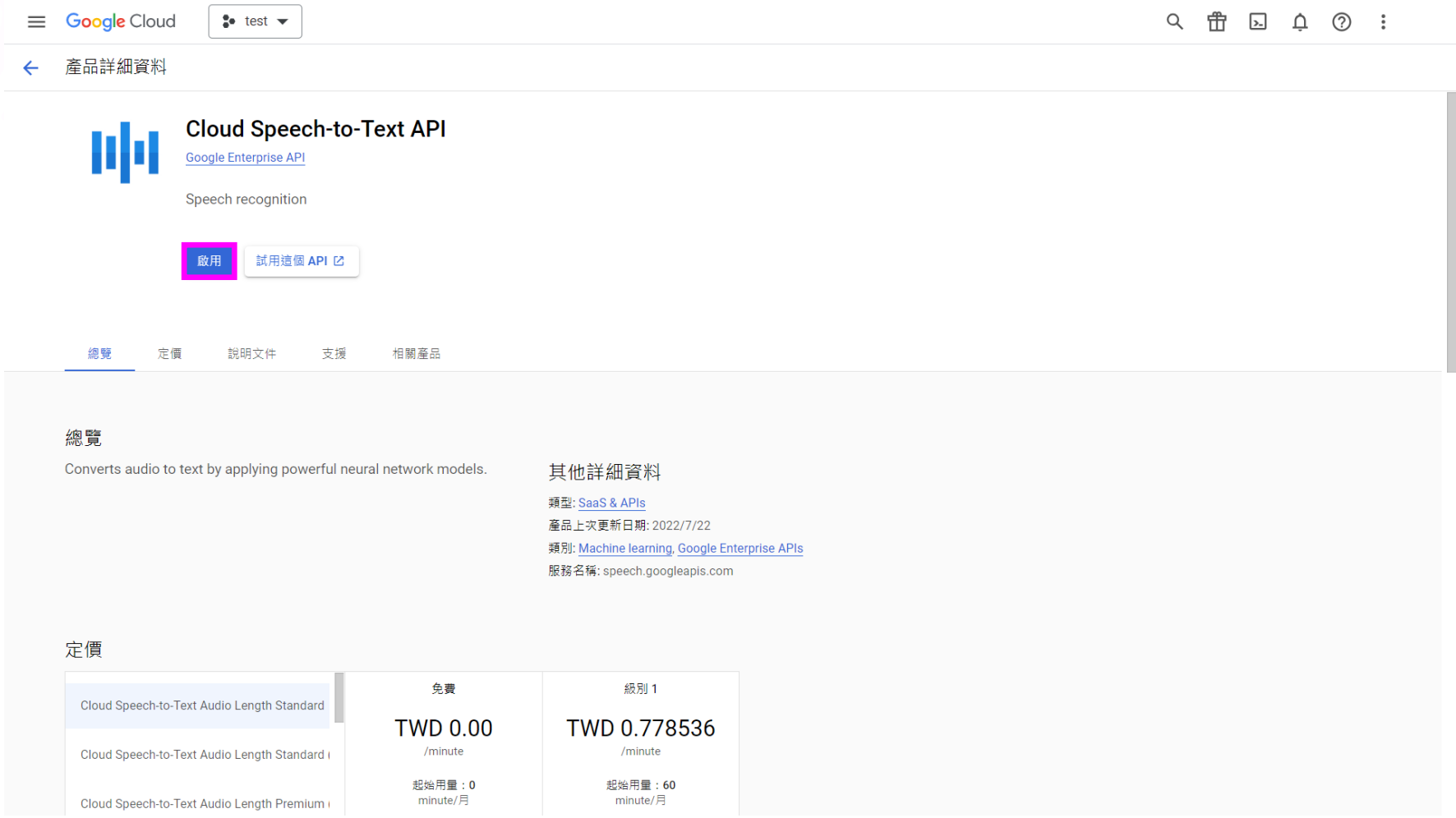
Cloud Speech-to-Text API

1. 啟用 API



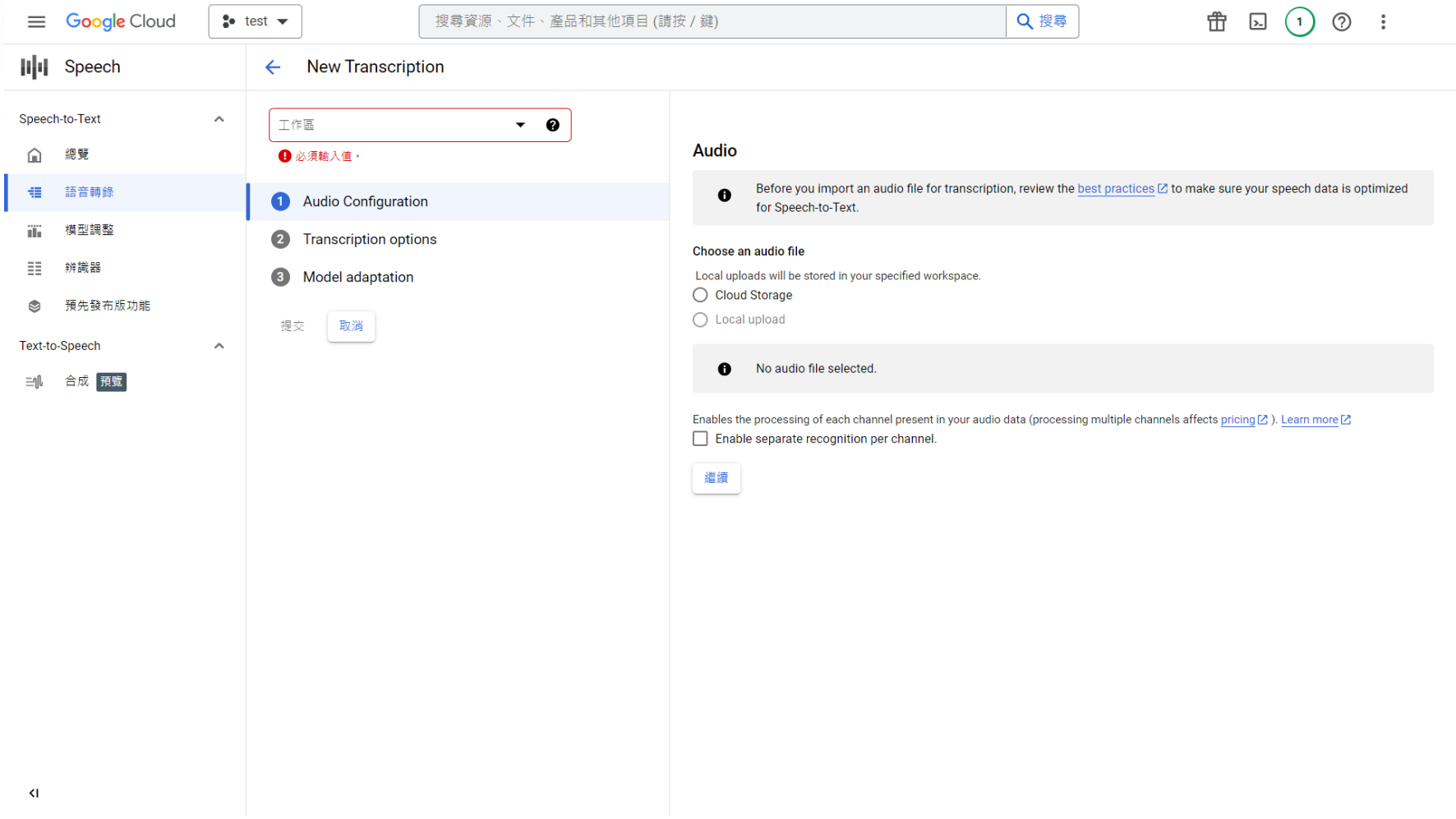
Cloud Speech-to-Text API

1. 啟用 API



Cloud Speech-to-Text API

1. 啟用 API

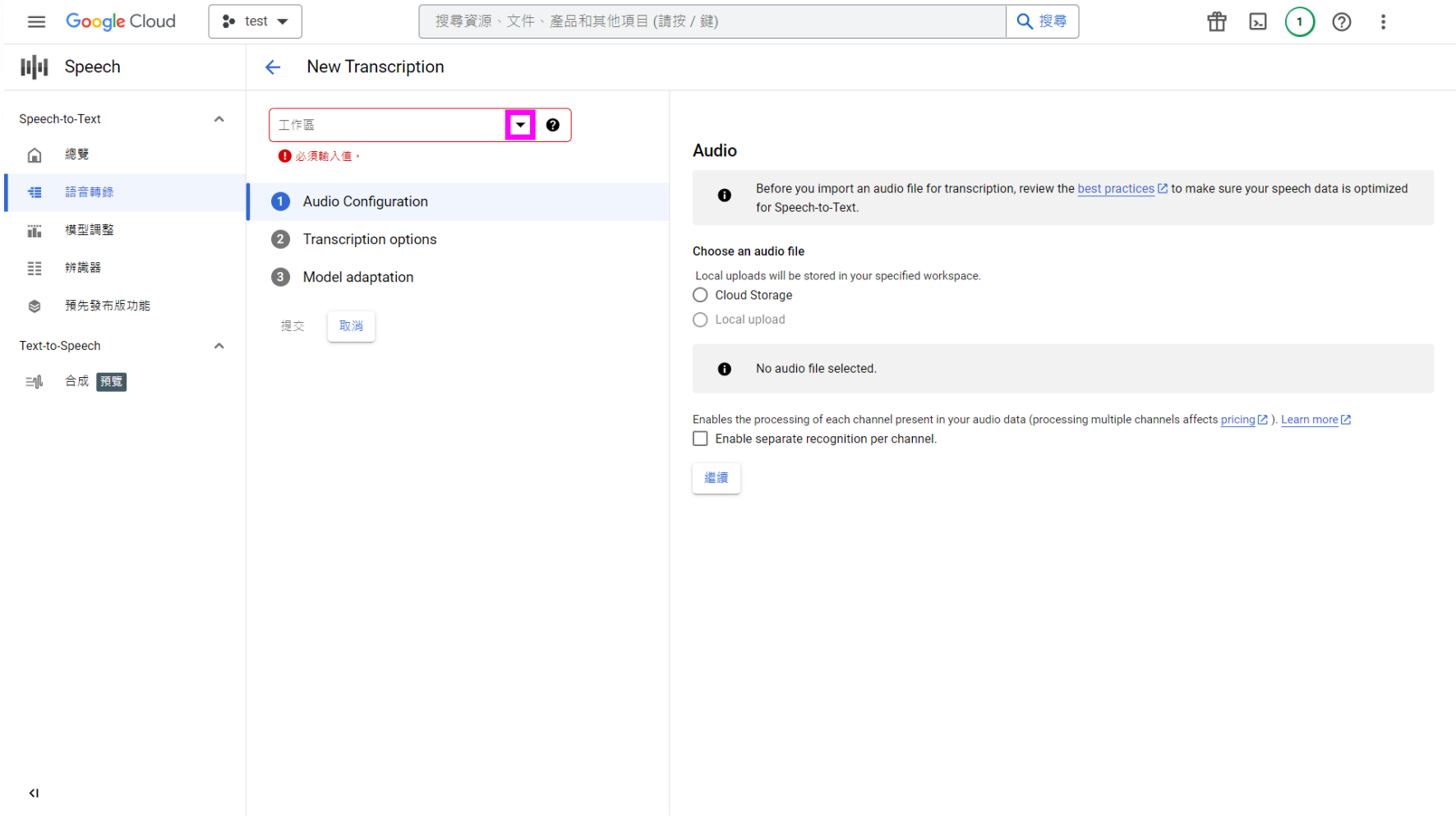


© 2026 Enos Chou

Cloud Speech-to-Text API

2. 建立轉錄工作

a. 建立工作區

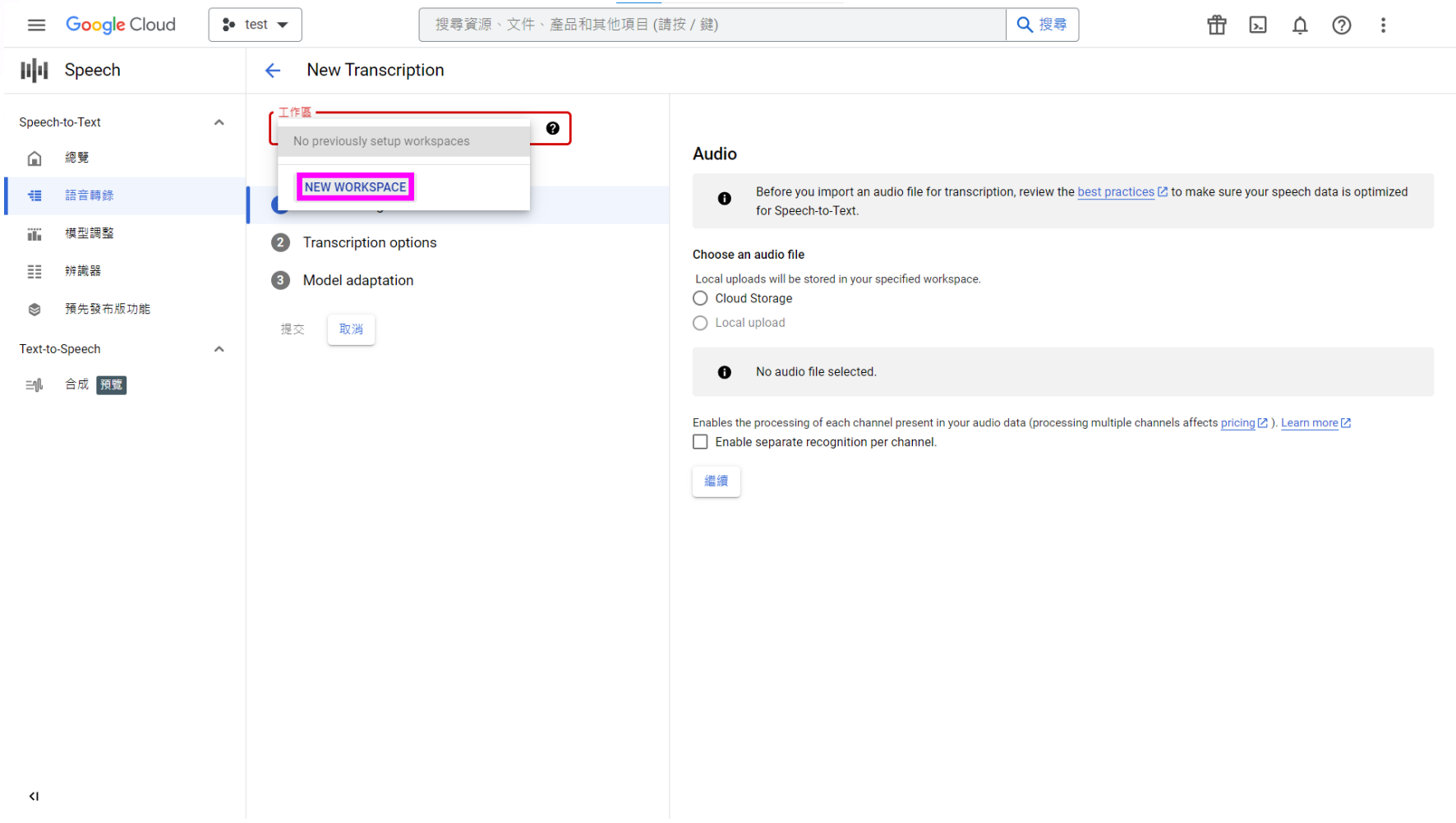


© 2026 Enos Chou

Cloud Speech-to-Text API

2. 建立轉錄工作

a. 建立工作區

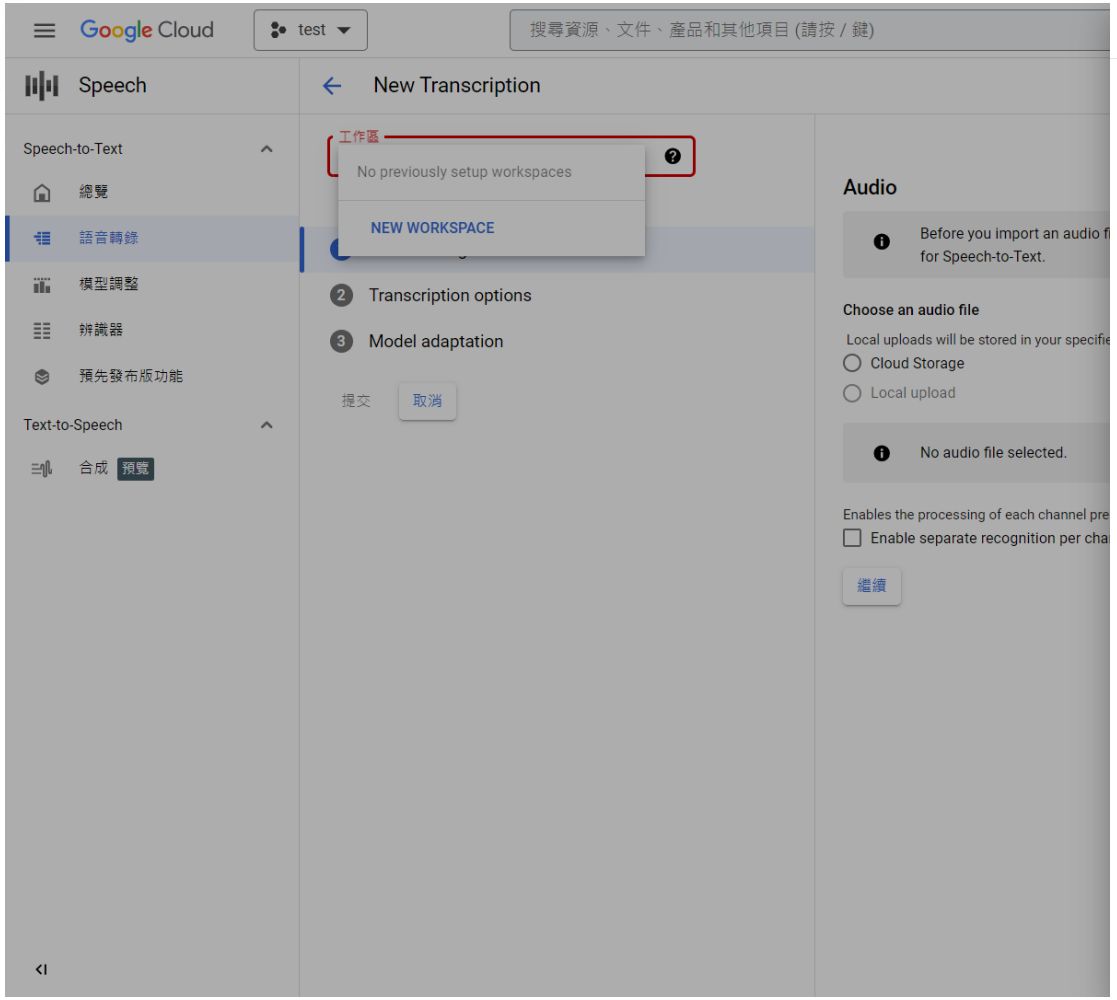


© 2026 Enos Chou

Cloud Speech-to-Text API

2. 建立轉錄工作

a. 建立工作區

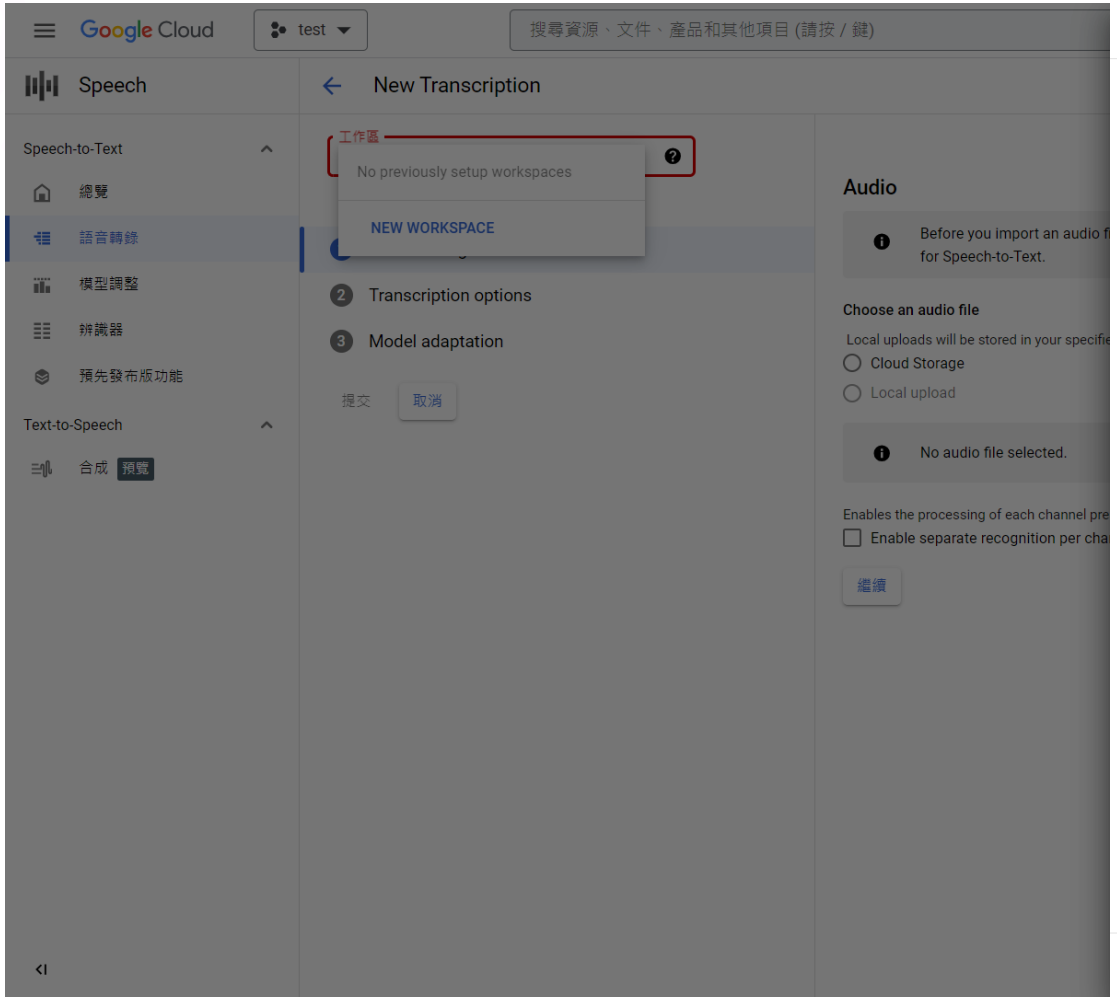


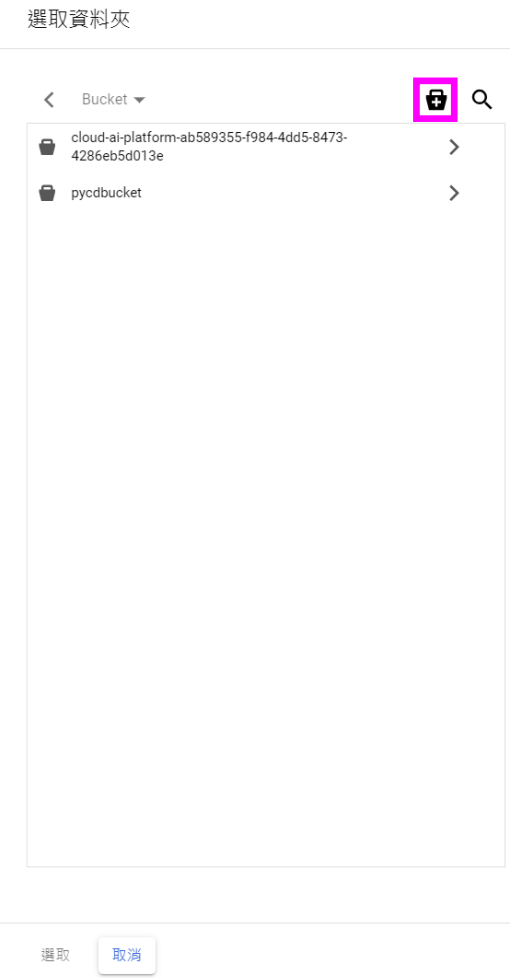
© 2026 Enos Chou

Cloud Speech-to-Text API

2. 建立轉錄工作

a. 建立工作區



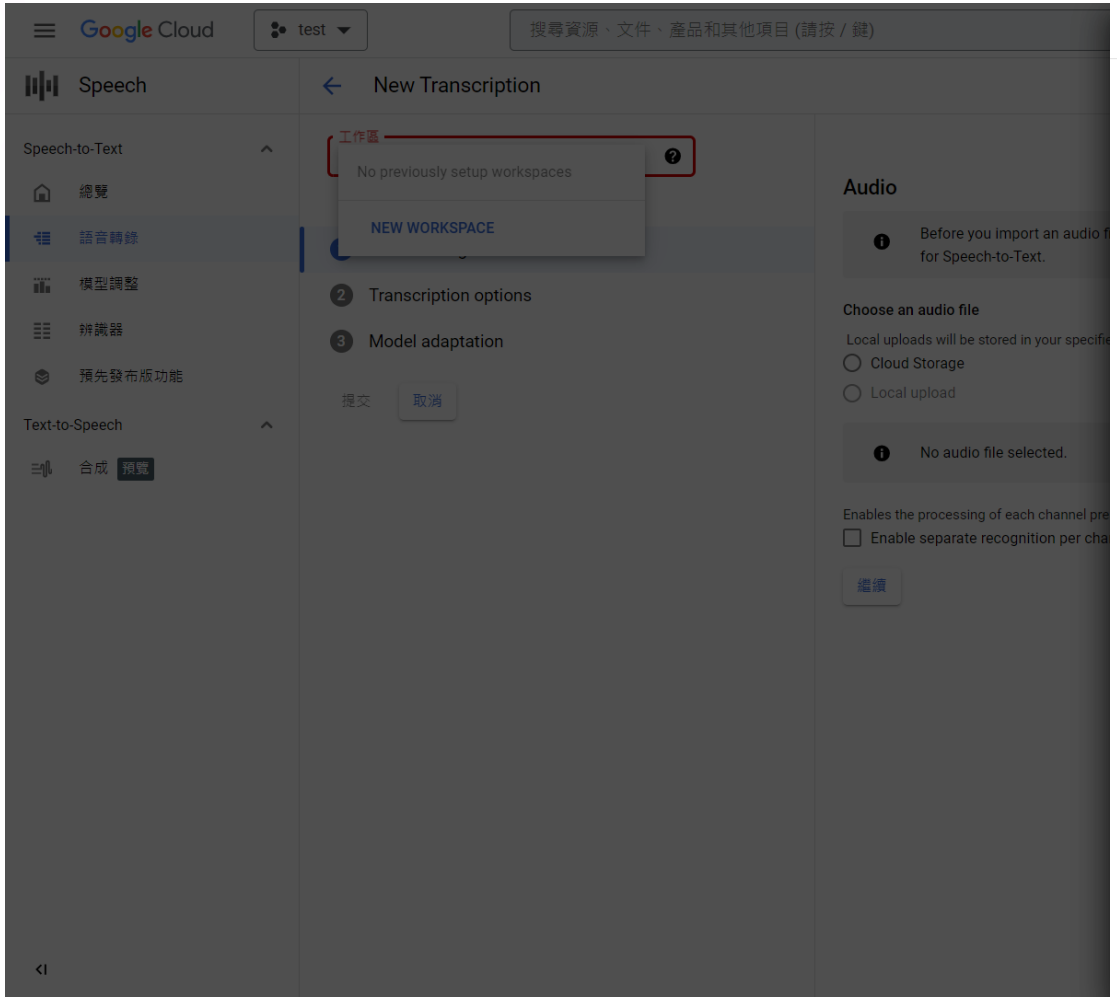


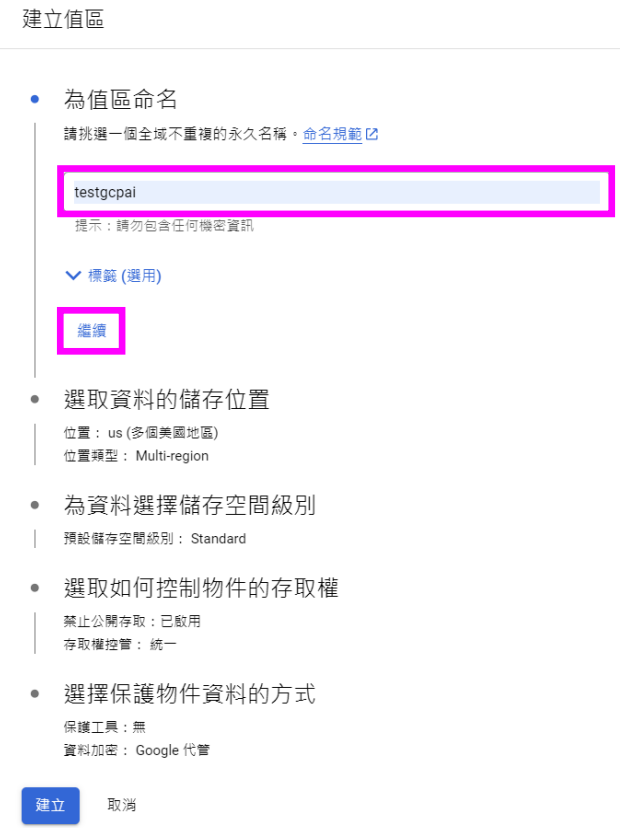
© 2026 Enos Chou

Cloud Speech-to-Text API

2. 建立轉錄工作

a. 建立工作區



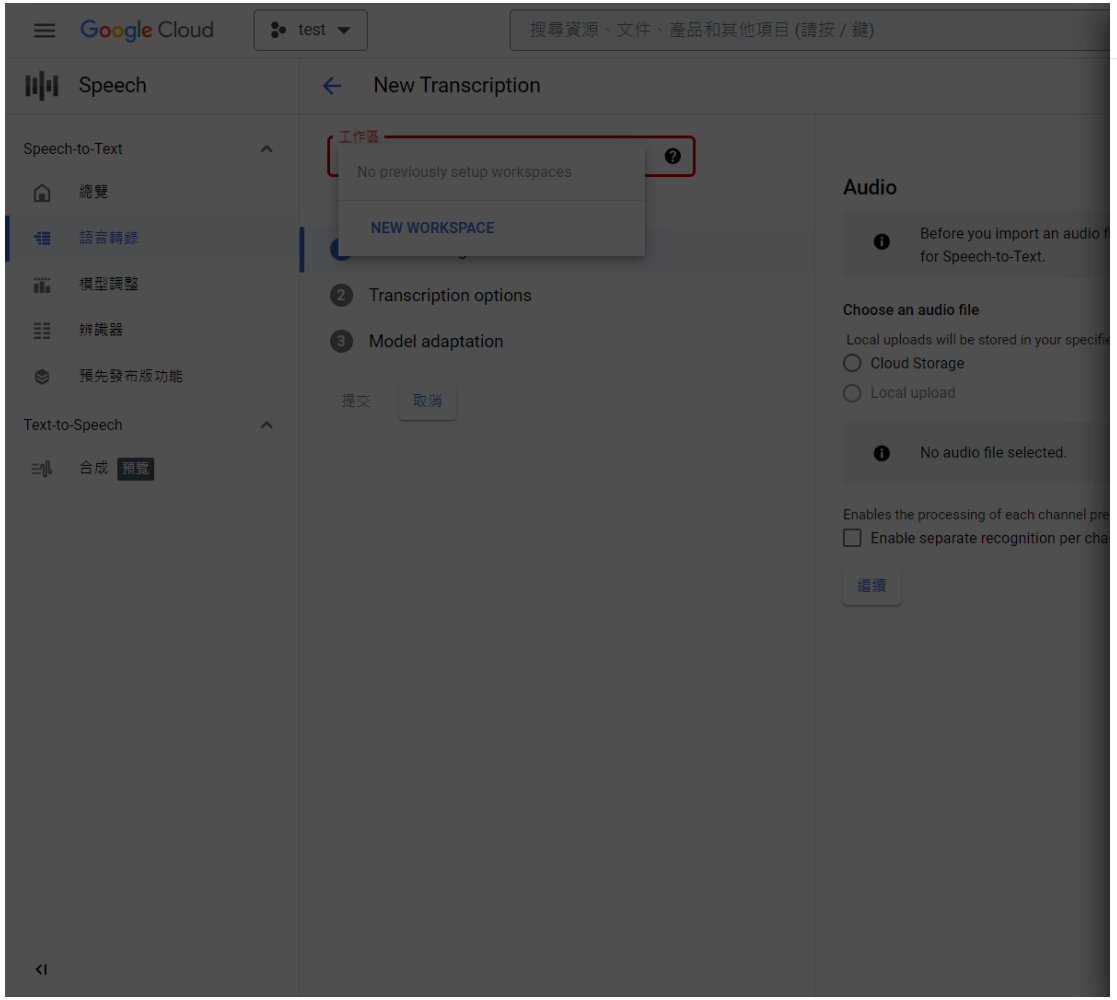


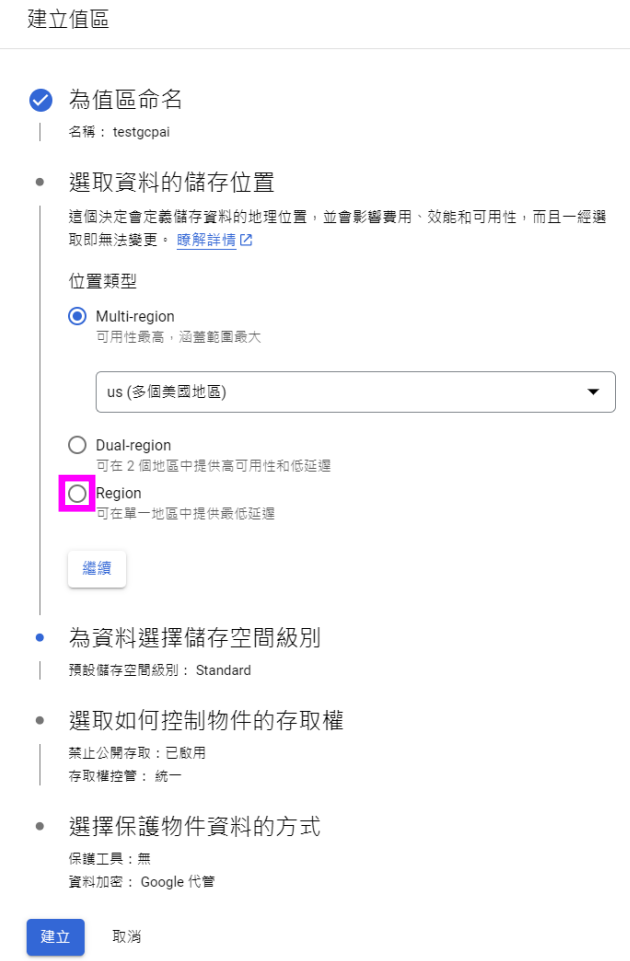
© 2026 Enos Chou

Cloud Speech-to-Text API

2. 建立轉錄工作

a. 建立工作區



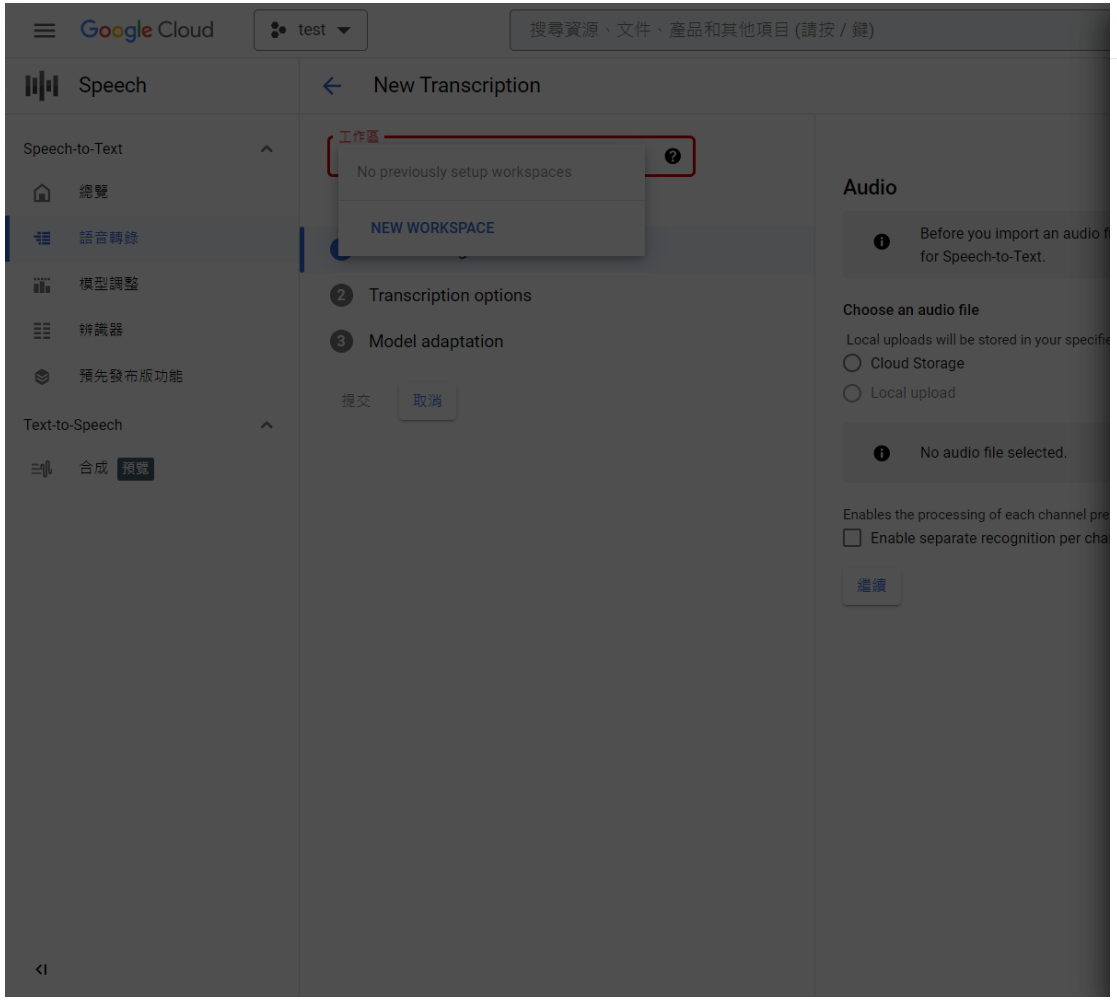


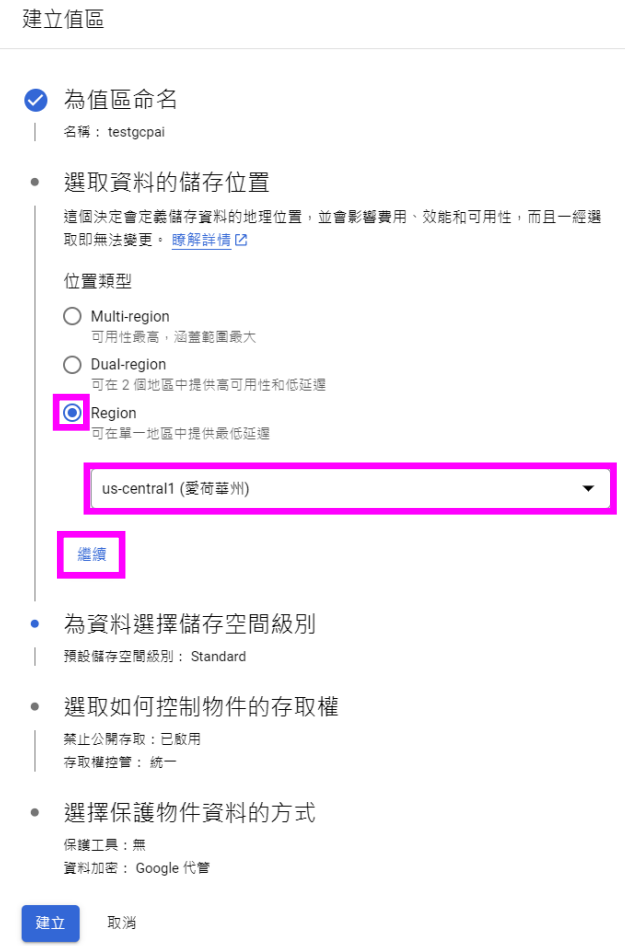
© 2026 Enos Chou

Cloud Speech-to-Text API

2. 建立轉錄工作

a. 建立工作區



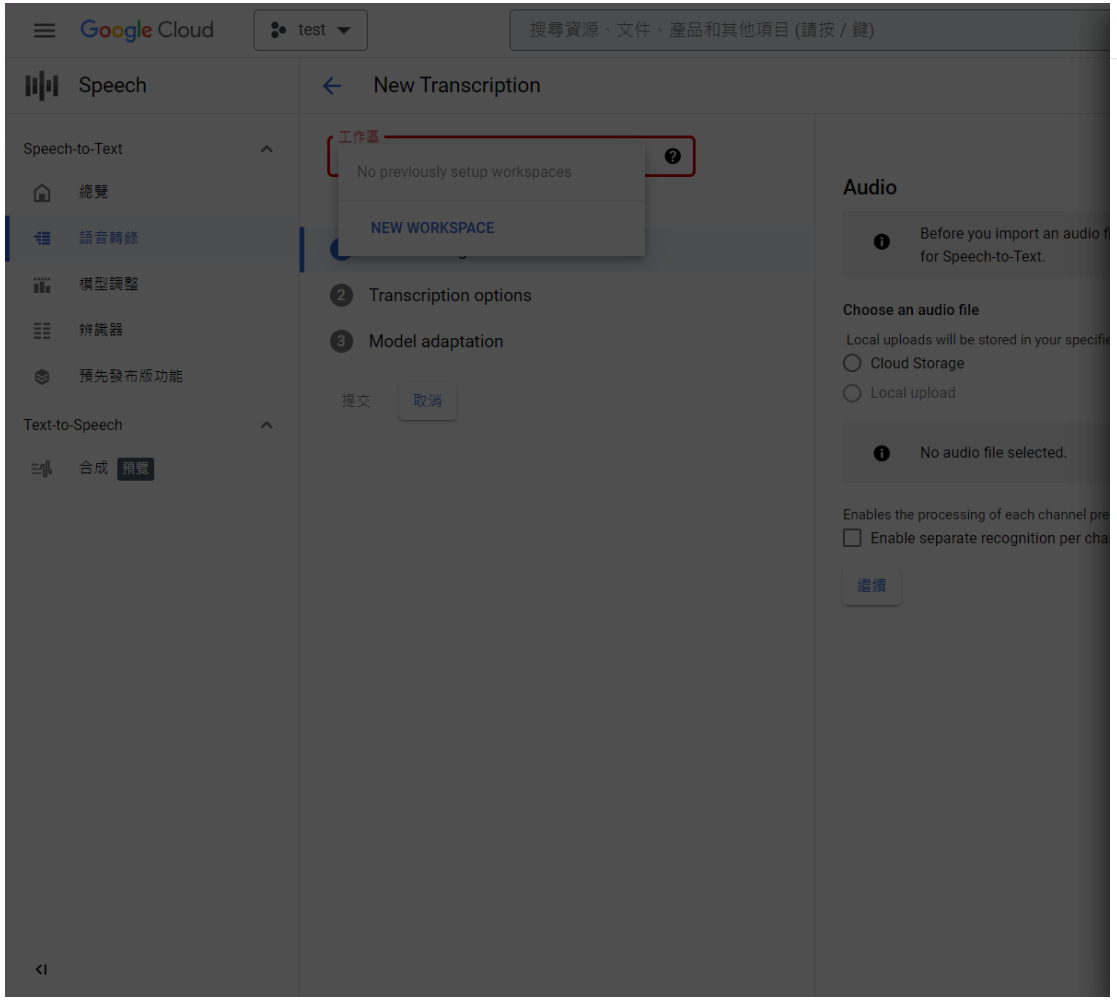


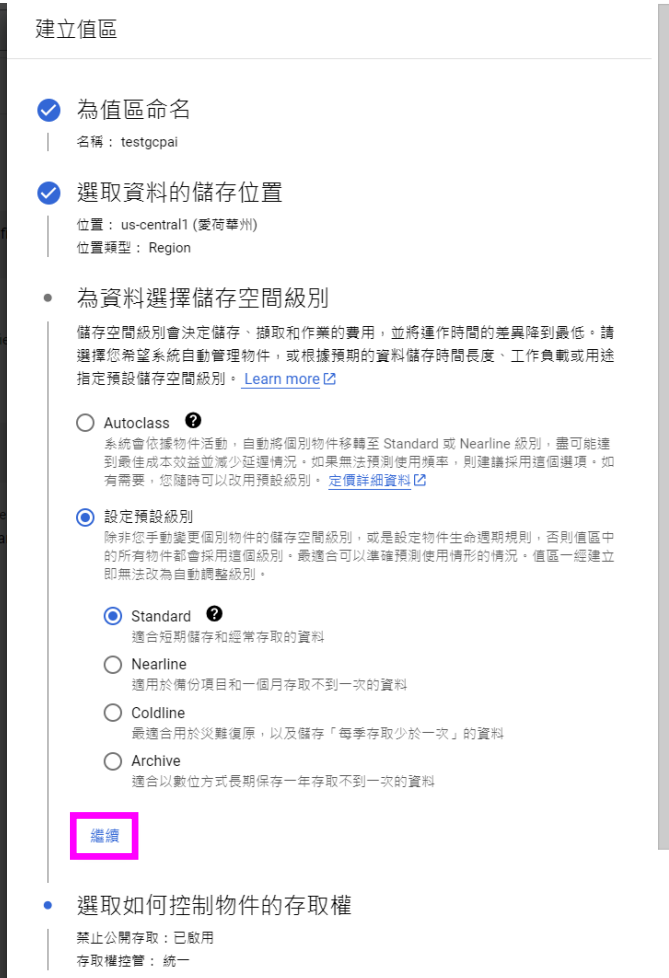
© 2026 Enos Chou

Cloud Speech-to-Text API

2. 建立轉錄工作

a. 建立工作區



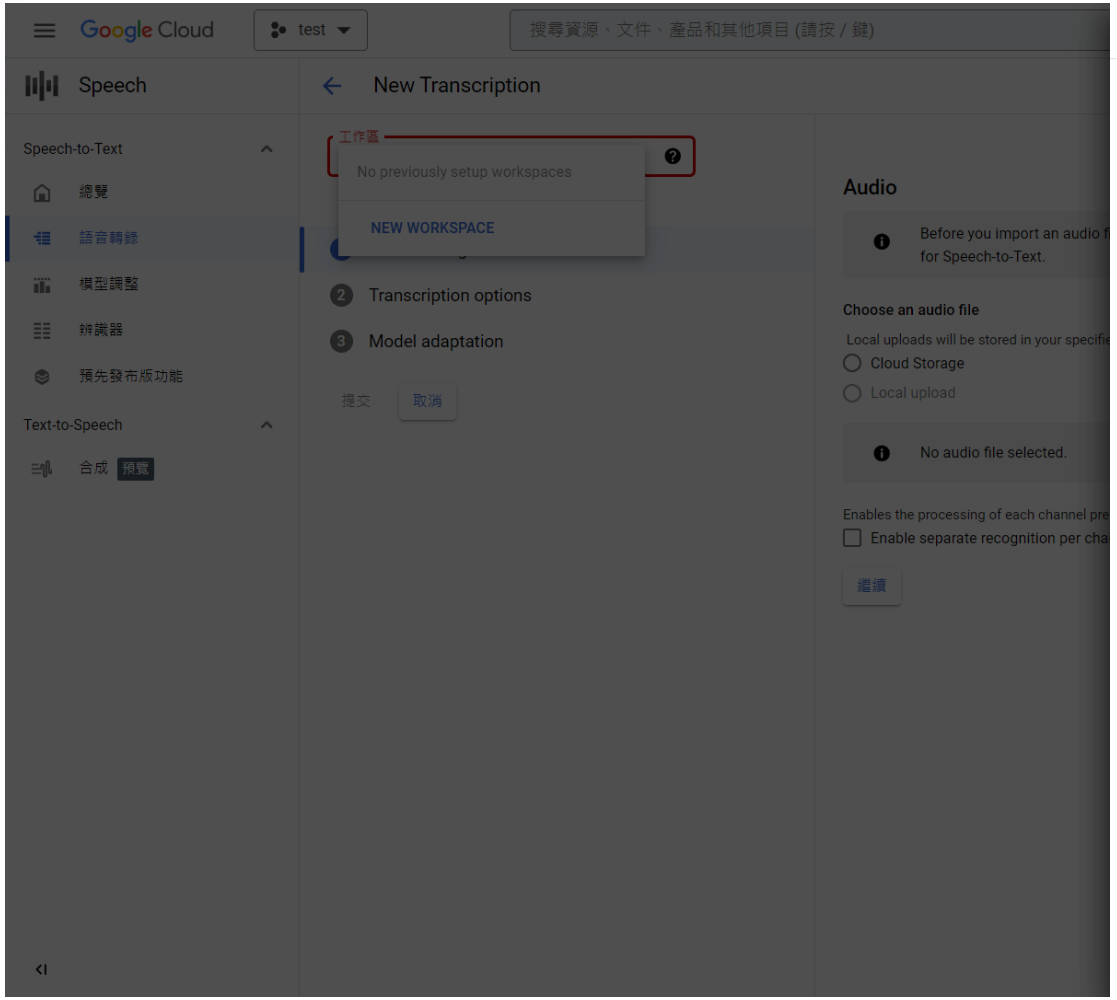


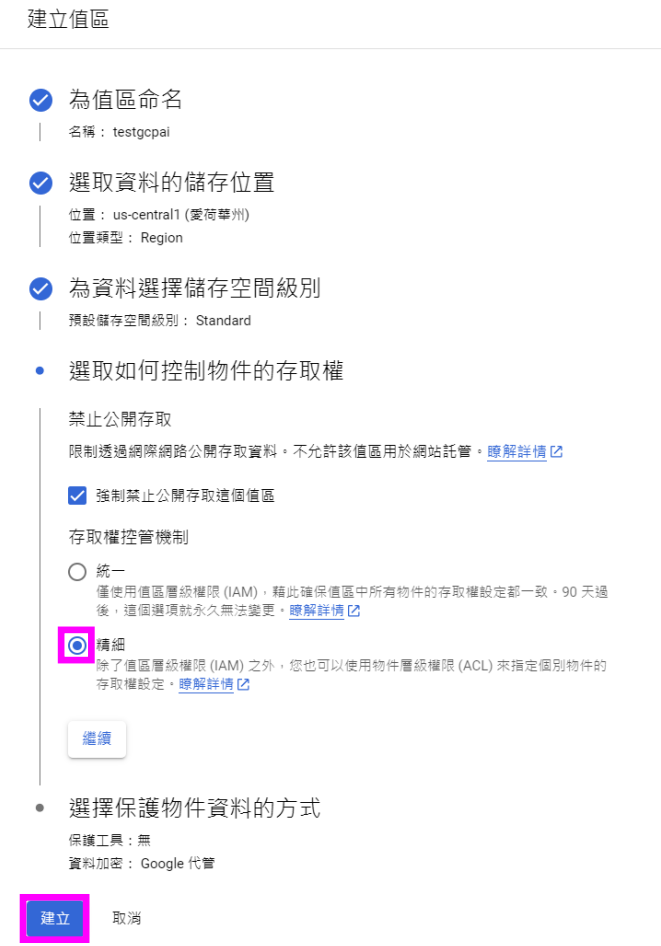
© 2026 Enos Chou

Cloud Speech-to-Text API

2. 建立轉錄工作

a. 建立工作區



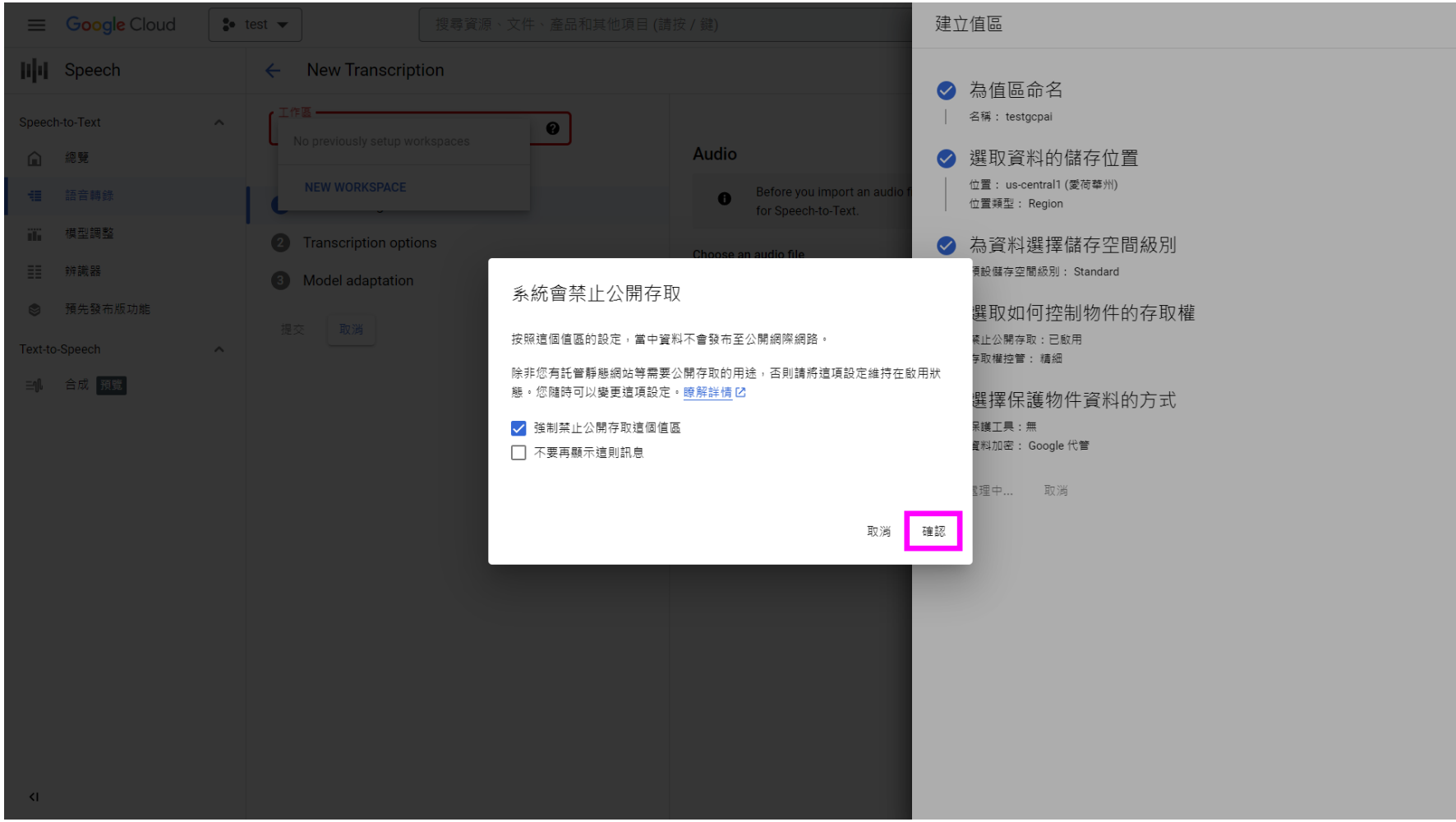


© 2026 Enos Chou

Cloud Speech-to-Text API

2. 建立轉錄工作

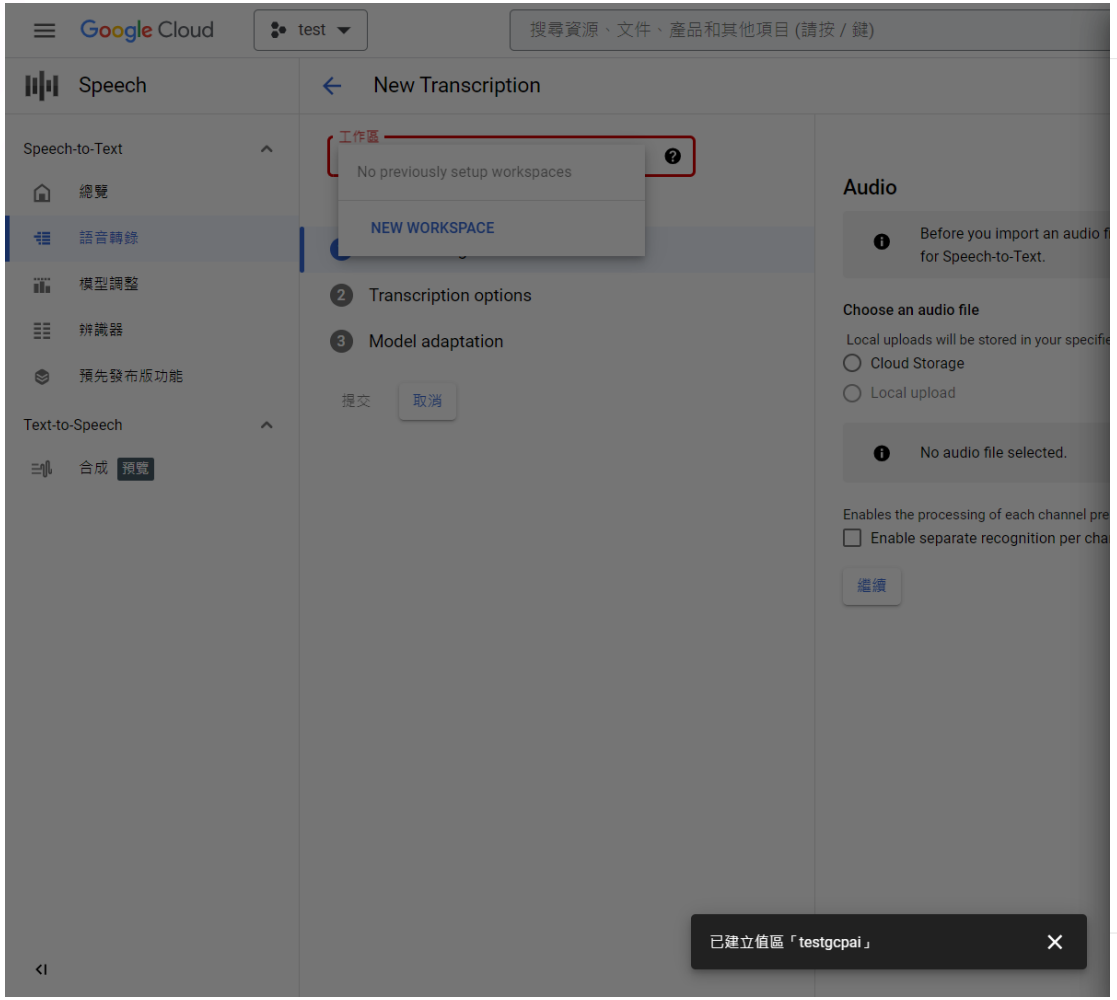
a. 建立工作區

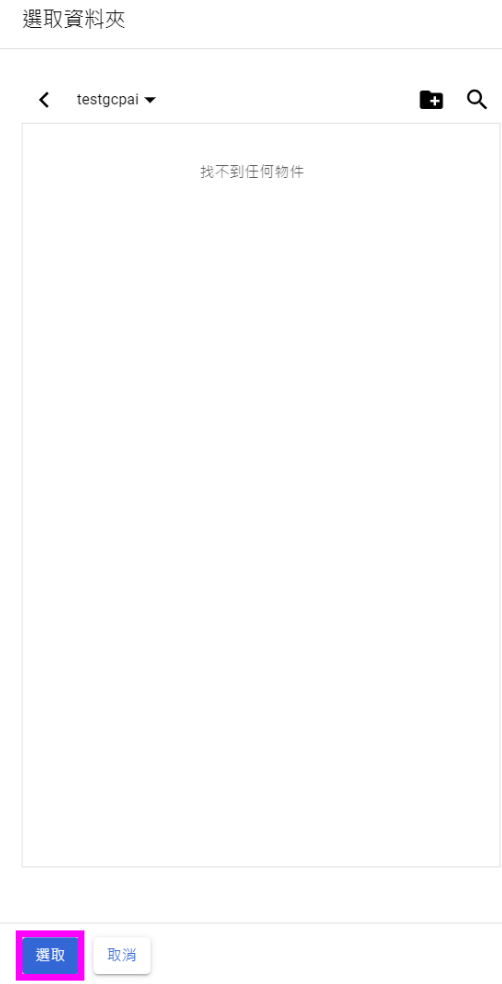


Cloud Speech-to-Text API

2. 建立轉錄工作

a. 建立工作區

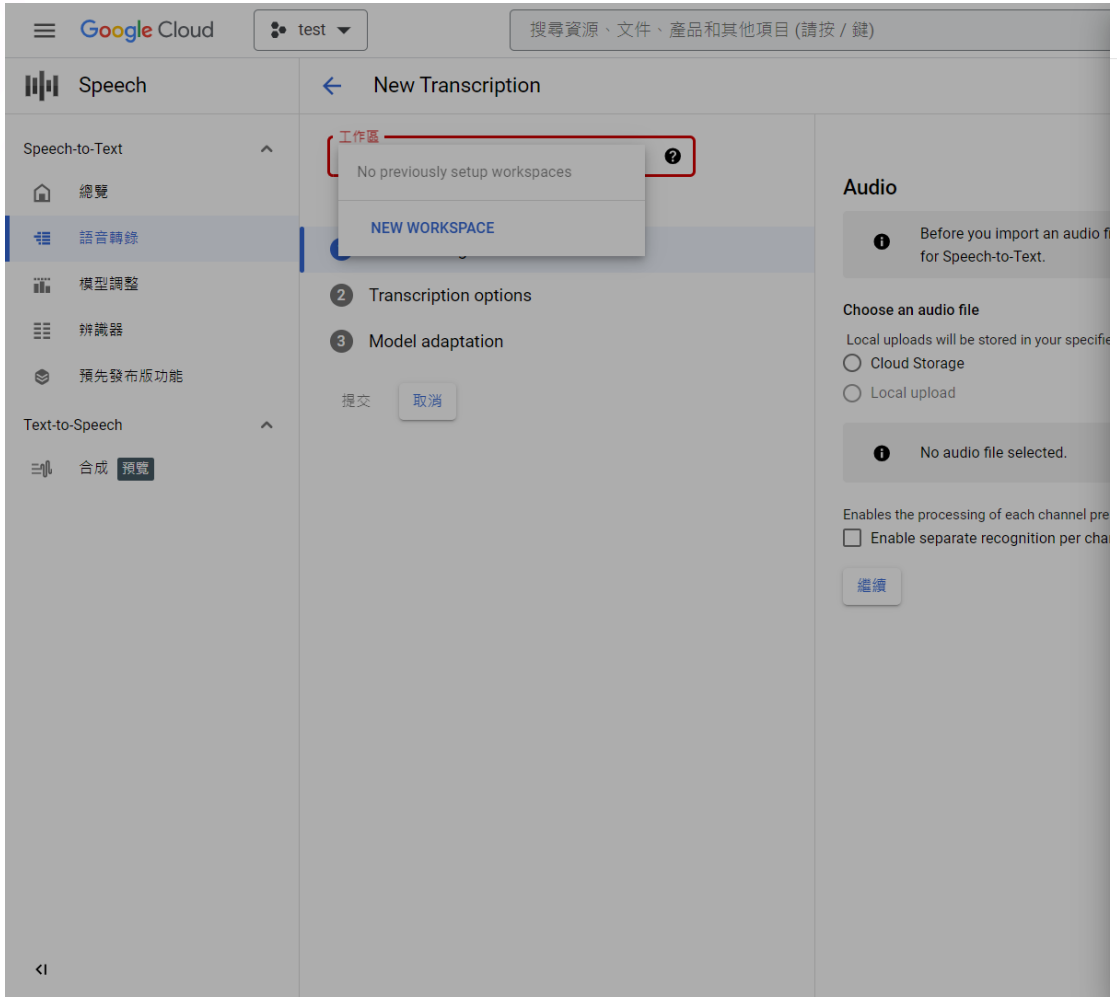


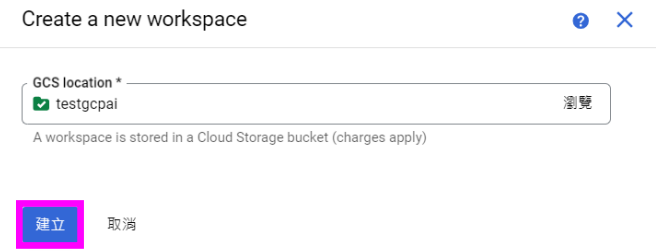


Cloud Speech-to-Text API

2. 建立轉錄工作

a. 建立工作區



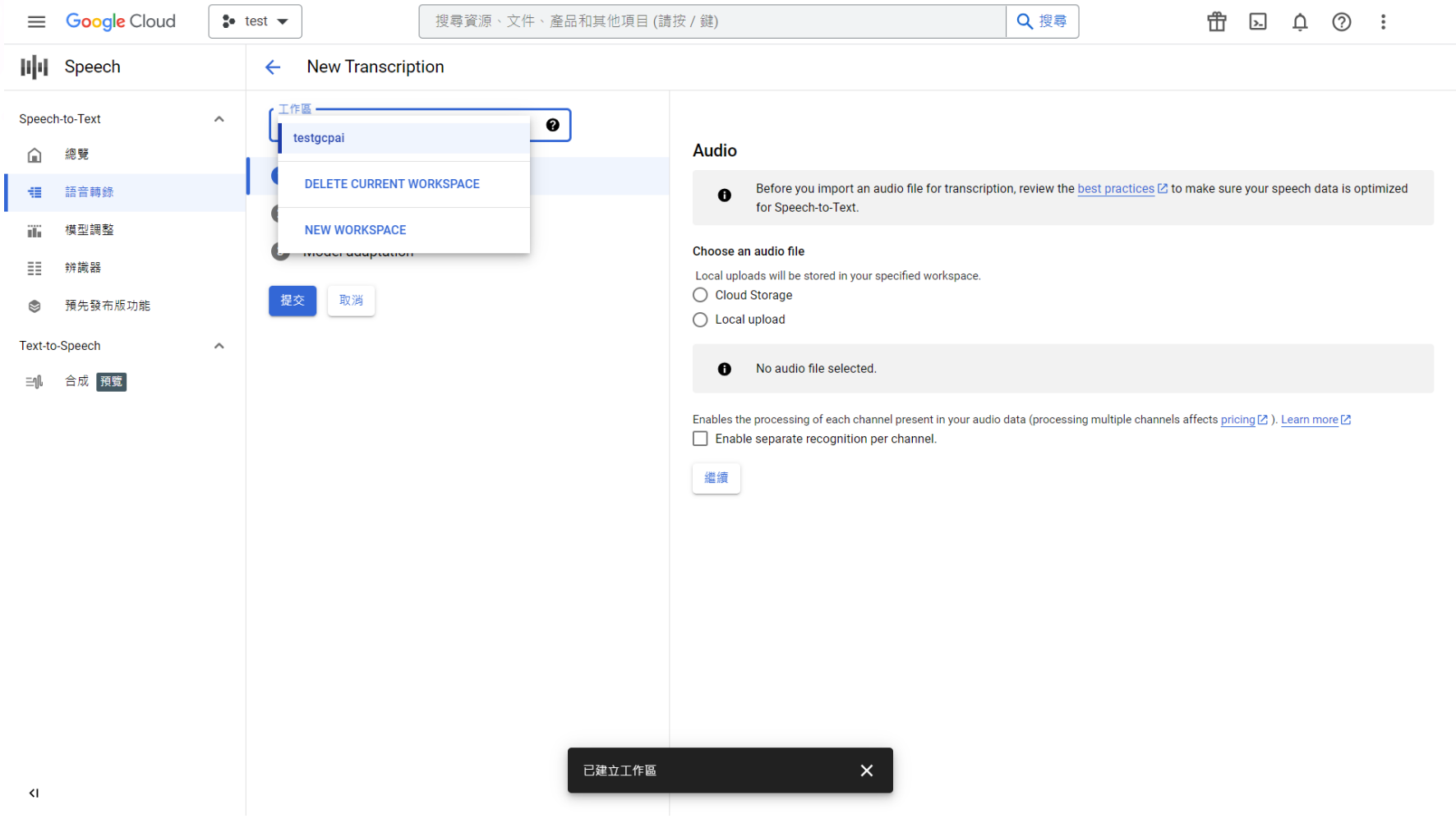


© 2026 Enos Chou

Cloud Speech-to-Text API

2. 建立轉錄工作

a. 建立工作區

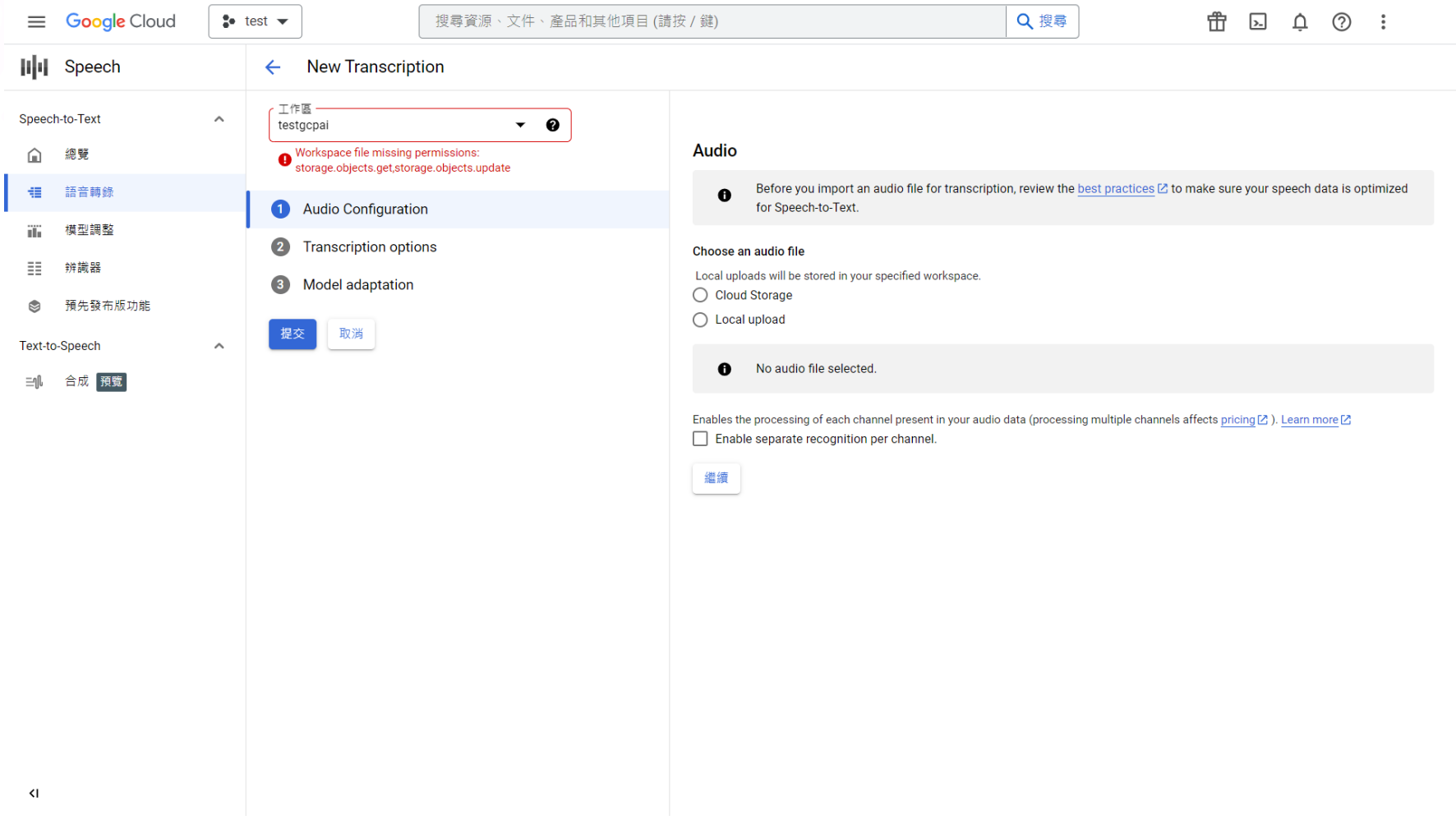


© 2026 Enos Chou

Cloud Speech-to-Text API

2. 建立轉錄工作

a. 建立工作區



© 2026 Enos Chou

Cloud Speech-to-Text API

2. 建立轉錄工作

b. (調整工作區權限)

Error
Workspace file missing permissions: storage.objects.get,storage.objects.update

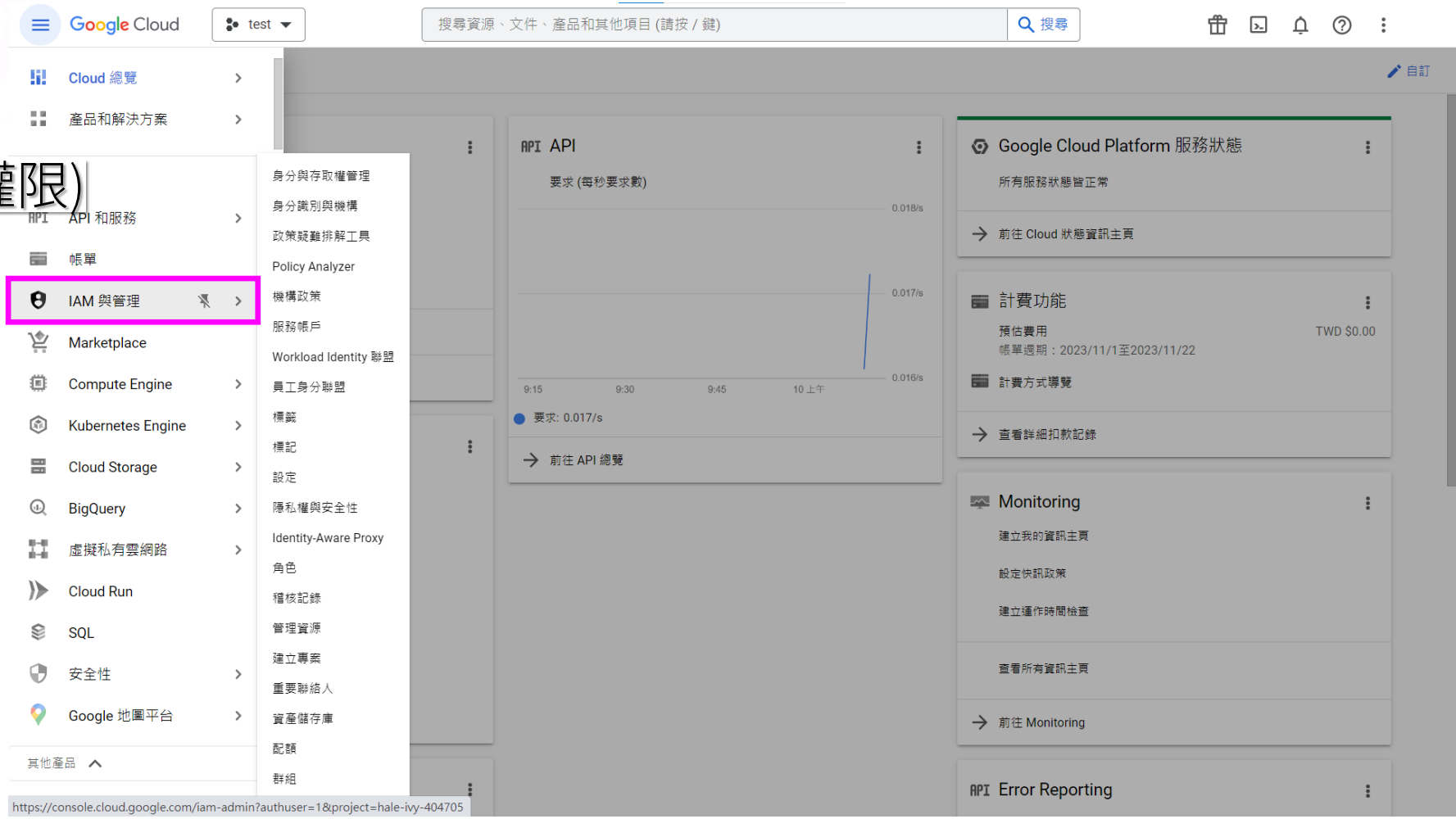
Solution
透過 "IAM 與管理" 針對使用對象增加 "Storage 物件使用者" 或 "Storage 管理員" 權限

© 2026 Enos Chou

Cloud Speech-to-Text API

2. 建立轉錄工作

b. (調整工作區權限)



Cloud Speech-to-Text API

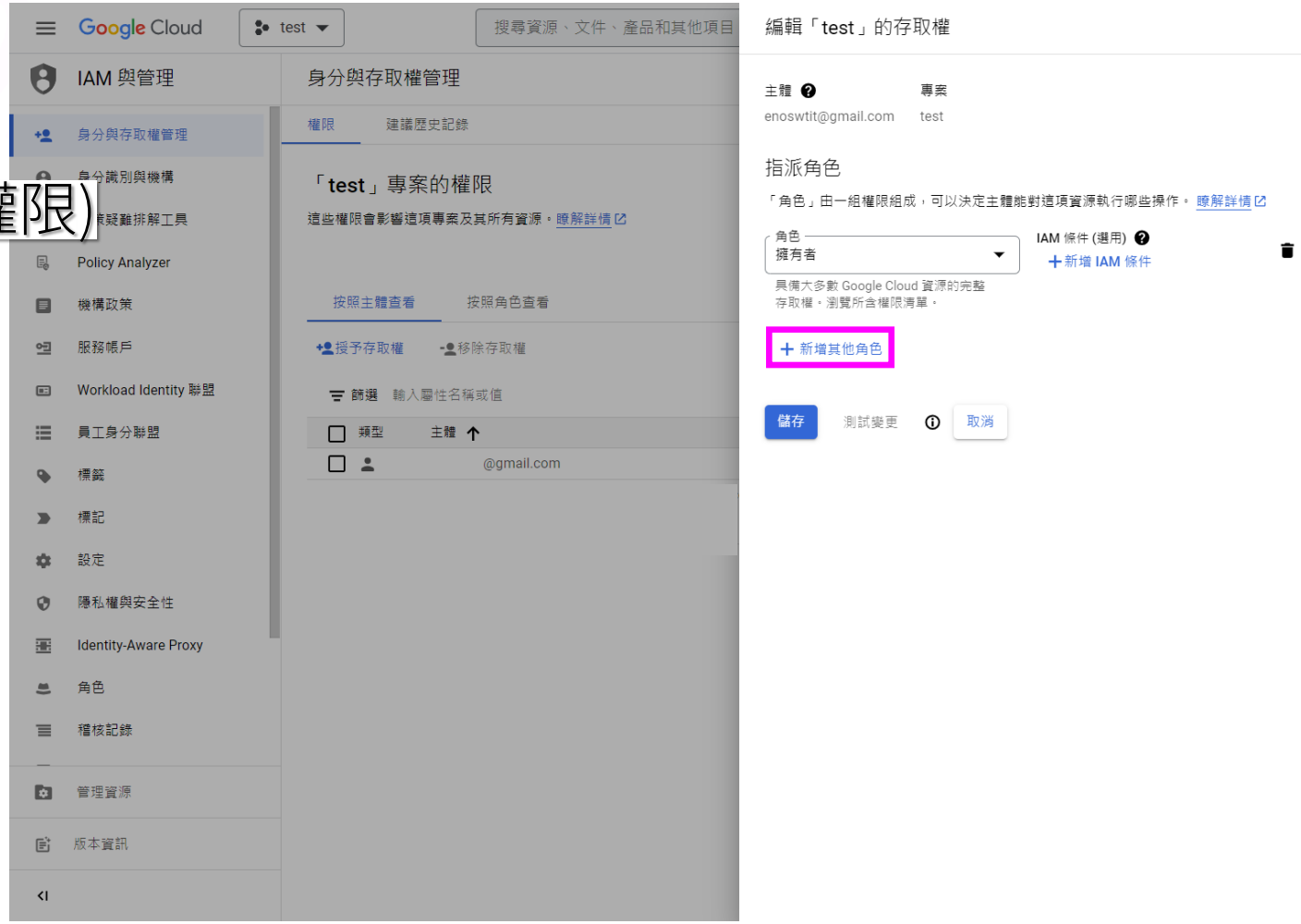
2. 建立轉錄工作

b. (調整工作區權限)

Cloud Speech-to-Text API

2. 建立轉錄工作

   b. (調整工作區權限)



Cloud Speech-to-Text API

2. 建立轉錄工作

   b. (調整工作區權限)

© 2026 Enos Chou

Cloud Speech-to-Text API

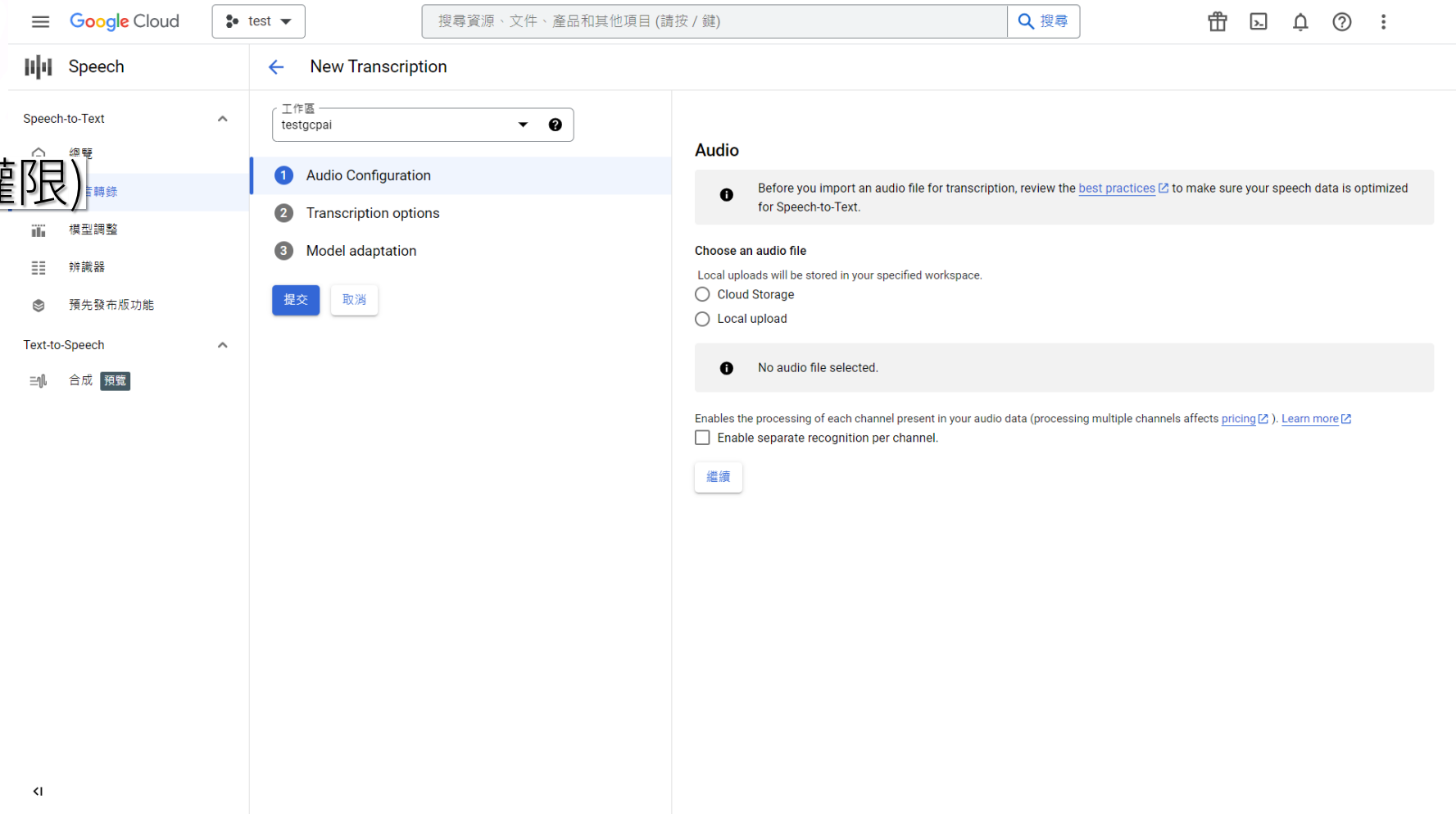
2. 建立轉錄工作

b. (調整工作區權限)

Cloud Speech-to-Text API

2. 建立轉錄工作

b. (調整工作區權限)

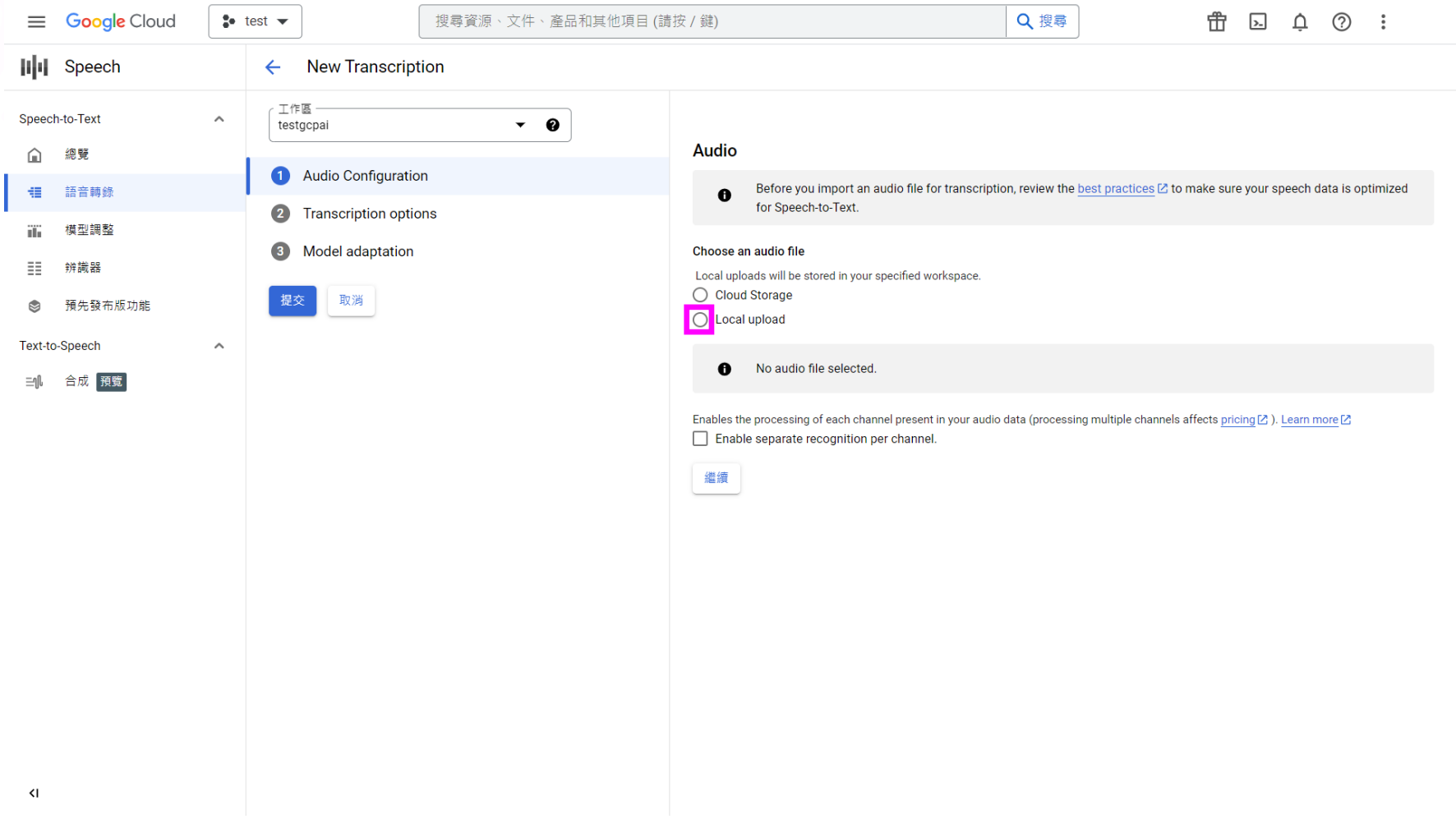


© 2026 Enos Chou

Cloud Speech-to-Text API

2. 建立轉錄工作

c. 設定音檔

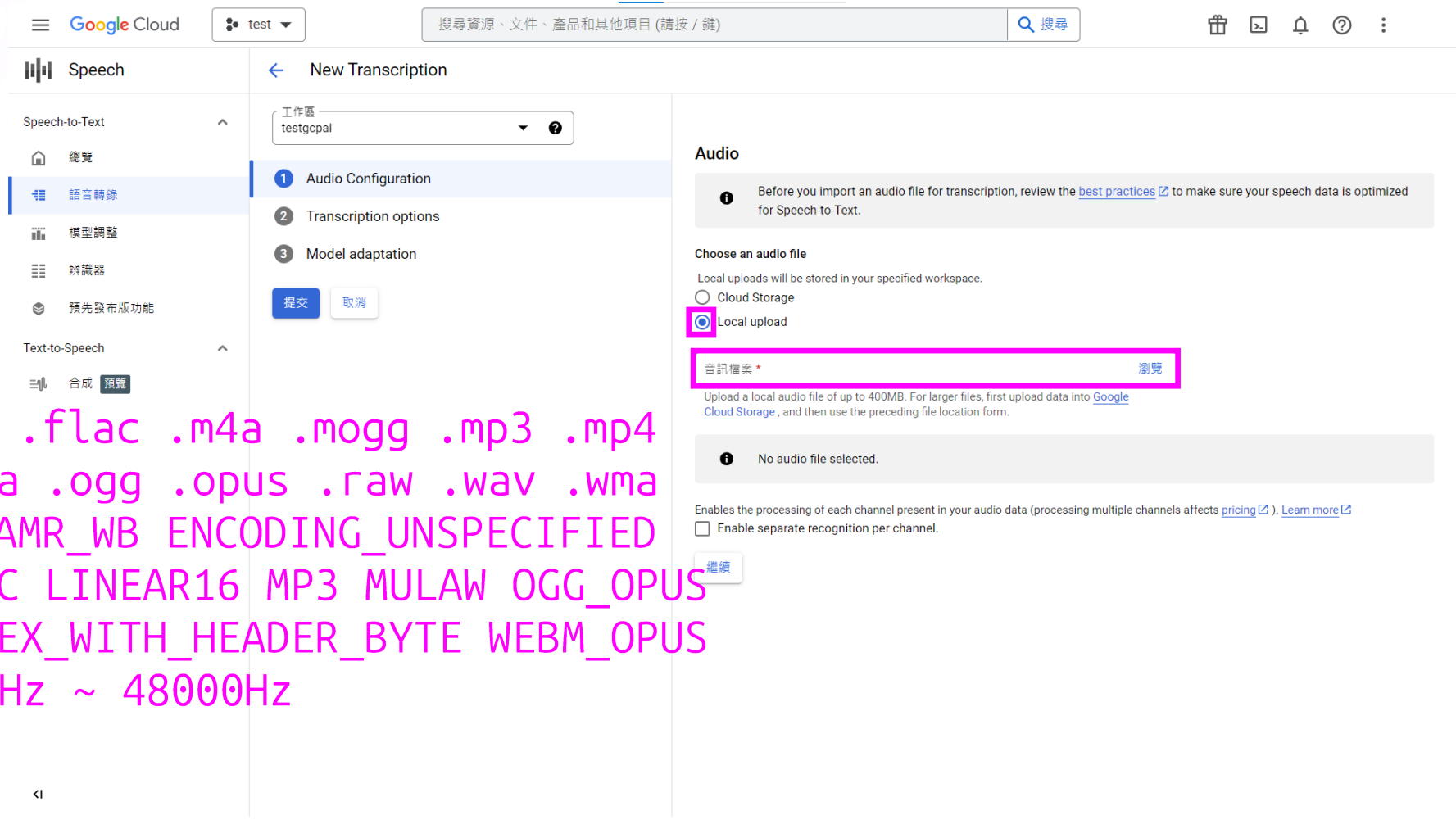


© 2026 Enos Chou

Cloud Speech-to-Text API

2. 建立轉錄工作

c. 設定音檔



Note

檔名限制：.aac .flac .m4a .mogg .mp3 .mp4
.oga .ogg .opus .raw .wav .wma

編碼限制：AMR AMR_WB ENCODING_UNSPECIFIED
FLAC LINEAR16 MP3 MULAW OGG_OPUS
SPEEX_WITH_HEADER_BYTE WEBM_OPUS

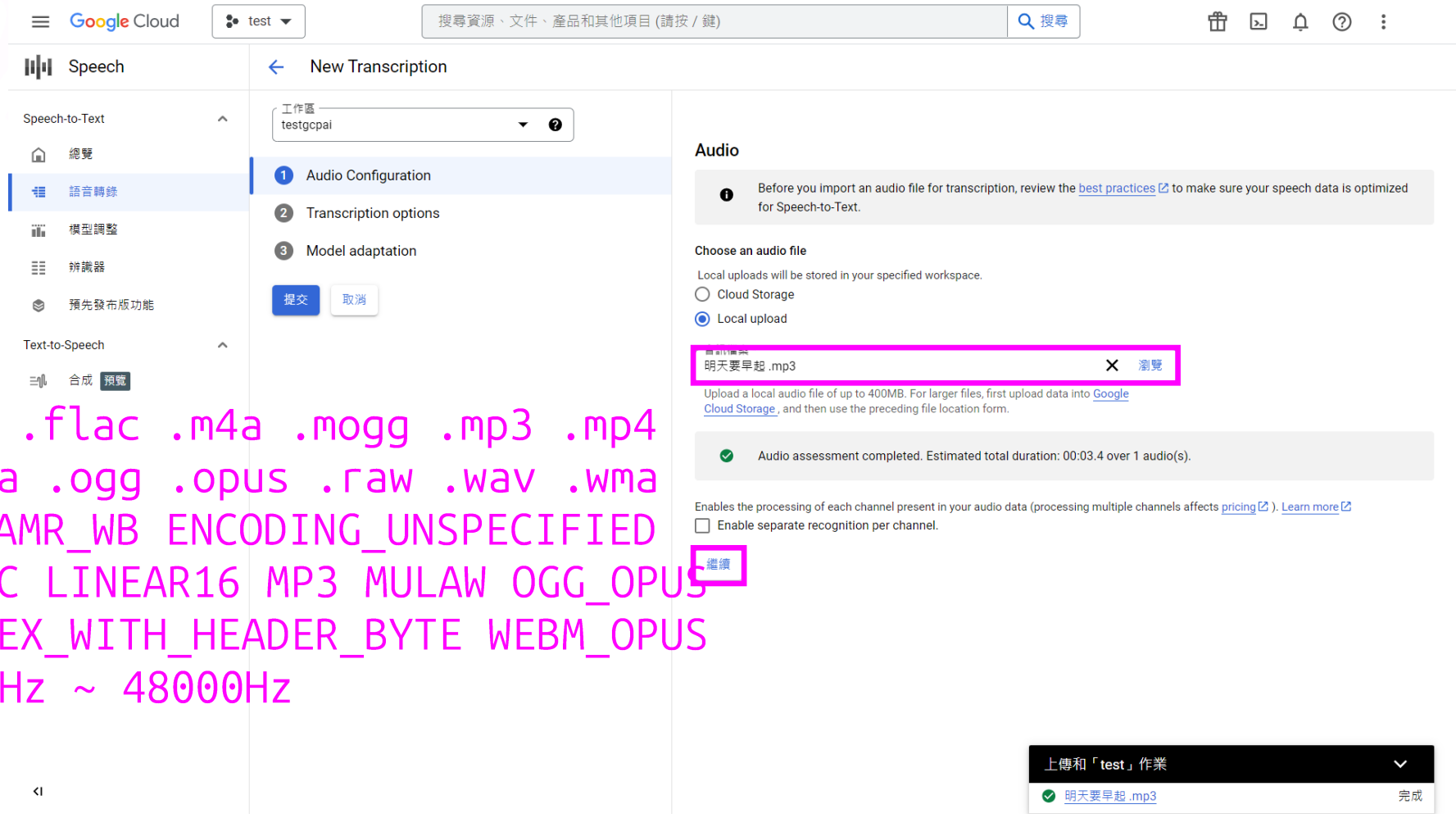
取樣率： 8000Hz ~ 48000Hz

© 2026 Enos Chou

Cloud Speech-to-Text API

2. 建立轉錄工作

c. 設定音檔



Note

檔名限制：.aac .flac .m4a .mogg .mp3 .mp4
.oga .ogg .opus .raw .wav .wma

編碼限制：AMR AMR_WB ENCODING_UNSPECIFIED
FLAC LINEAR16 MP3 MULAW OGG_OPUS
SPEEX_WITH_HEADER_BYTE WEBM_OPUS

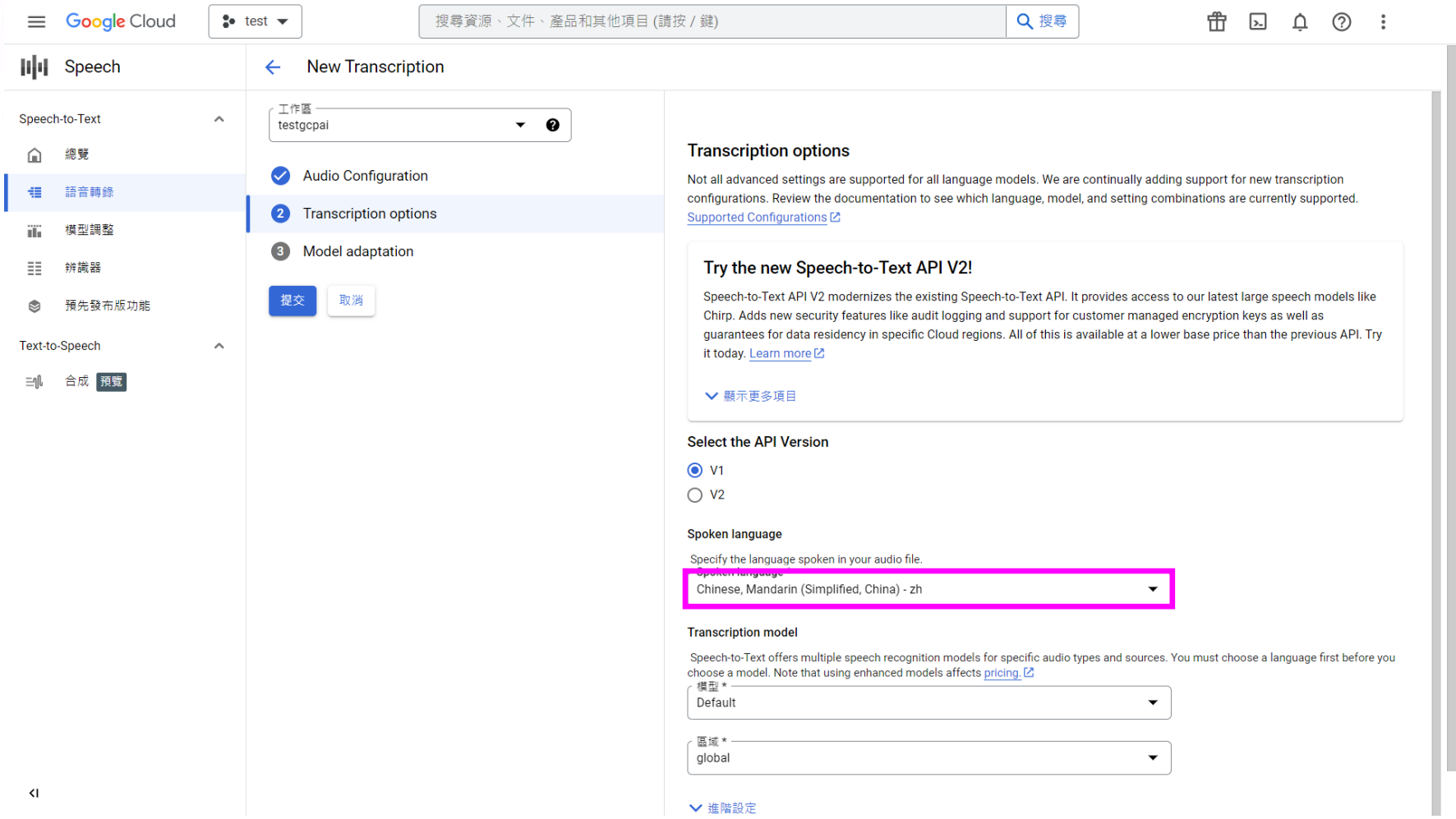
取樣率： 8000Hz ~ 48000Hz

© 2026 Enos Chou

Cloud Speech-to-Text API

2. 建立轉錄工作

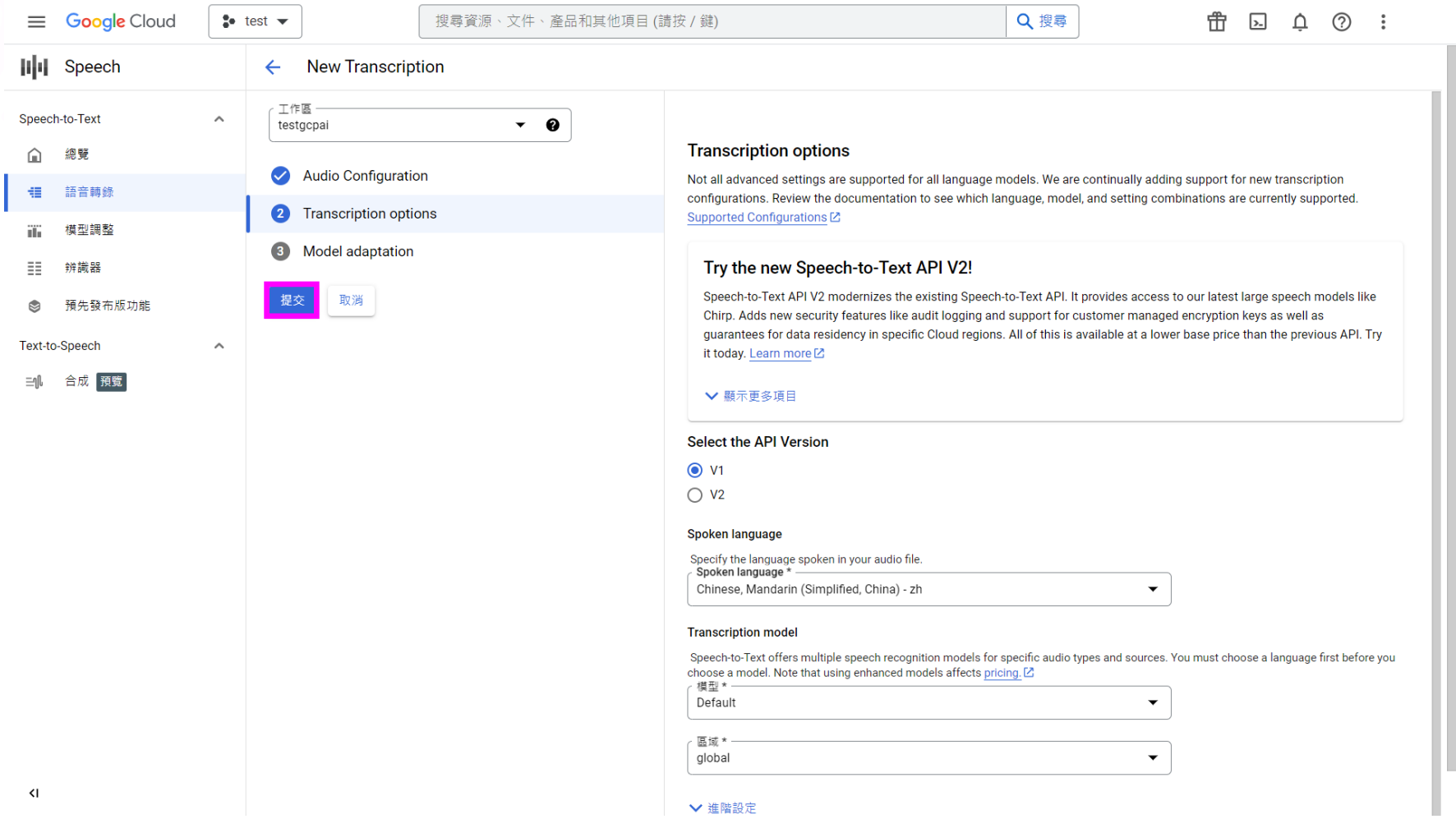
c. 設定音檔



Cloud Speech-to-Text API

2. 建立轉錄工作

d. 執行轉錄

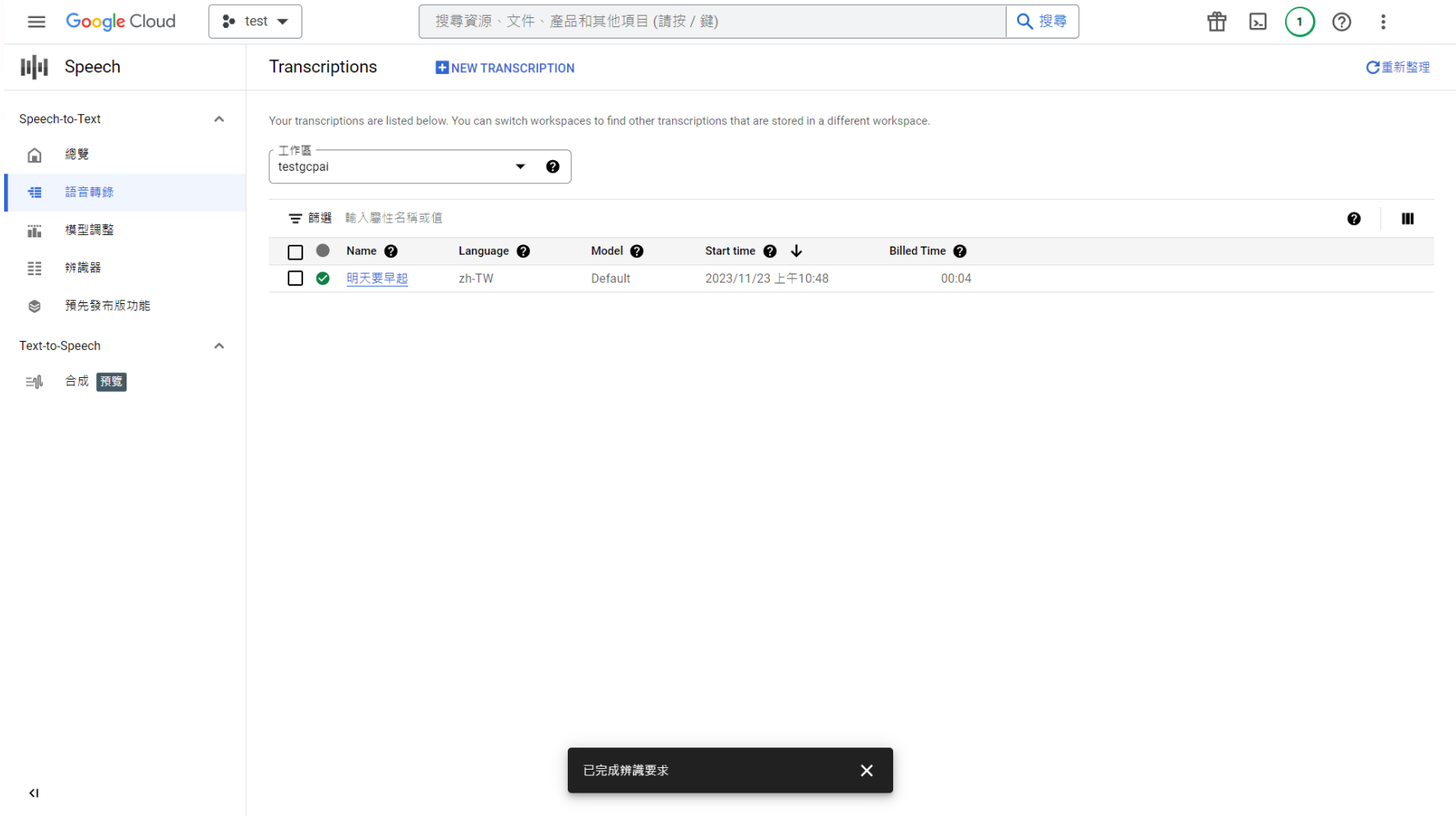


© 2026 Enos Chou

Cloud Speech-to-Text API

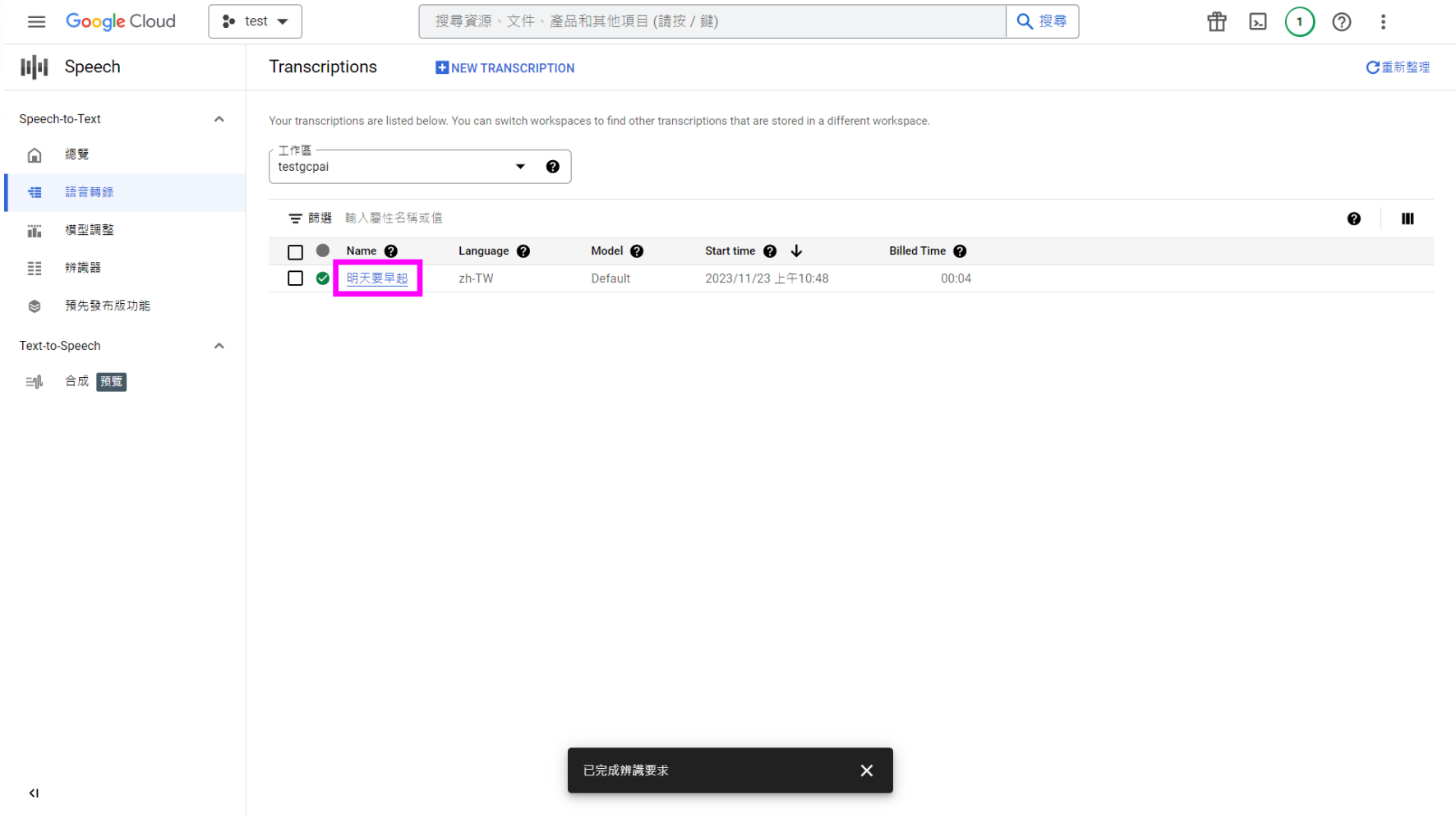
2. 建立轉錄工作

d. 執行轉錄



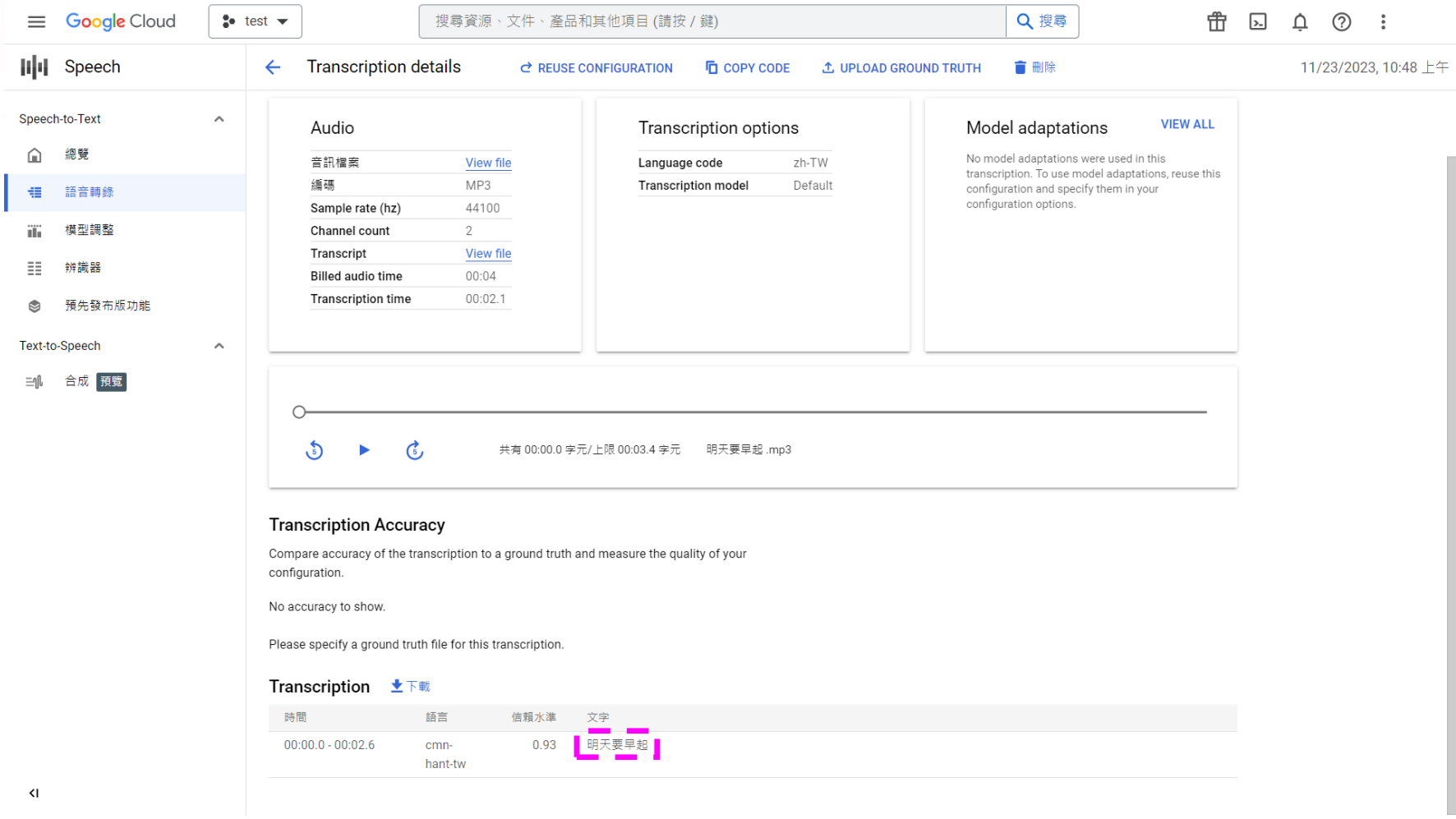
Cloud Speech-to-Text API

3. 檢視轉錄結果



Cloud Speech-to-Text API

3. 檢視轉錄結果



© 2026 Enos Chou

Speech-to-Text Application

叫用 API

a. 初始化
b. 準備音檔
c. 叫用 GCS API 上傳音檔
d. 叫用 Speech-to-Text API 語音轉文字
e. 處理結果

© 2026 Enos Chou

Speech-to-Text Application

叫用 API

a. 初始化

套件 Google Cloud Speech

```
pip install google-cloud-speech
```

© 2026 Enos Chou

Speech-to-Text Application

叫用 API

a. 初始化

case 1: 程式指定服務帳戶金鑰 in JSON

```
from google.cloud import speech

YOUR_SERVICE = service_account_key_file_name_str

speech_client = speech.SpeechClient.from_service_account_json(YOUR_SERVICE)
```

© 2026 Enos Chou

Speech-to-Text Application

叫用 API

a. 初始化

case 2: 環境變數指定服務帳戶金鑰 in JSON

```
import os
from google.cloud import speech

YOUR_SERVICE = service_account_key_file_name_str

os.environ['GOOGLE_APPLICATION_CREDENTIALS'] = YOUR_SERVICE
speech_client = speech.SpeechClient()
```

© 2026 Enos Chou

Speech-to-Text Application

叫用 API

a. 初始化

case 3: 服務運行於 GCP (如 GCE、Cloud Run)，直接於 console 設定服務帳戶

```
from google.cloud import speech

speech_client = speech.SpeechClient()
```

© 2026 Enos Chou

Speech-to-Text Application

叫用 API

b. 準備音檔

手動產生：同前
預先產生：略
即時產生：稍候

© 2026 Enos Chou

Speech-to-Text Application

叫用 API

c. 讀取音檔內容

case 1: one-shot upload

```
YOUR_AUDIO = audio_file_name_str

# prepare audio content
with open(YOUR_AUDIO, 'rb') as f:
    content = f.read()
audio = speech.RecognitionAudio(content=content)
```

© 2026 Enos Chou

Speech-to-Text Application

叫用 API

c. 讀取音檔內容

case 2: through GCS

```
from google.cloud import storage

YOUR_BUCKET = bucket_name_str
YOUR_AUDIO = audio_file_name_str
```

© 2026 Enos Chou

Speech-to-Text Application

叫用 API

c. 讀取音檔內容

case 2: through GCS

```
# prepare audio content
storage_client = storage.Client.from_service_account_json(YOUR_SERVICE)
bucket = storage_client.bucket(YOUR_BUCKET)
bucket.blob(YOUR_AUDIO).upload_from_filename(YOUR_AUDIO)
uri = f'gs://{YOUR_BUCKET}/{YOUR_AUDIO}'
audio = speech.RecognitionAudio(uri=uri)
```

© 2026 Enos Chou

Speech-to-Text Application

叫用 API

d. 叫用 Speech-to-Text API 語音轉文字

```
# transcript the audio
config = speech.RecognitionConfig(
    # omit the line below if WAV
    encoding=speech.RecognitionConfig.AudioEncoding.MP3,
    sample_rate_hertz=44100,
    audio_channel_count=2,  # take care, default is 1
    language_code="zh-TW"
)
response = speech_client.recognize(config=config, audio=audio)
```

© 2026 Enos Chou

Speech-to-Text Application

叫用 API

e. 處理結果

```
# get response
for r in response.results:
    print(f'{r.alternatives[0].transcript},
{r.alternatives[0].confidence:.3f}')
```

© 2026 Enos Chou

Challenge - Best Result

取得 Speech-to-Text API 回應的最佳結果

© 2026 Enos Chou

Speech-to-Text Application

叫用 API

b. 準備音檔 (即時產生)

套件 PyAudio

```
pip install pyaudio
```

© 2026 Enos Chou

Speech-to-Text Application

叫用 API

b. 準備音檔 (即時產生)

套件 PyAudio 範例程式

Search PyAudio > Record

© 2026 Enos Chou

Your Turn – Fixedtime Speech Recognition

讀取用戶語音，轉換為文字 (45 mins)

1. 固定語音秒數

Enos Chou

Your Turn – Arbitrarytime Speech Recognition

讀取用戶語音，轉換為文字 (45 mins)

2. 任意語音秒數 (optional)

Hints

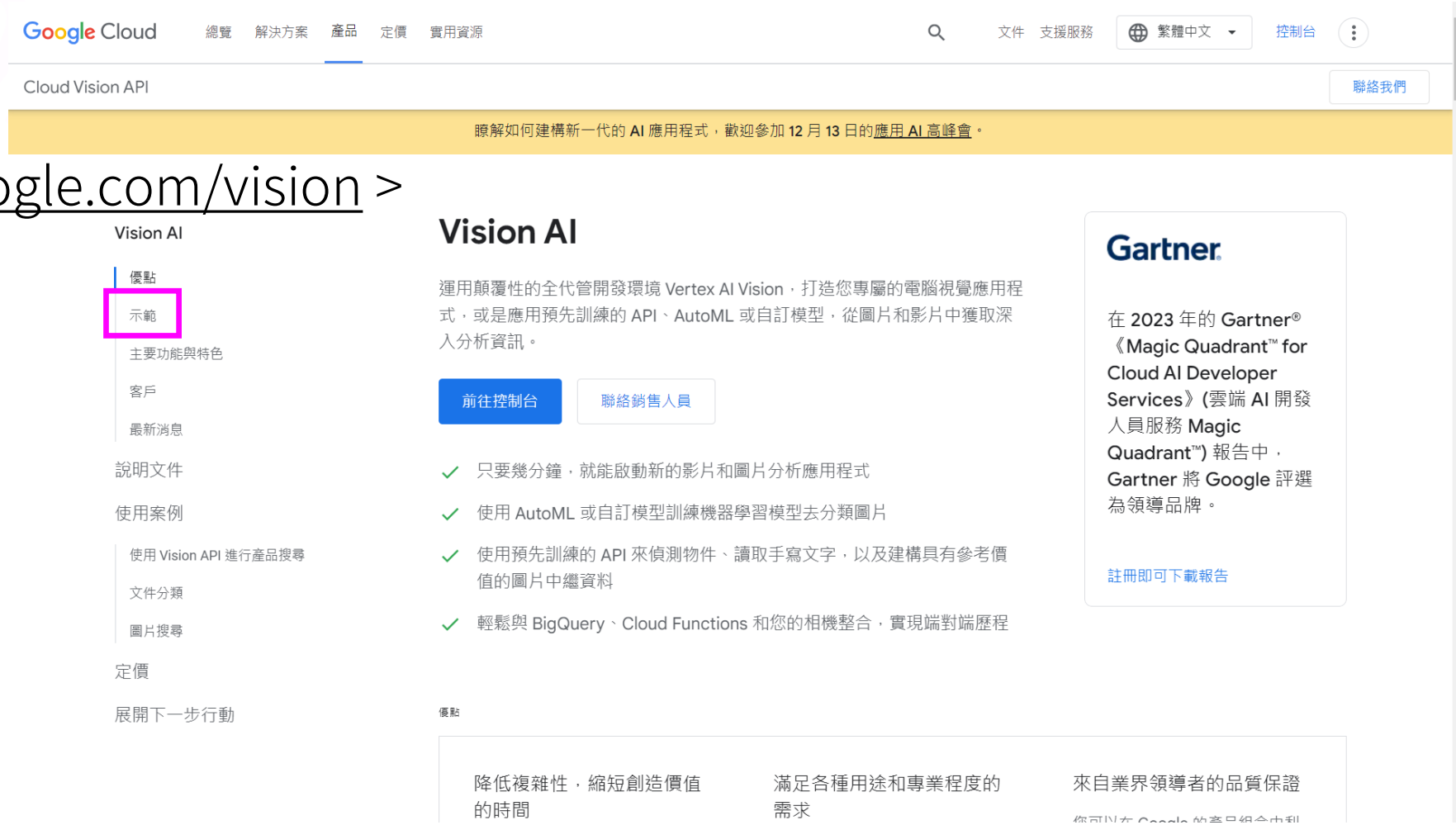
```
input() ft. threading
```

Enos Chou

Cloud Vision API

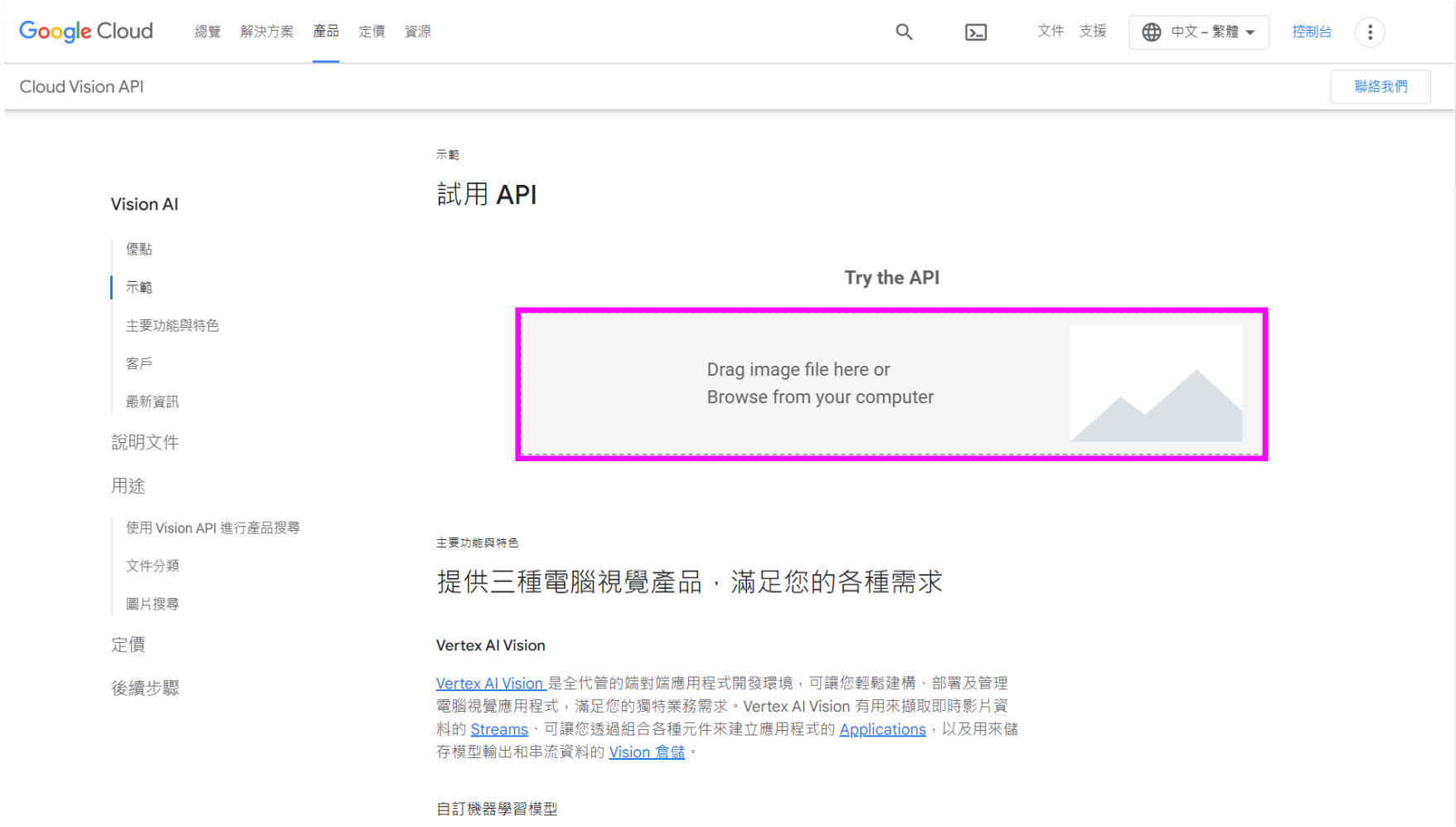
先試試

https://cloud.google.com/vision > 示範



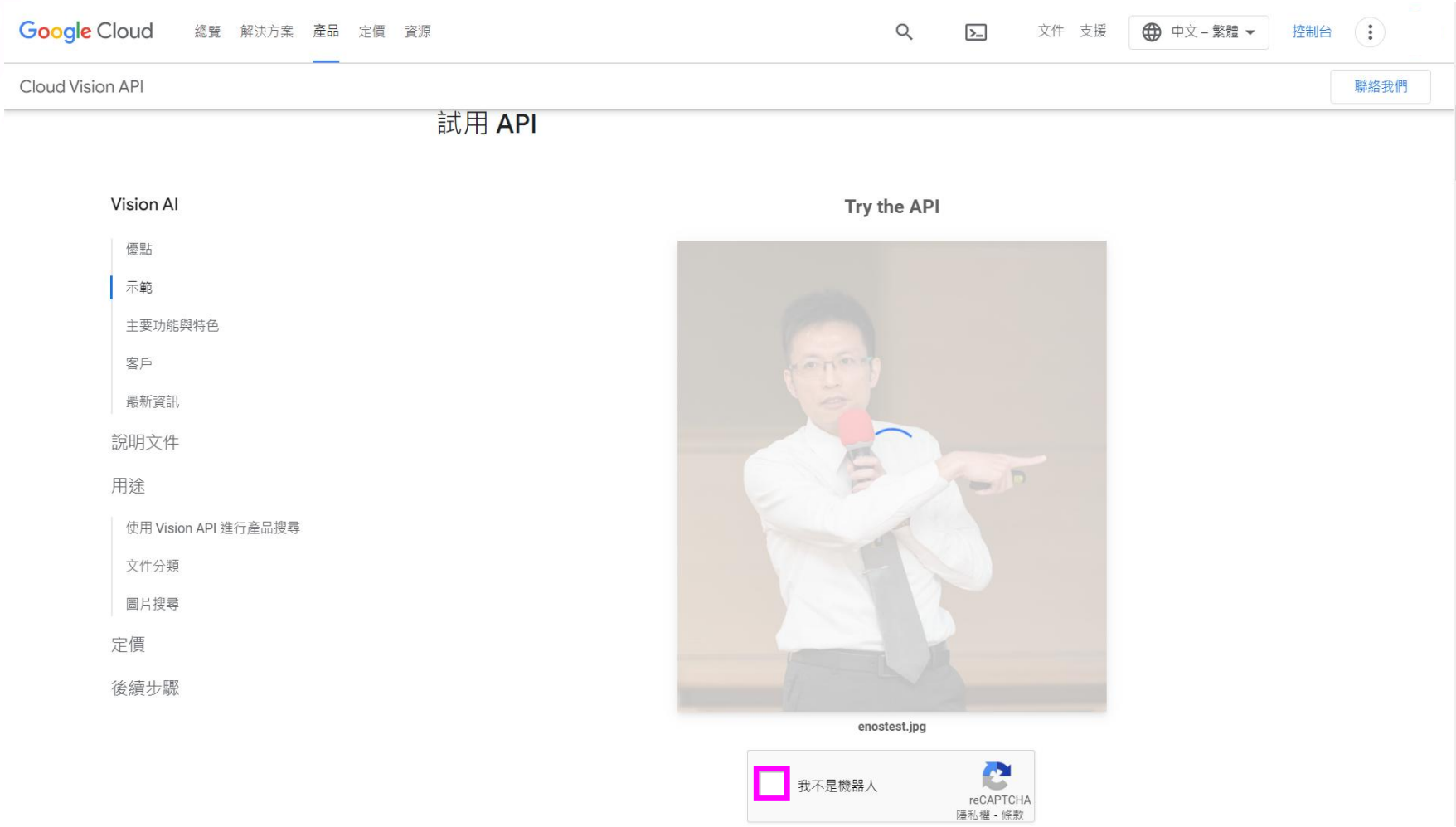
Cloud Vision API

先試試



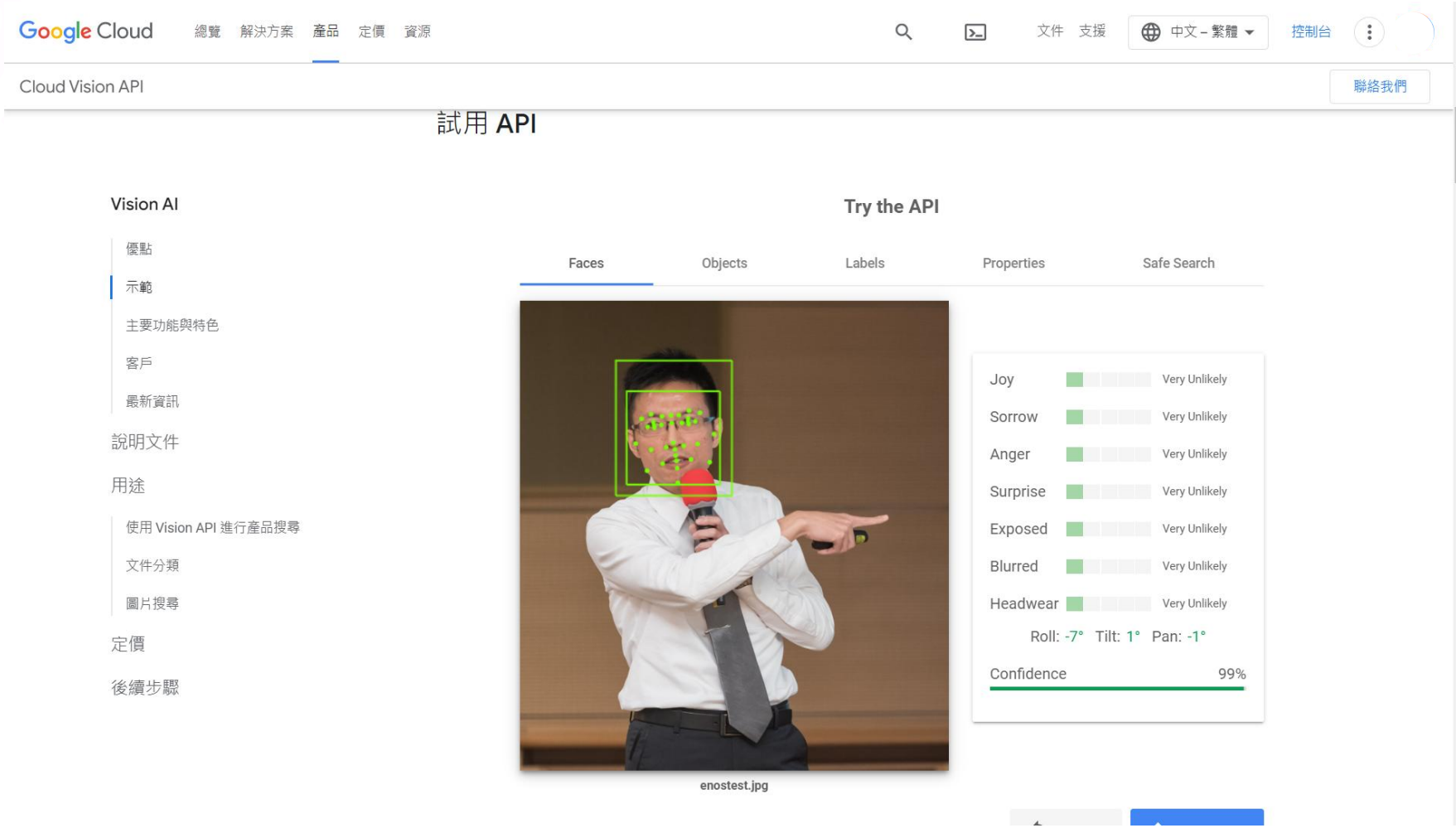
Cloud Vision API

先試試



Cloud Vision API

先試試



Cloud Vision API

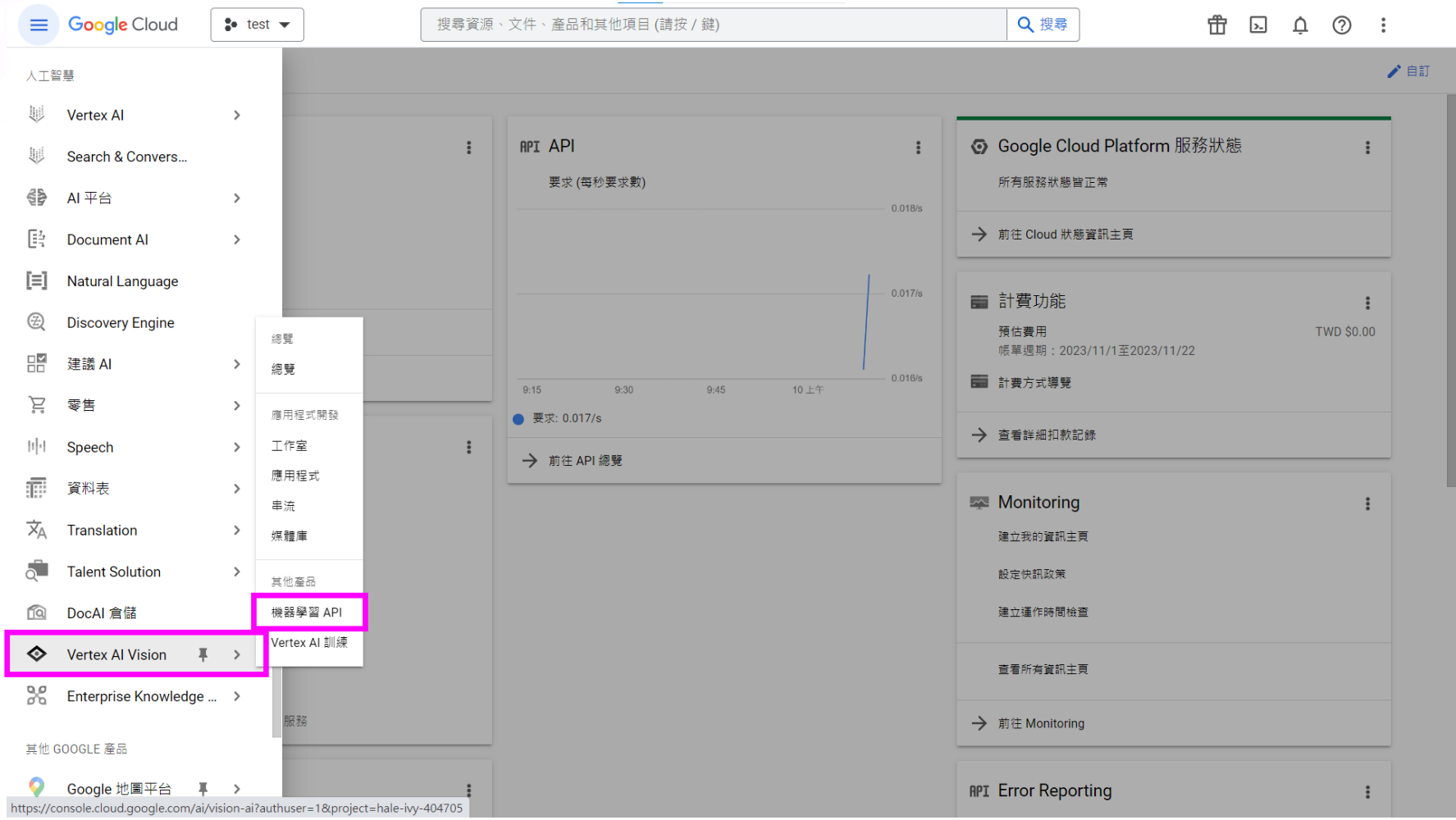
價格

https://cloud.google.com/vision >
定價 >
定價 >
Vision API 定價 >
Vertex AutoML 模型定價 (圖片資料)

© 2026 Enos Chou

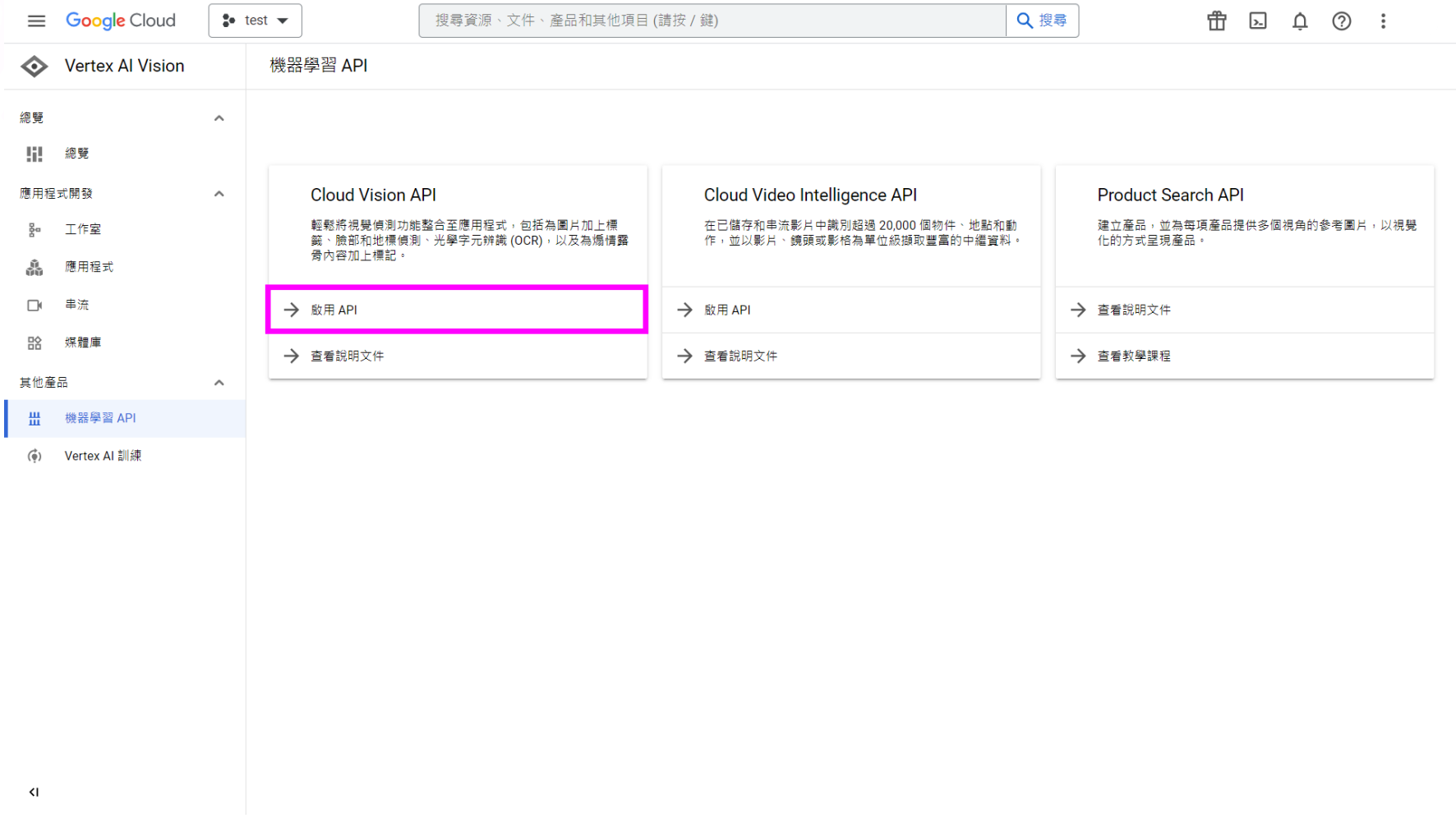
Cloud Vision API

啟用 Vision API



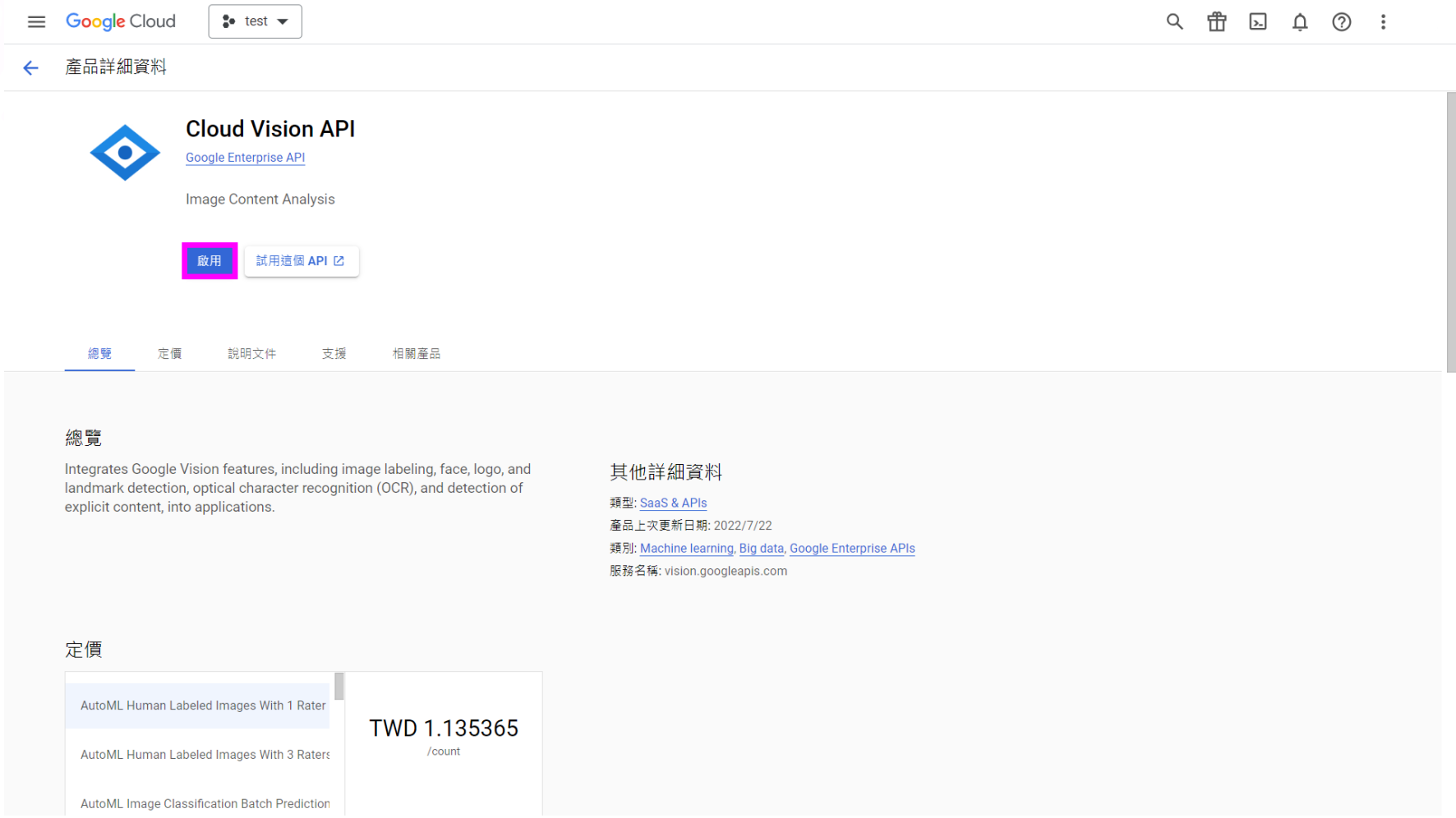
Cloud Vision API

啟用 Vision API



Cloud Vision API

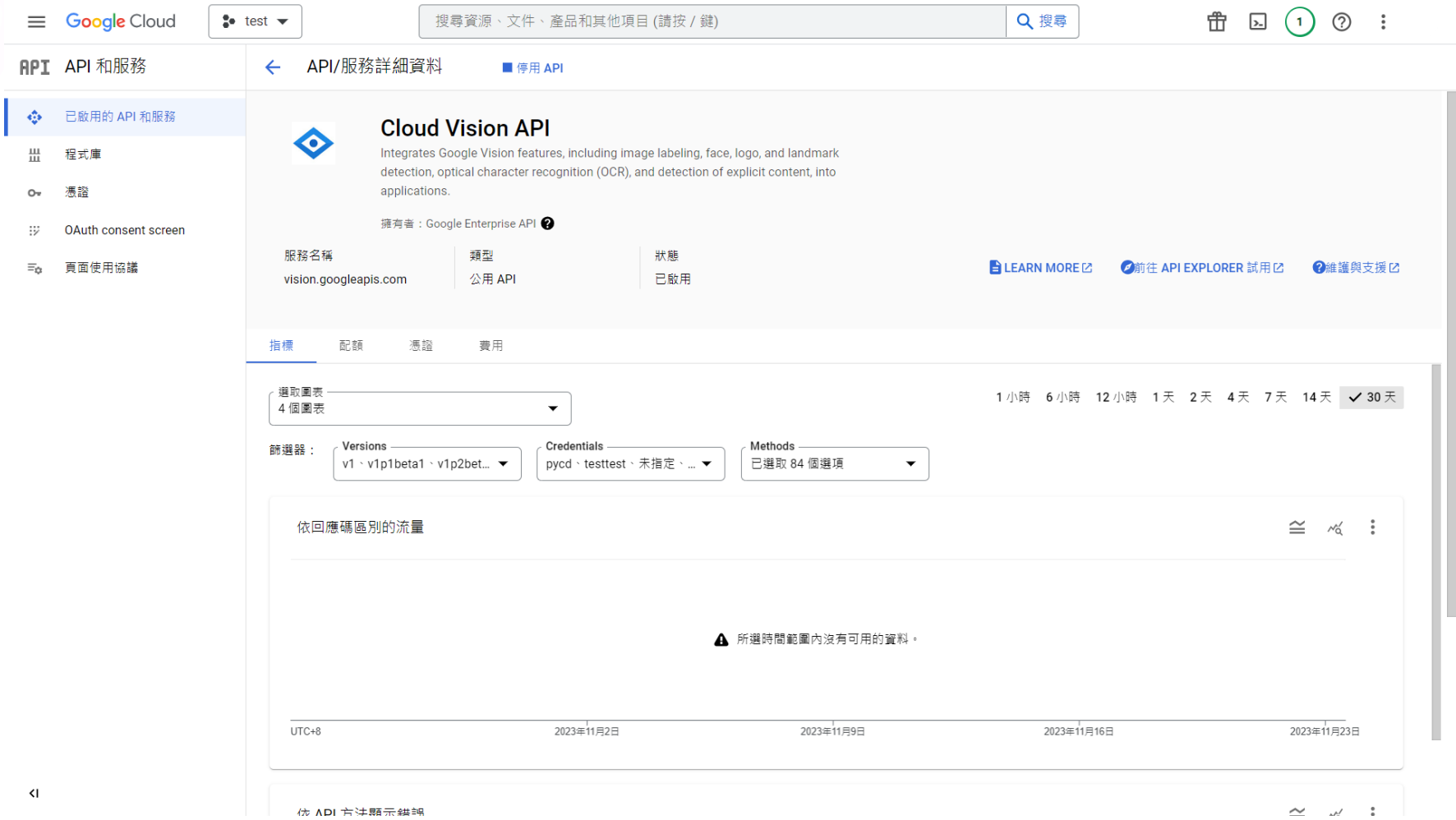
啟用 Vision API



© 2026 Enos Chou

Cloud Vision API

啟用 Vision API



Cloud Vision API

google.cloud.vision.ImageAnnotatorClient() major methods

```
label_detection()
face_detection()
text_detection()
document_text_detection()
object_localization()
landmark_detection()
web_detection()
logo_detection()
safe_search_detection()
product_search()
```

© 2026 Enos Chou

Vision Application

叫用 API

a. 初始化
b. 準備圖檔
c. 叫用 Vision API
d. 處理結果

© 2026 Enos Chou

Vision Application

叫用 API

a. 初始化

套件 Google Cloud Vision, Matplotlib, Pillow

```
pip install google-cloud-vision matplotlib pillow --upgrade
```

© 2026 Enos Chou

Vision Application

叫用 API

a. 初始化

case 1: 程式指定服務帳戶金鑰 in JSON

```
from google.cloud import vision

YOUR_SERVICE = service_account_key_file_name_str

vision_client = vision.ImageAnnotatorClient.from_service_account_json(YOUR_SERVICE)
```

© 2026 Enos Chou

Vision Application

叫用 API

a. 初始化

case 2: 環境變數指定服務帳戶金鑰 in JSON

```
import os
from google.cloud import vision

YOUR_SERVICE = service_account_key_file_name_str

os.environ['GOOGLE_APPLICATION_CREDENTIALS'] = YOUR_SERVICE
vision_client = vision.ImageAnnotatorClient()
```

© 2026 Enos Chou

Vision Application

叫用 API

a. 初始化

case 3: 服務運行於 GCP (如 GCE、Cloud Run)，於設定時指定服務帳戶

```
from google.cloud import vision

vision_client = vision.ImageAnnotatorClient()
```

© 2026 Enos Chou

Vision Application

叫用 API

b. 準備圖檔

case 1: one-shot upload

```
YOUR_IMAGE = image_name_str

with open(YOUR_IMAGE, 'rb') as image_file:
    content = image_file.read()
image = vision.Image(content=content)
```

© 2026 Enos Chou

Vision Application

叫用 API

b. 準備圖檔

case 2: through GCS

```
from google.cloud import storage

YOUR_BUCKET = bucket_name_str
YOUR_IMAGE = image_name_str
```

© 2026 Enos Chou

Vision Application

叫用 API

b. 準備圖檔

case 2: through GCS

```
storage_client = storage.Client()
bucket = storage_client.bucket(YOUR_BUCKET)
bucket.blob(YOUR_IMAGE).upload_from_filename(YOUR_IMAGE)
image_uri = f'gs://{YOUR_BUCKET}/{YOUR_IMAGE}'
source = vision.ImageSource(image_uri=image_uri)
image = vision.Image(source=source)
```

© 2026 Enos Chou

Vision Application

叫用 API

c. 叫用 Vision API

```
response = client.label_detection(image=image)
response = client.face_detection(image=image)
response = client.text_detection(image=image)
response = client.document_text_detection(image=image)
response = client.object_localization(image=image)
response = client.web_detection(image=image)
response = client.landmark_detection(image=image)
response = client.logo_detection(image=image)
response = client.safe_search_detection(image=image)
response = client.product_search(image=image)
```

© 2026 Enos Chou

Vision Application

叫用 API

d. 處理結果

`label_detection()`

- 物品種類

`face_detection()`

- 人臉座標
- 五官種類與位置
- 情緒
- 其他

© 2026 Enos Chou

Vision Application

叫用 API

d. 處理結果

`text_detection()`

- 文字位置與內容

`document_text_detection()`

- 文章位置與內容

© 2026 Enos Chou

Vision Application

叫用 API

d. 處理結果

`object_localization()`

- 物件位置與類別

`web_detection()`

- 爬網找物品
- 爬網找近似影像
- 給予照片標籤

© 2026 Enos Chou

Vision Application

叫用 API

d. 處理結果

`landmark_detection()`

- 地標位置、類別、經緯度

`logo_detection()`

- 商標名稱與位置

© 2026 Enos Chou

Vision Application

叫用 API

d. 處理結果

`safe_search_detection()`　　`product_search()`

- 五種風險內容等級

© 2026 Enos Chou

Your Turn - Car Plate Recognition

整合 Google Cloud Vision API，實作車牌辨識機制 (45 mins)

© 2026 Enos Chou

© 2026 Enos Chou

Gemini 整合應用

by google-genai SDK (generateContent API)

by LangChain SDK

Gemini Trial

Google AI Studio vs Google Cloud Vertex AI

© 2026 Enos Chou

Gemini Trial

1. 初次使用

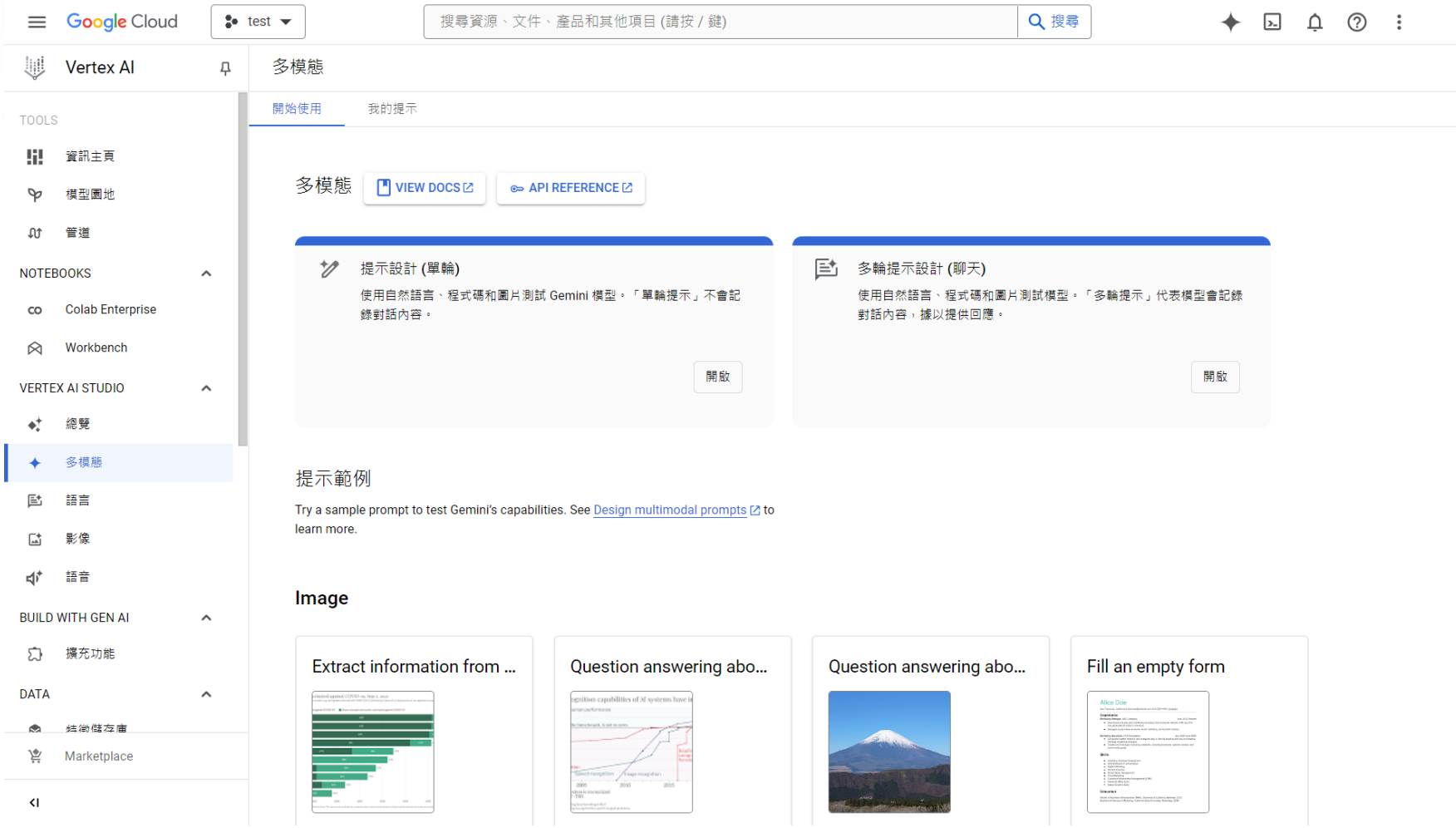
a. 進入 Gemini

① GCP 導覽選單 >
② Vertex AI >
③ VERTEX AI STUDIO 任意形式 >
④ 啟用 API (Vertex AI API/ Cloud Storage) 同意並繼續

Gemini Trial

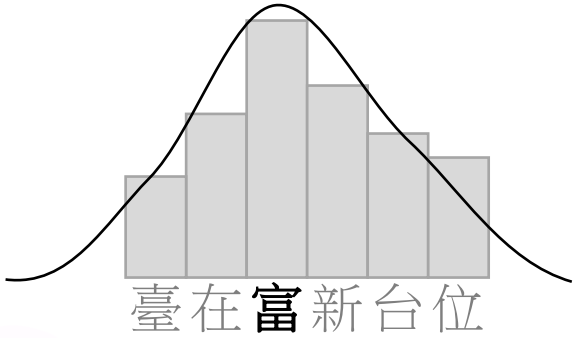
1. 初次使用

   b. 試用 Gemini

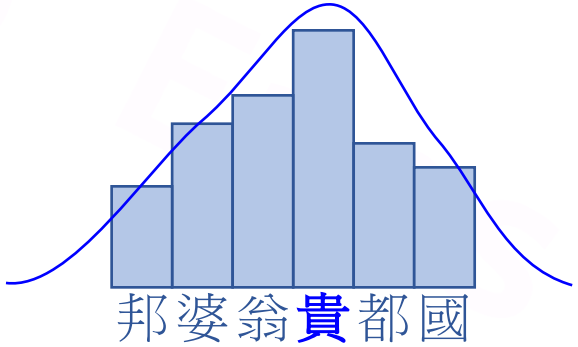


LLM 運作示意

台灣極北點在哪？

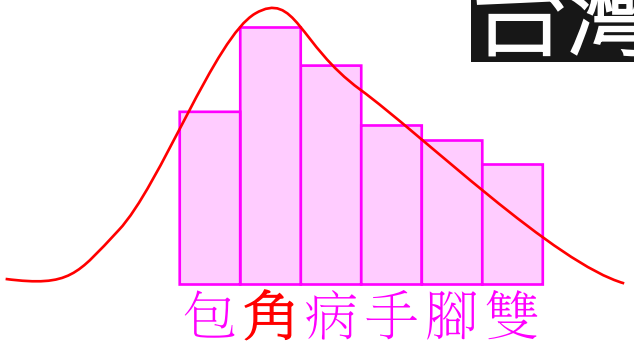


台灣極北點在哪？富



台灣極北點在哪？富貴角

台灣極北點在哪？富貴



© 2026 Enos Chou

Gemini Available MIME Types

Document

- application/json
- application/pdf
- text/css
- text/csv
- text/html
- text/javascript
- text/plain
- text/rtf
- text/xml

Audio

- audio/aac
- audio/flac
- audio/mp3
- audio/m4a
- audio/mpeg
- audio/mpga
- audio/mp4
- audio/opus
- audio/pcm
- audio/wav
- audio/webm

© 2026 Enos Chou

Gemini Available MIME Types

Image

- image/bmp
- image/jpeg
- image/png
- image/webp

Video

- video/3gpp
- video/avi
- video/mp4
- video/mpeg
- video/mpegps
- video/mpg
- video/quicktime
- video/webm
- video/wmv
- video/x-flv

© 2026 Enos Chou

Gemini Application

權限

啟用 API

`API和服務 > API程式庫 > Agent Platform API > 啟用`

© 2026 Enos Chou

Gemini Application

權限

服務帳戶角色

`Vertex AI: Agent Platform 使用者`

© 2026 Enos Chou

Gemini Application

用戶端開發

可用區域

https://cloud.google.com/vertex-ai/generative-ai/docs/learn/locations

© 2026 Enos Chou

Gemini Application

用戶端開發

安裝套件

```
pip install google-genai
```

© 2026 Enos Chou

Gemini Application

1. 車牌辨識

a. 初始化 ft. Service Account JSON

```
from google.oauth2 import service_account

YOUR_SERVICE = service_account_key_file_name_str

credentials = service_account.Credentials.from_service_account_file(
    YOUR_SERVICE,
    scopes=['https://www.googleapis.com/auth/cloud-platform']
)
```

© 2026 Enos Chou

Gemini Application

1. 車牌辨識

a. 初始化 ft. Service Account JSON

```
from google import genai

genai_client = genai.Client(
    enterprise=True,
    location='global',
    project=credentials.project_id,
    credentials=credentials
)
```

© 2026 Enos Chou

Gemini Application

1. 車牌辨識

a. 初始化 ft. API KEY

```
from google import genai

# 可透過 Google AI Studio 建立
# 可透過 GCP Console > API 和服務 > 憑證 > API 金鑰 取得
YOUR_API_KEY = api_key_str

genai_client = genai.Client(api_key=YOUR_API_KEY)
```

© 2026 Enos Chou

Gemini Application

1. 車牌辨識

b. 設定參數

```
config = {
    'max_output_tokens': 100,
    # 'temperature': 0,  # 0 ~ 2 間之浮點數，0 變化性最小，2 最大
                         # 官方在 gemini-3 建議使用預設值
}
```

© 2026 Enos Chou

Gemini Application

1. 車牌辨識

c. 設定提示詞 prompt

```
YOUR_PROMPT = '''車牌辨識並回2.遵循下列規則
1.
2.
'''

prompt = YOUR_PROMPT   # 1. 指定辨識內容
                       # 2. 詳述步驟
                       # 3. 律定輸出格式
```

© 2026 Enos Chou

Gemini Application

1. 車牌辨識

d. 準備資料

```
YOUR_JPG = image_file_name_str

with open(YOUR_JPG, 'rb') as f:
    data = f.read()
```

© 2026 Enos Chou

Gemini Application

1. 車牌辨識

d. 準備資料

```
contents = [
    {
        'role': 'user',
        'parts': [
            {'text': prompt},
            {'inline_data': {'data': data, 'mime_type': 'image/jpeg'}}
        ]
    }
]
```

© 2026 Enos Chou

Gemini Application

1. 車牌辨識

e. 叫用模型

```
YOUR_MODEL = 'gemini-3.5-flash-lite'  # can be changed

r = genai_client.models.generate_content(
    model=YOUR_MODEL,
    contents=contents,
    config=config
)
```

© 2026 Enos Chou

Gemini Application

1. 車牌辨識

f. 取得結果

```
print(r.text)
print(r.usage_metadata)
```

© 2026 Enos Chou

Gemini Application

關鍵參數

temperature：回應彈性 (Gemini 3 之後官方建議維持預設值 1 即可)

① 依模型而定，設定值範圍通常落於 0 ~ 1 或 0 ~ 2 之間
② 0： 最固定的回應，鑑別式應用通常設定此值
③ 最大值： 最彈性的回應
④ 中間值： 生成式應用通常依狀況設定此值

© 2026 Enos Chou

Gemini Application

關鍵參數

response_schema：結構化輸出

① 要求回應遵循指定格式，通常為 JSON
② 此 JSON schema 規格為 Google 自行設計
③ 可提供範例讓 Gemini 生成對應之 JSON schema 結構

© 2026 Enos Chou

Gemini Application

關鍵參數

response_schema：結構化輸出

範例 prompt

以 Python dict 建立 google.genai 使用的 response_schema 可以產出 {"plate": "ABC-123"}，"plate" 欄位為必須

© 2026 Enos Chou

Gemini Application

關鍵參數

response_schema：結構化輸出

```
wish_gemini_response = {
    "plate": "ABC-123"
}
```

```
response_schema = {
    "type": "object",
    "properties": {
        "plate": {
            "type": "string",
            "description": "車牌號碼"
        }
    },
    "required": ["plate"]
}
```

© 2026 Enos Chou

Gemini Application

關鍵參數

response_schema：結構化輸出

```
config = {
    'max_output_tokens': 100,
    'response_mime_type': 'application/json',
    'response_schema': response_schema
}
```

© 2026 Enos Chou

Gemini Application

關鍵參數

response_schema：結構化輸出

成功時

`r.parsed` 為 `dict` 內容

失敗時

`r.parsed` 為 `None`

© 2026 Enos Chou

Gemini Application

關鍵參數

thinking_config：思考設定

gemini-3 系列

① 不設定 thinking_config： 預設思考模式(各模型不同)
② thinking_config 為 None： 預設思考模式(各模型不同)
③ thinking_config 之 thinking_budget 為 0 關閉思考
④ thinking_config 之 thinking_budget 為指定值 有預算上限的思考

© 2026 Enos Chou

Gemini Application

關鍵參數

thinking_config：思考設定

gemini-3 系列

①	不設定 thinking_config：	預設思考模式(各模型不同)
②	thinking_config 為 None：	預設思考模式(各模型不同)
③	thinking_config 之 thinking_level 為 minimal	關閉思考 (3.6 以下)
④	thinking_config 之 thinking_level 為 low	開啟思考
⑤	thinking_config 之 thinking_level 為 medium	開啟思考
⑥	thinking_config 之 thinking_level 為 high	開啟思考

© 2026 Enos Chou

Gemini Application

關鍵參數

thinking_config：思考設定

```
config = {
    'max_output_tokens': 100,
    'thinking_config': {
        # 'thinking_budget': 0, # 擇一使用
        'thinking_level': 'minimal'
    }
}
```

© 2026 Enos Chou

Gemini Application

2. 健檢報告分析

a. 初始化

```
from google.oauth2 import service_account

YOUR_SERVICE = service_account_key_file_name_str

credentials = service_account.Credentials.from_service_account_file(
    YOUR_SERVICE,
    scopes=['https://www.googleapis.com/auth/cloud-platform']
)
```

© 2026 Enos Chou

Gemini Application

2. 健檢報告分析

a. 初始化

```
from google import genai

genai_client = genai.Client(
    enterprise=True,
    location='global',
    project=credentials.project_id,
    credentials=credentials
)
```

© 2026 Enos Chou

Gemini Application

2. 健檢報告分析

b. 設定參數

```
config = {
    'max_output_tokens': 2000,
    'thinking_config': {
        # 'thinking_budget': 0, # 擇一使用
        'thinking_level': 'minimal'
    },
    'response_mime_type': 'application/json'  # 指定輸出 MIME Type
}
```

© 2026 Enos Chou

Gemini Application

2. 健檢報告分析

c. 設定提示詞 prompt

```
YOUR_PROMPT = '''1. Analyze the health report and output to JSON.

                '''

prompt = YOUR_PROMPT   # 1. 指定辨識內容
                       # 2. 詳述步驟
                       # 3. 律定輸出格式
```

© 2026 Enos Chou

Gemini Application

2. 健檢報告分析

d. 準備資料

```
YOUR_JPG = image_file_name_str

with open(YOUR_JPG, 'rb') as f:
    data = f.read()
```

© 2026 Enos Chou

Gemini Application

2. 健檢報告分析

d. 準備資料

```
contents = [
    {
        'role': 'user',
        'parts': [
            {'text': prompt},
            {'inline_data': {'data': data, 'mime_type': 'image/jpeg'}}
        ]
    }
]
```

© 2026 Enos Chou

Gemini Application

2. 健檢報告分析

e. 叫用模型

```
YOUR_MODEL = 'gemini-3.5-flash-lite'  # can be changed

r = genai_client.models.generate_content(
    model=YOUR_MODEL,
    contents=contents,
    config=config
)
```

© 2026 Enos Chou

Gemini Application

2. 健檢報告分析

f. 取得結果

```
import json
from pprint import pprint

pprint(json.loads(r.text))
print(r.usage_metadata)
```

© 2026 Enos Chou

Gemini Application

3. 文件摘要

a. 初始化

```
from google.oauth2 import service_account

YOUR_SERVICE = service_account_key_file_name_str

credentials = service_account.Credentials.from_service_account_file(
    YOUR_SERVICE,
    scopes=['https://www.googleapis.com/auth/cloud-platform']
)
```

© 2026 Enos Chou

Gemini Application

3. 文件摘要

a. 初始化

```
from google import genai

genai_client = genai.Client(
    enterprise=True,
    location='global',
    project=credentials.project_id,
    credentials=credentials
)
```

© 2026 Enos Chou

Gemini Application

3. 文件摘要

b. 設定參數

```
config = {
    'max_output_tokens': 2000,
    'thinking_config': {
        # 'thinking_budget': 0, # 擇一使用
        'thinking_level': 'minimal'
    }
}
```

© 2026 Enos Chou

Gemini Application

3. 文件摘要

c. 設定提示詞 prompt

```
YOUR_PROMPT = '''以                                    '''

prompt = YOUR_PROMPT   # 1. 指定辨識內容
                       # 2. 詳述步驟
                       # 3. 律定輸出格式
```

© 2026 Enos Chou

Gemini Application

3. 文件摘要

d. 準備資料

```
YOUR_PDF = pdf_file_name_str

with open(YOUR_PDF, 'rb') as f:
    data = f.read()
```

© 2026 Enos Chou

Gemini Application

3. 文件摘要

d. 準備資料

```
contents = [
    {
        'role': 'user',
        'parts': [
            {'text': prompt},
            {'inline_data': {'data': data, 'mime_type': 'application/pdf'}}
        ]
    }
]
```

© 2026 Enos Chou

Gemini Application

3. 文件摘要

e. 叫用模型

```
YOUR_MODEL = 'gemini-3.5-flash-lite'  # can be changed

r = genai_client.models.generate_content(
    model=YOUR_MODEL,
    contents=contents,
    config=config
)
```

© 2026 Enos Chou

Gemini Application

3. 文件摘要

f. 取得結果

```
print(r.text)
print(r.usage_metadata)
```

© 2026 Enos Chou

Challenge - Expense Calculation

以 Gemini 讀取價目表影像檔之後計算消費金額

© 2026 Enos Chou

Gemini Application

4. 語音轉文字

a. 初始化

```
from google.oauth2 import service_account

YOUR_SERVICE = service_account_key_file_name_str

credentials = service_account.Credentials.from_service_account_file(
    YOUR_SERVICE,
    scopes=['https://www.googleapis.com/auth/cloud-platform']
)
```

© 2026 Enos Chou

Gemini Application

4. 語音轉文字

a. 初始化

```
from google import genai

genai_client = genai.Client(
    enterprise=True,
    location='global',
    project=credentials.project_id,
    credentials=credentials
)
```

© 2026 Enos Chou

Gemini Application

4. 語音轉文字

b. 設定參數

```
config = {
    'max_output_tokens': 1_024,
    'thinking_config': {
        # 'thinking_budget': 0, # 擇一使用
        'thinking_level': 'minimal'
    }
}
```

© 2026 Enos Chou

Gemini Application

4. 語音轉文字

c. 準備資料

```
YOUR_MP3 = audio_file_name_str

with open(YOUR_MP3, 'rb') as f:
    data = f.read()
```

© 2026 Enos Chou

Gemini Application

4. 語音轉文字

c. 準備資料

```
contents = [
    {
        'role': 'user',
        'parts': [
            {'text': 'transcribe to Traditional Chinese'},
            {'inline_data': {'data': data, 'mime_type': 'audio/mpeg'}}
        ]
    }
]
```

© 2026 Enos Chou

Gemini Application

4. 語音轉文字

d. 叫用模型

```
YOUR_MODEL = 'gemini-3.5-flash-lite'  # can be changed

r = genai_client.models.generate_content(
    model=YOUR_MODEL,
    contents=contents,
    config=config
)
```

© 2026 Enos Chou

Gemini Application

4. 語音轉文字

e. 取得結果

```
print(r.text)
print(r.usage_metadata)
```

© 2026 Enos Chou

Gemini Application

5. 影評歸納

a. 初始化

```
from google.oauth2 import service_account

YOUR_SERVICE = service_account_key_file_name_str

credentials = service_account.Credentials.from_service_account_file(
    YOUR_SERVICE,
    scopes=['https://www.googleapis.com/auth/cloud-platform']
)
```

© 2026 Enos Chou

Gemini Application

5. 影評歸納

a. 初始化

```
from google import genai

genai_client = genai.Client(
    enterprise=True,
    location='global',
    project=credentials.project_id,
    credentials=credentials
)
```

© 2026 Enos Chou

Gemini Application

5. 影評歸納

b. 設定參數

```
config = {
    'max_output_tokens': 1024,
    'thinking_config': {
        # 'thinking_budget': 0, # 擇一使用
        'thinking_level': 'minimal'
    },
    'response_mime_type': 'application/json'
}
```

© 2026 Enos Chou

Gemini Application

5. 影評歸納

c. 設定提示詞 prompt

```
YOUR_PROMPT = '''
    請分析影評，回傳給明於這部電影的負評內
    1.
    2.
    3.
    4.                                                    F"}
'''

prompt = YOUR_PROMPT
```

© 2026 Enos Chou

Gemini Application

5. 影評歸納

d. 準備資料

```
data = input('請輸入影評：')

contents = [
    {
        'role': 'user',
        'parts': [
            {'text': prompt},
            {'text': data}
        ]
    }
]
```

© 2026 Enos Chou

Gemini Application

5. 影評歸納

e. 叫用模型

```
YOUR_MODEL = 'gemini-3.5-flash-lite'  # can be changed

r = genai_client.models.generate_content(
    model=YOUR_MODEL,
    contents=contents,
    config=config
)
```

© 2026 Enos Chou

Gemini Application

5. 影評歸納

f. 取得結果

```
import json
from pprint import pprint

pprint(json.loads(r.text.strip('json`\n')))
print(r.usage_metadata)
```

© 2026 Enos Chou

Gemini Application

5. 影評歸納

g. (取得結果)

①
```
config = {
    'response_mime_type': 'application/json',
    'response_schema': ...
}
```
②
```
genai.Client.models.generate_content(...).parsed
```

© 2026 Enos Chou

Gemini Application

6. 文字聊天

a. 初始化

```
from google.oauth2 import service_account

YOUR_SERVICE = service_account_key_file_name_str

credentials = service_account.Credentials.from_service_account_file(
    YOUR_SERVICE,
    scopes=['https://www.googleapis.com/auth/cloud-platform']
)
```

© 2026 Enos Chou

Gemini Application

6. 文字聊天

a. 初始化

```
from google import genai

genai_client = genai.Client(
    enterprise=True,
    location='global',
    project=credentials.project_id,
    credentials=credentials
)
```

© 2026 Enos Chou

Gemini Application

6. 文字聊天

b. 設定系統提示詞 system instruction

```
YOUR_SYSTEM_INSTRUCTION = '          '
```

© 2026 Enos Chou

Gemini Application

6. 文字聊天

c. 設定參數

```
config = {
    'max_output_tokens': 1024,
    'thinking_config': {
        # 'thinking_budget': 0, # 擇一使用
        'thinking_level': 'minimal'
    },
    'system_instruction': YOUR_SYSTEM_INSTRUCTION
}
```

© 2026 Enos Chou

Gemini Application

6. 文字聊天

d. 準備資料

```
YOUR_CONTENT = chat_text_content_str

chat = YOUR_CONTENT
contents = [
    {'role': 'user', 'parts': [{'text': chat}]}
]
```

© 2026 Enos Chou

Gemini Application

6. 文字聊天

e. 叫用模型

```
YOUR_MODEL = 'gemini-3.5-flash'  # can be changed

r = genai_client.models.generate_content(
    model=YOUR_MODEL,
    contents=contents,
    config=config
)
```

© 2026 Enos Chou

Gemini Application

6. 文字聊天

f. 取得結果

```
print(r.text)
print(r.usage_metadata)
```

© 2026 Enos Chou

Gemini Application

6. 文字聊天

g. (設定參數 for 安全設定)

```
YOUR_SAFETY_SETTINGS = [
    {'category': 'HARM_CATEGORY_HATE_SPEECH', 'threshold': 'BLOCK_NONE'},
    {'category': 'HARM_CATEGORY_HARASSMENT', 'threshold': 'BLOCK_NONE'},
    {'category': 'HARM_CATEGORY_SEXUALLY_EXPLICIT', 'threshold': 'BLOCK_NONE'},
    {'category': 'HARM_CATEGORY_DANGEROUS_CONTENT', 'threshold': 'BLOCK_NONE'}
]

config = {
    'safety_settings': YOUR_SAFETY_SETTINGS
}
```

© 2026 Enos Chou

Gemini Application

6. 文字聊天

h. (準備資料 for 連續聊天)

```
YOUR_CONTENT = chat_text_content_str

previous_chat, previous_res = chat, r.text
chat = YOUR_CONTENT

contents = [
    {'role': 'user', 'parts': [{'text': previous_chat}]},
    {'role': 'model', 'parts': [{'text': previous_res}]},
    {'role': 'user', 'parts': [{'text': chat}]}
]
```

© 2026 Enos Chou

Gemini Application

7. 聊天查詢真實天氣

a. 初始化

```
from google.oauth2 import service_account

YOUR_SERVICE = service_account_key_file_name_str

credentials = service_account.Credentials.from_service_account_file(
    YOUR_SERVICE,
    scopes=['https://www.googleapis.com/auth/cloud-platform']
)
```

© 2026 Enos Chou

Gemini Application

7. 聊天查詢真實天氣

a. 初始化

```
from google import genai

genai_client = genai.Client(
    enterprise=True,
    location='global',
    project=credentials.project_id,
    credentials=credentials
)
```

© 2026 Enos Chou

Gemini Application

7. 聊天查詢真實天氣

b. 設定系統提示詞 system instruction

```
YOUR_SYSTEM_INSTRUCTION = '''
    你專注於查詢並回報台灣各地天氣，總是引導客戶詢問正確的地理位置，
    藉由工具計算的結果回覆天氣資訊
'''
```

© 2026 Enos Chou

Gemini Application

7. 聊天查詢真實天氣

c. 宣告函數

```
from google.genai.types import FunctionDeclaration

wea_fndec = FunctionDeclaration(
    name="get_taiwan_weather",
    description="Get current weather of a given location in Taiwan",
    parameters={"type": "object",
                "properties": {"site": {"type": "string",
                                        "description": "Location of Taiwa
                                        "enum": ["臺北", "苗栗"]}},
                "required": ["site"]}
)
```

© 2026 Enos Chou

Gemini Application

7. 聊天查詢真實天氣

d. 建立工具

```
from google.genai.types import Tool

tools = Tool(function_declarations=[wea_fndec])
```

© 2026 Enos Chou

Gemini Application

7. 聊天查詢真實天氣

e. 設定參數

```
config = {
    'max_output_tokens': 1024,
    'system_instruction': YOUR_SYSTEM_INSTRUCTION,
    'tools': [tools]
}
```

© 2026 Enos Chou

Gemini Application

7. 聊天查詢真實天氣

f. 準備資料

```
from google.genai.types import Part, Content

YOUR_CONTENT = chat_text_content_str

chat = Part.from_text(text=YOUR_CONTENT)

contents = [Content(role='user', parts=[chat])]
```

© 2026 Enos Chou

Gemini Application

7. 聊天查詢真實天氣

g. 叫用模型

```
YOUR_MODEL = 'gemini-3.5-flash-lite'  # can be changed

r = genai_client.models.generate_content(
    model=YOUR_MODEL,
    contents=contents,
    config=config
)
```

© 2026 Enos Chou

Gemini Application

7. 聊天查詢真實天氣

h. 若結果包含工具參數則呼叫函數

```
import wea

if r.function_calls:
    for fn in r.function_calls:
        if fn.name == 'get_taiwan_weather':
            wea_info = wea.tostr(wea.grab(**fn.args))
```

© 2026 Enos Chou

Gemini Application

7. 聊天查詢真實天氣

g. 重新準備回應資料

```
from google.genai.types import Part, Content

fn_response = Part.from_function_response(
    name=fn.name,
    response={'result': wea_info}
)
contents.append(Content(role='model’,
                        parts=[Part(function_call=fn)]))
contents.append(Content(role='user', parts=[fn_response]))
```

Gemini Application

7. 聊天查詢真實天氣

h. 再次叫用模型

```
YOUR_MODEL = 'gemini-3.5-flash'  # can be changed

r = genai_client.models.generate_content(
    model=YOUR_MODEL,
    contents=contents,
    config=config
)
```

© 2026 Enos Chou

Gemini Application

7. 聊天查詢真實天氣

i. 取得結果

```
print(r.text)
```

© 2026 Enos Chou

Gemini ft. LangChain

© 2026 Enos Chou

Gemini Application ft. LangChain

用戶端開發

安裝套件

```
pip install langchain langchain-google-genai
```

© 2026 Enos Chou

Gemini Application ft. LangChain

1. 車牌辨識

a. 初始化

```
from google.oauth2 import service_account

YOUR_SERVICE = service_account_key_file_name_str

credentials = service_account.Credentials.from_service_account_file(
    YOUR_SERVICE,
    scopes=['https://www.googleapis.com/auth/cloud-platform']
)
```

© 2026 Enos Chou

Gemini Application ft. LangChain

1. 車牌辨識

a. 初始化

```
from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleGenerativeAI(
    vertexai=True,
    location='global',
    credentials=credentials,
    model='gemini-3.5-flash-lite',  # can be changed
    thinking_level='minimal',
    max_tokens=100
)
```

© 2026 Enos Chou

Gemini Application ft. LangChain

1. 車牌辨識

b. 結構化

```
json_schema = {
    'type': 'object',
    'properties': {
        'plates': {
            'type': 'array',
            'items': {'type': 'string'},
            'description': '車牌號碼列表，找不到時為空'
        }
    },
    'required': ['plates'],
    'additionalProperties': False
}
```

Gemini Application ft. LangChain

1. 車牌辨識

b. 結構化

```
llm_json = llm.with_structured_output(
    json_schema,
    include_raw=True   # 回傳結果時附帶原始輸出內容
)
```

© 2026 Enos Chou

Gemini Application ft. LangChain

1. 車牌辨識

c. 準備資料

```
import base64

YOUR_JPG = image_file_name_str

with open(YOUR_JPG, 'rb') as f:
    image = f.read()

image_base64 = base64.b64encode(image).decode('utf-8')
```

© 2026 Enos Chou

Gemini Application ft. LangChain

1. 車牌辨識

c. 準備資料

```
contents = [{
    'role': 'user',
    'content': [{
        'type': 'image',
        'base64': image_base64,
        'mime_type': 'image/jpeg'  # can be changed
    }]
}]
```

© 2026 Enos Chou

Gemini Application ft. LangChain

1. 車牌辨識

d. 叫用模型

```
r = llm_json.invoke(contents)
```

© 2026 Enos Chou

Gemini Application ft. LangChain

1. 車牌辨識

e. 取得結果

```
print(r['parsed'])
print(r['raw'].usage_metadata)
```

© 2026 Enos Chou

Gemini Application ft. LangChain

2. 健檢報告分析

a. 初始化

```
from google.oauth2 import service_account

YOUR_SERVICE = service_account_key_file_name_str

credentials = service_account.Credentials.from_service_account_file(
    YOUR_SERVICE,
    scopes=['https://www.googleapis.com/auth/cloud-platform']
)
```

© 2026 Enos Chou

Gemini Application ft. LangChain

2. 健檢報告分析

a. 初始化

```
from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleGenerativeAI(
    vertexai=True,
    location='global',
    credentials=credentials,
    model='gemini-3.5-flash-lite',  # can be changed
    thinking_level='minimal',
    max_tokens=2_048
)
```

© 2026 Enos Chou

Gemini Application ft. LangChain

2. 健檢報告分析

b. 結構化

```
json_schema = {
    "type": "object",
    "properties": {
        "items": {
            "type": "array",
            "description": "檢查項目列表",
            "items": {
                "type": "object",
                "properties": {
                    "name": {"type": "string", "description": "檢查項目名稱"},
                    "value": {"type": "number", "description": "檢查項目的數值"},
                    "unit": {"type": "string", "description": "檢查項目的單位"}
                },
                "required": ["name", "value", "unit"]
            }
        }
    },
    "required": ["items"]
}
```

© 2026 Enos Chou

Gemini Application ft. LangChain

2. 健檢報告分析

b. 結構化

```
llm_json = llm.with_structured_output(
    json_schema,
    include_raw=True   # 回傳結果時附帶原始輸出內容
)
```

© 2026 Enos Chou

Gemini Application ft. LangChain

2. 健檢報告分析

c. 準備資料

```
import base64

YOUR_JPG = image_file_name_str

with open(YOUR_JPG, 'rb') as f:
    image = f.read()

image_base64 = base64.b64encode(image).decode('utf-8')
```

© 2026 Enos Chou

Gemini Application ft. LangChain

2. 健檢報告分析

c. 準備資料

```
contents = [{
    'role': 'user',
    'content': [{
        'type': image',
        'base64': image_base64,
        'mime_type': 'image/jpeg'  # can be changed
    }]
}]
```

© 2026 Enos Chou

Gemini Application ft. LangChain

2. 健檢報告分析

d. 叫用模型

```
r = llm_json.invoke(contents)
```

© 2026 Enos Chou

Gemini Application ft. LangChain

2. 健檢報告分析

e. 取得結果

```
from pprint import pprint

pprint(r['parsed'])
pprint(r['raw'].usage_metadata)
```

© 2026 Enos Chou

Gemini Application ft. LangChain

4. 語音轉文字

a. 初始化

```
from google.oauth2 import service_account

YOUR_SERVICE = service_account_key_file_name_str

credentials = service_account.Credentials.from_service_account_file(
    YOUR_SERVICE,
    scopes=['https://www.googleapis.com/auth/cloud-platform']
)
```

© 2026 Enos Chou

Gemini Application ft. LangChain

4. 語音轉文字

a. 初始化

```
from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleGenerativeAI(
    vertexai=True,
    location='global',
    credentials=credentials,
    model='gemini-3.5-flash-lite',  # can be changed
    thinking_level='minimal',
    max_tokens=2_048
)
```

© 2026 Enos Chou

Gemini Application ft. LangChain

4. 語音轉文字

b. 準備資料

```
import base64

YOUR_MP3 = audio_file_name_str

with open(YOUR_MP3, 'rb') as f:
    audio = f.read()

audio_base64 = base64.b64encode(audio).decode('utf-8')
```

© 2026 Enos Chou

Gemini Application ft. LangChain

4. 語音轉文字

b. 準備資料

```
contents = [{
    'role': 'user',
    'content': [
        {'type': 'text',
         'text': 'transcribe to Traditional Chinese'},
        {'type': 'audio',
         'base64': audio_base64,
         'mime_type': 'audio/mpeg'},  # can be changed
    ]
}]
```

© 2026 Enos Chou

Gemini Application ft. LangChain

4. 語音轉文字

c. 叫用模型

```
r = llm.invoke(contents)
```

© 2026 Enos Chou

Gemini Application ft. LangChain

4. 語音轉文字

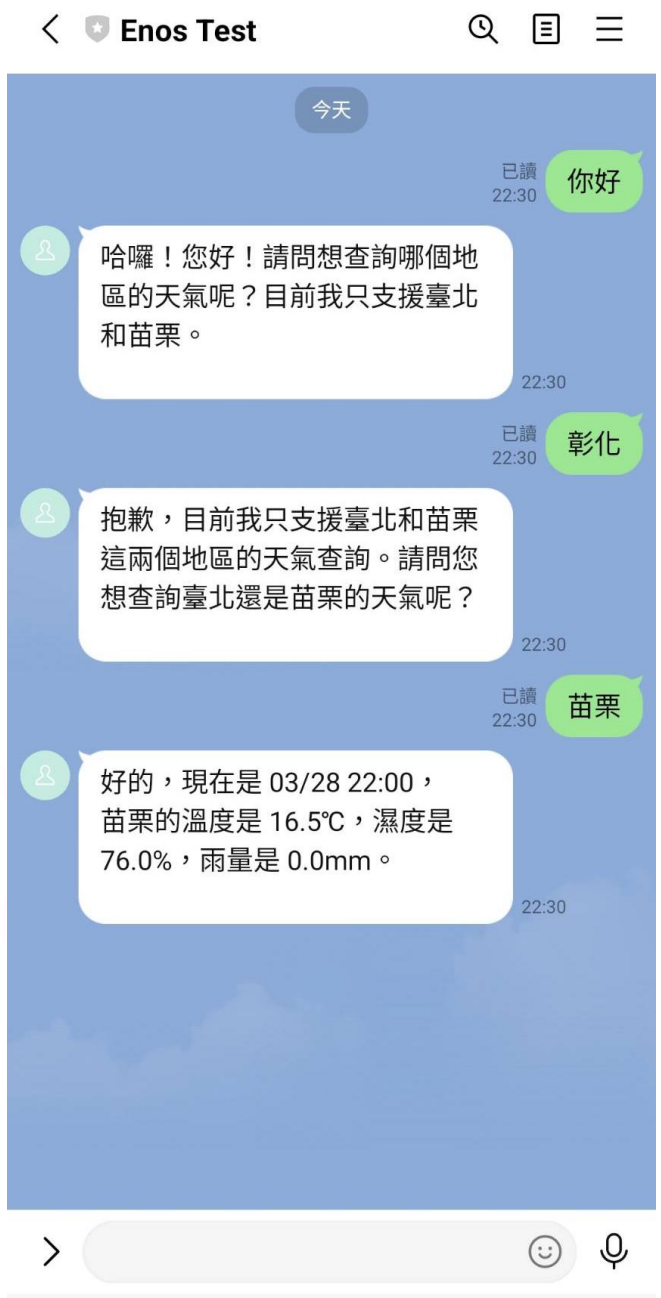
d. 取得結果

```
print(r.text)
print(r.usage_metadata)
```

© 2026 Enos Chou

Your Turn - 天氣對話機器人

完成能夠透過聊天查詢限定地點天氣的 LINE Bot (1 hr)



© 2026 Enos Chou

Azure OpenAI 整合應用

by openai SDK, Responses API

© 2026 Enos Chou

Azure OpenAI ft. Responses API

OpenAI Responses API 參考資料

範例說明

https://developers.openai.com/api/reference/responses/overview

API 整合

https://developers.openai.com/api/docs/guides/migrate-to-responses

© 2026 Enos Chou

Azure OpenAI Application

模型部署

透過 Azure Portal 的 Azure OpenAI 或 Microsoft Foundry 部署 LLM

© 2026 Enos Chou

Azure OpenAI Application

用戶端開發

安裝套件

```
# pydantic 為 optional，協助建立 JSON response schema
pip install openai pydantic
```

© 2026 Enos Chou

Azure OpenAI Application

1. 車牌辨識

a. 初始化

```
from urllib.parse import urljoin
from openai import OpenAI

# 透過 Azure Portal > Foundry > 金鑰與端點 取得以下參數
YOUR_KEY = azure_ai_key_str
YOUR_ENDPOINT = endpoint_name_str  # ex: https://xxx.openai.azure.com/

openai_client = OpenAI(api_key=YOUR_KEY,
                       base_url=urljoin(YOUR_ENDPOINT, '/openai/v1/'))
```

Azure OpenAI Application

關鍵參數

律定 response schema by hand

```
YOUR_RESPONSE_SCHEMA =

{'format': {'name': 'plates_schema',
            'schema': {'additionalProperties': False,
                       'properties': {'plates': {'description': '車牌號碼，允許多組，也可能為空陣列（代表未偵測到）',
                                                 'items': {'type': 'string'},
                                                 'title': 'Plates',
                                                 'type': 'array'}},
                       'required': ['plates'],
                       'title': 'PlatesSchema',
                       'type': 'object'},
            'strict': True,
            'type': 'json_schema'}}
```

© 2026 Enos Chou

Azure OpenAI Application

關鍵參數

律定 response schema by pydantic

```
from pydantic import BaseModel, Field, ConfigDict

class PlatesSchema(BaseModel):
    # "additionalProperties": False
    model_config = ConfigDict(extra='forbid')
    # ... means required
    plates: list[str] = Field(
        ...,
        description='車牌號碼，允許多組，也可能為空陣列（代表未偵測到）'
    )
```

© 2026 Enos Chou

Azure OpenAI Application

1. 車牌辨識

b. 設定 response schema

```
from pydantic import BaseModel, Field, ConfigDict

class PlatesSchema(BaseModel):
    # "additionalProperties": False
    model_config = ConfigDict(extra='forbid')
    # ... means required
    plates: list[str] = Field(
        ...,
        description='車牌號碼，允許多組，也可能為空陣列（代表未偵測到）'
    )
```

© 2026 Enos Chou

Azure OpenAI Application

1. 車牌辨識

b. 設定 response schema

```
YOUR_RESPONSE_SCHEMA = {
    "format": {
        "type": "json_schema",
        "name": "plates_schema",
        "strict": True,
        "schema": PlatesSchema.model_json_schema()
    }
}
```

© 2026 Enos Chou

Azure OpenAI Application

1. 車牌辨識

c. 設定參數

```
#YOUR_INSTRUCTIONS = system_instructions_str  # 視需要設定

config = {
    #'instructions': YOUR_INSTRUCTIONS,  # 視需要設定
    'text': YOUR_RESPONSE_SCHEMA,
    'reasoning': {'effort': 'none'}  # 不推理
}
```

© 2026 Enos Chou

Azure OpenAI Application

1. 車牌辨識

d. 準備資料

```
import base64

YOUR_JPG = 'YOUR_JPG'

with open(YOUR_JPG, 'rb') as f:
    base64_image = base64.b64encode(f.read()).decode("utf-8")
content = [
    {"type": "input_image",
     "image_url": f"{base64_image}"}
]
```

© 2026 Enos Chou

Azure OpenAI Application

1. 車牌辨識

d. 準備資料

```
messages = [{'role': 'user', 'content': content}]
```

© 2026 Enos Chou

Azure OpenAI Application

1. 車牌辨識

e. 叫用模型

```
YOUR_MODEL = deployment_name_str  # ex: gpt-5.4

r = openai_client.responses.create(
    model=YOUR_MODEL,
    input=messages,
    **config
)
```

© 2026 Enos Chou

Azure OpenAI Application

1. 車牌辨識

f. 取得結果

```
import json
from pprint import pprint

pprint(json.loads(r.output_text))
print(r.usage)
```

© 2026 Enos Chou

Azure OpenAI Application

2. 文件摘要

a. 初始化

```
from urllib.parse import urljoin
from openai import OpenAI

# 透過 Azure Portal > Foundry > 金鑰與端點 取得以下參數
YOUR_KEY = azure_ai_key_str
YOUR_ENDPOINT = endpoint_name_str  # ex: https://xxx.openai.azure.com/

openai_client = OpenAI(api_key=YOUR_KEY,
                       base_url=urljoin(YOUR_ENDPOINT, '/openai/v1/'))
```

© 2026 Enos Chou

Azure OpenAI Application

2. 文件摘要

b. 設定 response schema

```
from pydantic import BaseModel, Field, ConfigDict

class ArticleSchema(BaseModel):
    # "additionalProperties": False
    model_config = ConfigDict(extra='forbid')
    # ... means required
    title: str = Field(..., description='內容標題')
    description: str = Field(..., description='重點描述')
    date: str = Field(..., description='文章時間')
```

© 2026 Enos Chou

Azure OpenAI Application

2. 文件摘要

b. 設定 response schema

```
YOUR_RESPONSE_SCHEMA = {
    "format": {
        "type": "json_schema",
        "name": "article_schema",
        "strict": True,
        "schema": ArticleSchema.model_json_schema()
    }
}
```

© 2026 Enos Chou

Azure OpenAI Application

2. 文件摘要

c. 設定參數

```
#YOUR_INSTRUCTIONS = system_instructions_str  # 視需要設定

config = {
    #'instructions': YOUR_INSTRUCTIONS,  # 視需要設定
    'text': YOUR_RESPONSE_SCHEMA,
    'reasoning': {'effort': 'none'}  # 不推理
}
```

© 2026 Enos Chou

Azure OpenAI Application

2. 文件摘要

d. 準備資料

```
YOUR_CONTENT = chat_text_content_str

messages = [{'role': 'user', 'content': YOUR_CONTENT}]
```

© 2026 Enos Chou

Azure OpenAI Application

2. 文件摘要

e. 叫用模型

```
YOUR_MODEL = deployment_name_str  # ex: gpt-5.4

r = openai_client.responses.create(
    model=YOUR_MODEL,
    input=messages,
    **config
)
```

© 2026 Enos Chou

Azure OpenAI Application

2. 文件摘要

f. 取得結果

```
import json
from pprint import pprint

pprint(json.loads(r.output_text))
print(r.usage)
```

© 2026 Enos Chou

© 2026 Enos Chou

RAG & Vector DB 實作

Vector DB ft. Firestore + Gemini

Vector DB ft. Azure Cosmos DB + OpenAI

RAG ft. Google File Search

RAG 在 LLM 的主要目的

1. Grounding
2. Cost Down

© 2026 Enos Chou

RAG 流程

1. 將所有必要資料 embed 為向量儲存
2. 用戶查詢時先將問題 embed 為向量，再以語意搜尋儲存中最相關向量
3. 將最相關向量對應的實際資料提供 LLM 參考以回覆用戶

© 2026 Enos Chou

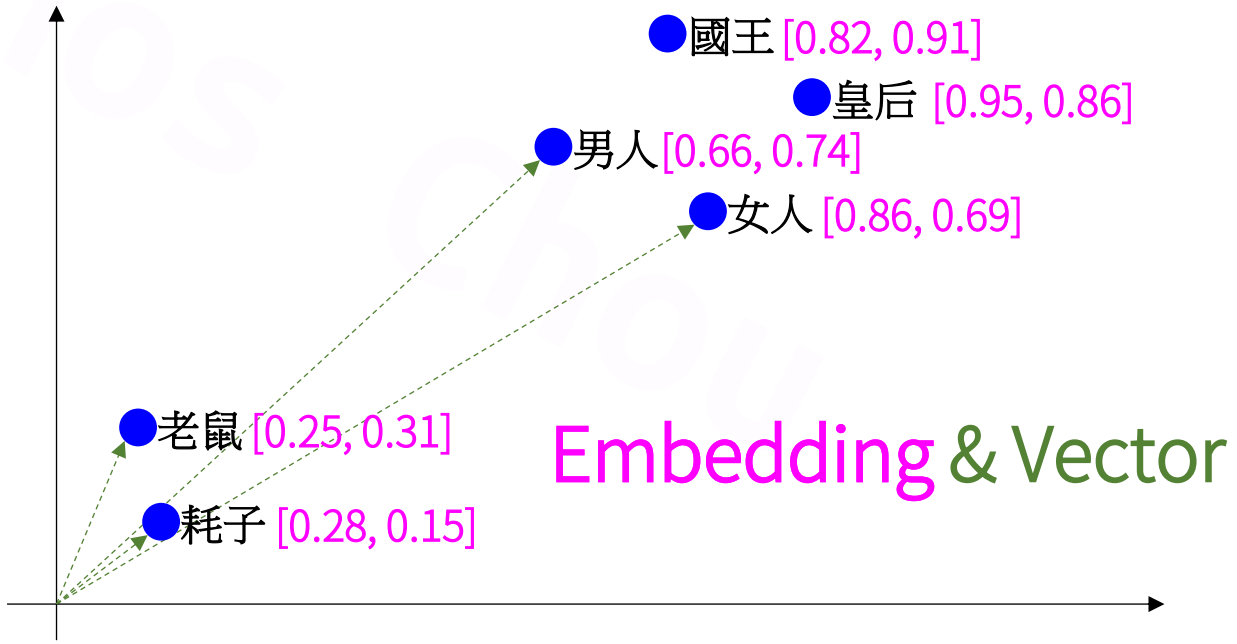
語意搜尋與向量儲存

文字比對

老鼠 ≠ 耗子

語意比對

老鼠 ≒ 耗子



© 2026 Enos Chou

向量儲存

文字儲存方式

RDB or NoSQL

關聯式資料庫 或 非關聯式資料庫

向量儲存方式

Vector DB or NoSQL

向量資料庫 或 非關聯式資料庫

© 2026 Enos Chou

向量儲存

向量資料庫 (Vector DB)

Google Cloud Vector Search
Azure AI Search
Qdrant
Pinecone
Faiss
...

非關連資料庫 (NoSQL)

Google Cloud Firestore
Azure Cosmos DB
MongoDB Atlas
...

© 2026 Enos Chou

Google Cloud Firestore for Vector

1. 建立 Firestore Database
2. 準備資料並 embed
3. 上傳資料與 embeddings
4. 建立 index
5. 語意搜尋
6. 整合至 LINE Bot

© 2026 Enos Chou

Google Cloud Firestore for Vector

1. 建立 Firestore Database

   via the console

© 2026 Enos Chou

Google Cloud Firestore for Vector

2. 準備資料並 embed

服務帳戶角色

`Vertex AI: Agent Platform 使用者`

Enos Chou

Google Cloud Firestore for Vector

2. 準備資料並 embed

套件

```
pip install google-genai google-cloud-firestore
```

© 2026 Enos Chou

Google Cloud Firestore for Vector

2. 準備資料並 embed

初始化 genai

```
from google.oauth2 import service_account

YOUR_SERVICE = service_account_key_file_name_str

credentials = service_account.Credentials.from_service_account_file(
    YOUR_SERVICE,
    scopes=['https://www.googleapis.com/auth/cloud-platform']
)
```

© 2026 Enos Chou

Google Cloud Firestore for Vector

2. 準備資料並 embed

初始化 genai

```
from google import genai

YOUR_REGION = region_str  # 指定模型區域，新模型通常位於 global or
                          # us-central1
genai_client = genai.Client(
    enterprise=True,
    location=YOUR_REGION,
    project=credentials.project_id,
    credentials=credentials
)
```

© 2026 Enos Chou

Google Cloud Firestore for Vector

2. 準備資料並 embed

初始化 genai

© 2026 Enos Chou

Google Cloud Firestore for Vector

2. 準備資料並 embed

準備資料

```
data = [...,
        {'title': '理賠計算機',
         'text': '搭配 Gemini 與 LINE Bot 實作保險理賠計算機',
         'author': ['Jenny Wu', 'Jacob Chiu', 'Jim Hsiao'],
         'date': '20250712'},
        {'title': '資安檢測機器人',
         'text': '混和 AI Agent 與 RAG 技術，實作全自動資安檢測機器人',
         'author': ['Joe Li', 'Jim Hsiao'],
         'date': '20251011'}]
```

© 2026 Enos Chou

Google Cloud Firestore for Vector

2. 準備資料並 embed

準備資料

```
contents = [d['text'] for d in data]
```

© 2026 Enos Chou

Google Cloud Firestore for Vector

2. 準備資料並 embed

embedding

```
YOUR_MODEL = 'gemini-embedding-2'  # 維度上限 3,072
YOUR_DIM = 768  # Firestore Vector 維度上限 2,048

config = {
    'task_type': 'RETRIEVAL_DOCUMENT',
    'output_dimensionality': YOUR_DIM
}
```

© 2026 Enos Chou

Google Cloud Firestore for Vector

2. 準備資料並 embed

embedding

```
r = genai_client.models.embed_content(
    model=YOUR_MODEL,
    contents=contents,
    config=config
)

print(r.embeddings)
```

Enos Chou

Google Cloud Firestore for Vector

2. 準備資料並 embed

embedding

```
from google.cloud.firestore_v1.vector import Vector

YOUR_EMBEDDING_KEY = embedding_key_str

docs = [d | {'id': f'{i:03}', YOUR_EMBEDDING_KEY: Vector(e.values)} \
        for i, (d, e) in enumerate(zip(data, r.embeddings))]
```

© 2026 Enos Chou

Google Cloud Firestore for Vector

3. 上傳資料與 embeddings

服務帳戶角色

Datastore: Cloud Datastore 使用者

Enos Chou

Google Cloud Firestore for Vector

3. 上傳資料與 embeddings

套件

```
pip install google-cloud-firestore
```

© 2026 Enos Chou

Google Cloud Firestore for Vector

3. 上傳資料與 embeddings

初始化 Firestore

```
YOUR_SERVICE = service_account_key_file_name_str
YOUR_DATABASE = database_name_str  # None 為預設 DB
YOUR_COLLECTION = collection_name_str
```

© 2026 Enos Chou

Google Cloud Firestore for Vector

3. 上傳資料與 embeddings

初始化 Firestore

```
from google.cloud import firestore

firestore_client = firestore.Client.from_service_account_json(
    YOUR_SERVICE,
    database=YOUR_DATABASE
)
collection = firestore_client.collection(YOUR_COLLECTION)
```

© 2026 Enos Chou

Google Cloud Firestore for Vector

3. 上傳資料與 embeddings

上傳資料

```
for d in docs:
    collection.document(d['id']).set(d)
```

© 2026 Enos Chou

Google Cloud Firestore for Vector

4. 建立索引

gcloud CLI

① 登入認證

```
gcloud auth login
```

© 2026 Enos Chou

Google Cloud Firestore for Vector

4. 建立索引

gcloud CLI

② 設定專案代號

```
gcloud config set project YOUR_PROJECTID
```

© 2026 Enos Chou

Google Cloud Firestore for Vector

4. 建立索引

gcloud CLI

③ 建立 index （初次使用或異動 dimension 值都須重新建立 index）

```
gcloud firestore indexes composite create --project=YOUR_PROJECTID --
database=YOUR_DATABASE --collection-group=YOUR_COLLECTION --query-
scope=COLLECTION --field-config=vector-
config="{'dimension':YOUD_DIM,'flat':{}}",field-path=YOUR_EMBEDDING_KEY
```

```
gcloud firestore indexes composite create --project=my-proj-001 --
database=test --collection-group=hello --field-config=vector-
config="{'dimension':768,'flat':{}}",field-path=embedding
```

© 2026 Enos Chou

Google Cloud Firestore for Vector

5. 語意搜尋

Embed 搜尋描述

```
YOUR_SEARCH = content_to_search_str

config = {'task_type': 'RETRIEVAL_DOCUMENT',
          'output_dimensionality': YOUR_DIM}

r = genai_client.models.embed_content(
    model=YOUR_MODEL,
    contents=[YOUR_SEARCH],
    config=config)
)
```

© 2026 Enos Chou

Google Cloud Firestore for Vector

5. 語意搜尋

KNN 向量搜尋

```
vector_search = collection.find_nearest(
    vector_field=YOUR_EMBEDDING_KEY,
    query_vector=Vector(r.embeddings[0].values),
    #distance_measure=DistanceMeasure.EUCLIDEAN,    # 愈小愈接近
    #distance_measure=DistanceMeasure.COSINE,       # 愈小愈接近
    distance_measure=DistanceMeasure.DOT_PRODUCT,   # 愈大愈接近
    limit=5,
    distance_result_field='d'   # 保留距離欄位
)
```

© 2026 Enos Chou

Google Cloud Firestore for Vector

5. 語意搜尋

KNN 混合搜尋 (需要調整 index)

```
vector_search = collection.where('author', 'array_contains', 'Jim Hsiao').find_nearest(
    vector_field=YOUR_EMBEDDING_KEY,
    query_vector=Vector(r.embeddings[0].values),
    #distance_measure=DistanceMeasure.EUCLIDEAN,    # 愈小愈接近
    #distance_measure=DistanceMeasure.COSINE,       # 愈小愈接近
    distance_measure=DistanceMeasure.DOT_PRODUCT,  # 愈大愈接近
    limit=5,
    distance_result_field='d'  # 保留距離欄位
)
```

© 2026 Enos Chou

Google Cloud Firestore for Vector

5. 語意搜尋

KNN 混合搜尋 (需要調整 index)

```
gcloud firestore indexes composite create --project=YOUR_PROJECTID --
database=YOUR_DATABASE --collection-group=YOUR_COLLECTION --query-
scope=COLLECTION --field-config=vector-
config="{'dimension':YOUD_DIM,'flat':{}}",field-path=YOUR_EMBEDDING_KEY
```

```
gcloud firestore indexes composite create --project=my-proj-001 --
database=test --collection-group=hello --field-config=array-
config=contains,field-path=author --field-config=vector-
config="{'dimension':768,'flat':{}}",field-path=embedding
```

© 2026 Enos Chou

Google Cloud Firestore for Vector

5. 語意搜尋

KNN 搜尋

```
nearest = [v.to_dict() for v in vector_search.get()]
```

© 2026 Enos Chou

Google Cloud Firestore for Vector

5. 語意搜尋

KNN 搜尋

```
from pprint import pprint

pprint([(a['text'], a['d']) for a in nearest])
```

© 2026 Enos Chou

Azure Cosmos DB Vector Search

1. 建立 Azure Cosmos DB account
2. 啟動向量搜尋
3. 建立向量索引 Container
4. 建立 Azure OpenAI Embedding Model
5. 準備資料並 embed
6. 上傳資料與 embeddings
7. 語意搜尋
8. 整合至 LINE Bot

© 2026 Enos Chou

Azure Cosmos DB Vector Search

1. 建立 Azure Cosmos DB account

   via the portal

© 2026 Enos Chou

Azure Cosmos DB Vector Search

2. 啟動向量搜尋

進入指定 Azure Cosmos DB account >
設定 > 功能 > Vector Search for NoSQL API > Enable

© 2026 Enos Chou

Azure Cosmos DB Vector Search

3. 建立向量索引 Container

依正常程序 New Container >

Container Vector Policy >

Add vector embedding >

Path:	/embedding (自行命名)
Data type:	float32 (float32, uint8, int8 擇一)
Distance function:	dotproduct (euclidean, cosine, dotproduct 擇一)
Dimension:	768 (0 ~ 4,096)
Index type:	quantizedFlat (flat, diskANN, quantizedFlat 擇一)
	(flat: max 505, others: 4,096)

> OK

© 2026 Enos Chou

Azure Cosmos DB Vector Search

4. 建立 Azure OpenAI Embedding Model

via the portal >
建立 Azure AI Fountry >
部署 text-embedding-3-large >

Note
text-embedding-3-large 預設與最大維度皆為 3,072

© 2026 Enos Chou

Azure Cosmos DB Vector Search

5. 準備資料並 embed

套件

```
pip install openai
```

© 2026 Enos Chou

Azure Cosmos DB Vector Search

5. 準備資料並 embed

初始化 openai

```
from openai import AzureOpenAI

YOUR_KEY = azure_ai_key_str  # Azure AI Foundry > 資源管理 > 金鑰與端點
YOUR_ENDPOINT = endpoint_name_str  # 同上,
                                   # ex: https://xxx.openai.azure.com/

model = AzureOpenAI(api_key=YOUR_KEY,
                    azure_endpoint=YOUR_ENDPOINT,
                    api_version='2024-02-01')
```

© 2026 Enos Chou

Azure Cosmos DB Vector Search

5. 準備資料並 embed

準備資料

```
data = [...,
        {'title': '理賠計算機',
         'text': '搭配 Gemini 與 LINE Bot 實作保險理賠計算機',
         'author': ['Jenny Wu', 'Jacob Chiu', 'Jim Hsiao'],
         'date': '20250712',
         'course': 'CCC456'},
        {'title': '資安檢測機器人',
         'text': '混和 AI Agent 與 RAG 技術，實作全自動資安檢測機器人',
         'author': ['Joe Li', 'Jim Hsiao'],
         'date': '20251011',
         'course': 'CCC456'}]
```

© 2026 Enos Chou

Azure Cosmos DB Vector Search

5. 準備資料並 embed

embedding

```
YOUR_MODEL = 'text-embedding-3-large'   # 預設與最大維度 3,072
YOUR_DIM = 768
```

© 2026 Enos Chou

Azure Cosmos DB Vector Search

5. 準備資料並 embed

embedding

```
contents = [d['text'] for d in data]

r = model.embeddings.create(
    input=contents,
    model=YOUR_MODEL,
    dimensions=YOUR_DIM
)
```

© 2026 Enos Chou

Azure Cosmos DB Vector Search

5. 準備資料並 embed

embedding

```
print(r.data[0].embedding)
```

© 2026 Enos Chou

Azure Cosmos DB Vector Search

5. 準備資料並 embed

embedding

```
YOUR_EMBEDDING_KEY = embedding_key_str

docs = [d | {'id': f'{i:03}', YOUR_EMBEDDING_KEY: r.data[i].embedding} \
        for i, d in enumerate(data)]
```

© 2026 Enos Chou

Azure Cosmos DB Vector Search

6. 上傳資料與 embeddings

套件

```
pip install azure-cosmos
```

© 2026 Enos Chou

Azure Cosmos DB Vector Search

6. 上傳資料與 embeddings

初始化 Cosmos

```
from azure.cosmos import CosmosClient

YOUR_CONNSTR = connection_string_str  # Cosmos DB account > 設定 >
                                      # 索引鍵
YOUR_DATABASE = database_name_str
YOUR_CONTAINER = container_name_str

cosmos_client = CosmosClient.from_connection_string(YOUR_CONNSTR)
db = cosmos_client.get_database_client(database=YOUR_DATABASE)
container = db.get_container_client(YOUR_CONTAINER)
```

© 2026 Enos Chou

Azure Cosmos DB Vector Search

6. 上傳資料與 embeddings

上傳資料

```
for d in docs:
    container.upsert_item(d)
```

© 2026 Enos Chou

Azure Cosmos DB Vector Search

7. 語意搜尋

Embed 搜尋描述

```
YOUR_SEARCH = content_to_search_str

r = model.embeddings.create(
    input=[YOUR_SEARCH],
    model=YOUR_MODEL,
    dimensions=YOUR_DIM
)

search_embedding = r.data[0].embedding
```

© 2026 Enos Chou

Azure Cosmos DB Vector Search

7. 語意搜尋

向量搜尋

```
query = f'''
SELECT TOP 3 c.text, VectorDistance(c.{YOUR_EMBEDDING_KEY}, {search_embedding})
AS SimilarityScore
FROM c
ORDER BY VectorDistance(c.{YOUR_EMBEDDING_KEY}, {search_embedding})
'''

nearest = list(container.query_items(query=query,
                                     enable_cross_partition_query=True))
```

© 2026 Enos Chou

Google File Search

功能與特色

- 使用者直接上傳檔案，系統自動完成 embedding & 向量儲存
- 上傳檔案完成後即可詢問，將回覆 RAG 建議以及 grounding source
- 只要負擔建立 (檔案上傳) 時的 embedding 費用；查詢 embedding 免費

限制

目前只能透過 Google AI Studio 使用

© 2026 Enos Chou

Google File Search

參考資料

https://ai.google.dev/gemini-api/docs/file-search?hl=zh-tw

© 2026 Enos Chou

Google File Search

1. 初始化
2. 建立
3. 維護
4. 查詢

© 2026 Enos Chou

Google File Search

1. 初始化

安裝套件

```
pip install google-genai
```

© 2026 Enos Chou

Google File Search

1. 初始化

初始化 genai

```
from google import genai

YOUR_KEY = google_ai_studio_key_str  # Google AI Studio or
                                     # "GCP API 和服務" > 憑證

genai_client = genai.Client(api_key=YOUR_KEY)
```

© 2026 Enos Chou

Google File Search

2. 建立

建立 file search store

```
file_search_store = genai_client.file_search_stores.create(
    config={'display_name': 'mystore'}
)

print(file_search_store)
```

© 2026 Enos Chou

Google File Search

2. 建立

上傳檔案至 file search store

```
# 一次上傳多檔範例
files = {
    '1141-1.pdf': {'original': '114年度上半年產業人才投資計畫', 'pages': 112},
    '1141-2.pdf': {'original': '114年度上半年提升勞工自主學習計畫', 'pages': 60},
    '1142-1.pdf': {'original': '114年度下半年產業人才投資計畫', 'pages': 110},
    '1142-2.pdf': {'original': '114年度下半年提升勞工自主學習計畫', 'pages': 56}
}
```

© 2026 Enos Chou

Google File Search

2. 建立

上傳檔案至 file search store

```
# 上傳多個檔案範例，極簡版
uploads = [None] * len(files)
for i, (file, meta) in enumerate(files.items()):
    uploads[i] = genai_client.file_search_stores.upload_to_file_search_store(
        file=file,  # 須為 ascii
        file_search_store_name=file_search_store.name
    )
```

© 2026 Enos Chou

Google File Search

2. 建立

上傳檔案至 file search store

```
uploads = [None] * len(files)
for i, (file, meta) in enumerate(files.items()):
    uploads[i] = genai_client.file_search_stores.upload_to_file_search_store(
        file=file,
        file_search_store_name=file_search_store.name,
        config={
            'display_name': file,
            'custom_metadata': [
                {'key': 'original', 'string_value': meta['original']},
                {'key': 'pages', 'numeric_value': meta['pages']}
            ],
            'chunking_config': {
               'white_space_config': {
                    'max_tokens_per_chunk': 200,
                    'max_overlap_tokens': 20
               }
            }
        }
    )
```

```
# 上傳多個案檔範例，
# 含進階參數，包含
# chunking 與 meta
```

Google File Search

2. 建立

上傳檔案至 file search store

```
operation = genai_client.file_search_stores.upload_to_file_search_store(
    file='1141-1.pdf',
    file_search_store_name=file_serach_store.name,
    config={
        'display_name': '1141-1.pdf',
        'mime_type': 'application/pdf',
        'custom_metadata': [
            {'key': 'original', 'string_value': '114年度上半年產業人才投資計畫'},
            {'key': 'pages', 'numeric_value': 112}
        ]
    }
)
print(operation)
```

```
# 上傳單一檔案範例
```

© 2026 Enos Chou

Google File Search

2. 建立

(等待上傳完成)

```
import time

operation = genai_client.operations.get(operation)
while not operation.done:
    time.sleep(5)
    operation = genai_client.operations.get(operation)

print(operation)
```

© 2026 Enos Chou

Google File Search

3. 維護

列出所有 file search stores

```
for stores in genai_client.file_search_stores.list():
    print(stores)
```

© 2026 Enos Chou

Google File Search

3. 維護

刪除指定 file search store

```
from google.genai.types import DeleteFileSearchStoreConfig

YOUR_STORE_NAME = file_search_store_name_str

# 刪除空的 file search store
genai_client.file_search_stores.delete(name=YOUR_STORE_NAME)

# 強制刪除 file search store，非空也能夠刪除
genai_client.file_search_stores.delete(
    name=YOUR_STORE_NAME,
    config=DeleteFileSearchStoreConfig(force=True)
)
```

© 2026 Enos Chou

Google File Search

3. 維護

列出 file search store 中所有 documents

```
YOUR_STORE_NAME = file_search_store_name_str

for doc in genai_client.file_search_stores.documents.list(parent=YOUR_STORE_NAME):
    print(doc)
```

© 2026 Enos Chou

Google File Search

3. 維護

刪除指定 document

```
from google.genai.types import DeleteDocumentConfig

YOUR_DOCUMENT_NAME = document_name_str

# 刪除空的 document
genai_client.file_search_stores.documents.delete(name=YOUR_DOCUMENT_NAME)

# 強制刪除指定 document，非空也能夠刪除
genai_client.file_search_stores.documents.delete(
    name=YOUR_DOCUMENT_NAME,
    config=DeleteDocumentConfig(force=True)
)
```

© 2026 Enos Chou

Google File Search

3. 維護

刪除 file search store 中所有 documents

```
from google.genai.types import DeleteDocumentConfig

YOUR_STORE_NAME = file_search_store_name_str

for doc in genai_client.file_search_stores.documents.list(parent=YOUR_STORE_NAME):
    genai_client.file_search_stores.documents.delete(
        name=doc.name,
        config=DeleteDocumentConfig(force=True)
    )
    print(f'delete {doc.name}: {doc.display_name} done')
```

© 2026 Enos Chou

Google File Search

4. 查詢

RAG 查詢

```
from google import genai

YOUR_KEY = google_ai_studio_key_str

genai_client = genai.Client(api_key=YOUR_KEY)
```

© 2026 Enos Chou

Google File Search

4. 查詢

RAG 查詢

```
from google.genai.types import Tool, FileSearch

YOUR_STORE_NAME = file_search_store_name_str

search_tool = Tool(
    file_search=FileSearch(
        file_search_store_names=[YOUR_STORE_NAME],
        metadataFilter='original=114年度下半年產業人才投資計畫'   # optional
    )
)
```

© 2026 Enos Chou

Google File Search

4. 查詢

RAG 查詢

```
from google.genai.types import GenerateContentConfig

YOUR_MODEL = model_name_str  # ex: gemini-3.5-flash
YOUR_SEARCH = content_to_search_str  # ex: 找找看 Python 47 小時的課程

response = genai_client.models.generate_content(
    model=YOUR_MODEL,
    contents=YOUR_QUESTION,
    config=GenerateContentConfig(tools=[search_tool])
)
```

© 2026 Enos Chou

Google File Search

4. 查詢

RAG 查詢

```
# 取得 RAG 回應
print(response.text)
```

© 2026 Enos Chou

Google File Search

4. 查詢

RAG 查詢

```
# 取得 grounding sources
if g := response.candidates[0].grounding_metadata:
    sources = {c.retrieved_context.title for c in g.grounding_chunks}
    print('\n'.join(sources))
```

© 2026 Enos Chou

Your Turn - 專題導覽員

完成能夠透過聊天查詢專題內容的機器人 (1 hr)

© 2026 Enos Chou

© 2026 Enos Chou

AI Agent 實作

ft. LangGraph + Gemini

AI Agent

能夠自行完成任務的代理人

1. 自行依任務規劃執行步驟，不須人為介入
2. 若任務缺乏關鍵資訊會主動詢問
3. 自行利用工具完成各項步驟，不須人為介入
4. 當結果不如預期能夠自行調整步驟，不須人為介入

© 2026 Enos Chou

AI Agent 常用框架

1. LangGraph
2. Google ADK
3. LlamaIndex
4. ...

© 2026 Enos Chou

天氣查詢 AI Agent ft. LangGraph

需求

當用戶詢問某台灣地名之目前天氣資訊，AI Agent 能夠自行調用

a. 座標查詢工具 (地名轉座標)
b. 目前天氣查詢工具 (座標取得天氣)

以取得正確天氣資訊

© 2026 Enos Chou

天氣查詢 AI Agent ft. LangGraph

1. 準備環境
2. 驗證工具調用
3. 驗證自行調整
4. 整合至 LINE Bot

© 2026 Enos Chou

天氣查詢 AI Agent ft. LangGraph

1. 準備環境

啟用 API

`Gemini` or `Generative Language API`

© 2026 Enos Chou

天氣查詢 AI Agent ft. LangGraph

1. 準備環境

服務帳戶角色

`Vertex AI: Agent Platform 使用者`

© 2026 Enos Chou

天氣查詢 AI Agent ft. LangGraph

1. 準備環境

套件

public

```
langchain langchain-google-genai
geopy
```

proprietary

```
wea
```

NOTE

`langchain`: 自帶 AI Agent 框架 LangGraph
`langchain-google-genai`: Gemini API
`geopy`: 地名轉座標

`wea`: 天氣查詢

© 2026 Enos Chou

天氣查詢 AI Agent ft. LangGraph

2. 驗證工具調用

初始化 LLM

```
import os
from langchain_google_genai import ChatGoogleGenerativeAI

YOUR_SERVICE = service_account_key_file_name_str
YOUR_MODEL = model_name_str

os.environ['GOOGLE_APPLICATION_CREDENTIALS'] = YOUR_SERVICE
llm = ChatGoogleGenerativeAI(model=YOUR_MODEL, vertexai=True, location='global')
```

© 2026 Enos Chou

天氣查詢 AI Agent ft. LangGraph

2. 驗證工具調用

準備工具

```
from langchain_core.tools import tool
```

© 2026 Enos Chou

天氣查詢 AI Agent ft. LangGraph

2. 驗證工具調用

準備工具

```
from geopy.geocoders import Nominatim

geolocator = Nominatim(user_agent='user_agent', timeout=5)

@tool
def get_coordinates(location: str) -> dict[str, float] | str:
    '''輸入地點名稱，取得地點座標'''
    if loc := geolocator.geocode(location):
        return {'latitude': loc.latitude, 'longitude': loc.longitude}
    return f"找不到 {location}"
```

© 2026 Enos Chou

天氣查詢 AI Agent ft. LangGraph

2. 驗證工具調用

準備工具

```
import wea

@tool
def get_weather_by_coordinates(latitude: float, longitude: float) -> str:
    '''輸入地點座標，取得天氣資訊'''
    return wea.tostr(wea.grab((latitude, longitude)))
```

© 2026 Enos Chou

天氣查詢 AI Agent ft. LangGraph

2. 驗證工具調用

建立 ReAct AI Agent

```
from langchain.agents import create_agent

tools = [
    get_coordinates,
    get_weather_by_coordinates
]

agent = create_agent(model=llm, tools=tools, system_prompt=YOUR_PROMPT)
```

© 2026 Enos Chou

天氣查詢 AI Agent ft. LangGraph

2. 驗證工具調用

建立叫用函數

```
def ask(something: str) -> str:
    '''ask AI Agent something
    something - asking
    return    - answer from AI Agent
    '''
    messages = [{"role": "user", "content": something}]
    result = agent.invoke({"messages": messages})
    ans = result['messages'][-1].content
    if type(ans) != str: ans = ans[0]['text']
    return ans
```

© 2026 Enos Chou

天氣查詢 AI Agent ft. LangGraph

2. 驗證工具調用

測試

```
ask('擎天崗天氣如何？')
```

© 2026 Enos Chou

天氣查詢 AI Agent ft. LangGraph

2. 驗證工具調用

觀察 LLM ReAct 行為

```
print(result)
```

© 2026 Enos Chou

天氣查詢 AI Agent ft. LangGraph

3. 驗證自行調整

加入新工具

```
@tool
def get_weather_by_location(location: str) -> str:
    '''輸入地點名稱，取得天氣資訊'''
    return wea.tostr(wea.grab(location))
```

© 2026 Enos Chou

天氣查詢 AI Agent ft. LangGraph

3. 驗證自行調整

建立 AI Agent

```
tools = [
    get_coordinates,
    get_weather_by_coordinates,
    get_weather_by_location
]

agent = create_agent(model=llm, tools=tools, system_prompt=YOUR_PROMPT)
```

© 2026 Enos Chou

天氣查詢 AI Agent ft. LangGraph

3. 驗證自行調整

測試

```
ask('擎天崗天氣如何？')
```

© 2026 Enos Chou

天氣查詢 AI Agent ft. LangGraph

3. 驗證自行調整

觀察

```
print(result)
```

© 2026 Enos Chou

天氣查詢 AI Agent ft. LangGraph

3. 驗證自行調整

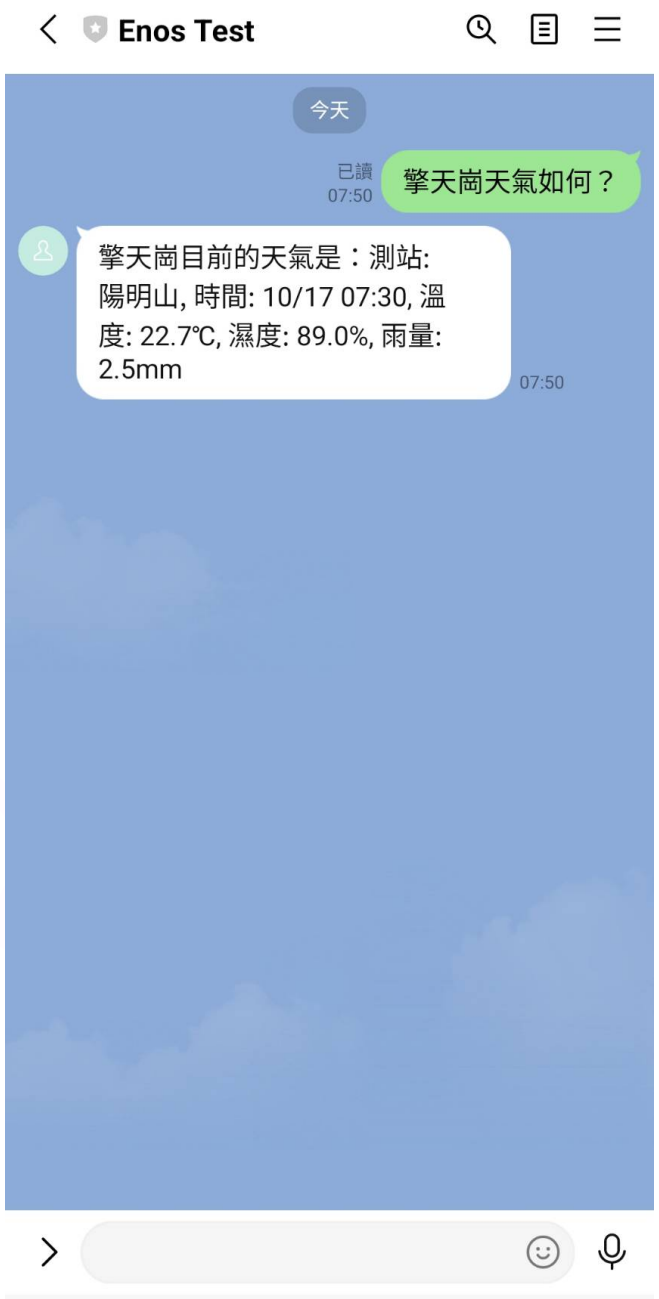
強化自行調整能力

① 調整 `prompt`，要求 `AI Agent` 遇到工具回覆不如預期時須嘗試其他工具而非直接回覆錯誤訊息
② 調整工具 `docstring`，提供必要指引
③ 調整模型，選擇推理能力較佳者
④ 調整 `thinking_budget`，提供較高的推理預算

© 2026 Enos Chou

Your Turn - 天氣代理人

完成能夠透過聊天查詢任意地點天氣的 LINE Bot (1 hr)



© 2026 Enos Chou

© 2026 Enos Chou

GCP AI 模型訓練 & 整合應用

AutoML

AutoML Edge

Teachable Machine

AutoML by Vertex AI

價格

https://cloud.google.com/vertex-ai/ >
定價 >
定價 >
總價費率

© 2026 Enos Chou

AutoML

雲端建模雲端服務

a. 上傳資料
b. 訓練模型
c. 部署模型
d. 叫用模型

© 2026 Enos Chou

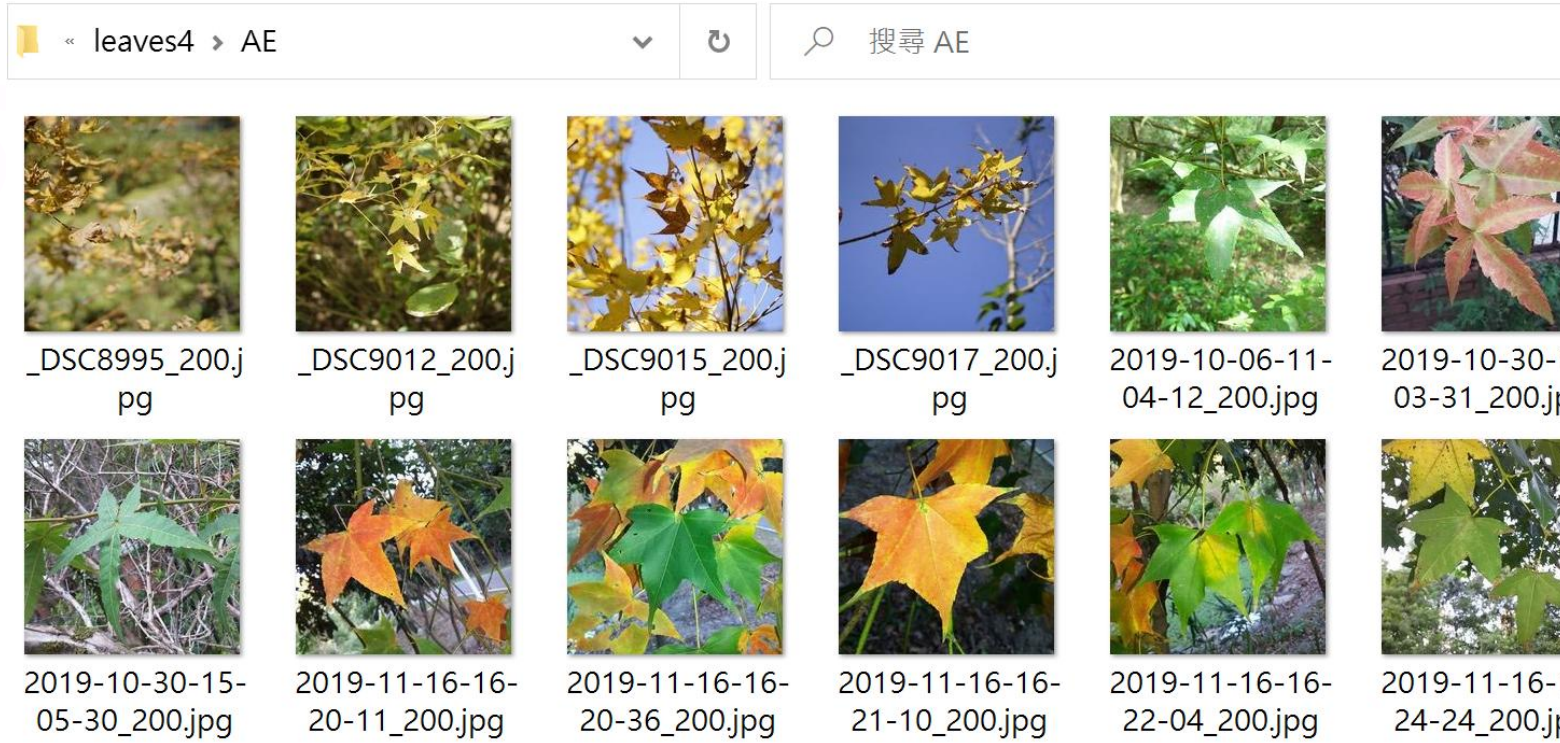
Modeling ft. AutoML

雲端建模雲端服務

a. 上傳資料

① dataset leaves4

unzip leaves4.zip



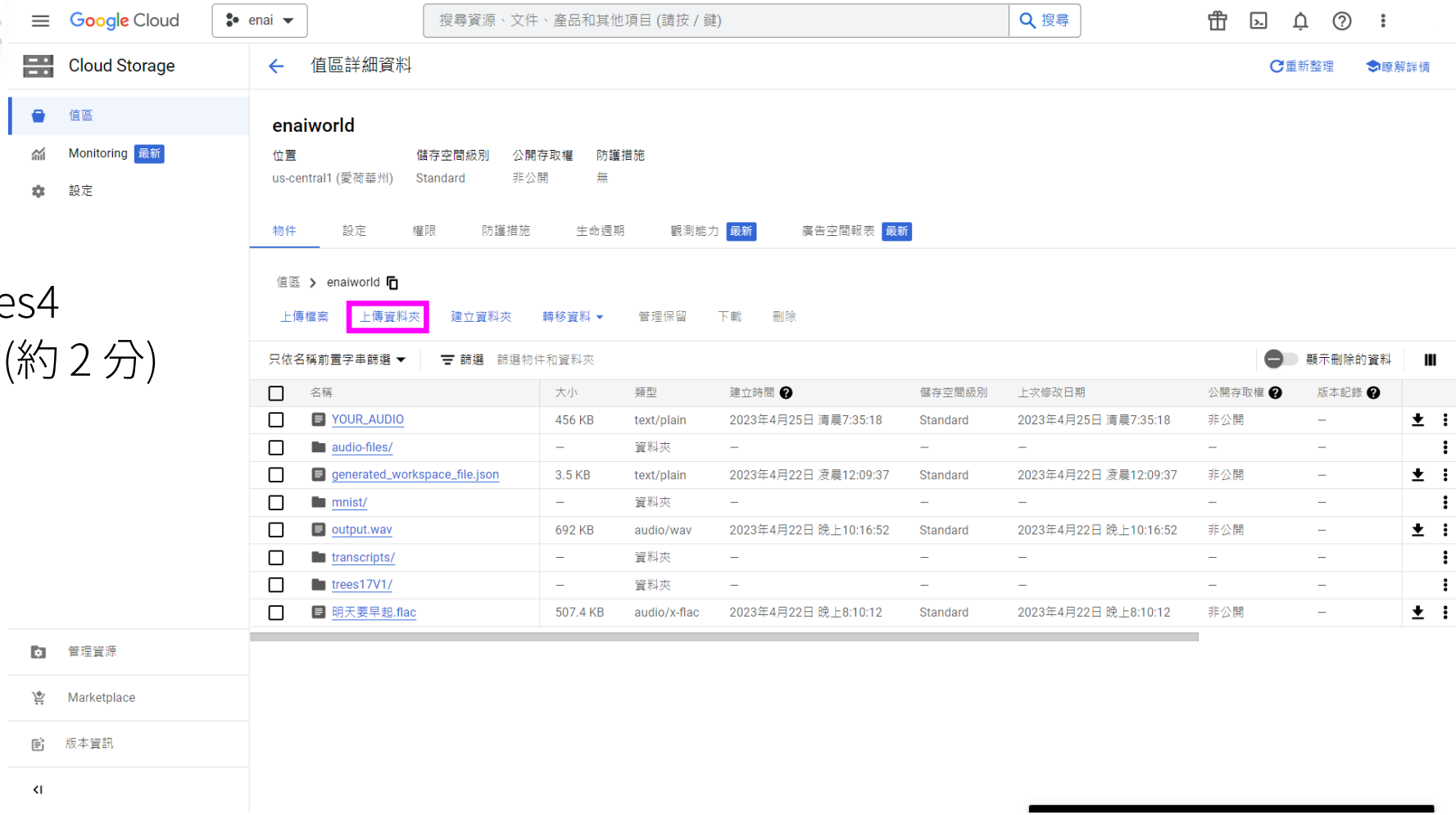
© 2026 Enos Chou

Modeling ft. AutoML

雲端建模雲端服務

a. 上傳資料

① dataset leaves4
② local to GCS (約 2 分)



© 2026 Enos Chou

Modeling ft. AutoML

雲端建模雲端服務

a. 上傳資料

① dataset leaves4
② local to GCS (約 2 分)

© 2026 Enos Chou

Modeling ft. AutoML

雲端建模雲端服務

a. 上傳資料

① dataset leaves4
② local to GCS (約 2 分)
③ annotation

```
training, gs://enaiworld/leaves4/FF/IMG_1842_200.JPG, FF
test, gs://enaiworld/leaves4/FF/IMG_4946_200.JPG, FF
validation, gs://enaiworld/leaves4/FF/IMG_5159_200.JPG, FF
```

(選用) 資料用途　　資料位置　　(選用) Label

© 2026 Enos Chou

Modeling ft. AutoML

雲端建模雲端服務

a. 上傳資料

① dataset leaves4
② local to GCS (約 2 分)
③ annotation

```
python anno.py -p YOUR_DATAPATH -b YOUR_BUCKETPATH -o YOUR_OUTPUTFILE
python anno.py -p leaves4 -b gs://enaiworld/leaves4 -o anno.csv
```


© 2026 Enos Chou

Modeling ft. AutoML

雲端建模雲端服務

a. 上傳資料

① dataset leaves4
② local to GCS
③ annotation
④ GCS to Vertex AI

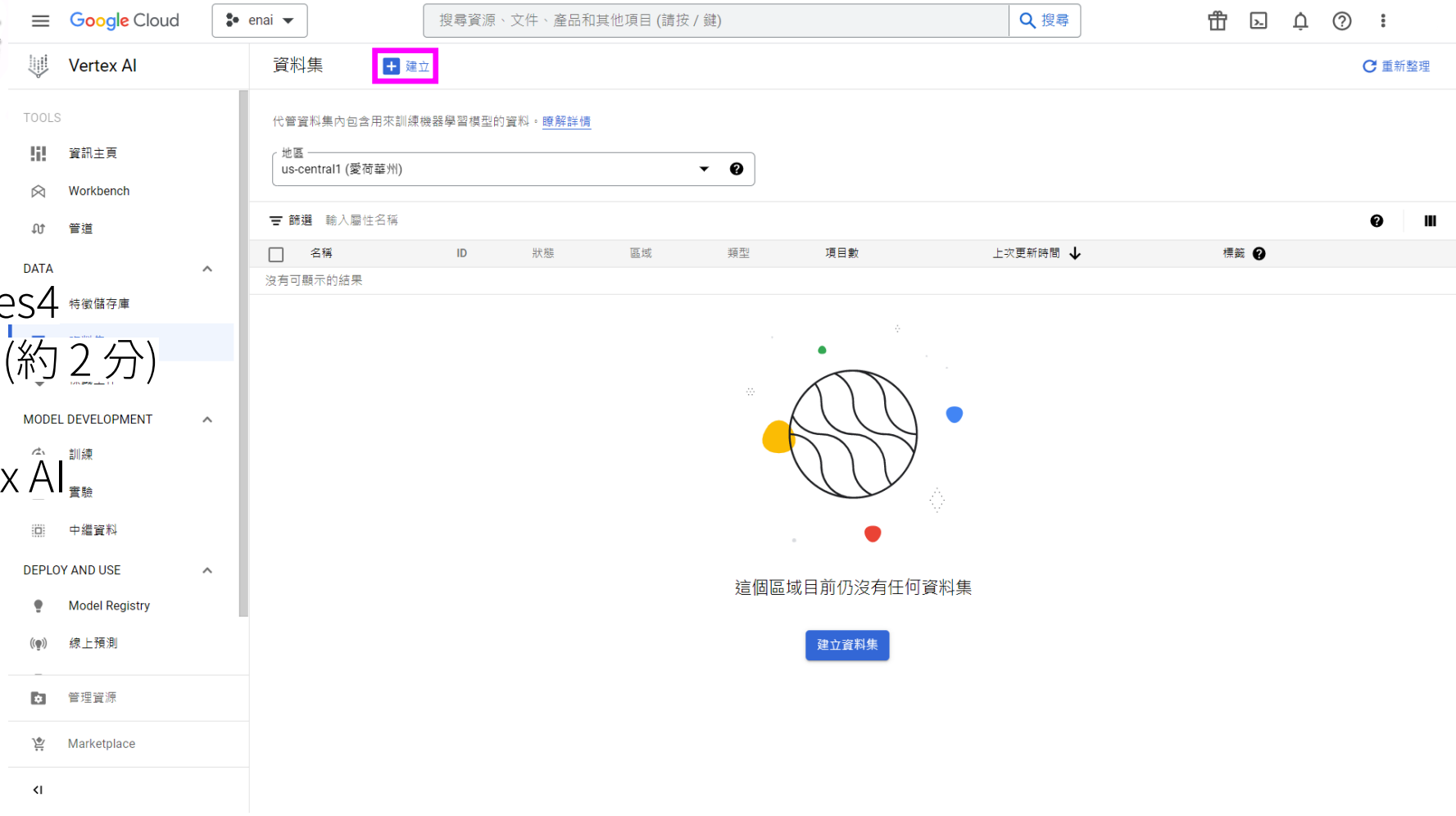
© 2026 Enos Chou

Modeling ft. AutoML

雲端建模雲端服務

a. 上傳資料

① dataset leaves4
② local to GCS (約 2 分)
③ annotation
④ GCS to Vertex AI



© 2026 Enos Chou

Modeling ft. AutoML

雲端建模雲端服務

a. 上傳資料

① dataset leaves4
② local to GCS (約 2 分)
③ annotation
④ GCS to Vertex AI

Modeling ft. AutoML

雲端建模雲端服務

a. 上傳資料

① dataset leaves4
② local to GCS (約 2 分)
③ annotation
④ GCS to Vertex AI

Modeling ft. AutoML

雲端建模雲端服務

a. 上傳資料

① dataset leaves4
② local to GCS (約 2 分)
③ annotation
④ GCS to Vertex AI

© 2026 Enos Chou

Modeling ft. AutoML

雲端建模雲端服務

a. 上傳資料

① dataset leaves4
② local to GCS (約 2 分)
③ annotation
④ GCS to Vertex AI

© 2026 Enos Chou

Modeling ft. AutoML

雲端建模雲端服務

a. 上傳資料

① dataset leaves4
② local to GCS (約 2 分)
③ annotation
④ GCS to Vertex AI

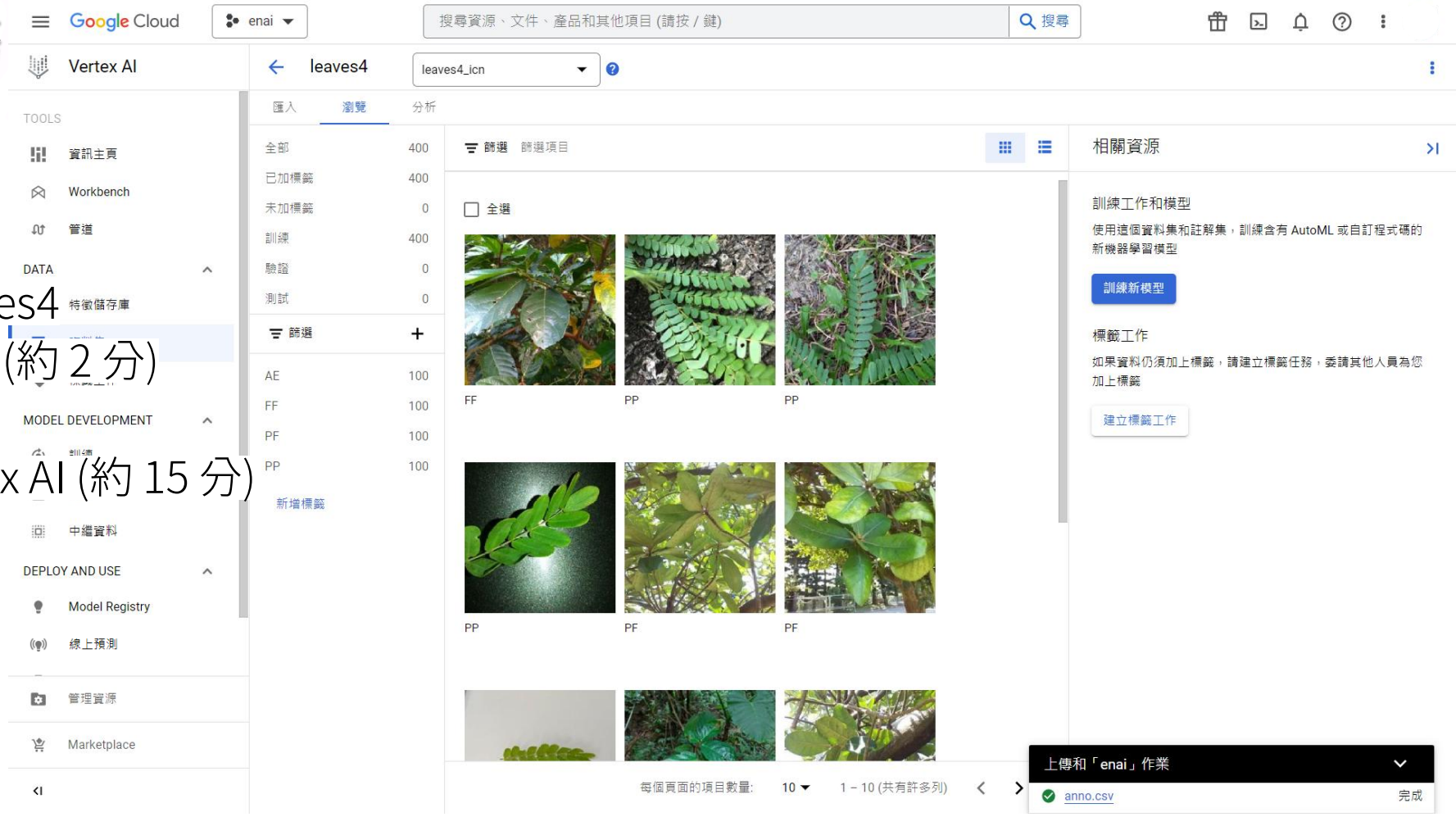
© 2026 Enos Chou

Modeling ft. AutoML

雲端建模雲端服務

a. 上傳資料

① dataset leaves4
② local to GCS (約 2 分)
③ annotation
④ GCS to Vertex AI (約 15 分)

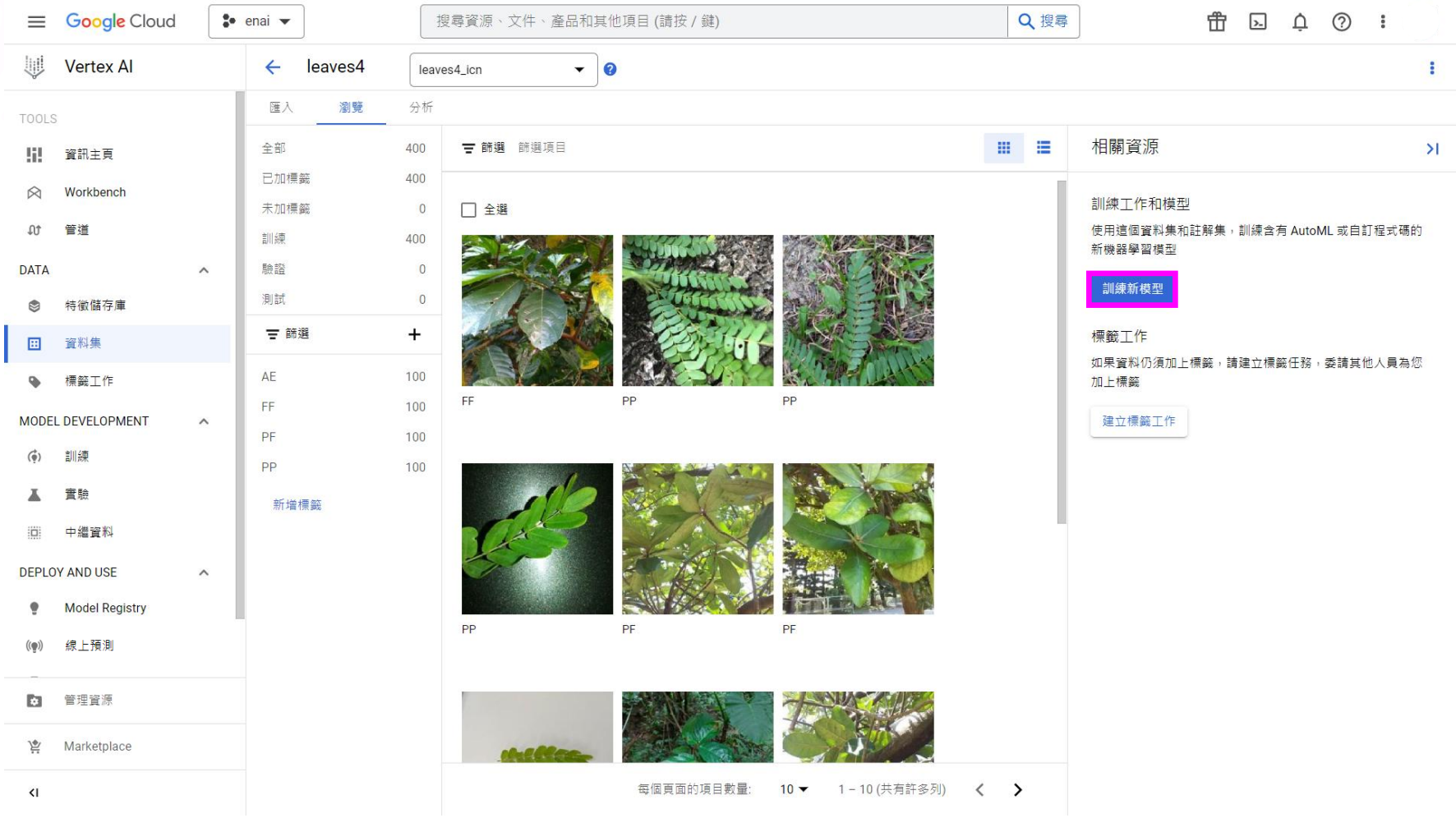


© 2026 Enos Chou

Modeling ft. AutoML

雲端建模雲端服務

b. 訓練模型



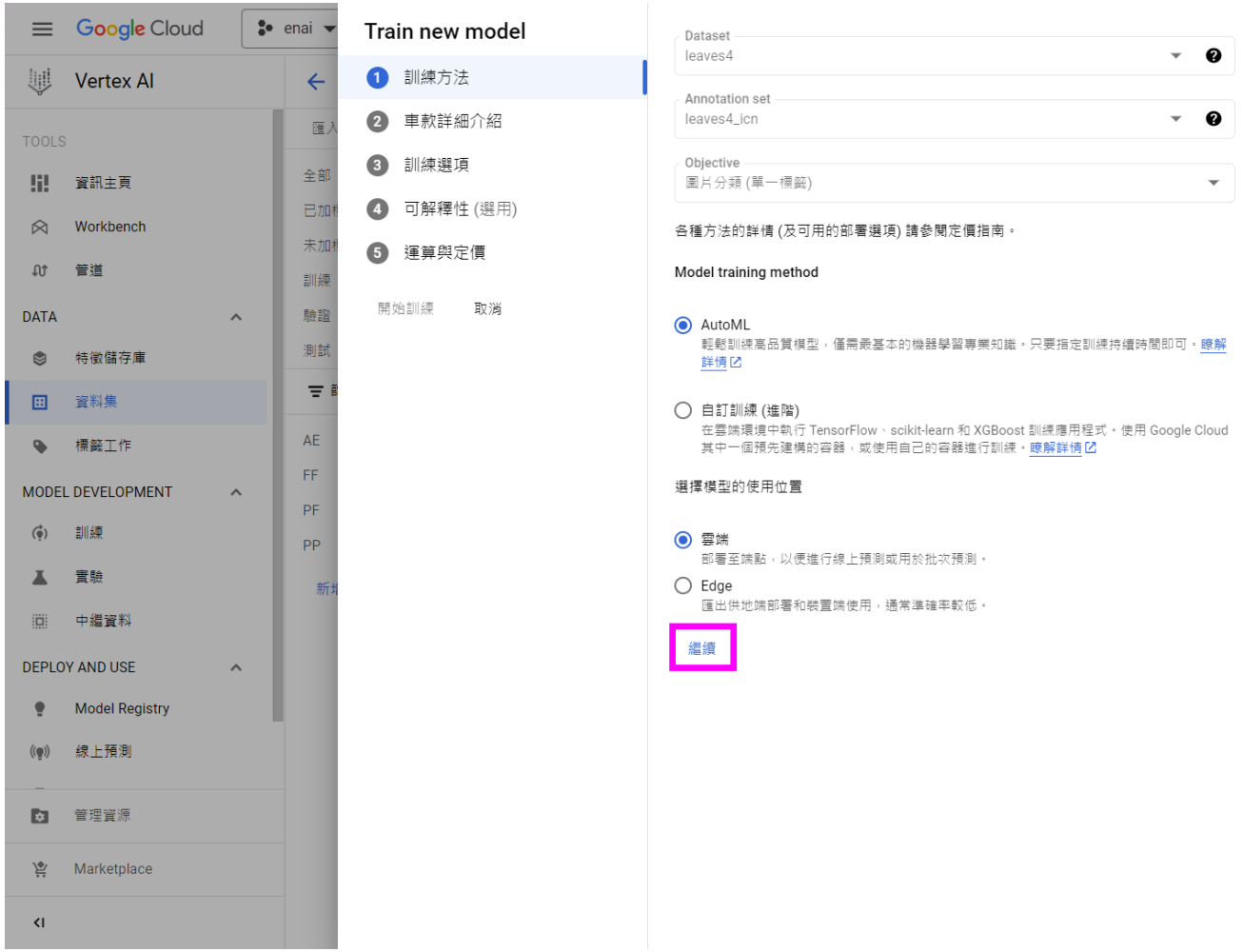
© 2026 Enos Chou

Modeling ft. AutoML

雲端建模雲端服務

b. 訓練模型

① 訓練方法



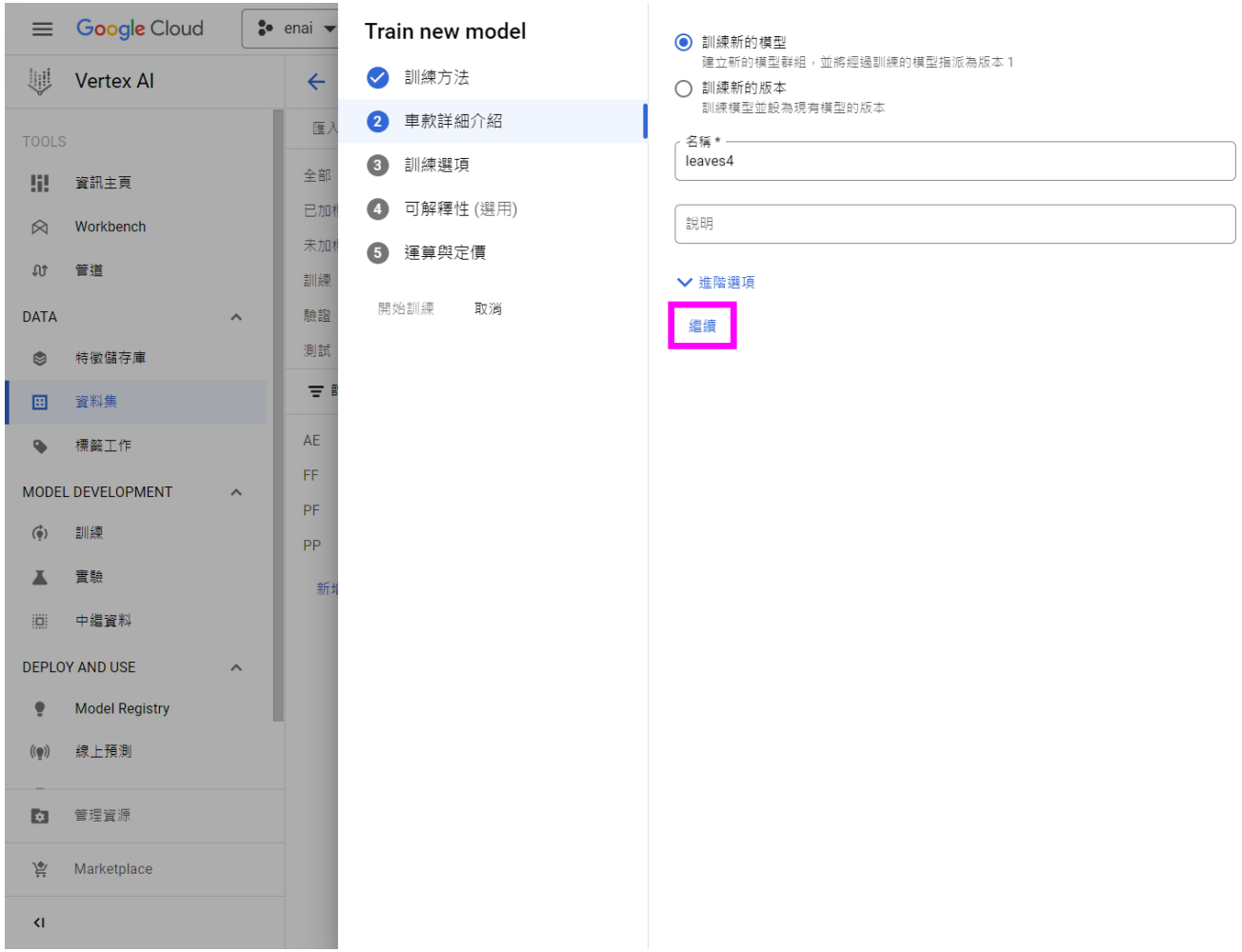
© 2026 Enos Chou

Modeling ft. AutoML

雲端建模雲端服務

b. 訓練模型

① 訓練方法
② 名稱



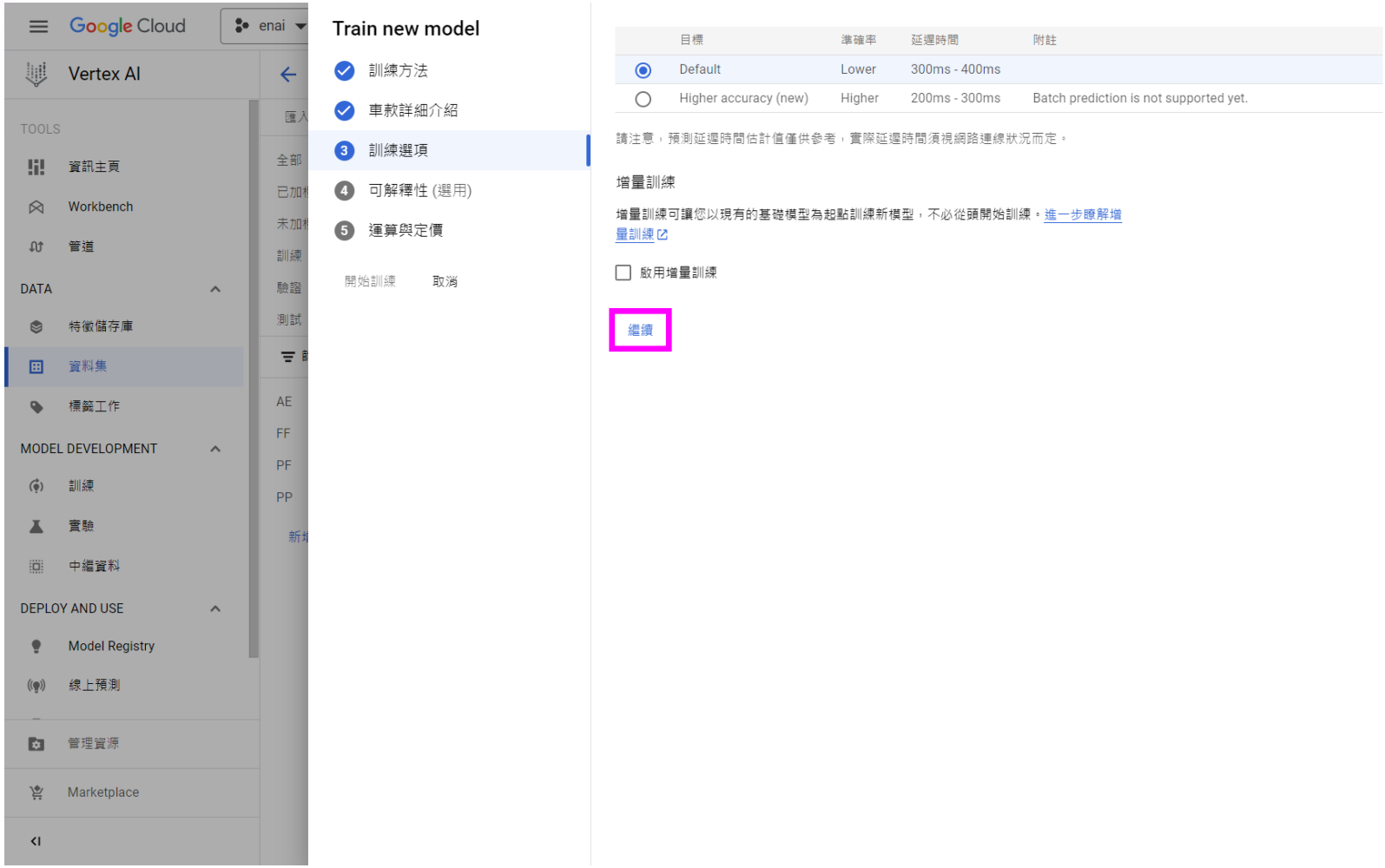
© 2026 Enos Chou

Modeling ft. AutoML

雲端建模雲端服務

b. 訓練模型

① 訓練方法
② 名稱
③ 精確度



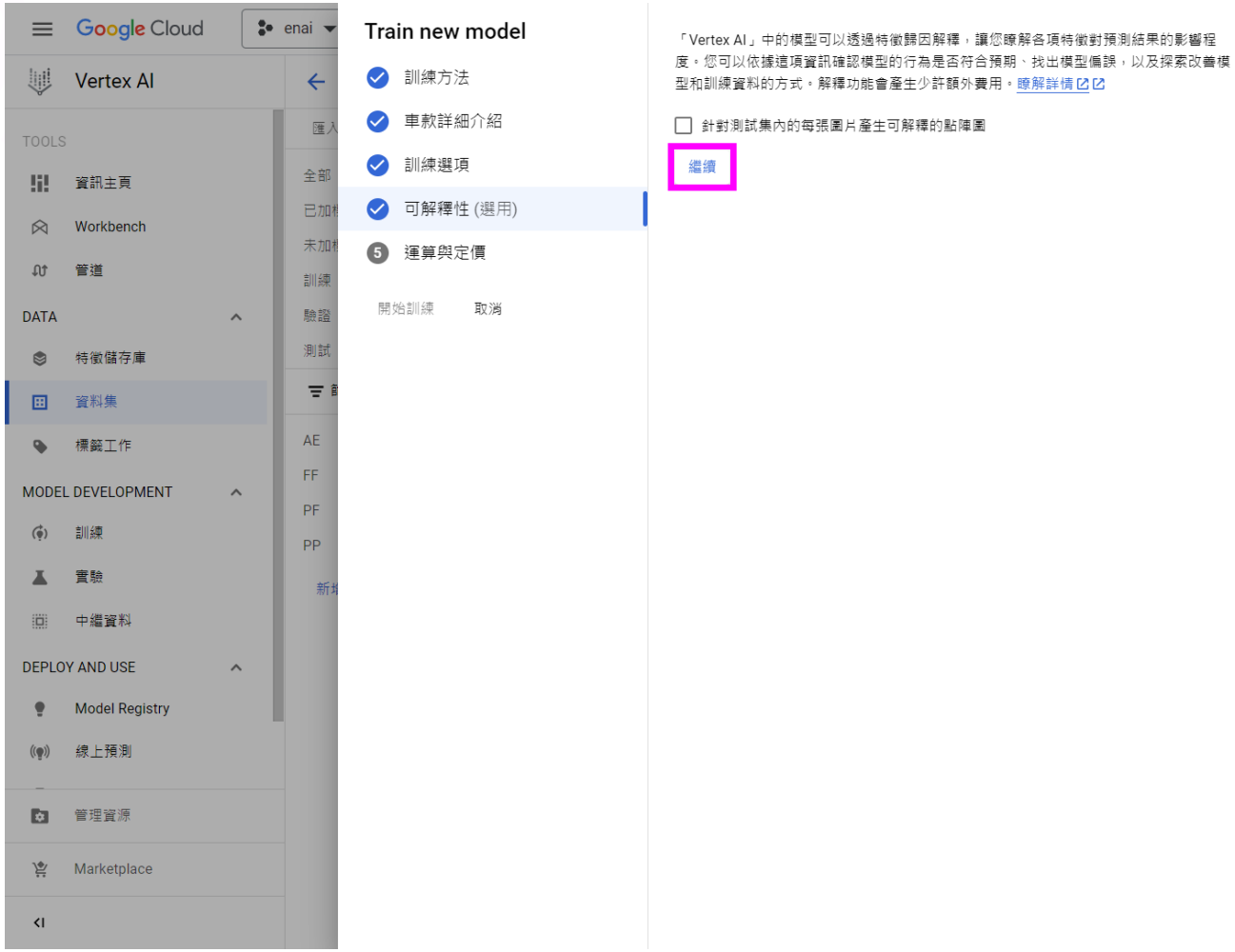
© 2026 Enos Chou

Modeling ft. AutoML

雲端建模雲端服務

b. 訓練模型

① 訓練方法
② 名稱
③ 精確度
④ 可解釋性



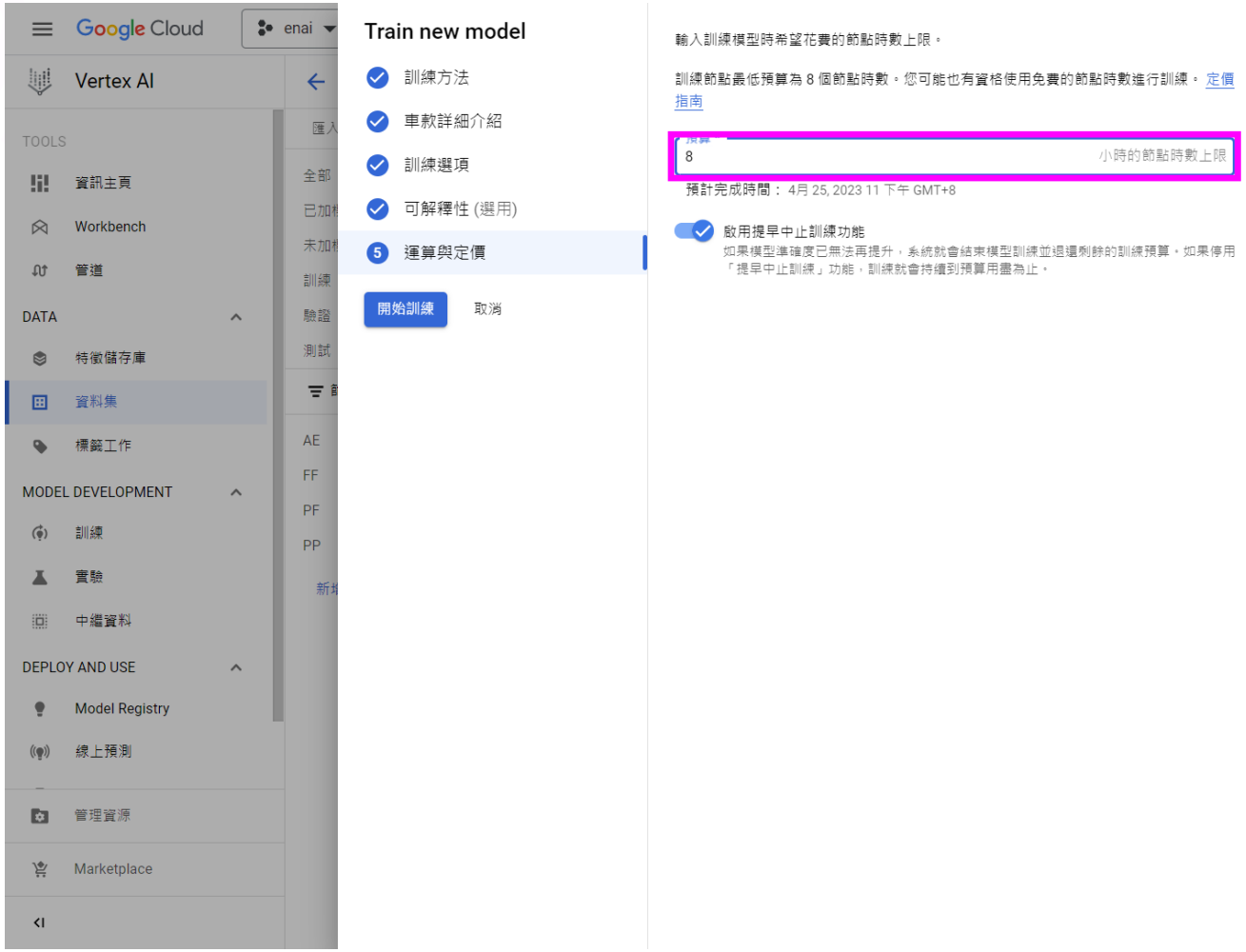
© 2026 Enos Chou

Modeling ft. AutoML

雲端建模雲端服務

b. 訓練模型

① 訓練方法
② 名稱
③ 精確度
④ 可解釋性
⑤ 訓練預算



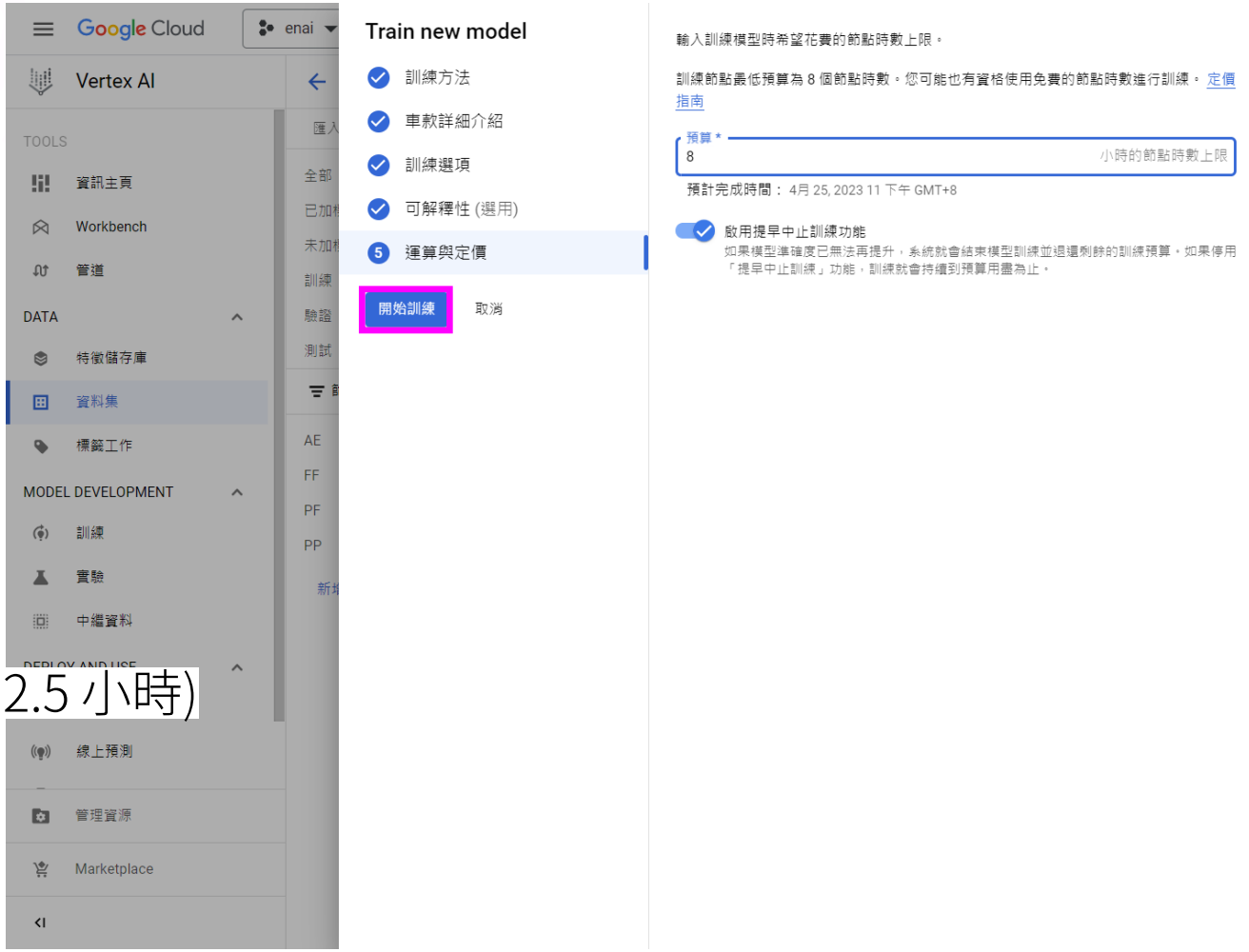
© 2026 Enos Chou

Modeling ft. AutoML

雲端建模雲端服務

b. 訓練模型

① 訓練方法
② 名稱
③ 精確度
④ 可解釋性
⑤ 訓練預算
⑥ 開始訓練 (約 2.5 小時)



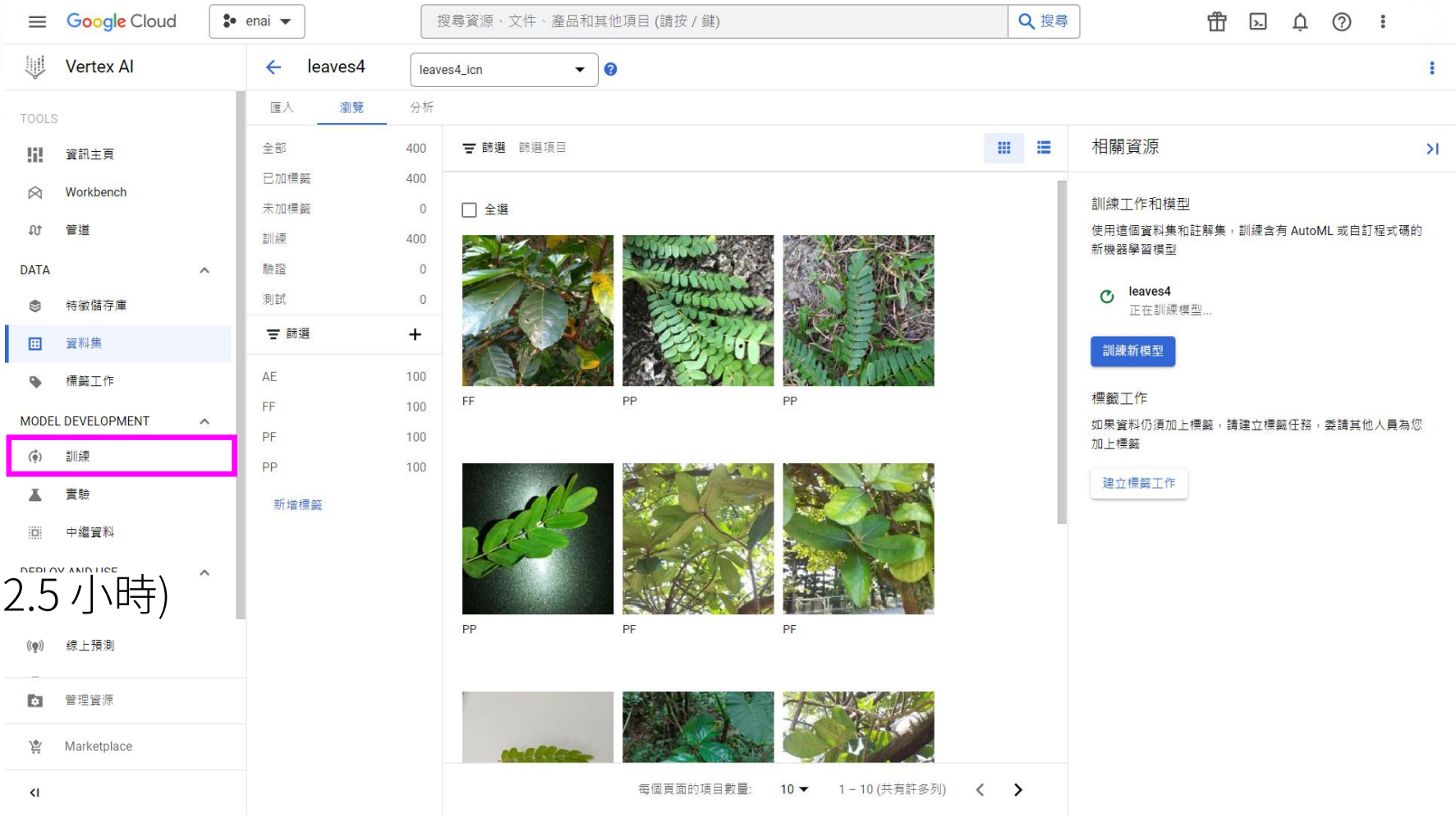
© 2026 Enos Chou

Modeling ft. AutoML

雲端建模雲端服務

b. 訓練模型

① 訓練方法
② 名稱
③ 精確度
④ 可解釋性
⑤ 訓練預算
⑥ 開始訓練 (約 2.5 小時)



© 2026 Enos Chou

Modeling ft. AutoML

雲端建模雲端服務

b. 訓練模型

① 訓練方法
② 名稱
③ 精確度
④ 可解釋性
⑤ 訓練預算
⑥ 開始訓練 (約 2.5 小時)

© 2026 Enos Chou

Modeling ft. AutoML

雲端建模雲端服務

b. 訓練模型

① 訓練方法
② 名稱
③ 精確度
④ 可解釋性
⑤ 訓練預算
⑥ 開始訓練 (約 2.5 小時)
⑦ 評價模型

© 2026 Enos Chou

Modeling ft. AutoML

雲端建模雲端服務

b. 訓練模型

① 訓練方法
② 名稱
③ 精確度
④ 可解釋性
⑤ 訓練預算
⑥ 開始訓練 (約 2.5 小時)
⑦ 評價模型

Deploying ft. AutoML

雲端建模雲端服務

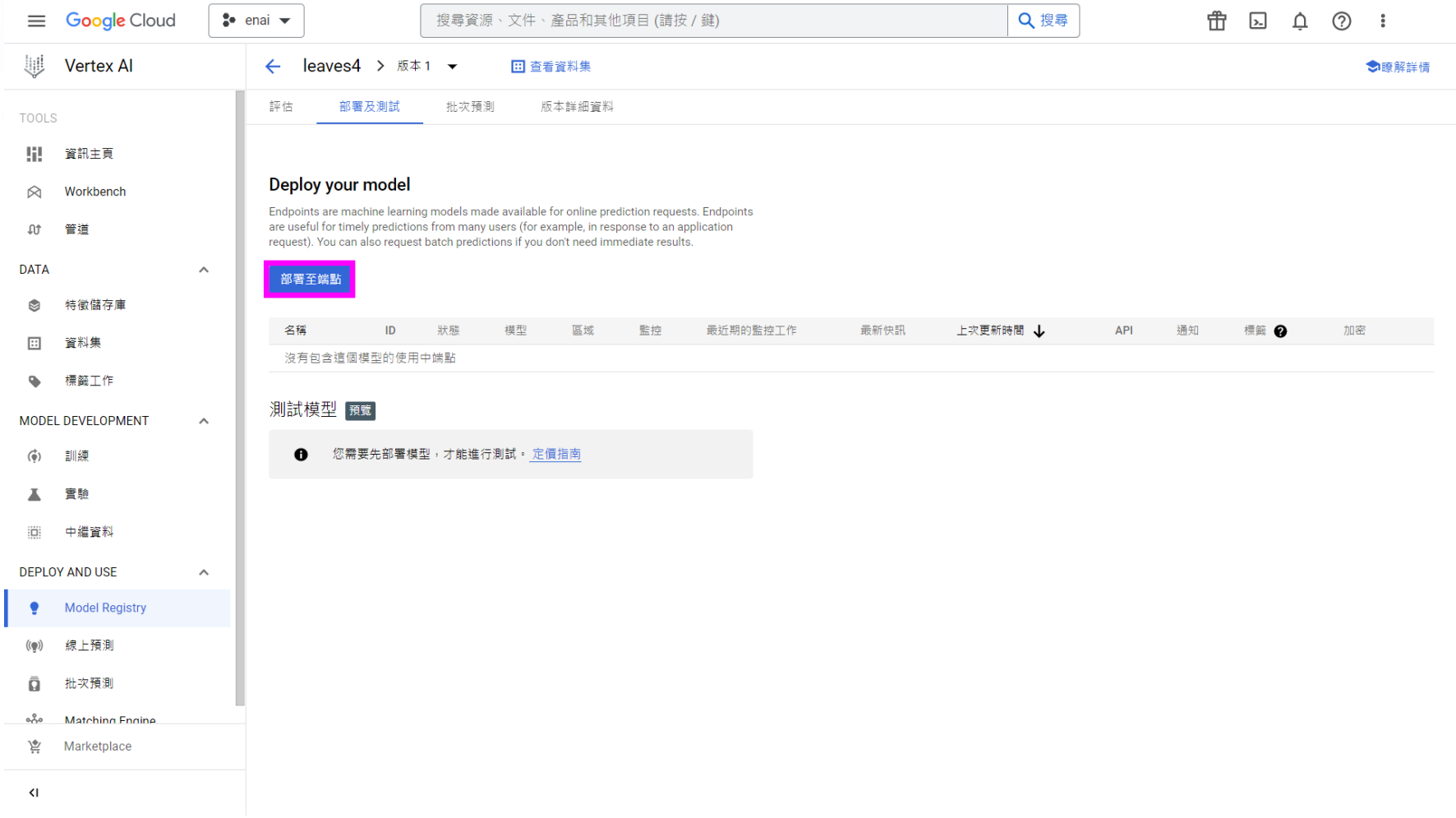
c. 部署模型

Deploying ft. AutoML

雲端建模雲端服務

c. 部署模型

① 部署至端點

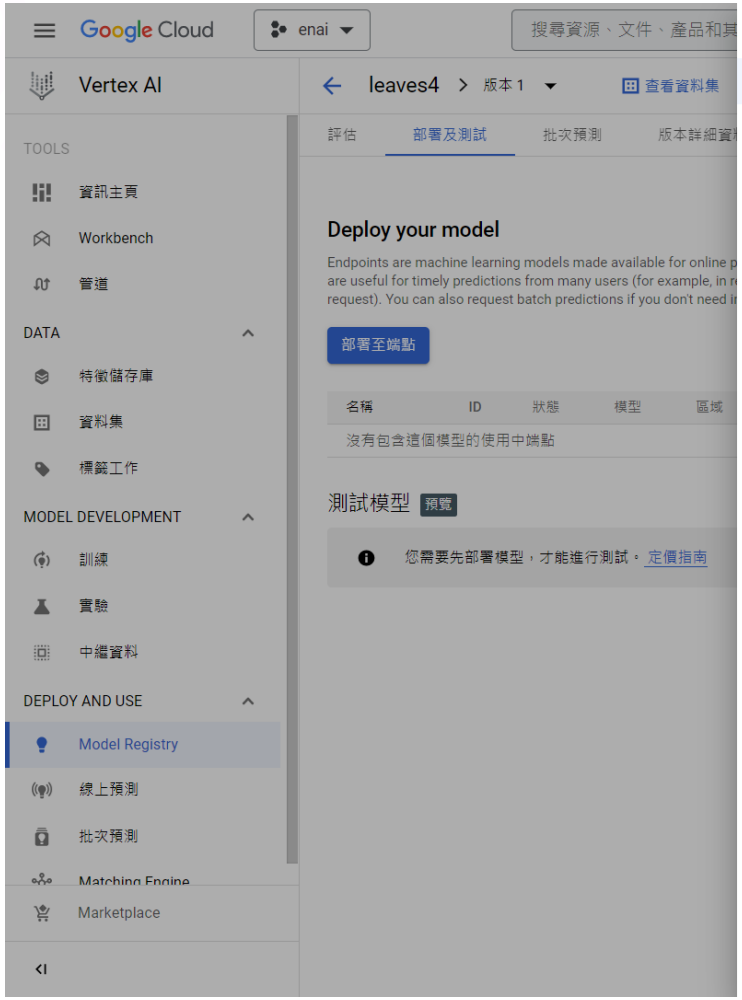


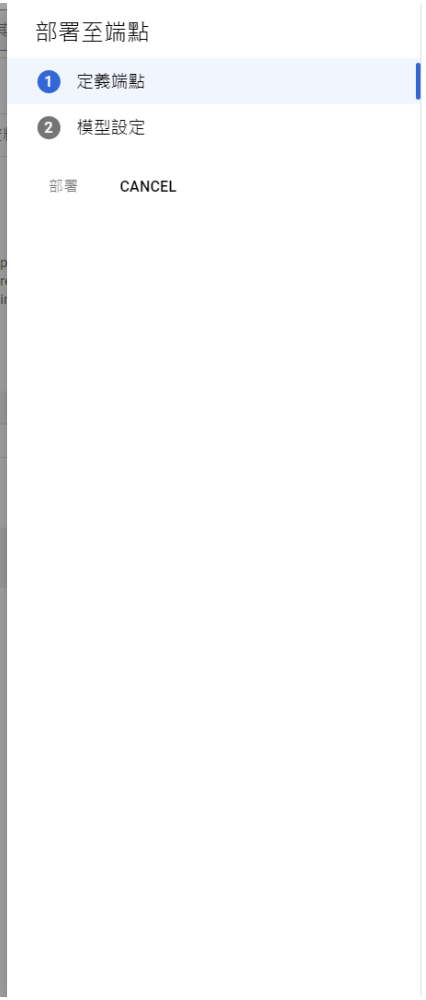
Deploying ft. AutoML

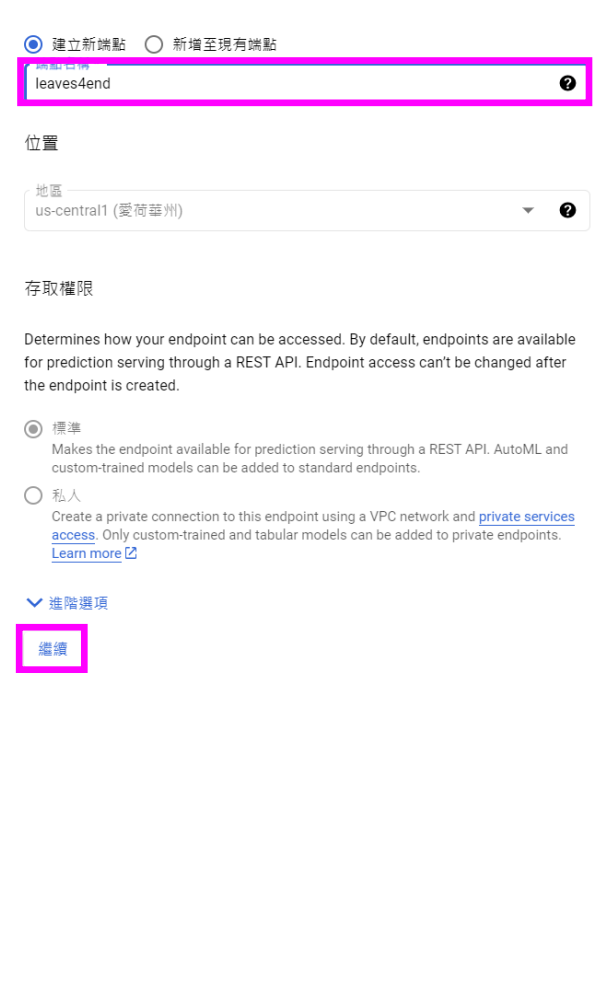
雲端建模雲端服務

c. 部署模型

① 部署至端點
② 端點名稱







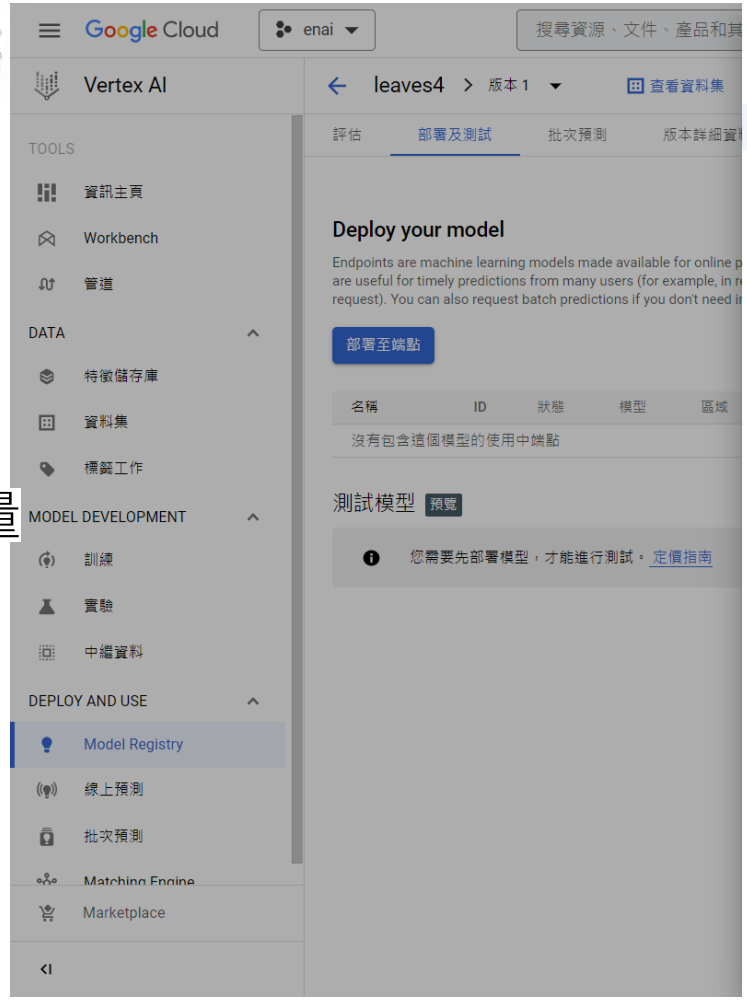
© 2026 Enos Chou

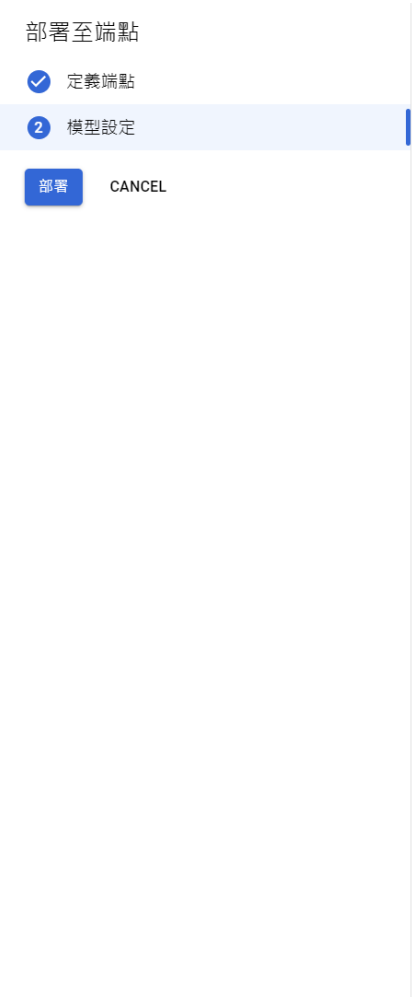
Deploying ft. AutoML

雲端建模雲端服務

c. 部署模型

① 部署至端點
② 端點名稱
③ 運算節點數量





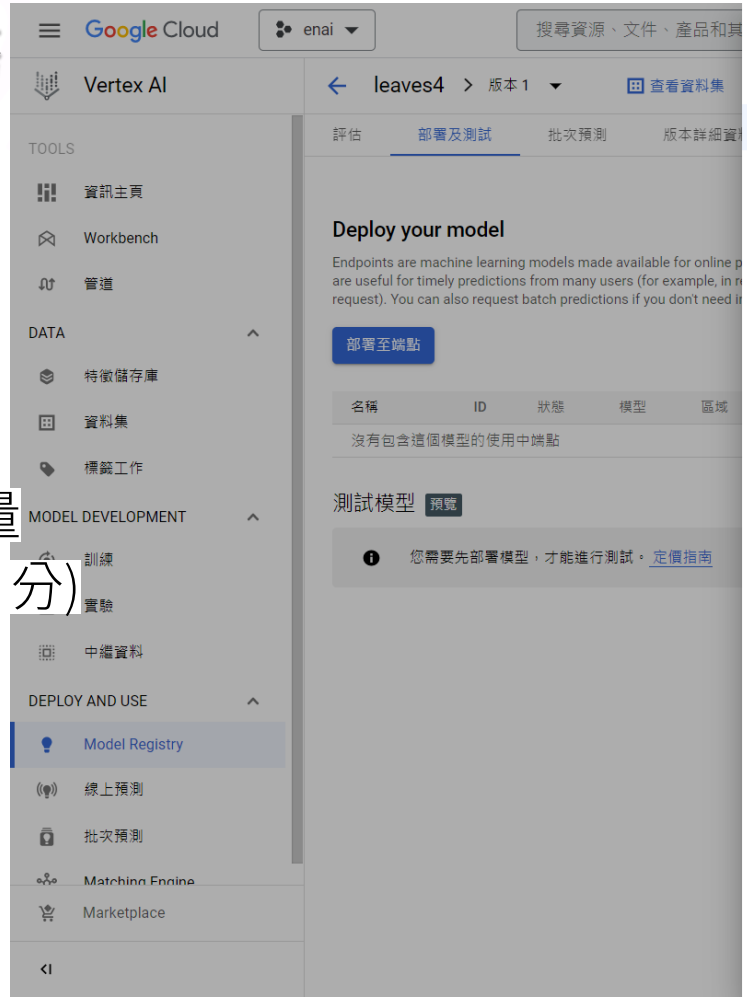
© 2026 Enos Chou

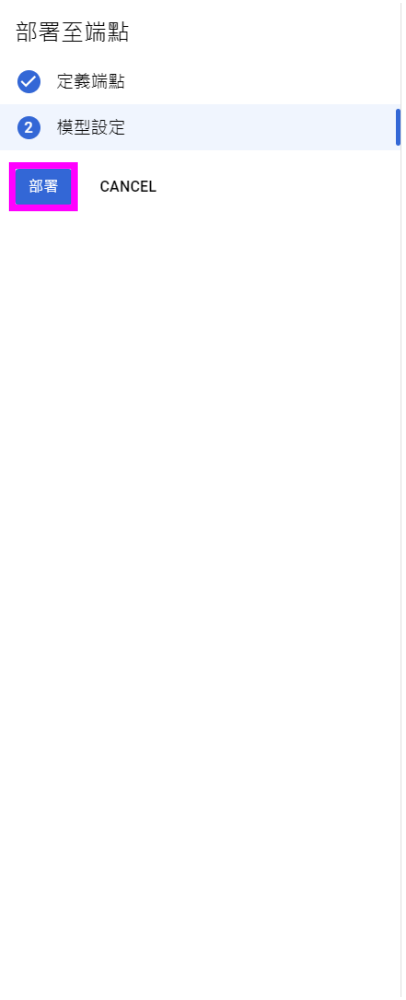
Deploying ft. AutoML

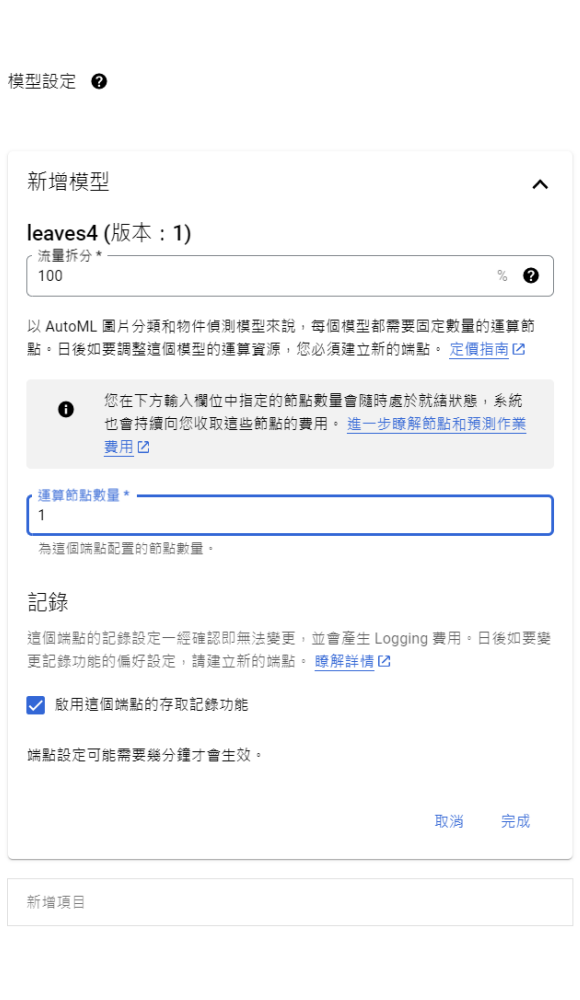
雲端建模雲端服務

c. 部署模型

① 部署至端點
② 端點名稱
③ 運算節點數量
④ 部署 (約 10.5 分)







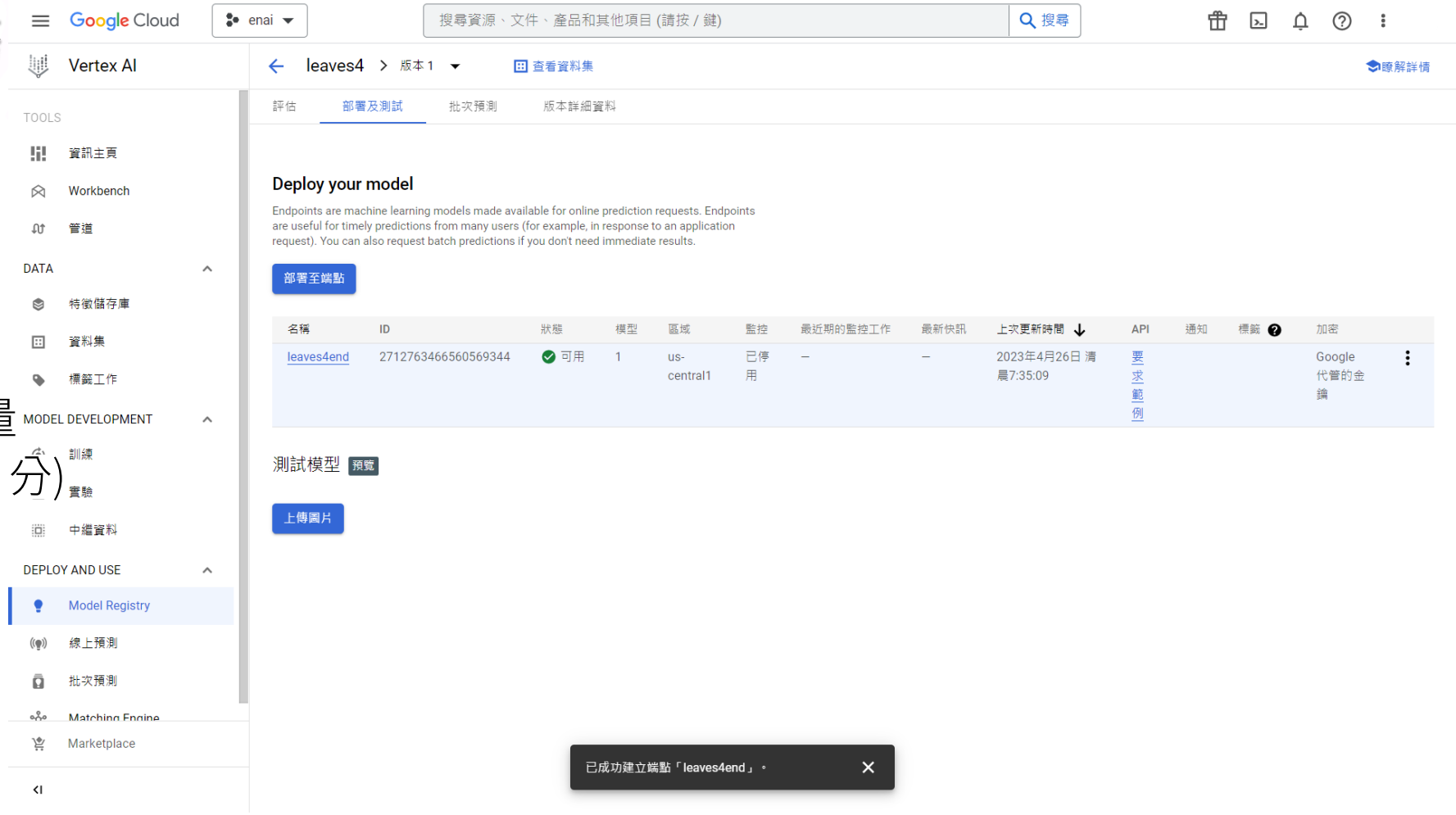
© 2026 Enos Chou

Deploying ft. AutoML

雲端建模雲端服務

c. 部署模型

① 部署至端點
② 端點名稱
③ 運算節點數量
④ 部署 (約 10.5 分)

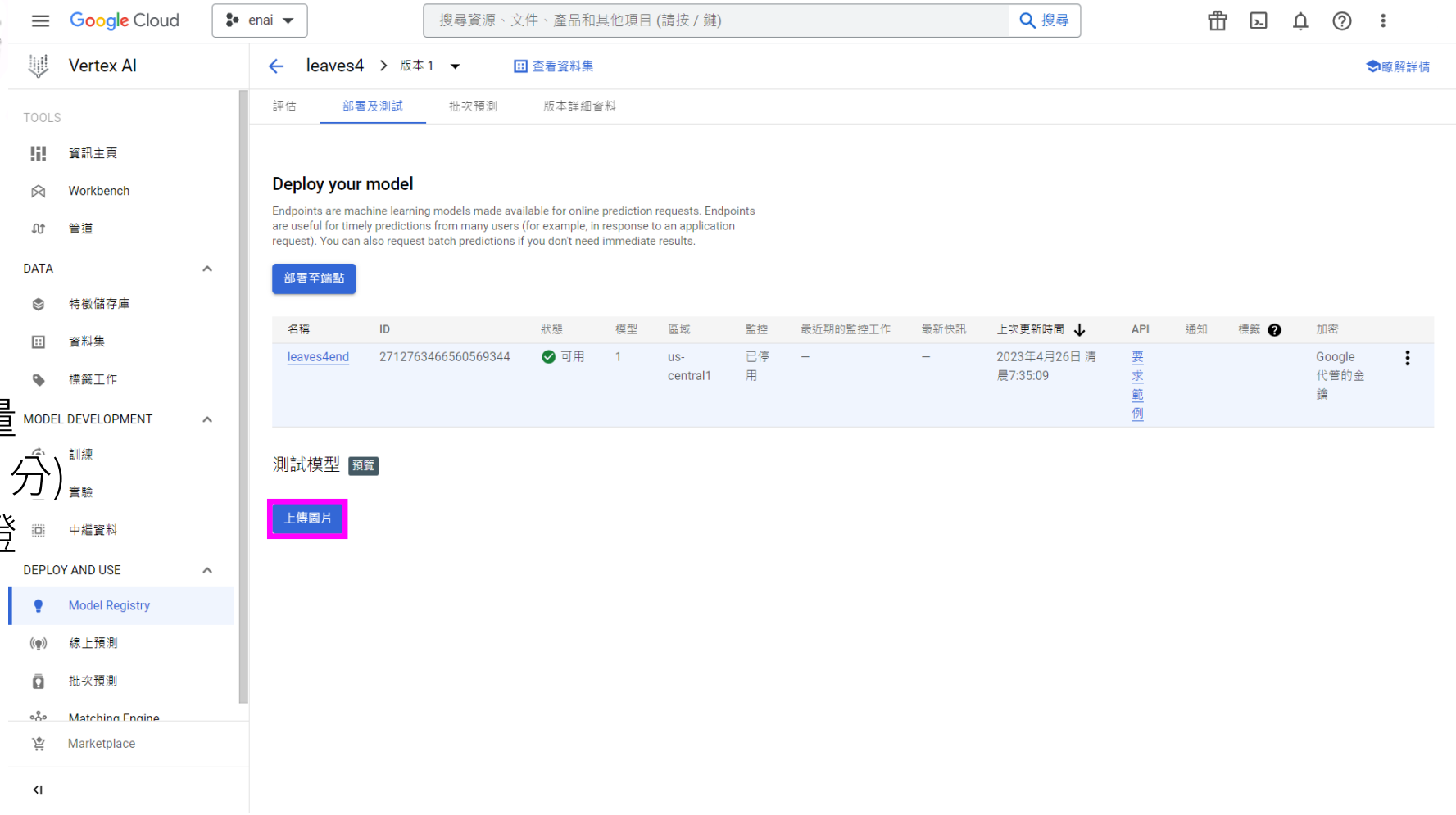


Deploying ft. AutoML

雲端建模雲端服務

c. 部署模型

① 部署至端點
② 端點名稱
③ 運算節點數量
④ 部署 (約 10.5 分)
⑤ 上傳圖片驗證



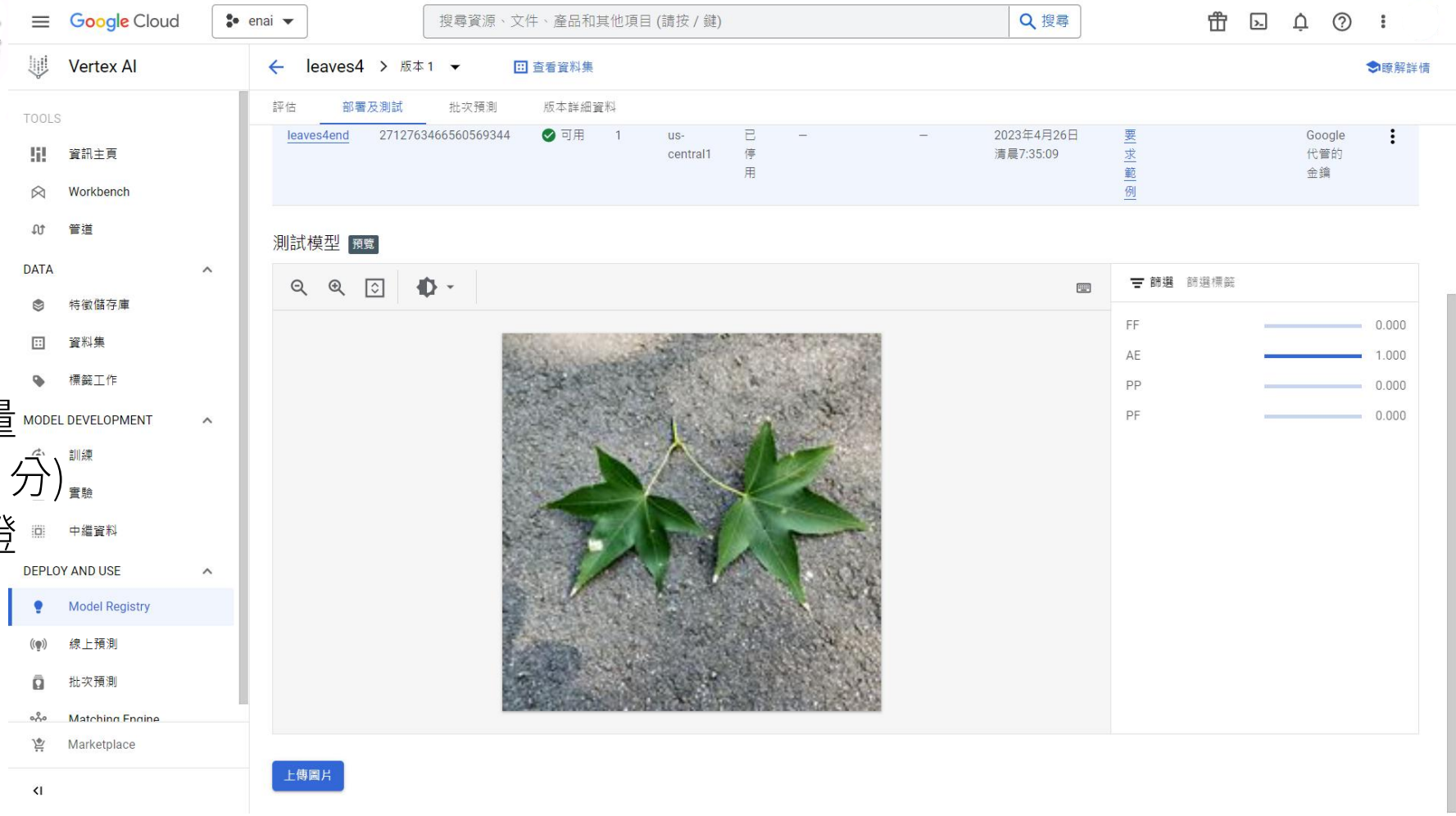
Deploying ft. AutoML

雲端建模雲端服務

c. 部署模型

① 部署至端點
② 端點名稱
③ 運算節點數量
④ 部署 (約 10.5 分)
⑤ 上傳圖片驗證

ae.jpg 青楓



© 2026 Enos Chou

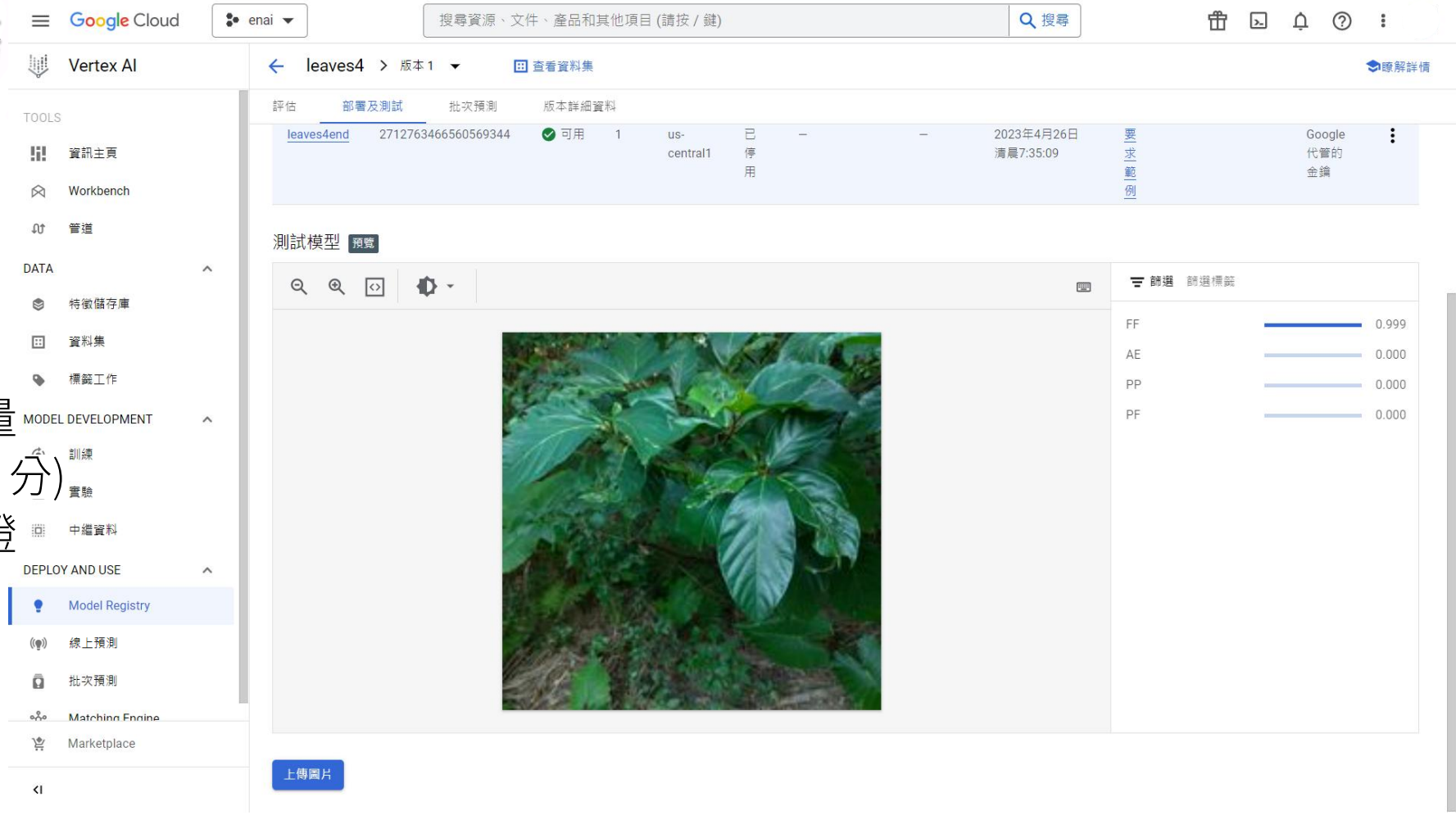
Deploying ft. AutoML

雲端建模雲端服務

c. 部署模型

① 部署至端點
② 端點名稱
③ 運算節點數量
④ 部署 (約 10.5 分)
⑤ 上傳圖片驗證

ff.jpg 水同木



© 2026 Enos Chou

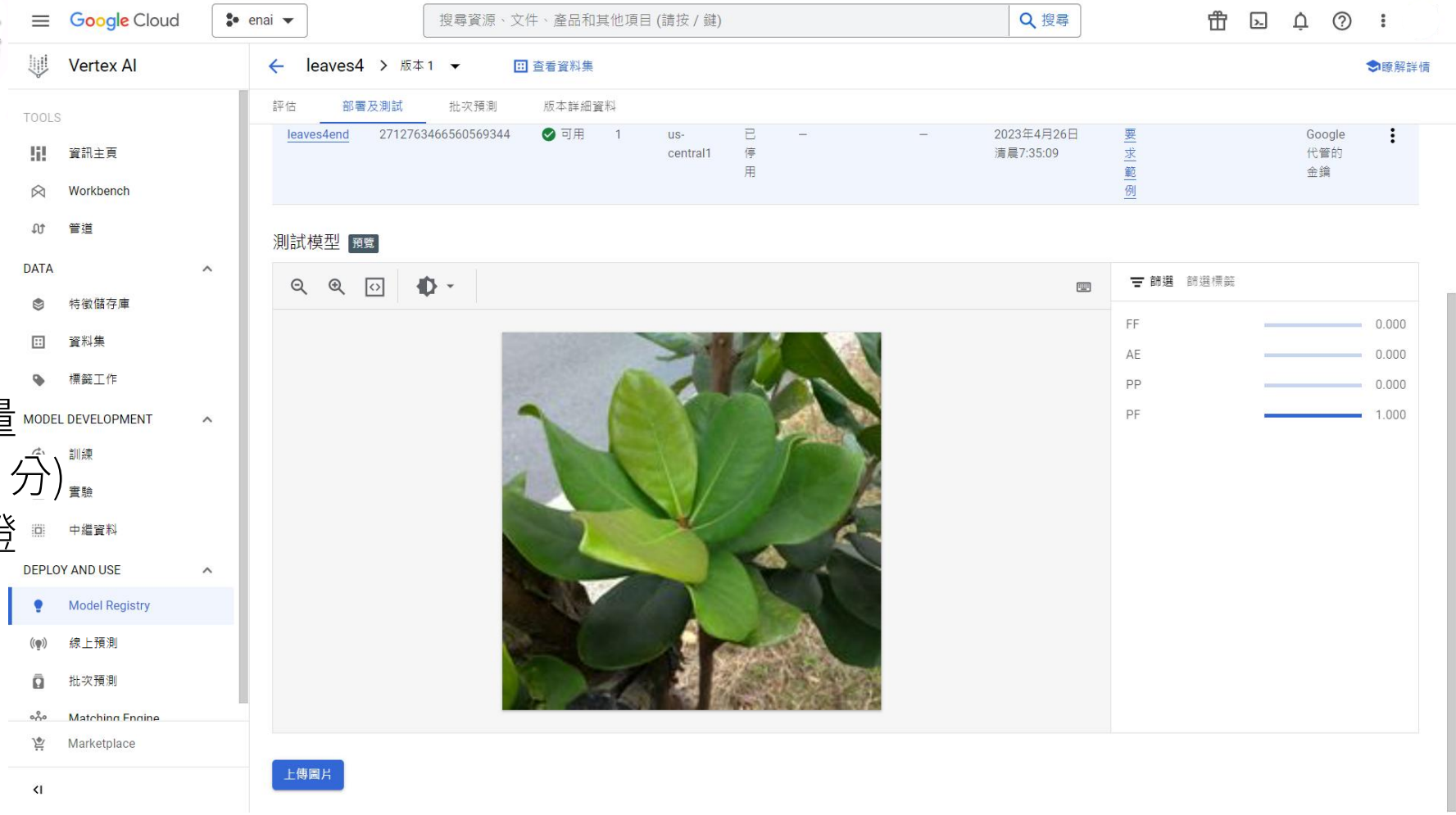
Deploying ft. AutoML

雲端建模雲端服務

c. 部署模型

① 部署至端點
② 端點名稱
③ 運算節點數量
④ 部署 (約 10.5 分)
⑤ 上傳圖片驗證

pf.jpg 大葉山欖



© 2026 Enos Chou

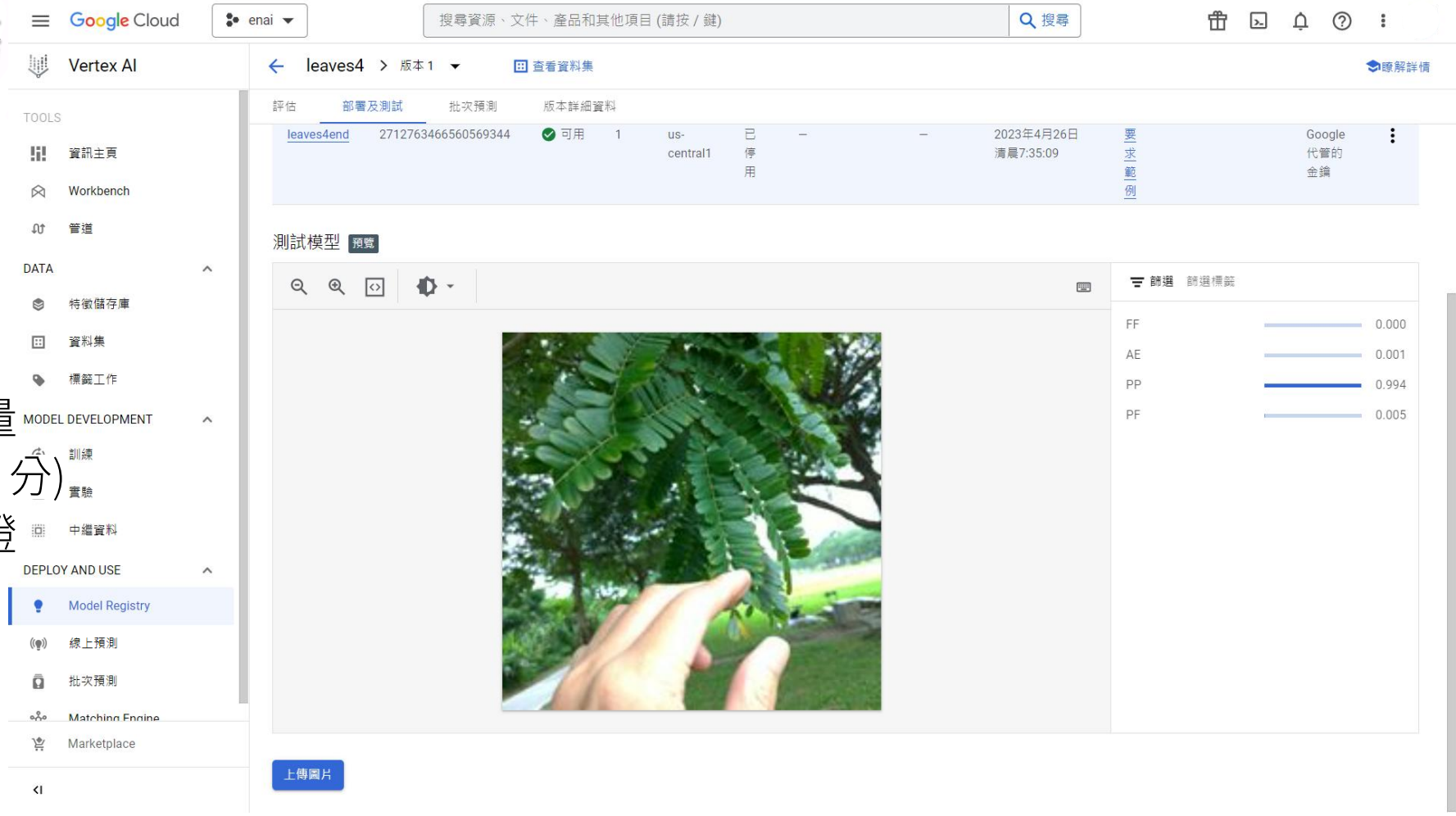
Deploying ft. AutoML

雲端建模雲端服務

c. 部署模型

① 部署至端點
② 端點名稱
③ 運算節點數量
④ 部署 (約 10.5 分)
⑤ 上傳圖片驗證

pp.jpg 盾柱木



© 2026 Enos Chou

Application ft. AutoML

雲端建模雲端服務

d. 叫用模型

① 下載官方叫用函式

https://github.com/googleapis/ >
Find a repository: python-aiplatform >
python-aiplatform >
samples >
snippets >
prediction_service >
predict_image_classification_sample.py

© 2026 Enos Chou

Application ft. AutoML

雲端建模雲端服務

d. 叫用模型

① 下載官方叫用函式
② 叫用雲端模型

Core

```
import os
import predict_image_classification_sample as p

os.environ['GOOGLE_APPLICATION_CREDENTIALS'] = YOUR_SERVICE
```


© 2026 Enos Chou

Application ft. AutoML

雲端建模雲端服務

d. 叫用模型

① 下載官方叫用函式
② 叫用雲端模型

Core

```
p.predict_image_classification_sample(
    YOUT_PROJECT, YOUR_ENDPOINTID, YOUR_IMAGE, YOUR_REGION, YOUR_APIENDPOINT
)
```


© 2026 Enos Chou

Application ft. AutoML

雲端建模雲端服務

d. 叫用模型

① 下載官方叫用函式
② 叫用雲端模型

```
python treeserving_infer.py -a AutoML -p YOUR_PROJECTID -s YOUR_SERVICE -e YOUR_ENDPOINTID -R YOUR_REGION YOUR_IMAGES
python treeserving_infer.py -a AutoML -p YOUR_PROJECTID -s YOUR_SERVICE -e YOUR_ENDPOINTID -R YOUR_REGION ae.jpg
```


© 2026 Enos Chou

AutoML Edge

雲端建模地端應用

a. 上傳資料
b. 訓練模型
c. 匯出模型
d. 下載模型
e. 使用模型

© 2026 Enos Chou

Modeling ft. AutoML Edge

雲端建模地端應用

a. 上傳資料

略

© 2026 Enos Chou

Modeling ft. AutoML Edge

雲端建模地端應用

b. 訓練模型

① 進入 Vertex AI > 訓練

Modeling ft. AutoML Edge

雲端建模地端應用

b. 訓練模型

① 進入 Vertex AI > 訓練

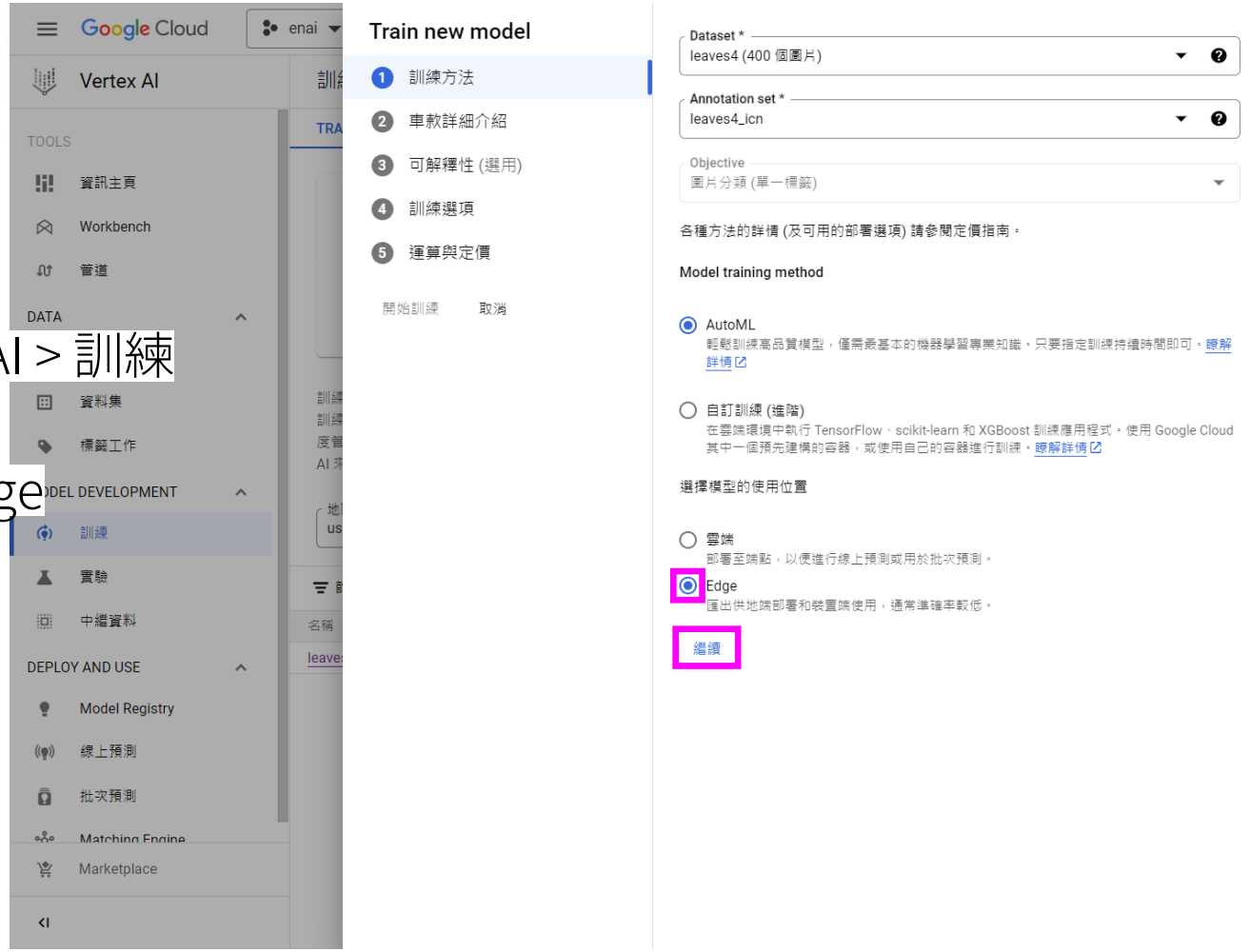
② 建立

Modeling ft. AutoML Edge

雲端建模地端應用

b. 訓練模型

① 進入 Vertex AI > 訓練
② 建立
③ 訓練方法 Edge



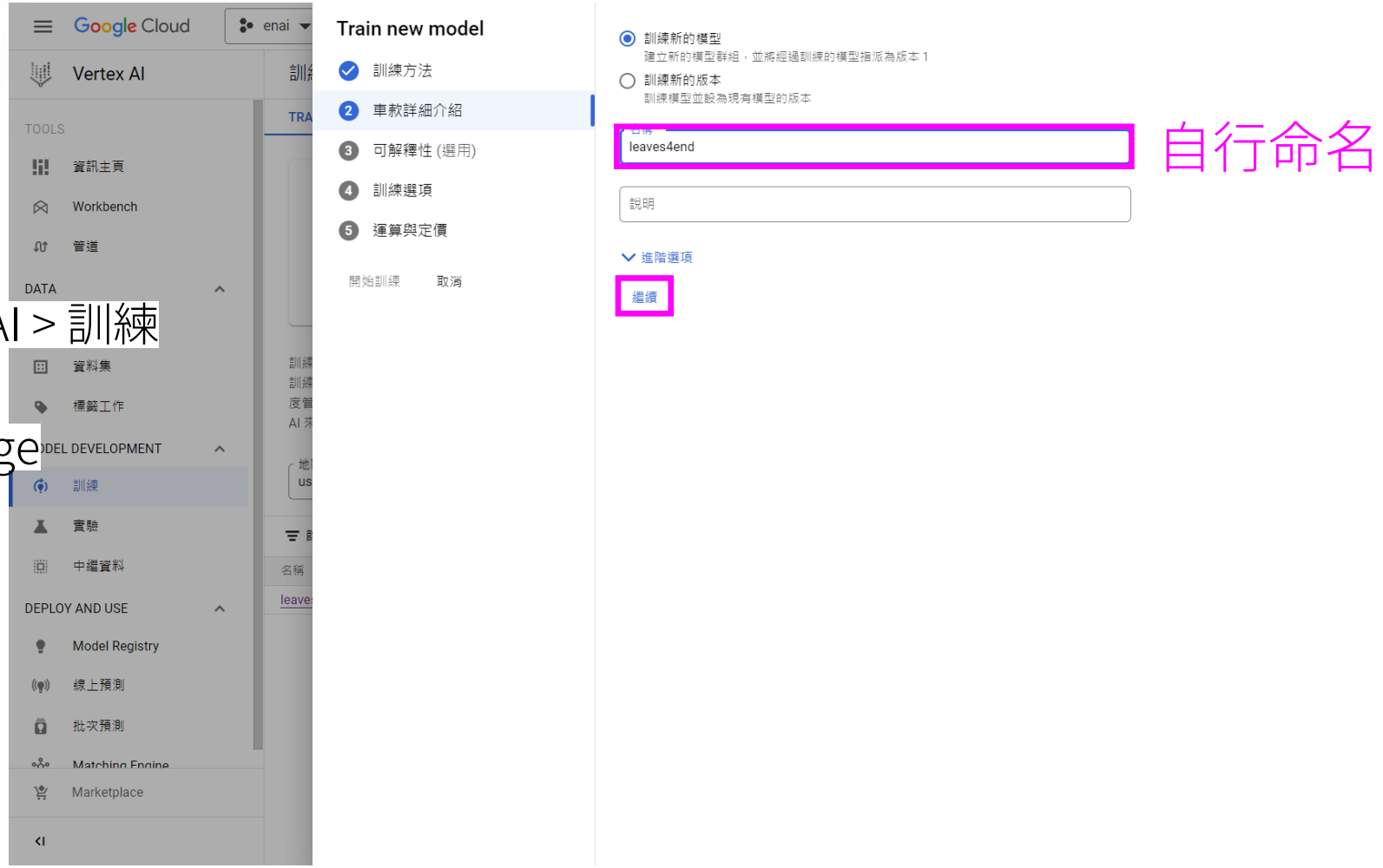
© 2026 Enos Chou

Modeling ft. AutoML Edge

雲端建模地端應用

b. 訓練模型

① 進入 Vertex AI > 訓練
② 建立
③ 訓練方法 Edge
④ 模型名稱

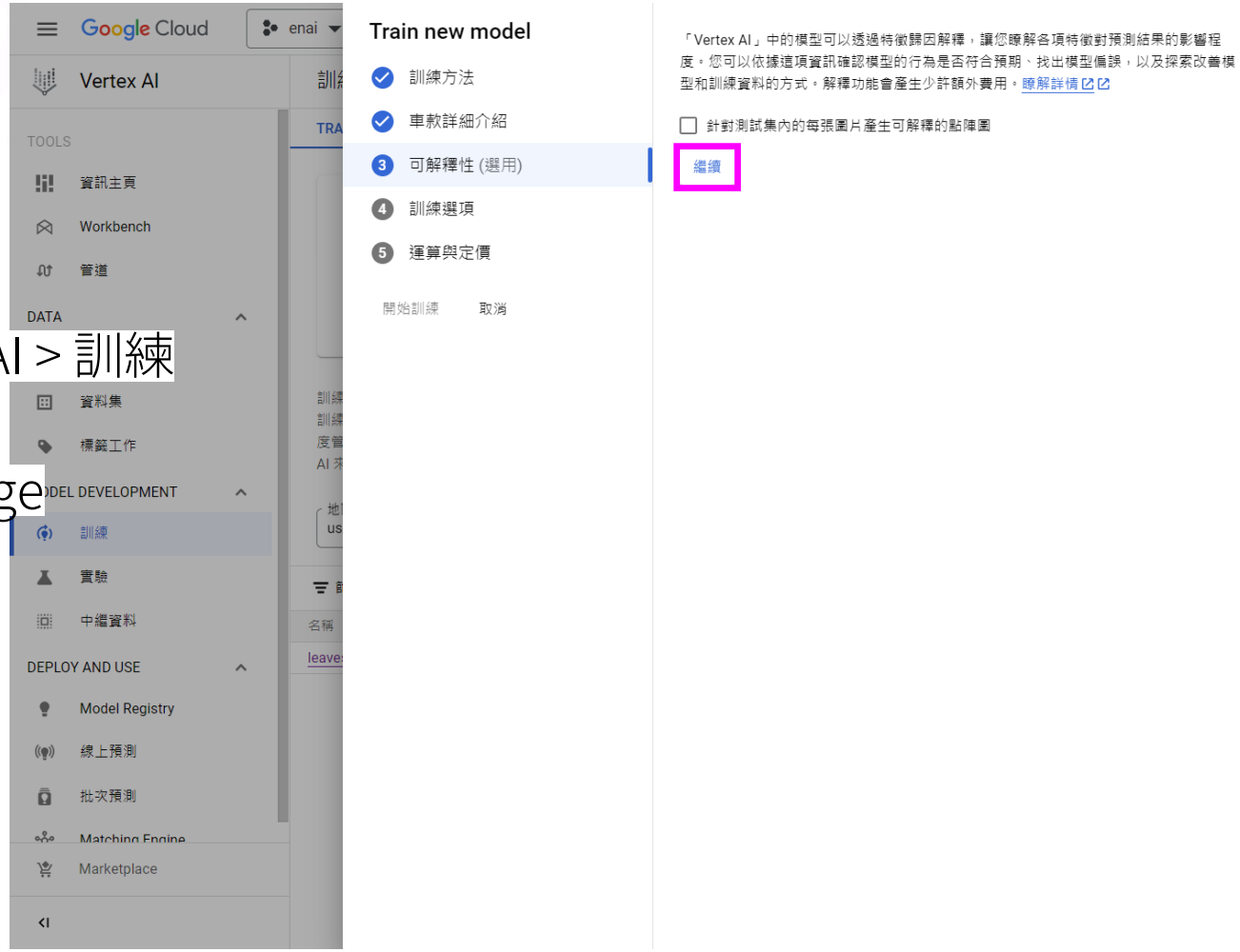


Modeling ft. AutoML Edge

雲端建模地端應用

b. 訓練模型

① 進入 Vertex AI > 訓練
② 建立
③ 訓練方法 Edge
④ 模型名稱
⑤ 可解釋性



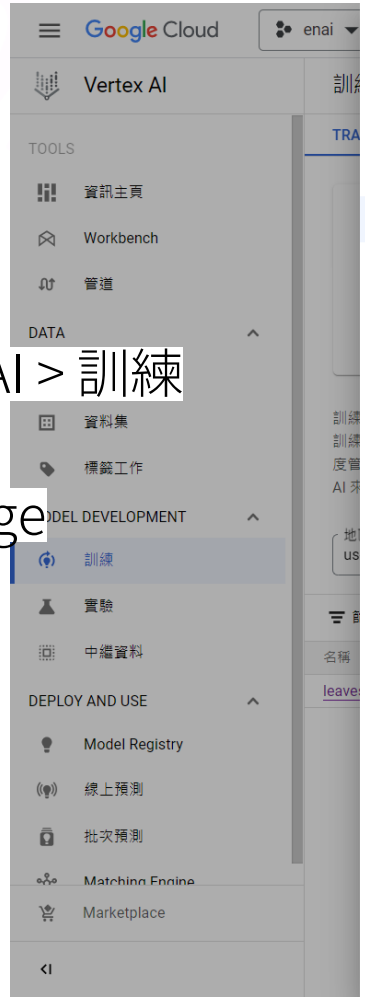
© 2026 Enos Chou

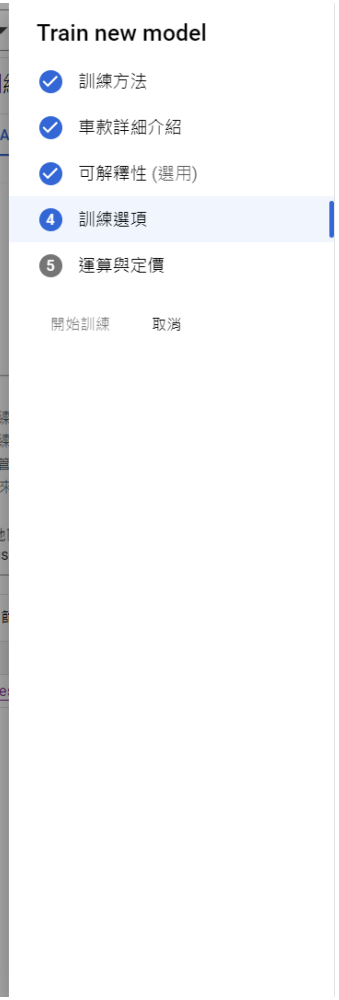
Modeling ft. AutoML Edge

雲端建模地端應用

b. 訓練模型

① 進入 Vertex AI > 訓練
② 建立
③ 訓練方法 Edge
④ 模型名稱
⑤ 可解釋性
⑥ 精確度





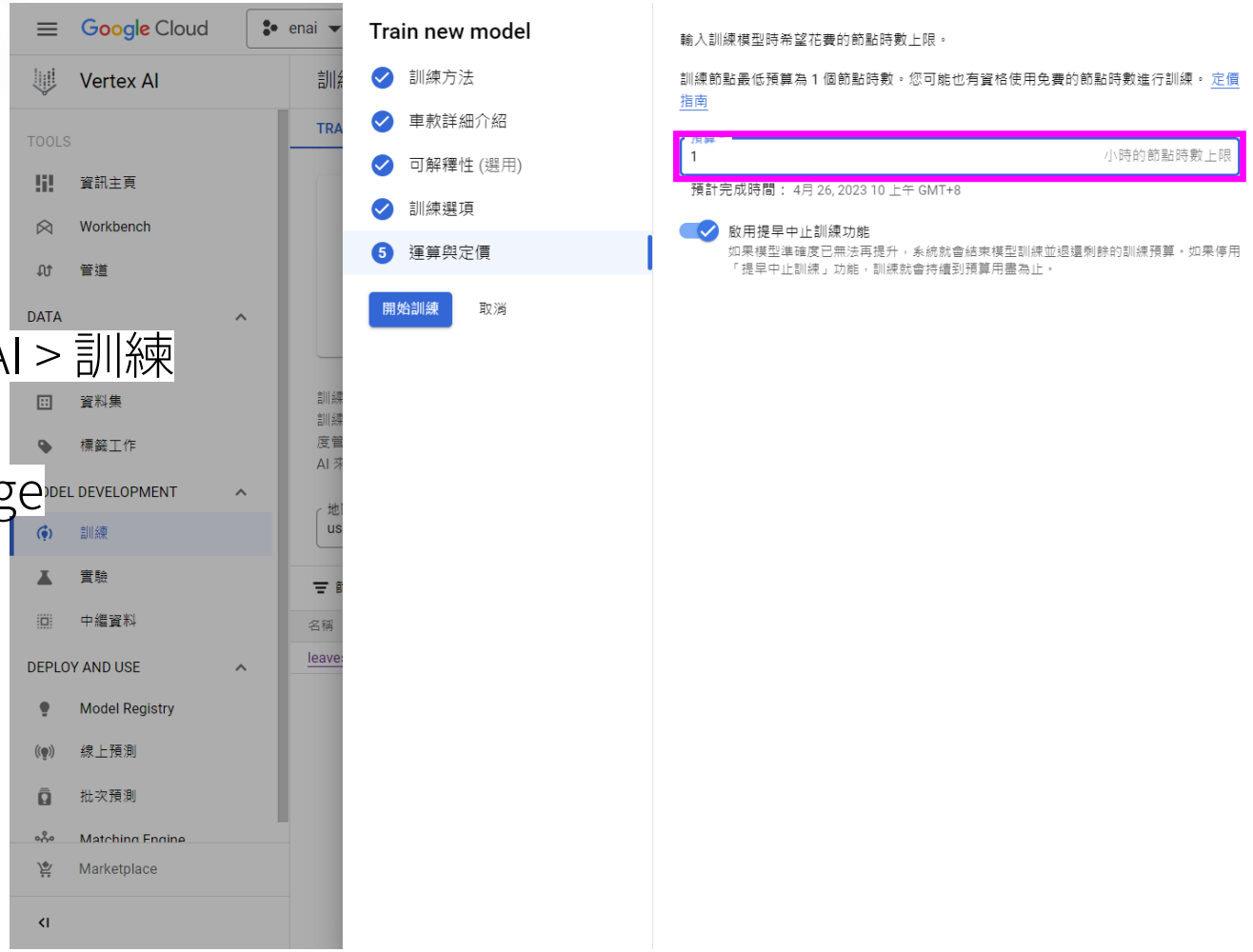
© 2026 Enos Chou

Modeling ft. AutoML Edge

雲端建模地端應用

b. 訓練模型

① 進入 Vertex AI > 訓練
② 建立
③ 訓練方法 Edge
④ 模型名稱
⑤ 可解釋性
⑥ 精確度
⑦ 訓練預算



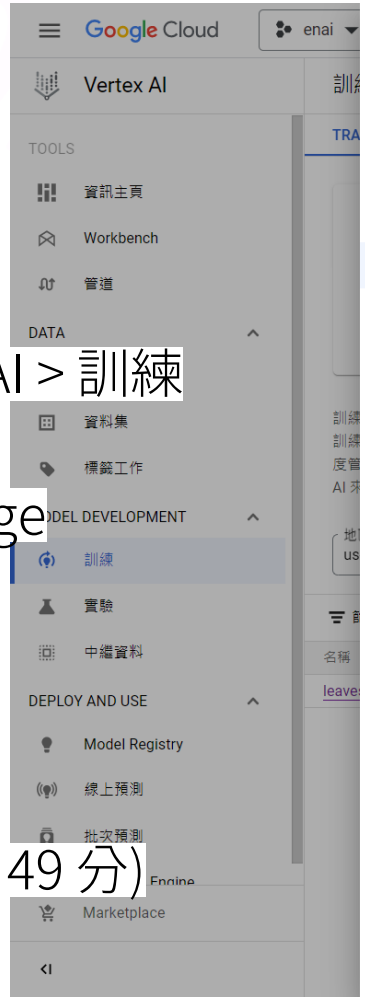
© 2026 Enos Chou

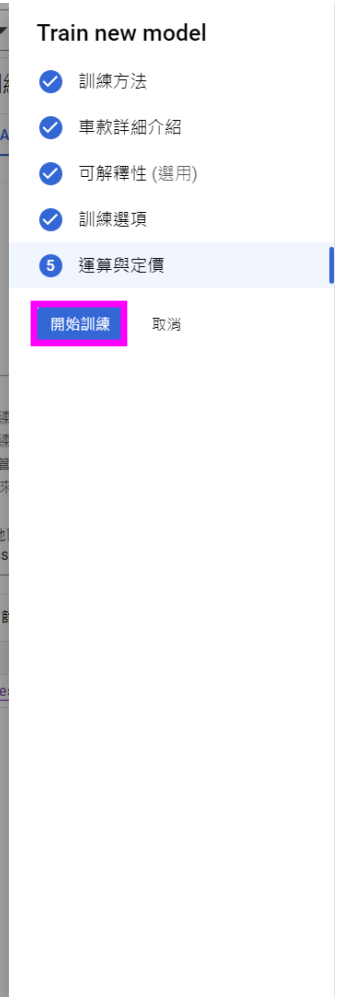
Modeling ft. AutoML Edge

雲端建模地端應用

b. 訓練模型

① 進入 Vertex AI > 訓練
② 建立
③ 訓練方法 Edge
④ 模型名稱
⑤ 可解釋性
⑥ 精確度
⑦ 訓練預算
⑧ 開始訓練 (約 49 分)





© 2026 Enos Chou

Modeling ft. AutoML Edge

雲端建模地端應用

b. 訓練模型

① 進入 Vertex AI > 訓練
② 建立
③ 訓練方法 Edge
④ 模型名稱
⑤ 可解釋性
⑥ 精確度
⑦ 訓練預算
⑧ 開始訓練 (約 49 分)

© 2026 Enos Chou

Modeling ft. AutoML Edge

雲端建模地端應用

b. 訓練模型

① 進入 Vertex AI > 訓練
② 建立
③ 訓練方法 Edge
④ 模型名稱
⑤ 可解釋性
⑥ 精確度
⑦ 訓練預算
⑧ 開始訓練 (約 49 分)
⑨ 評價模型

© 2026 Enos Chou

Modeling ft. AutoML Edge

雲端建模地端應用

b. 訓練模型

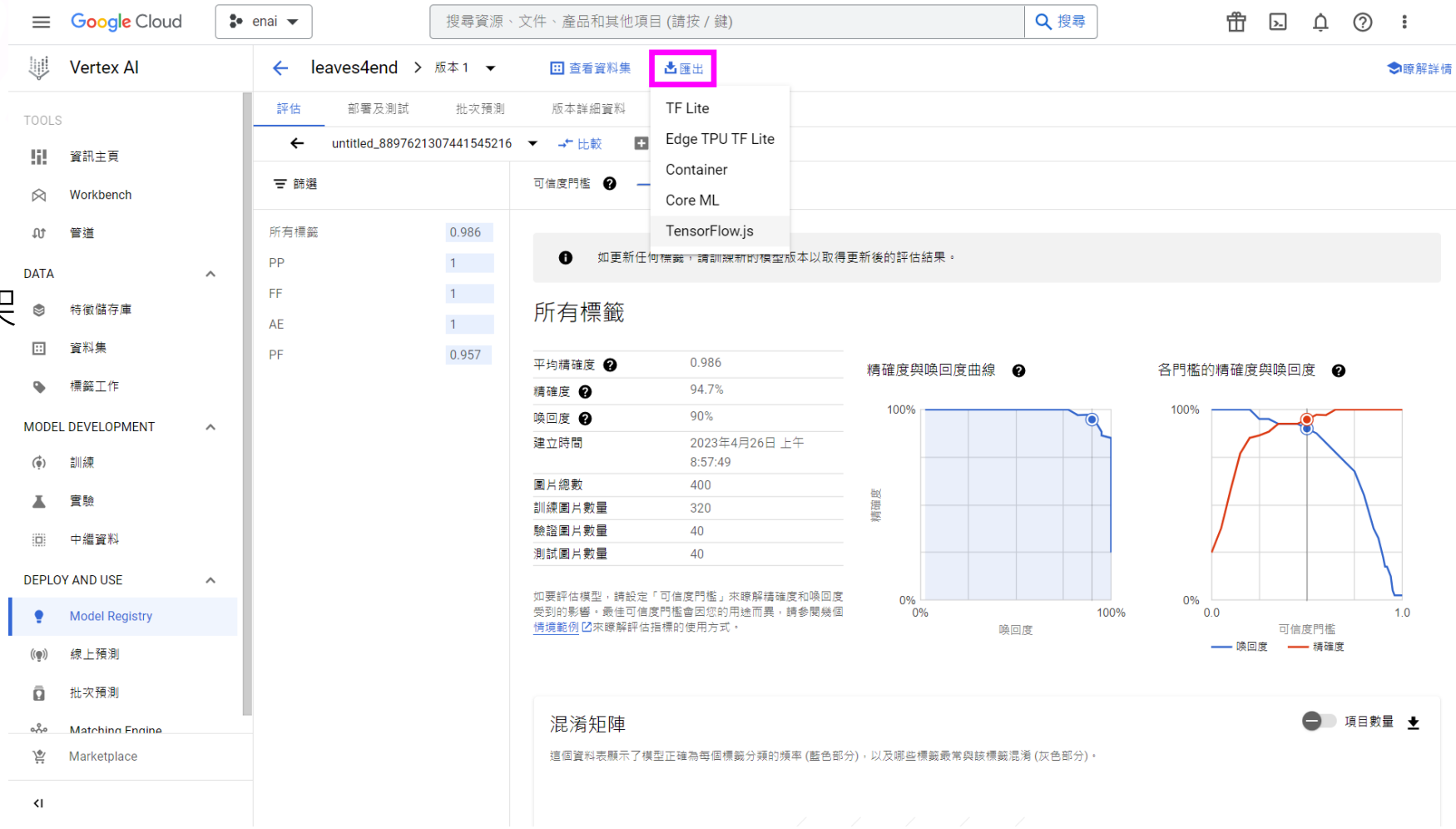
① 進入 Vertex AI > 訓練
② 建立
③ 訓練方法 Edge
④ 模型名稱
⑤ 可解釋性
⑥ 精確度
⑦ 訓練預算
⑧ 開始訓練 (約 49 分)
⑨ 評價模型

Application ft. AutoML Edge

雲端建模地端應用

c. 匯出模型

① 指定模型框架

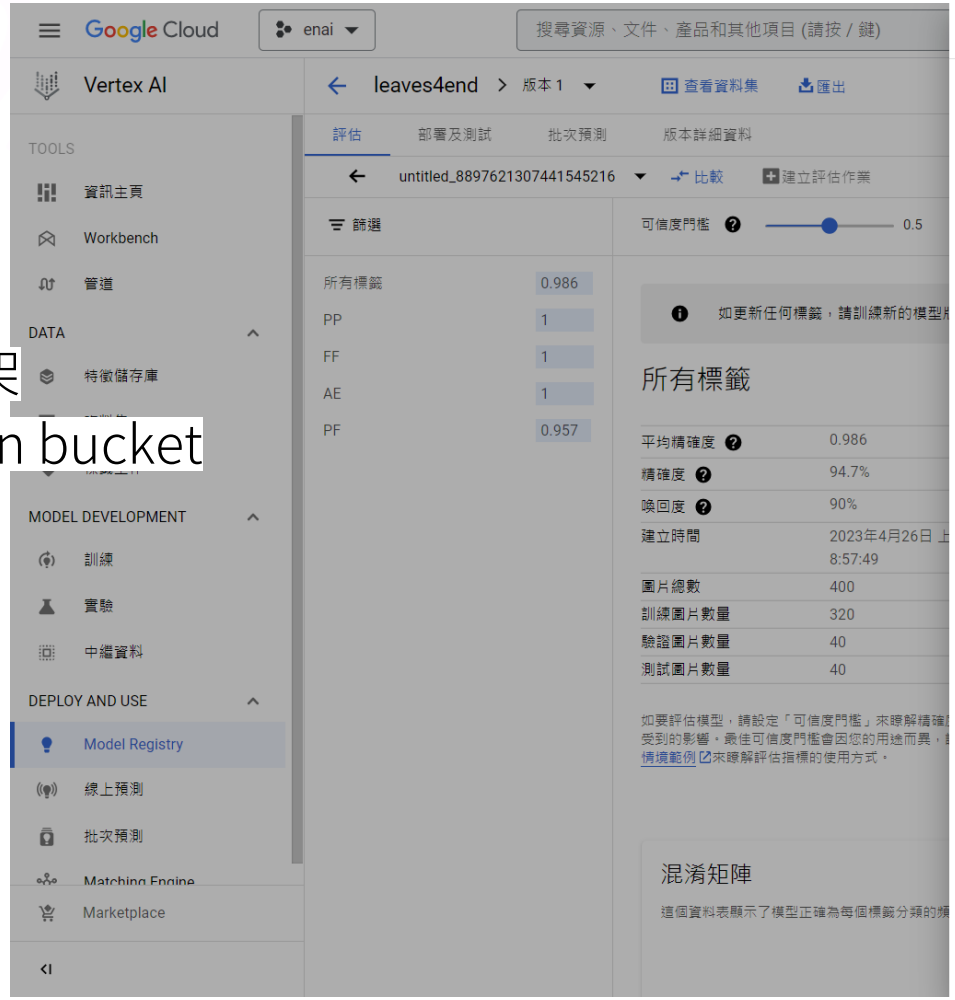


Application ft. AutoML Edge

雲端建模地端應用

c. 匯出模型

① 指定模型框架
② 指定同 region bucket

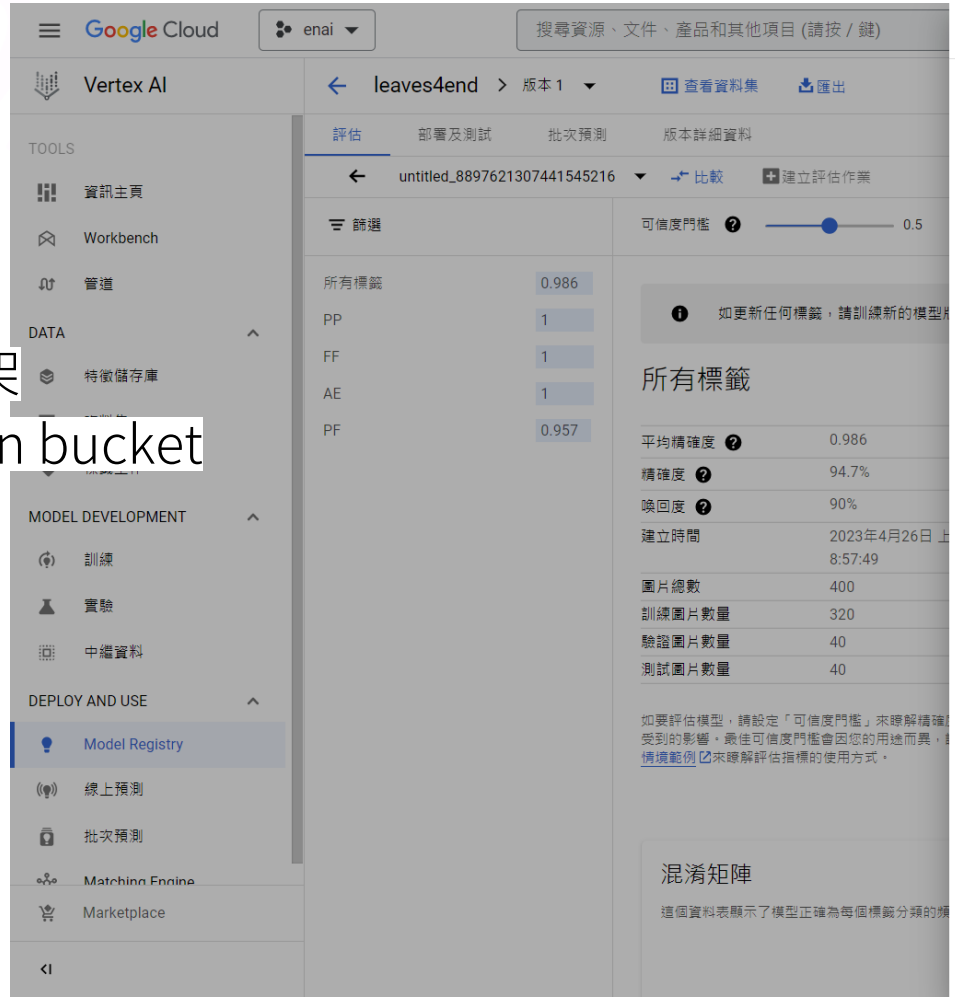


Application ft. AutoML Edge

雲端建模地端應用

c. 匯出模型

① 指定模型框架
② 指定同 region bucket
③ 匯出

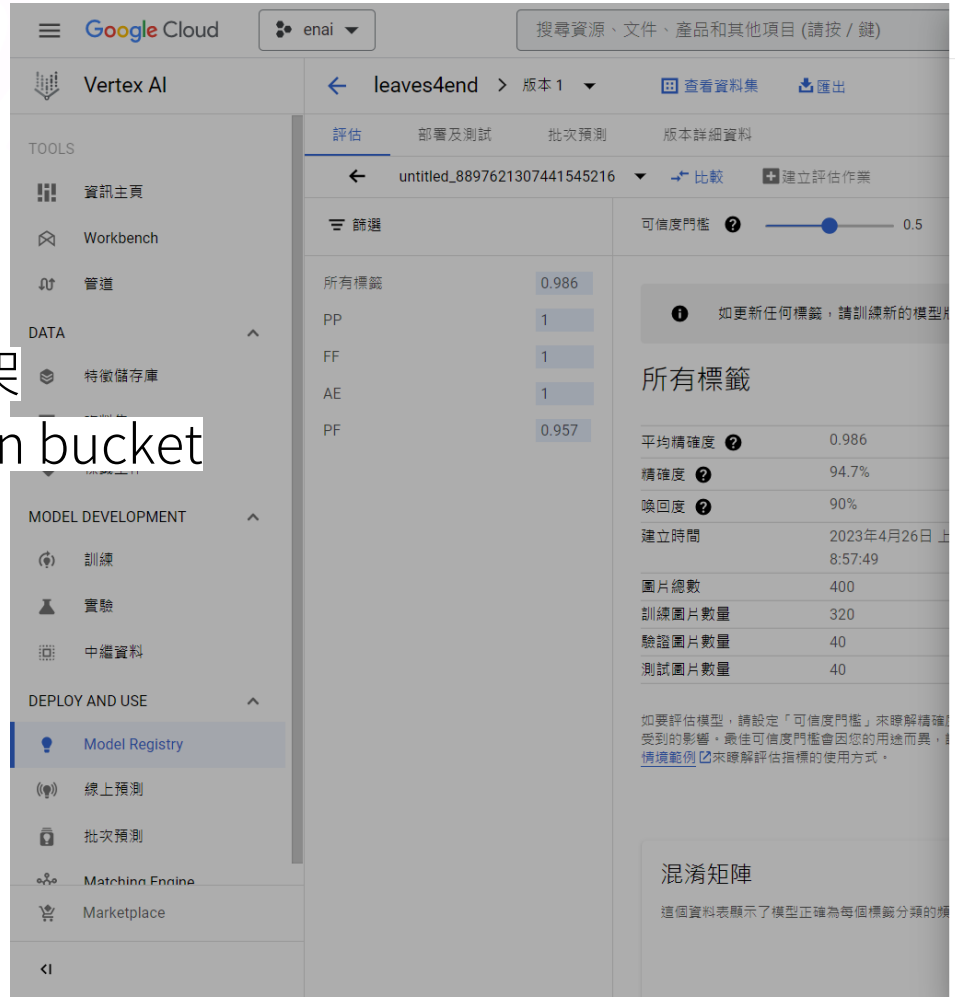


Application ft. AutoML Edge

雲端建模地端應用

c. 匯出模型

① 指定模型框架
② 指定同 region bucket
③ 匯出

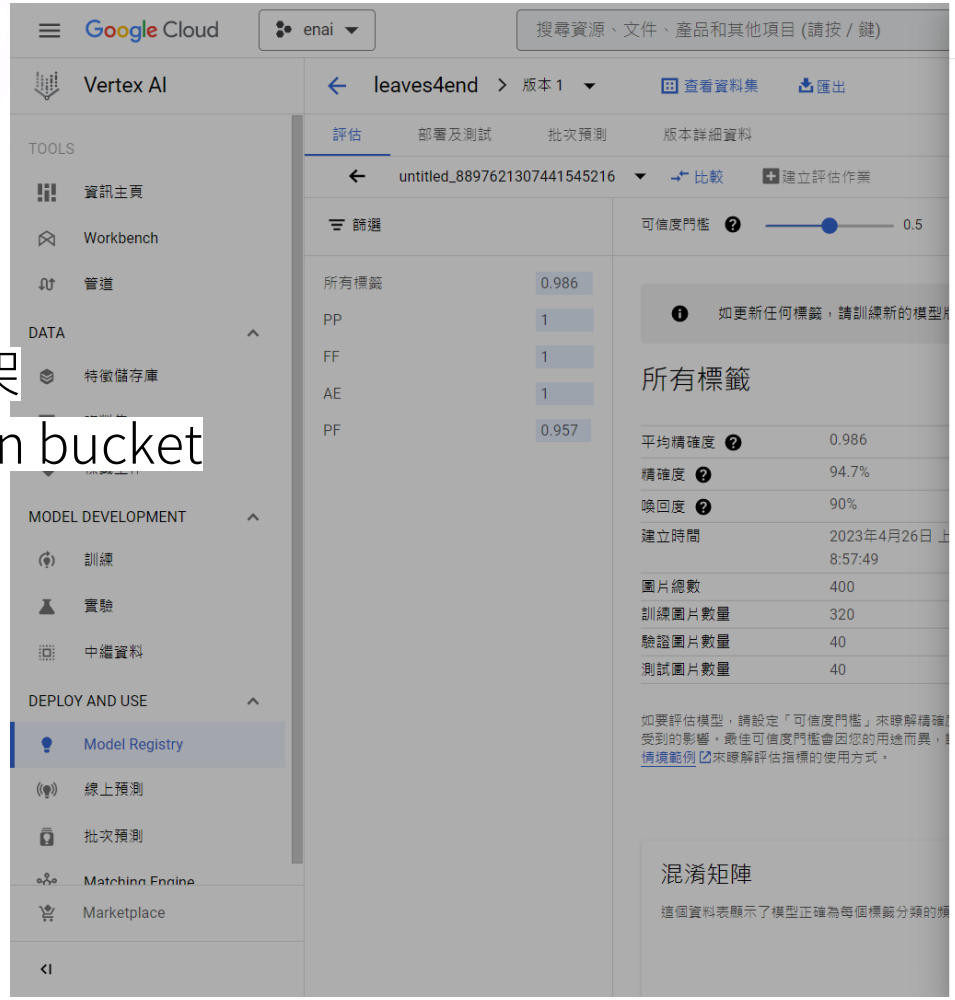


Application ft. AutoML Edge

雲端建模地端應用

c. 匯出模型

① 指定模型框架
② 指定同 region bucket
③ 匯出
④ 查看資料夾



Application ft. AutoML Edge

雲端建模地端應用

c. 匯出模型

① 指定模型框架
② 指定同 region bucket
③ 匯出
④ 查看資料夾

© 2026 Enos Chou

Application ft. AutoML Edge

雲端建模地端應用

c. 匯出模型

① 指定模型框架
② 指定同 region bucket
③ 匯出
④ 查看資料夾

© 2026 Enos Chou

Application ft. AutoML Edge

雲端建模地端應用

d. 下載模型

① 啟用 Cloud Shell

© 2026 Enos Chou

Application ft. AutoML Edge

雲端建模地端應用

d. 下載模型

① 啟用 Cloud Shell

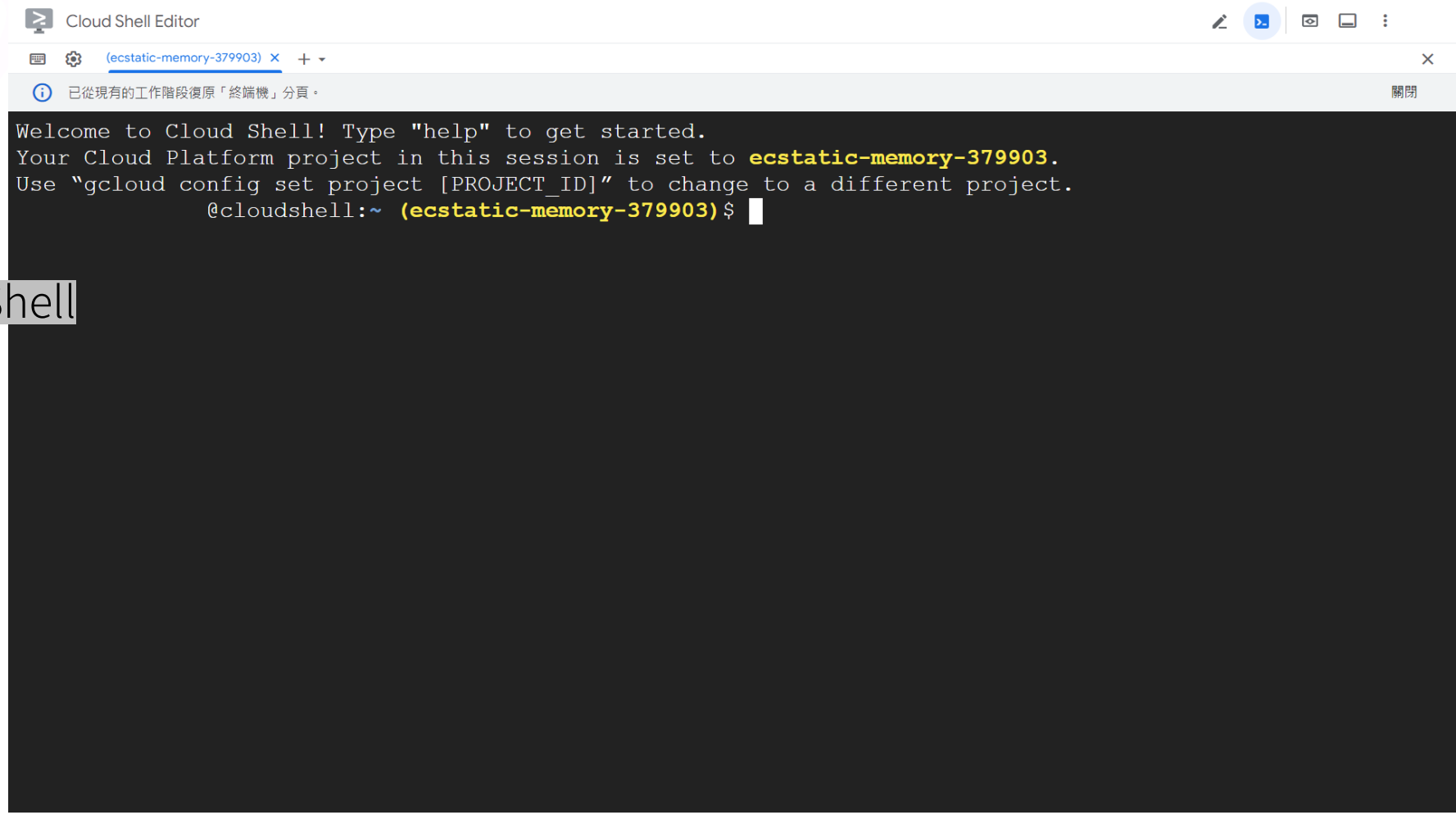
© 2026 Enos Chou

Application ft. AutoML Edge

雲端建模地端應用

d. 下載模型

① 啟用 Cloud Shell



© 2026 Enos Chou

Application ft. AutoML Edge

雲端建模地端應用

d. 下載模型

Issue

直接以 Vertex AI 建議的方式下載模型會失敗

```
gsutil cp -r gs://YOUR_GCS_MODEL_PATH ./download_dir
```

Root Cause

① Windows 不允許含 ":" 的目錄或檔名；直接自 GCS 下載模型目錄至 Windows 會失敗

② GCS 中的檔案或目錄無法直接更名，僅能複製至新目錄 `gsutil cp` 或以 `gsutil mv` 更名

Solution

先以 `gsutil cp` 的方式將原模型複製至新目錄，再行下載

© 2026 Enos Chou

Application ft. AutoML Edge

雲端建模地端應用

d. 下載模型

Command Example

```
gsutil cp -r gs://enaiworld/model-8482532097175584768/core-ml/2022-11-25T07:05:39.632518Z gs://enaiworld/leaves4edge/core-ml
gsutil cp -r gs://enaiworld/model-8482532097175584768/edgetpu-tflite/2022-11-25T07:05:39.632518Z gs://enaiworld/leaves4edge/edgetpu-tflite
gsutil cp -r gs://enaiworld/model-8482532097175584768/tf-js/2022-11-25T07:05:39.632518Z gs://enaiworld/leaves4edge/tf-js
gsutil cp -r gs://enaiworld/model-8482532097175584768/tf-saved-model/2022-11-25T07:05:39.632518Z gs://enaiworld/leaves4edge/tf-saved-model
gsutil cp -r gs://enaiworld/model-8482532097175584768/tflite/2022-11-25T07:05:39.632518Z gs://enaiworld/leaves4edge/tflite
```

© 2026 Enos Chou

Application ft. AutoML Edge

雲端建模地端應用

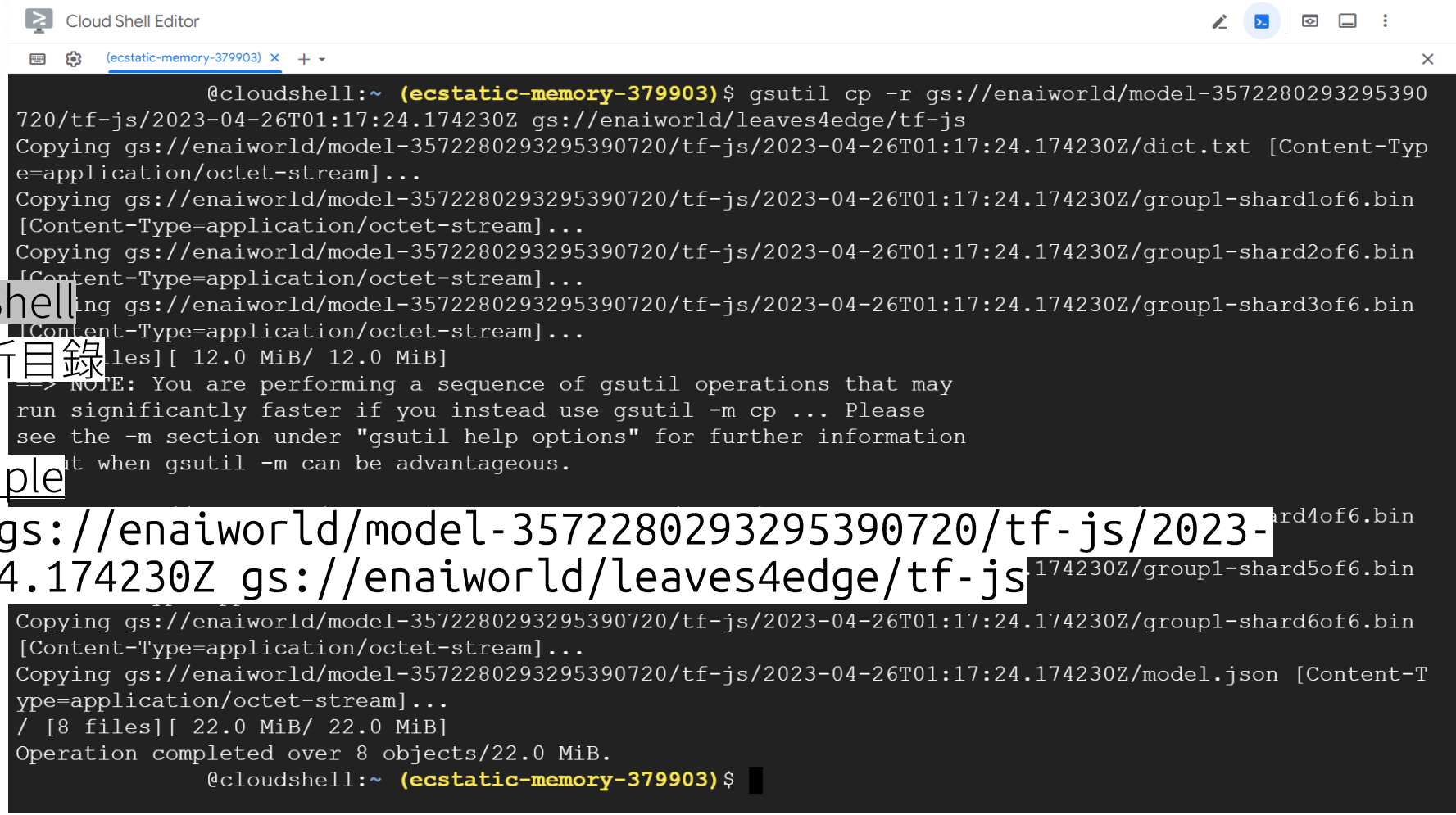
d. 下載模型

① 啟用 Cloud Shell

② 複製模型至新目錄

Command Example

```
gsutil cp -r gs://enaiworld/model-3572280293295390720/tf-js/2023-
04-26T01:17:24.174230Z gs://enaiworld/leaves4edge/tf-js
```



Application ft. AutoML Edge

雲端建模地端應用

d. 下載模型

① 啟用 Cloud Shell
② 複製模型至新目錄

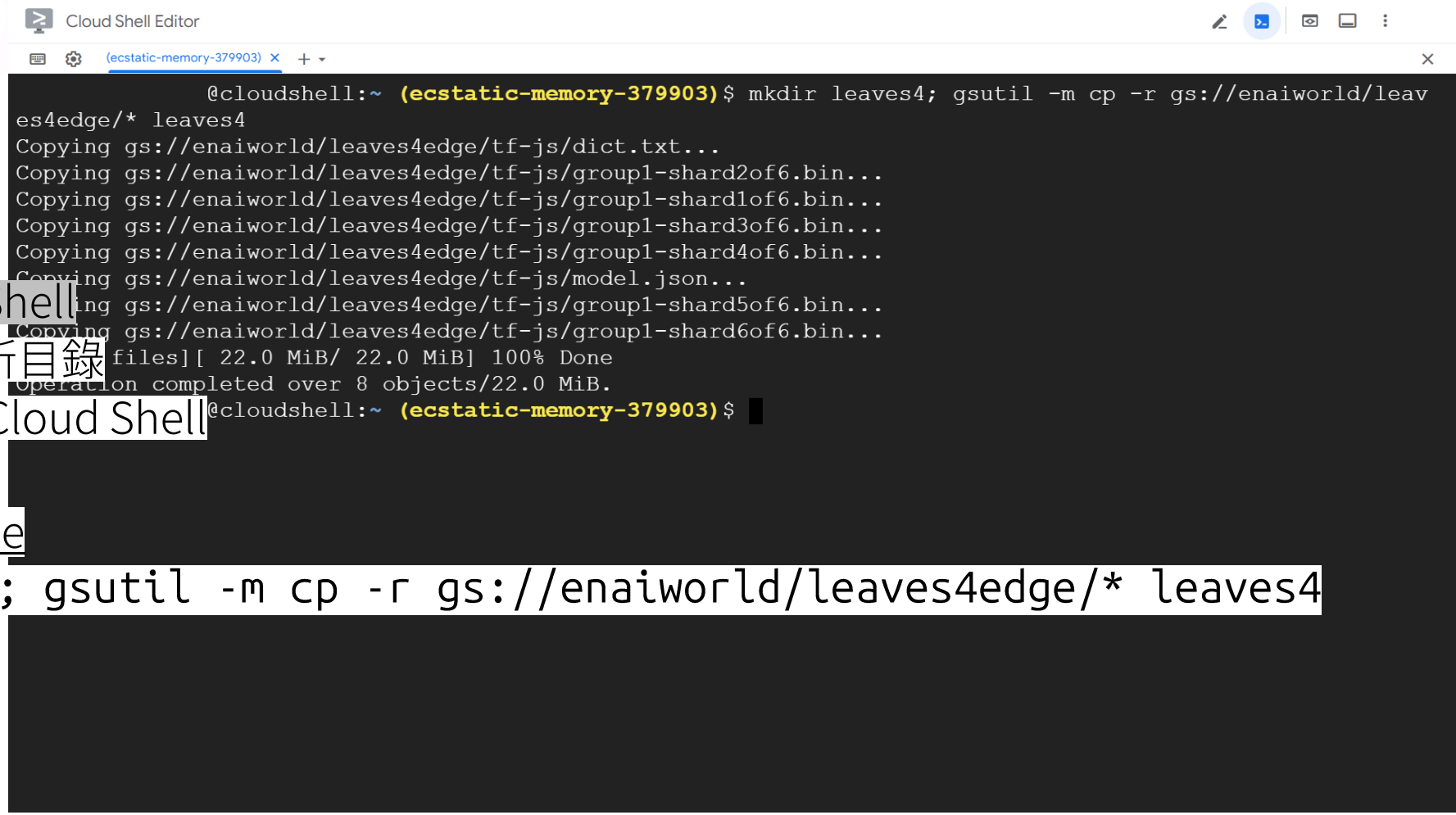
© 2026 Enos Chou

Application ft. AutoML Edge

雲端建模地端應用

d. 下載模型

① 啟用 Cloud Shell
② 複製模型至新目錄
③ 下載模型至 Cloud Shell



Comand Example

```
mkdir leaves4; gsutil -m cp -r gs://enaiworld/leaves4edge/* leaves4
```

© 2026 Enos Chou

Application ft. AutoML Edge

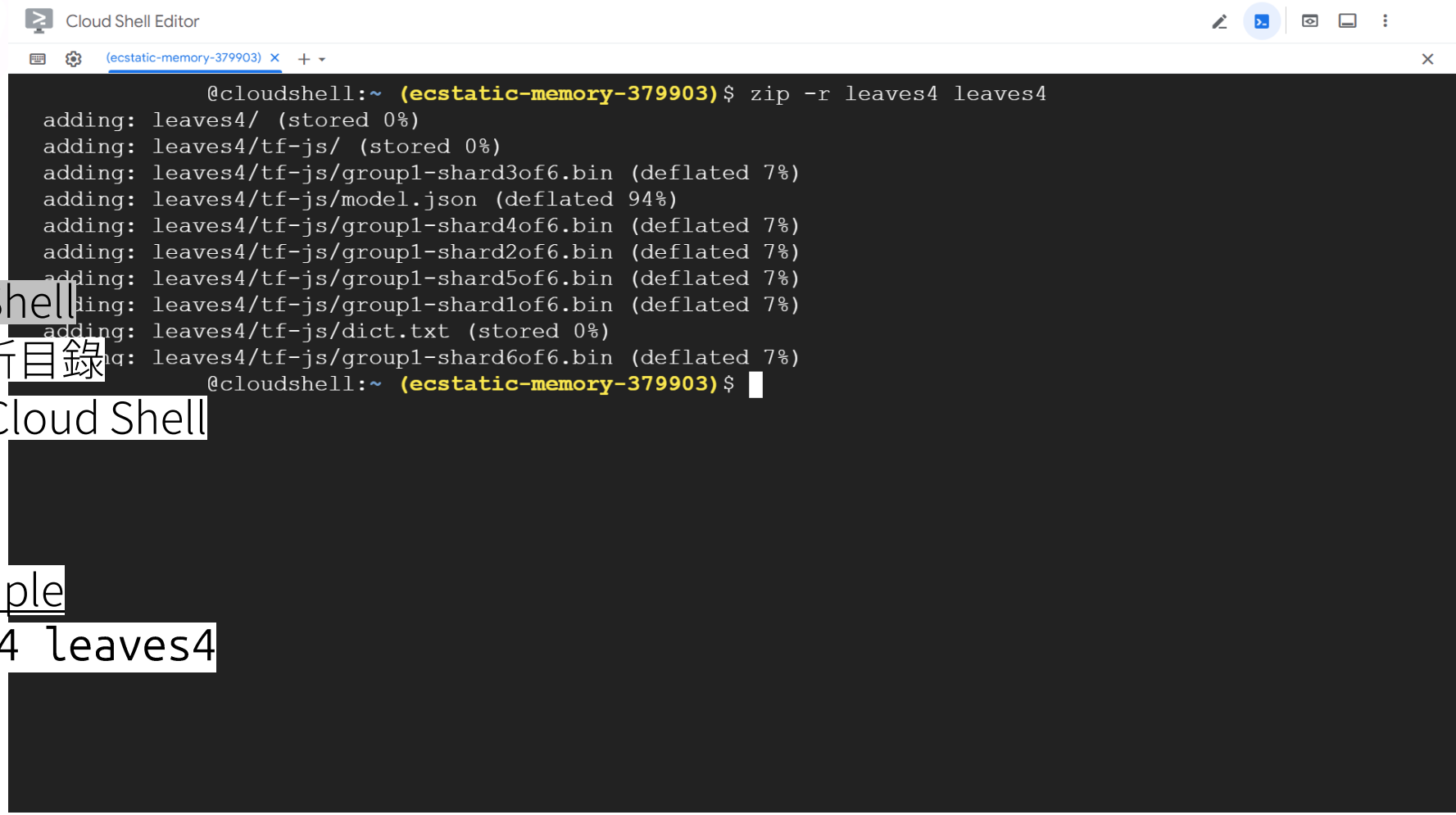
雲端建模地端應用

d. 下載模型

① 啟用 Cloud Shell
② 複製模型至新目錄
③ 下載模型至 Cloud Shell
④ 壓縮模型

Command Example

```
zip -r leaves4 leaves4
```



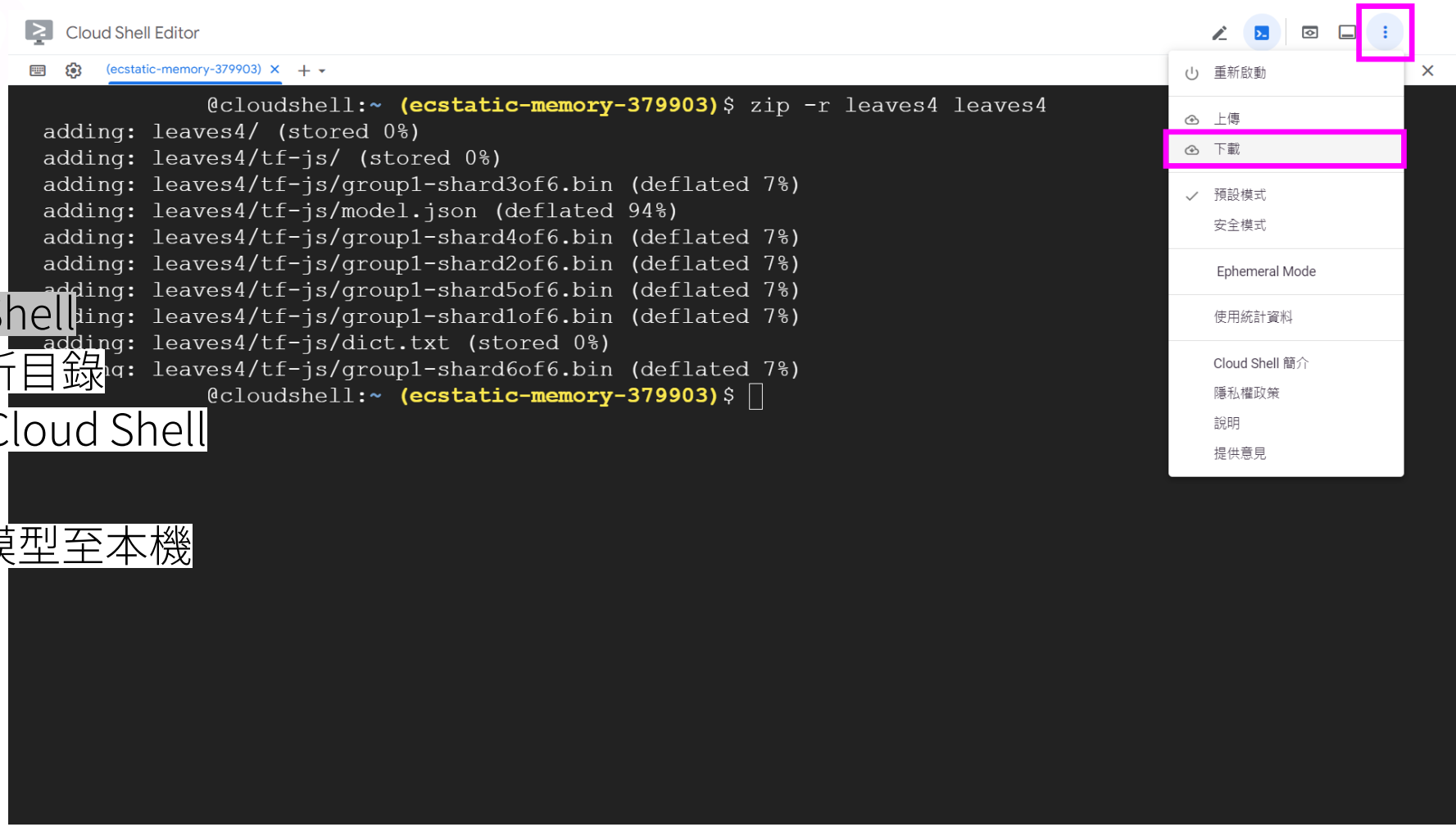
© 2026 Enos Chou

Application ft. AutoML Edge

雲端建模地端應用

d. 下載模型

① 啟用 Cloud Shell
② 複製模型至新目錄
③ 下載模型至 Cloud Shell
④ 壓縮模型
⑤ 下載壓縮之模型至本機



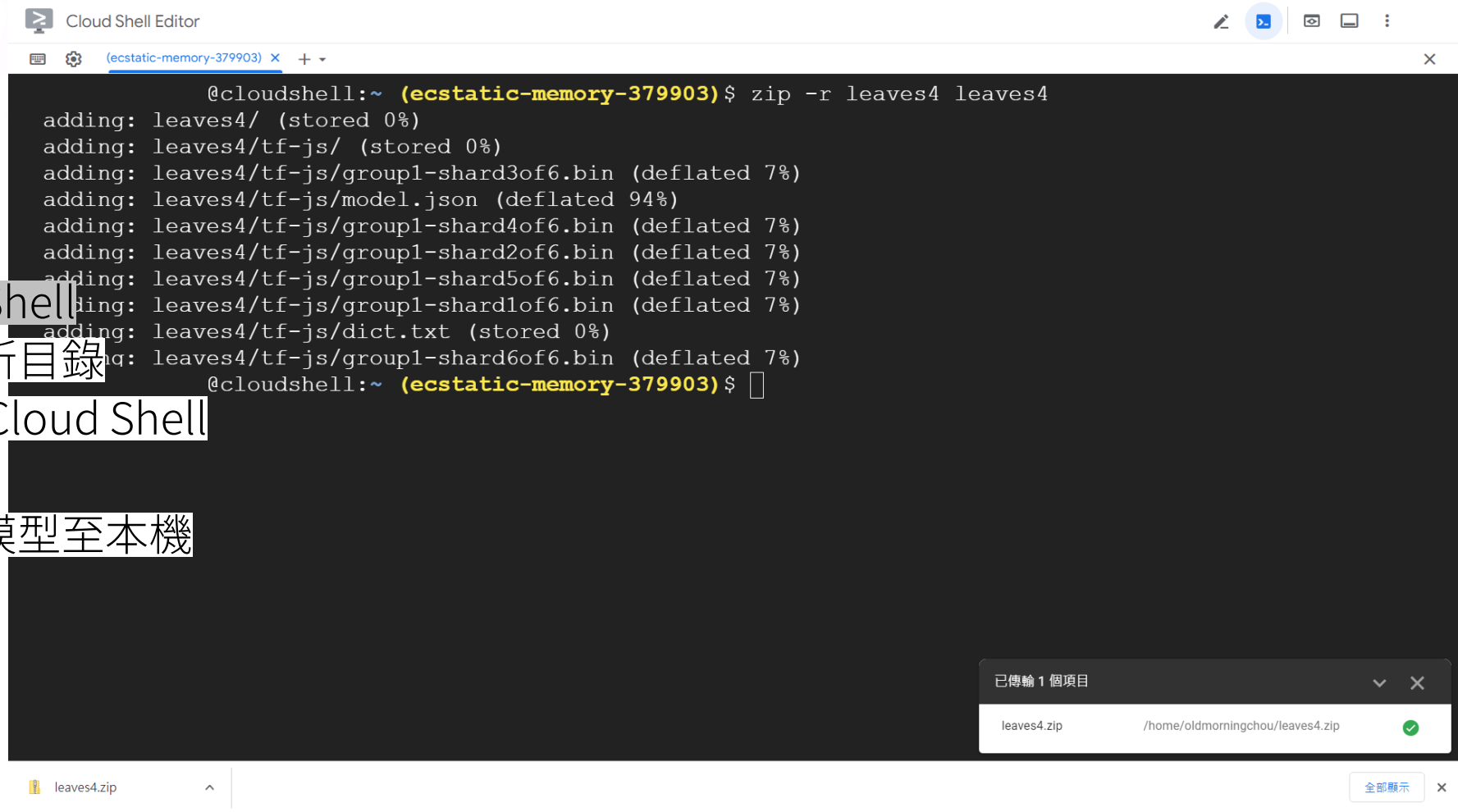
© 2026 Enos Chou

Application ft. AutoML Edge

雲端建模地端應用

d. 下載模型

① 啟用 Cloud Shell
② 複製模型至新目錄
③ 下載模型至 Cloud Shell
④ 壓縮模型
⑤ 下載壓縮之模型至本機

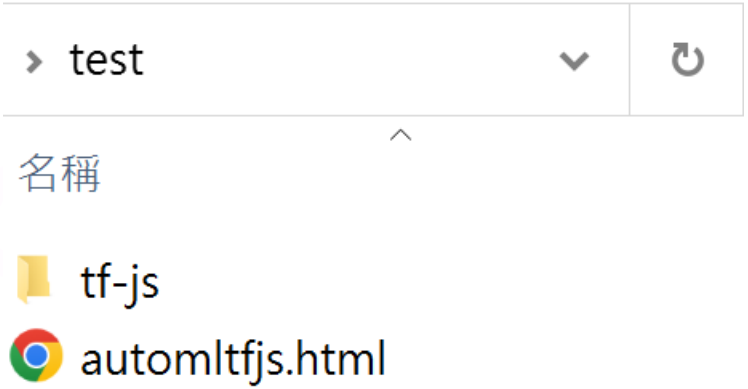


Application ft. AutoML Edge

雲端建模地端應用

e. 應用模型 (tf-js)

① 解壓縮模型，將 tf-js 置於工作目錄
② 部署 automltfjs.html 於工作目錄



© 2026 Enos Chou

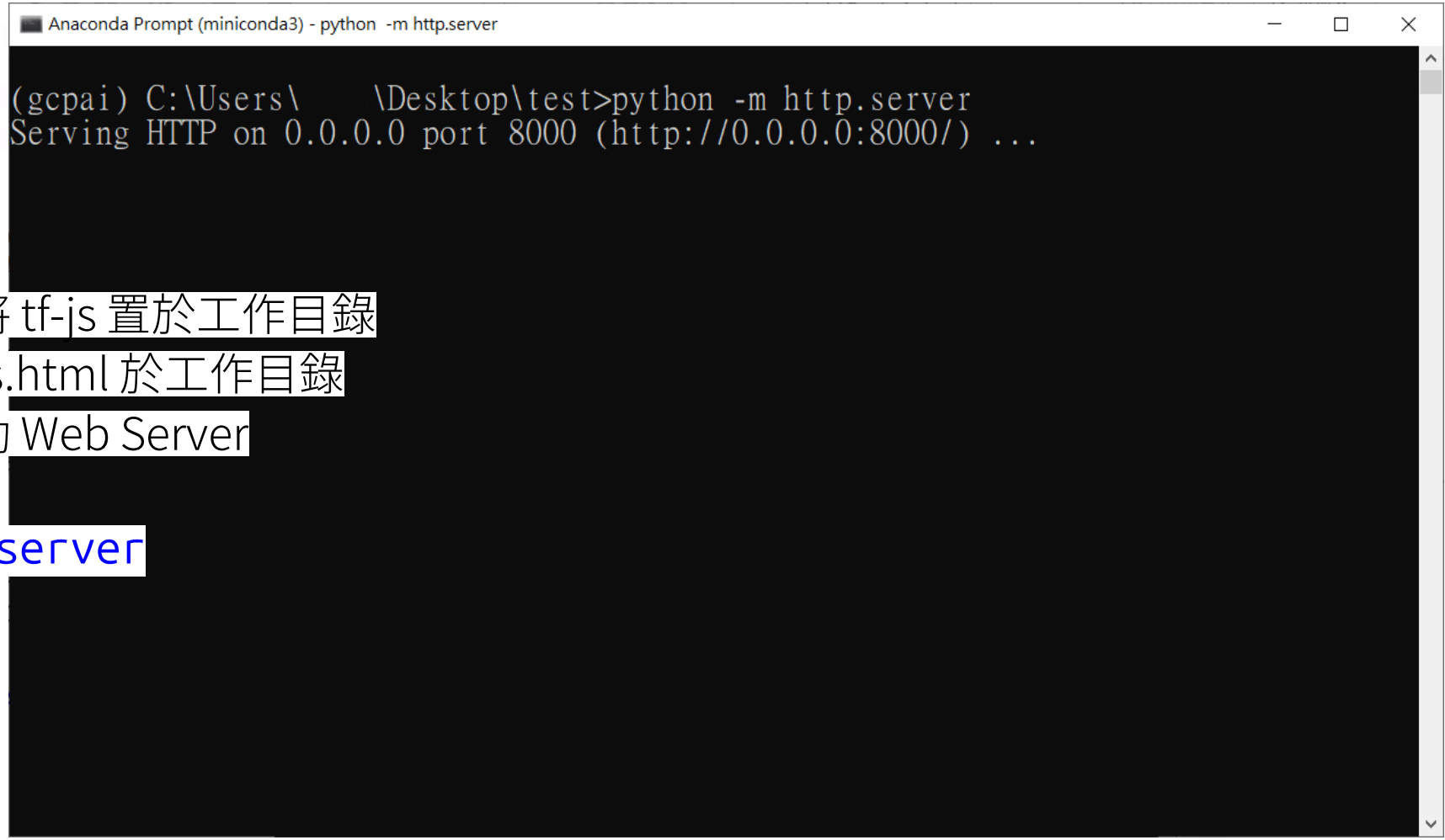
Application ft. AutoML Edge

雲端建模地端應用

e. 應用模型 (tf-js)

① 解壓縮模型，將 tf-js 置於工作目錄
② 部署 automltfjs.html 於工作目錄
③ 於工作目錄啟動 Web Server

```
python -m http.server
```



© 2026 Enos Chou

Application ft. AutoML Edge

雲端建模地端應用

e. 應用模型 (tf-js)

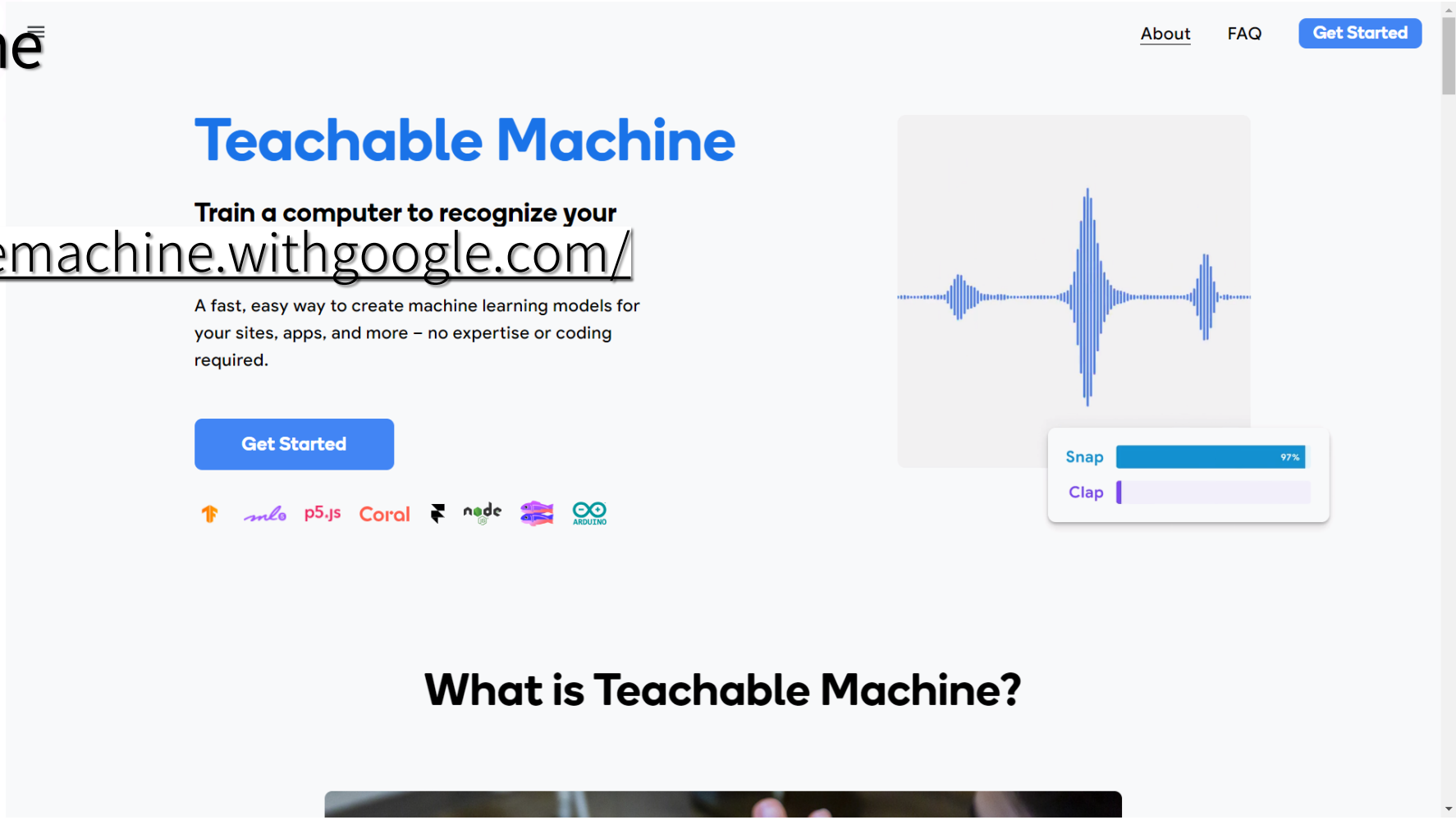
① 解壓縮模型，將 tf-js 置於工作目錄
② 部署 automltfjs.html 於工作目錄
③ 於工作目錄啟動 Web Server
④ 測試

http://localhost:8000/automltfjs.html

Teachable Machine

Teachable Machine

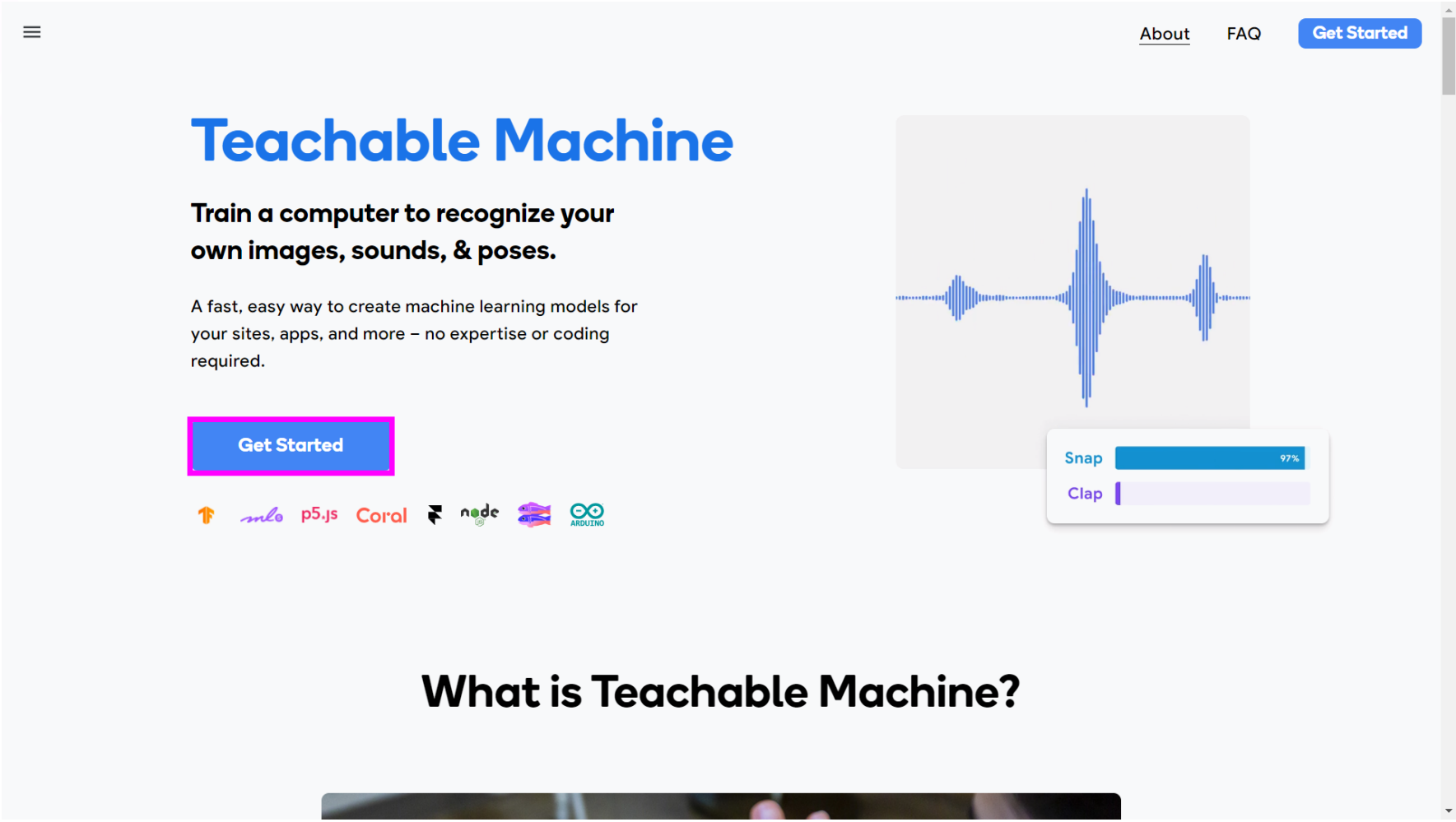
https://teachablemachine.withgoogle.com/



Modeling ft. Teachable Machine

1. 建立模型

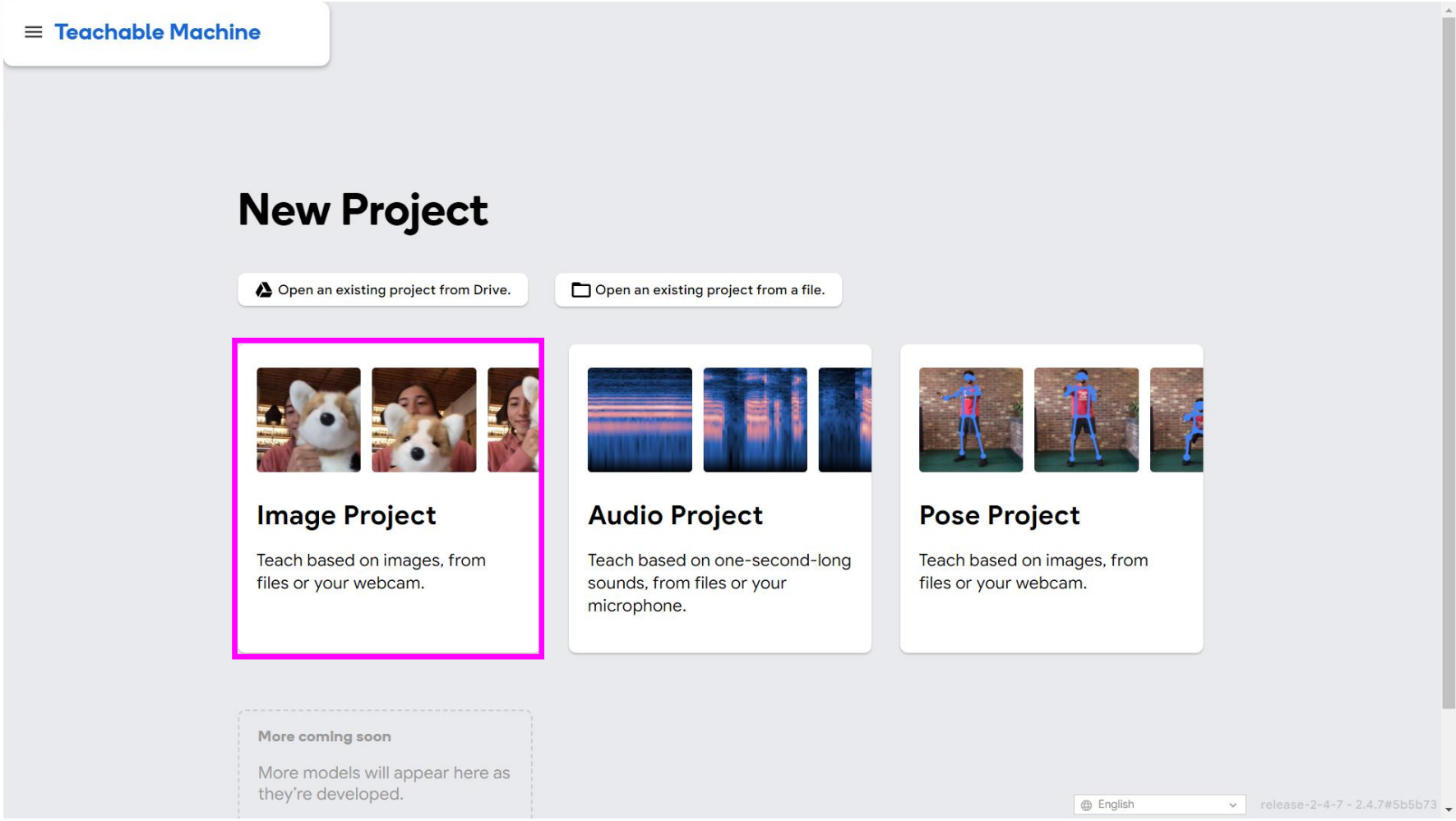
開始使用



Modeling ft. Teachable Machine

1. 建立模型

開始使用

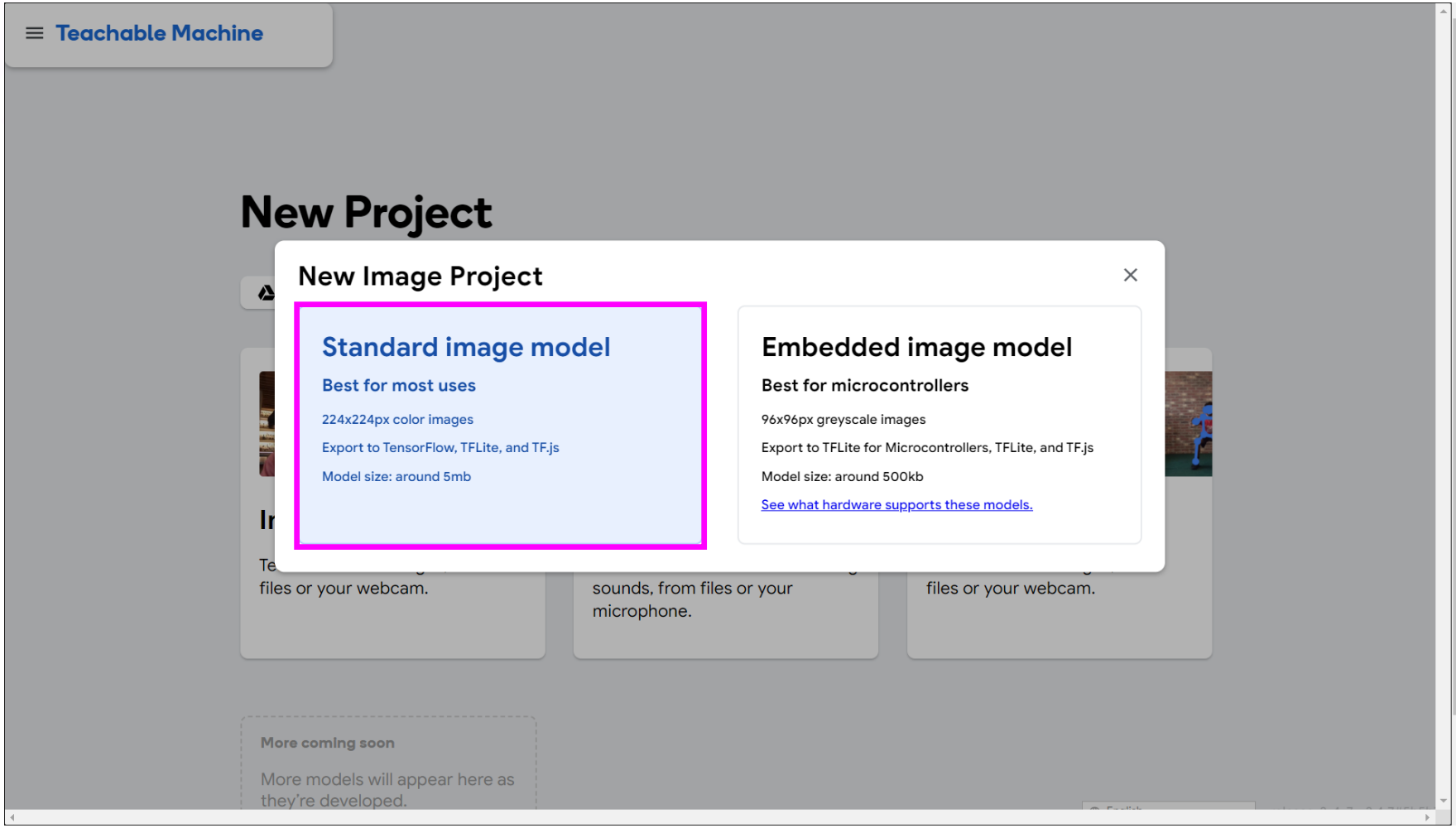


© 2026 Enos Chou

Modeling ft. Teachable Machine

1. 建立模型

開始使用

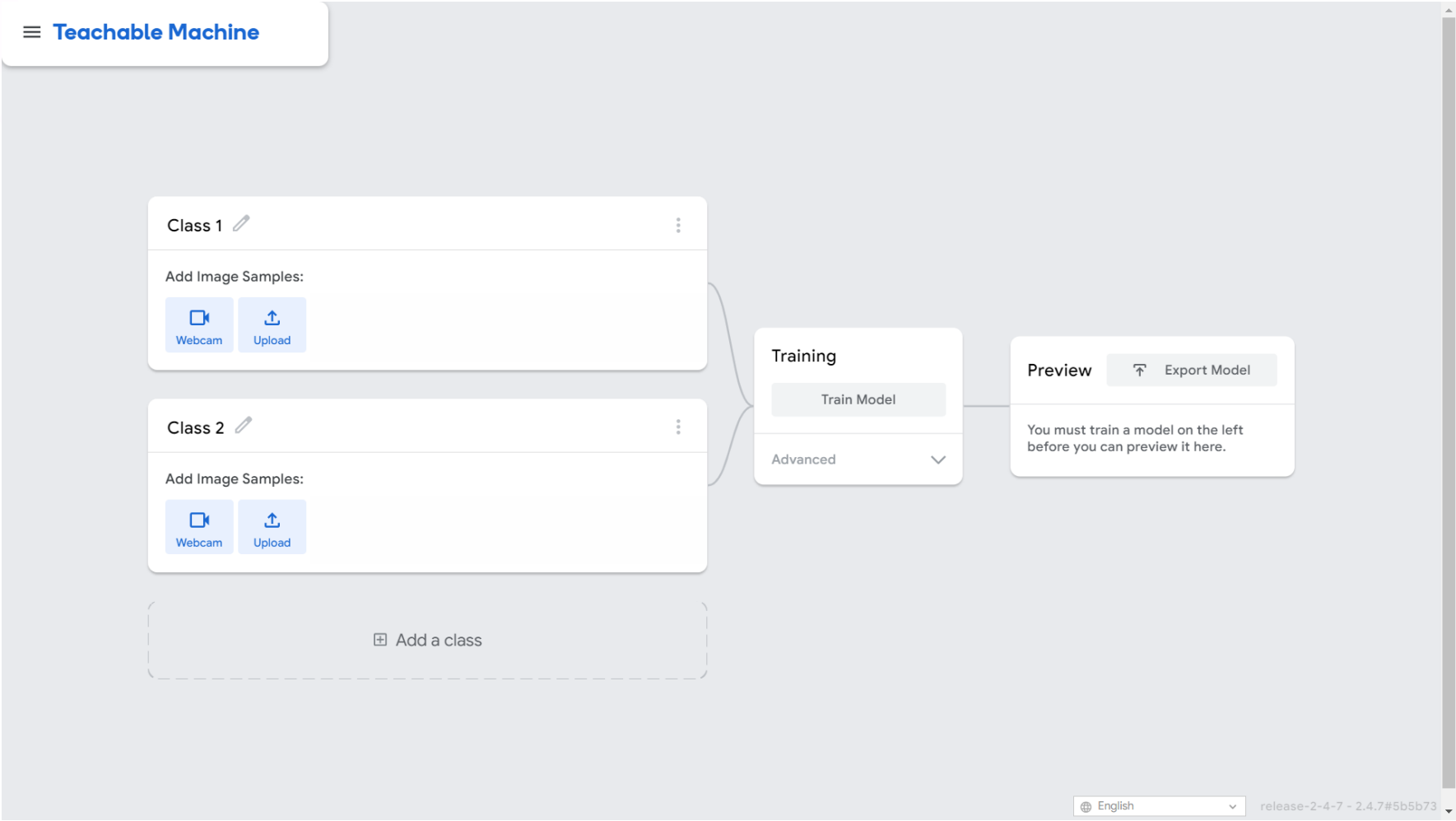


© 2026 Enos Chou

Modeling ft. Teachable Machine

1. 建立模型

開始使用



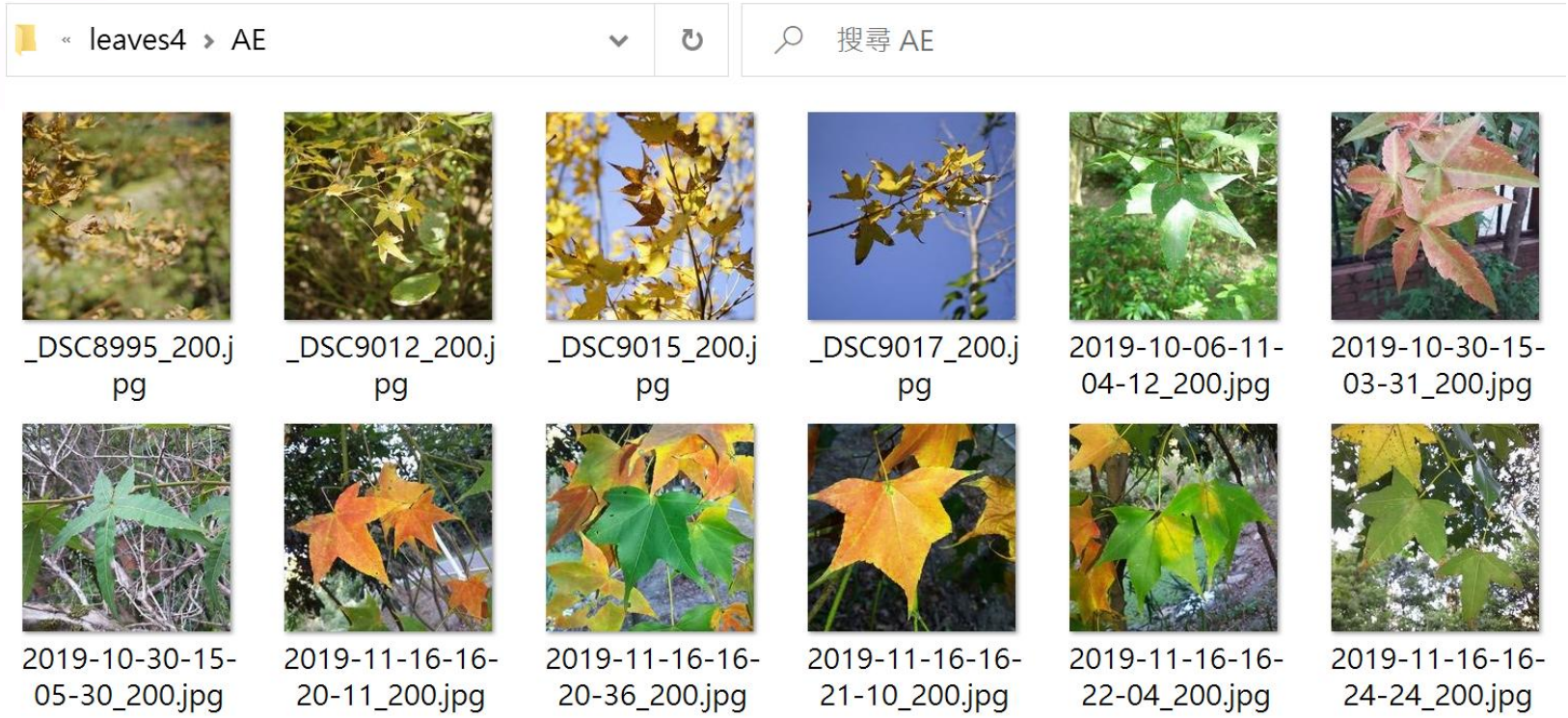
© 2026 Enos Chou

Modeling ft. Teachable Machine

1. 建立模型

a. 準備資料集

unzip leaves4.zip



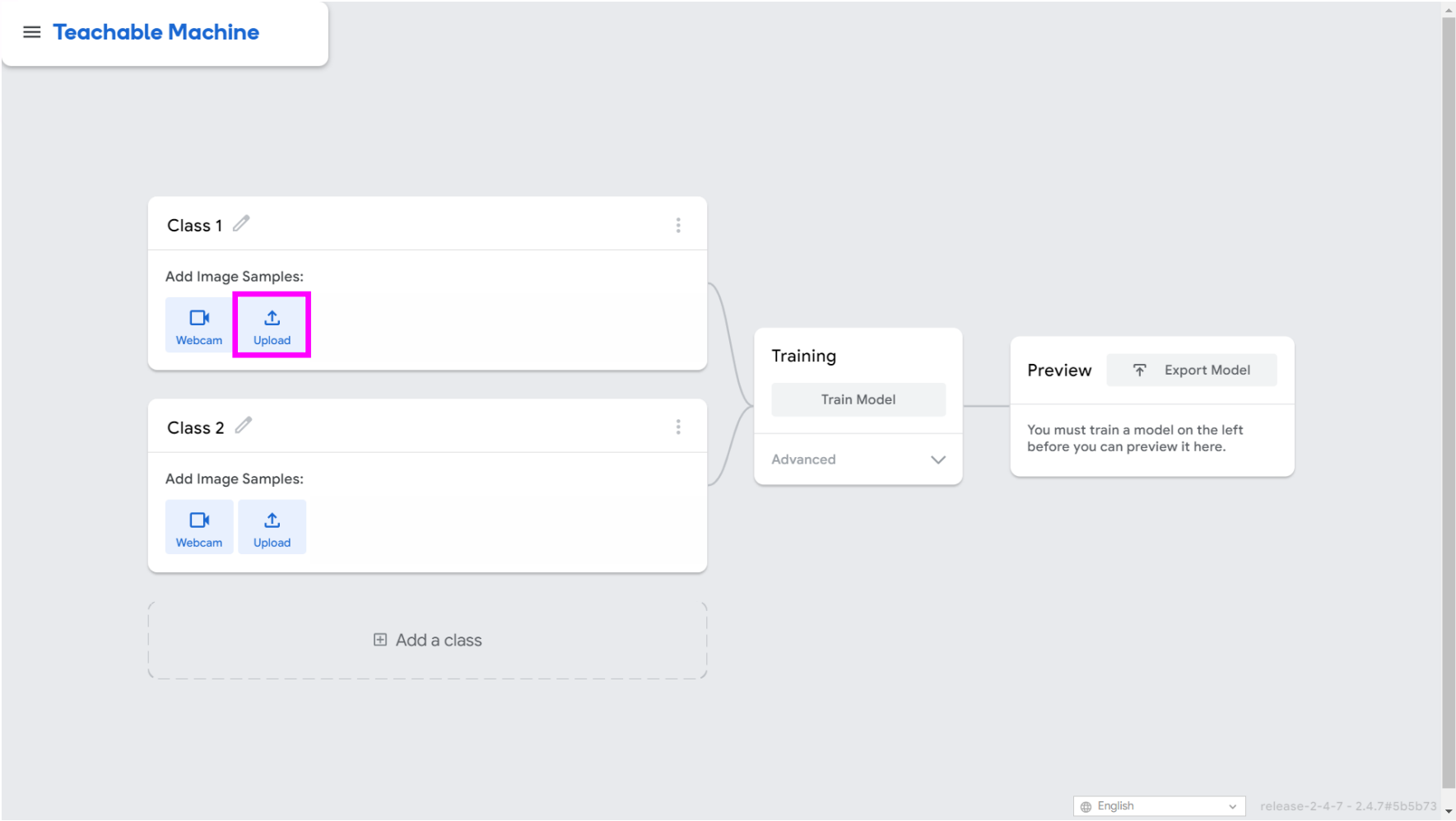
© 2026 Enos Chou

Modeling ft. Teachable Machine

1. 建立模型

   b. 匯入資料

      AE



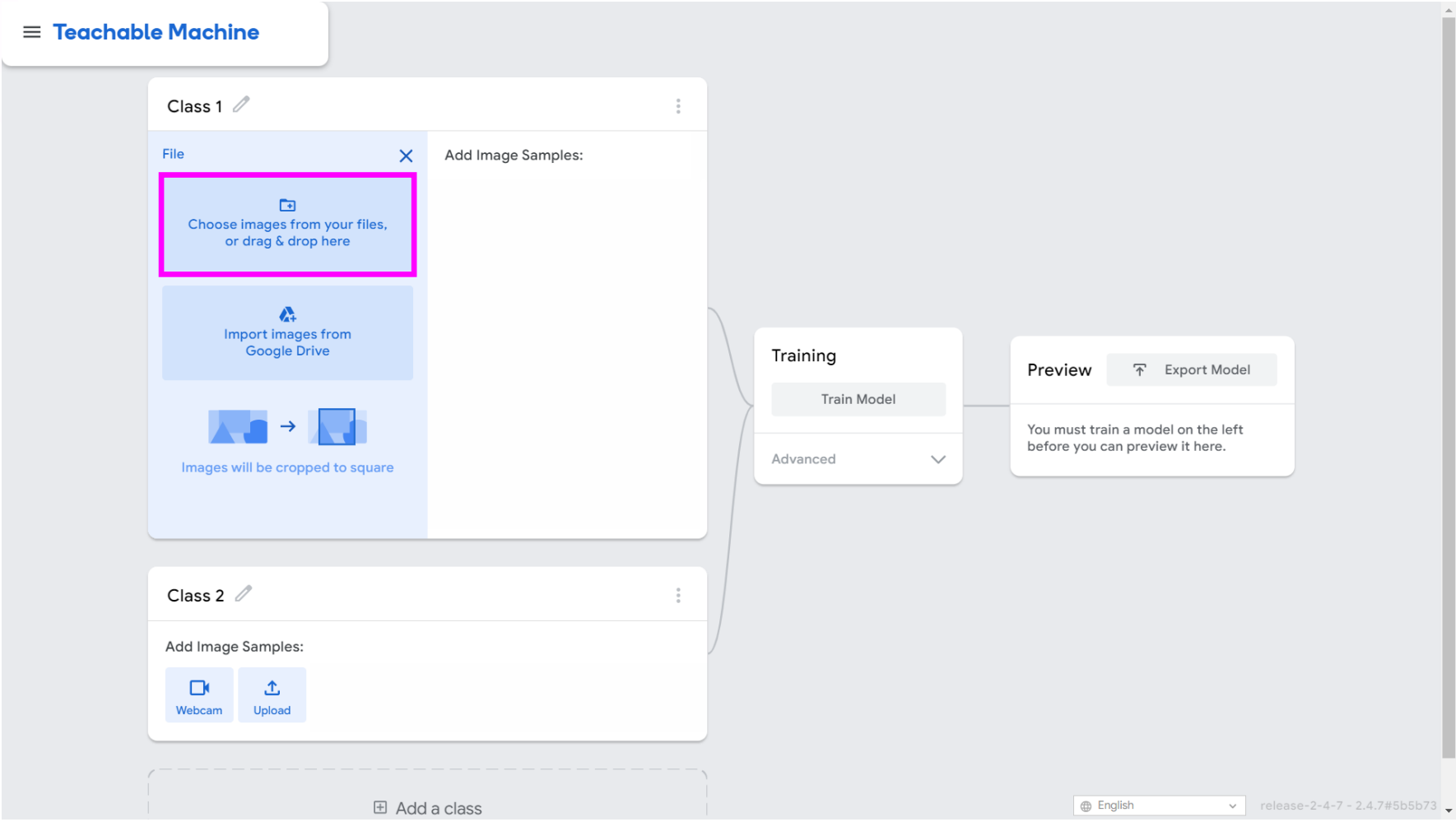
© 2026 Enos Chou

Modeling ft. Teachable Machine

1. 建立模型

   b. 匯入資料

      AE

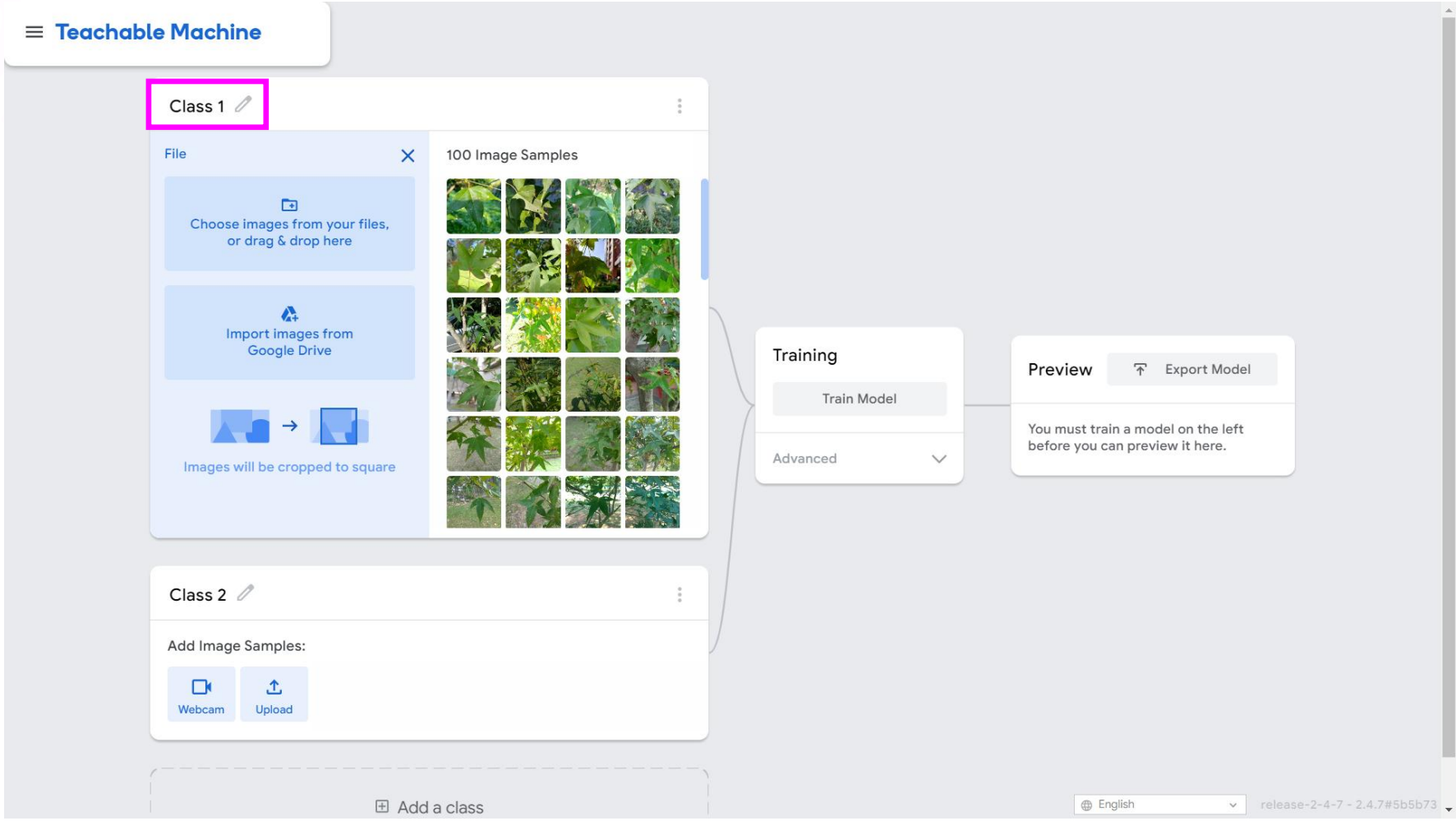


Modeling ft. Teachable Machine

1. 建立模型

   b. 匯入資料

      AE



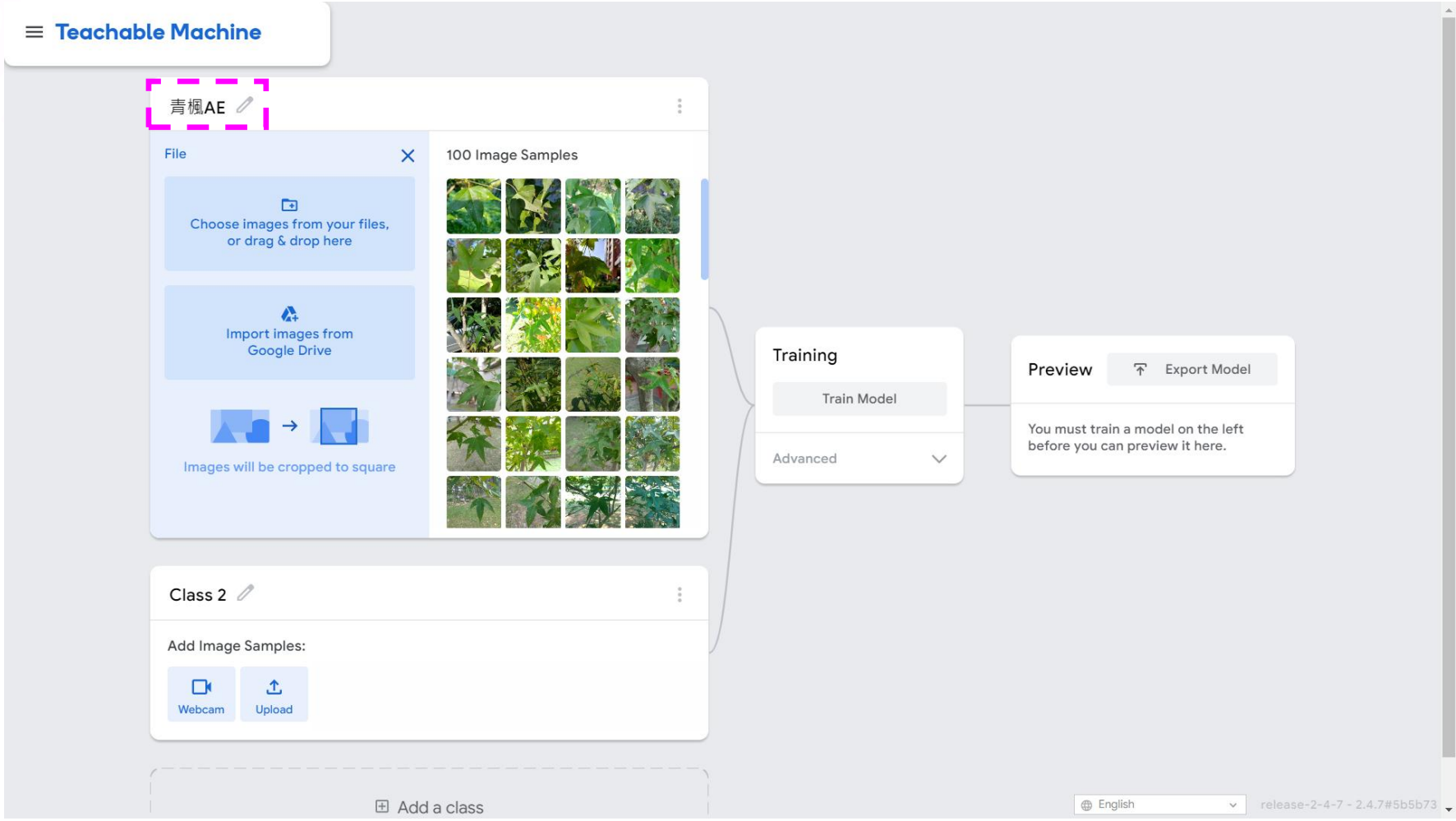
© 2026 Enos Chou

Modeling ft. Teachable Machine

1. 建立模型

   b. 匯入資料

      AE

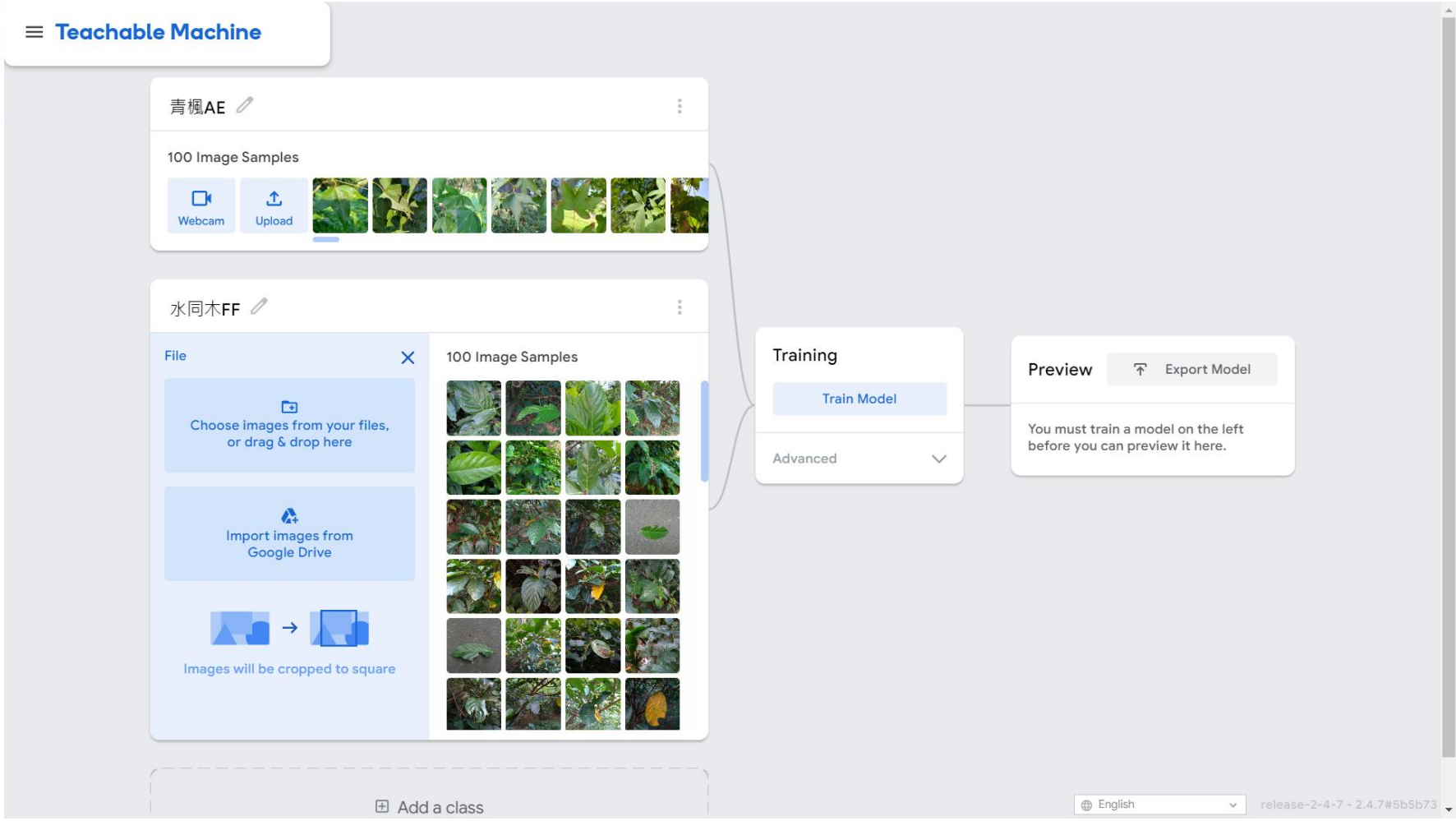


Modeling ft. Teachable Machine

1. 建立模型

   b. 匯入資料

      FF

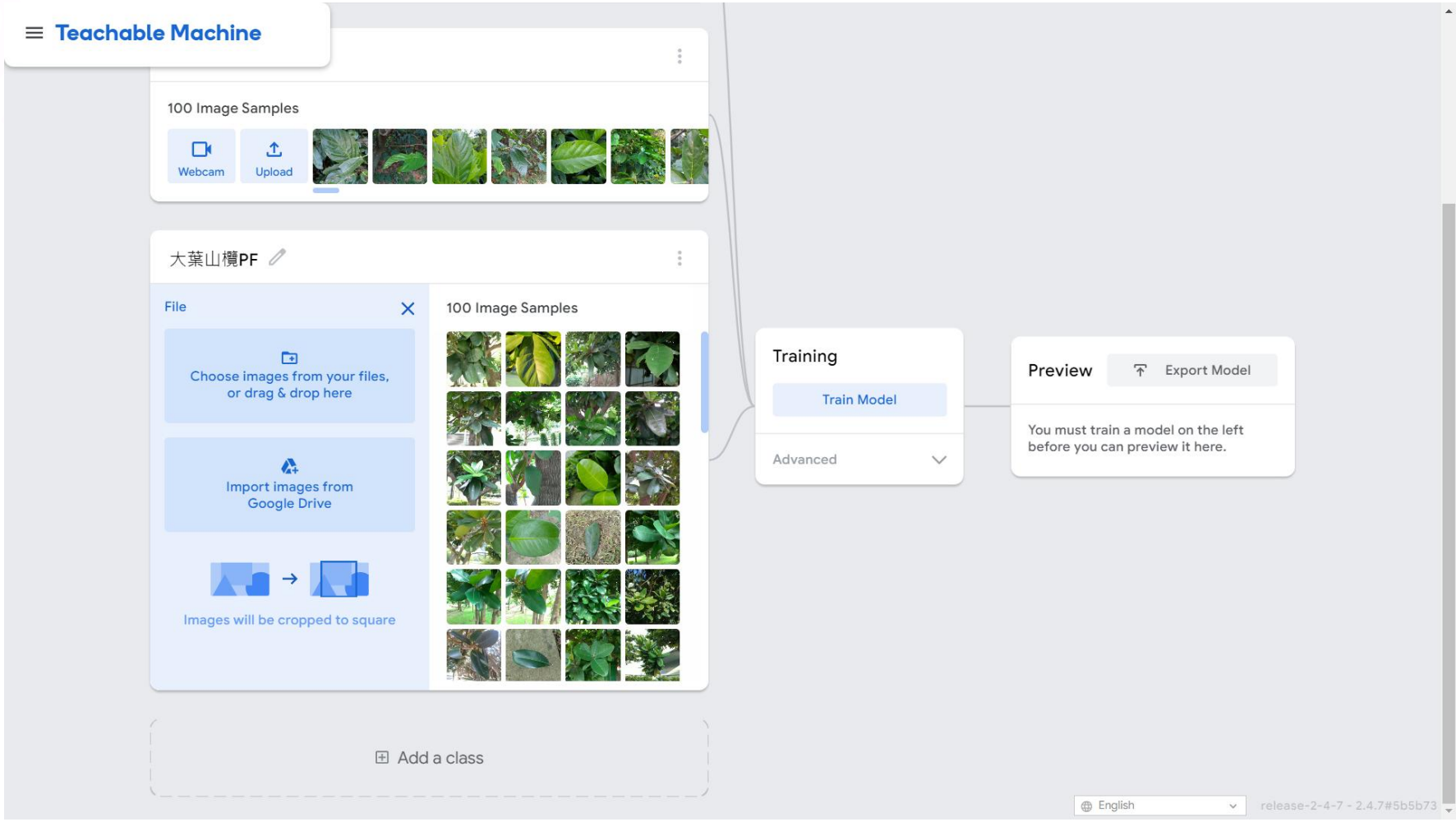


Modeling ft. Teachable Machine

1. 建立模型

   b. 匯入資料

      PF

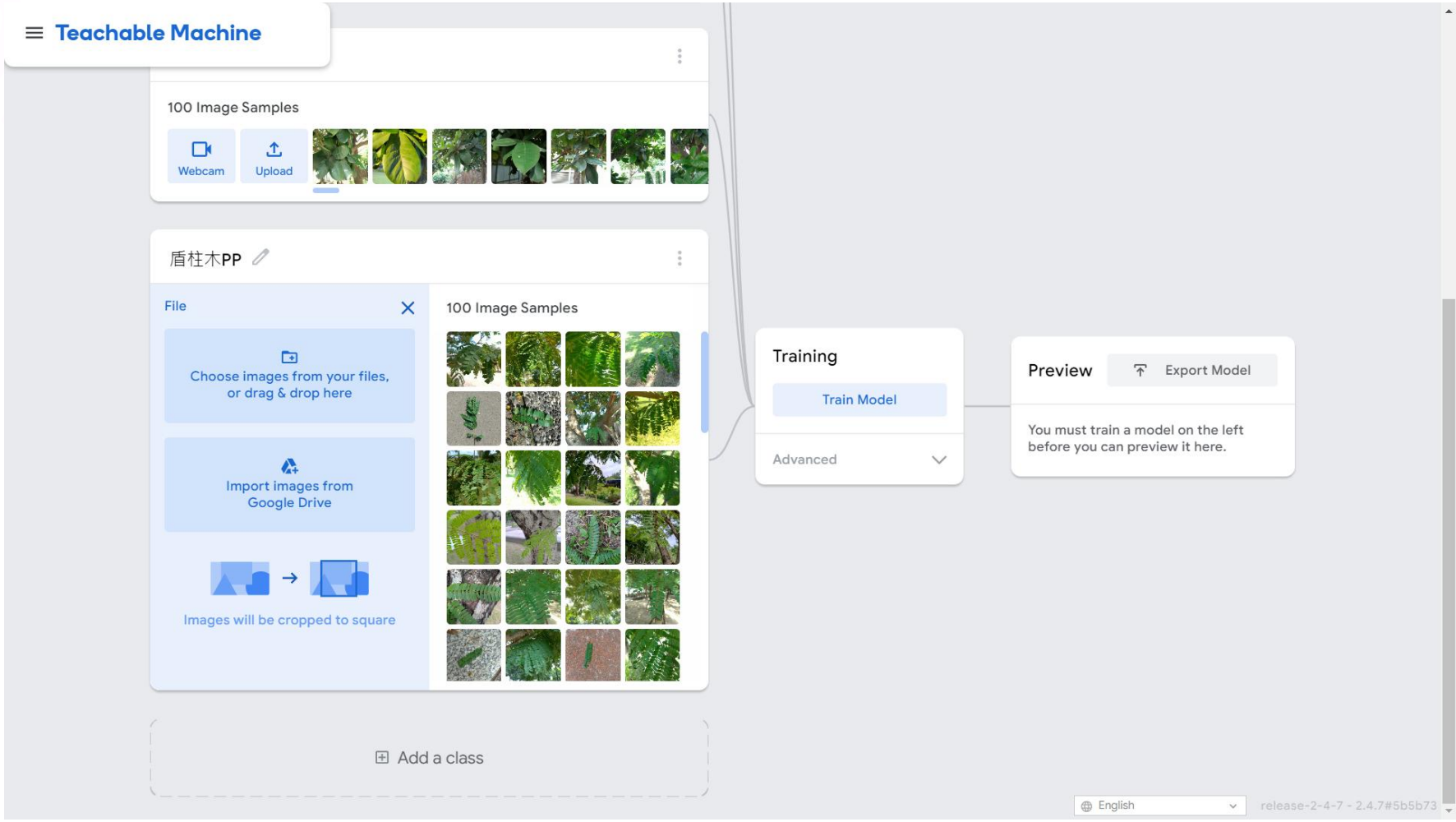


Modeling ft. Teachable Machine

1. 建立模型

   b. 匯入資料

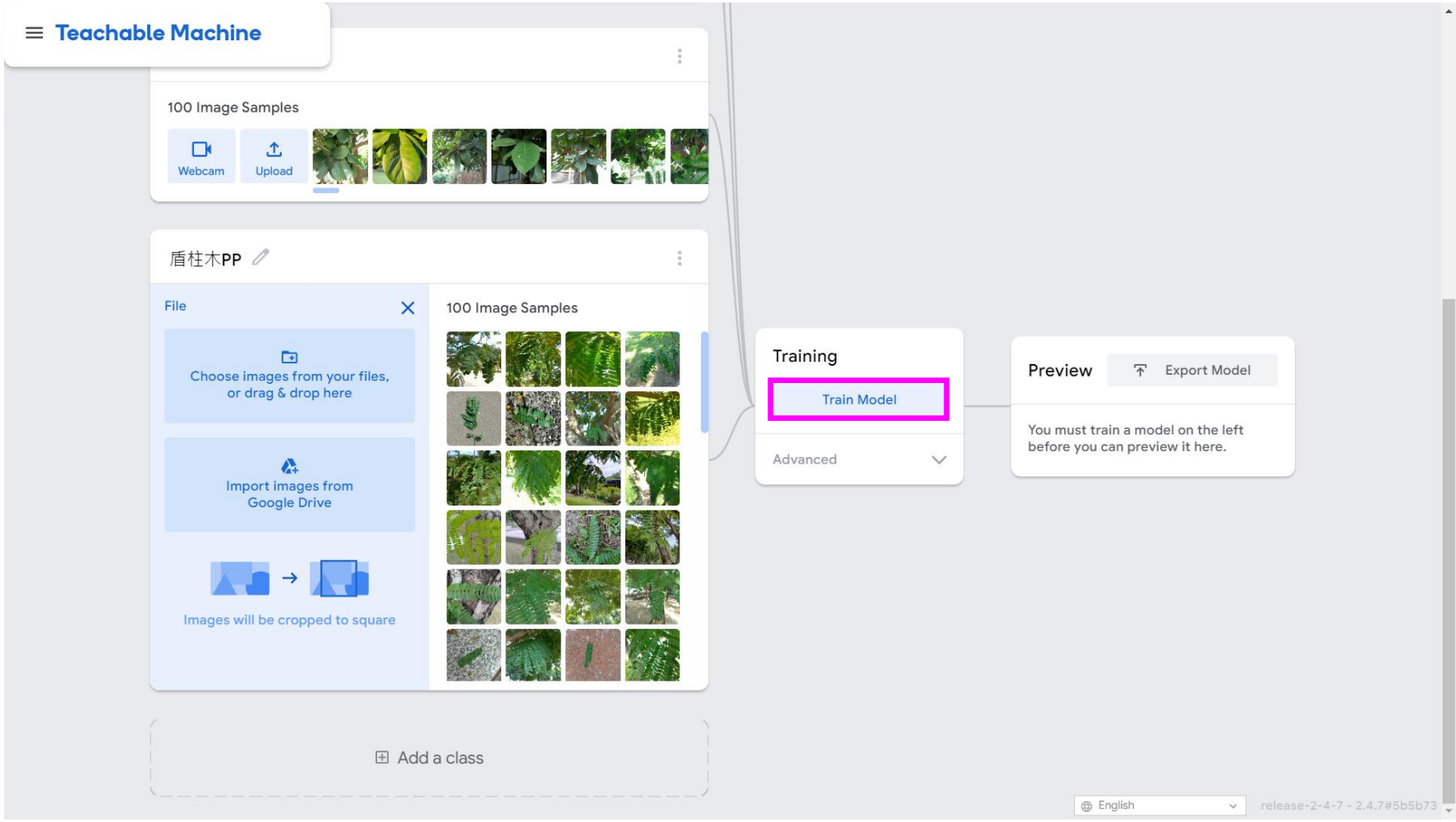
      PP



Modeling ft. Teachable Machine

1. 建立模型

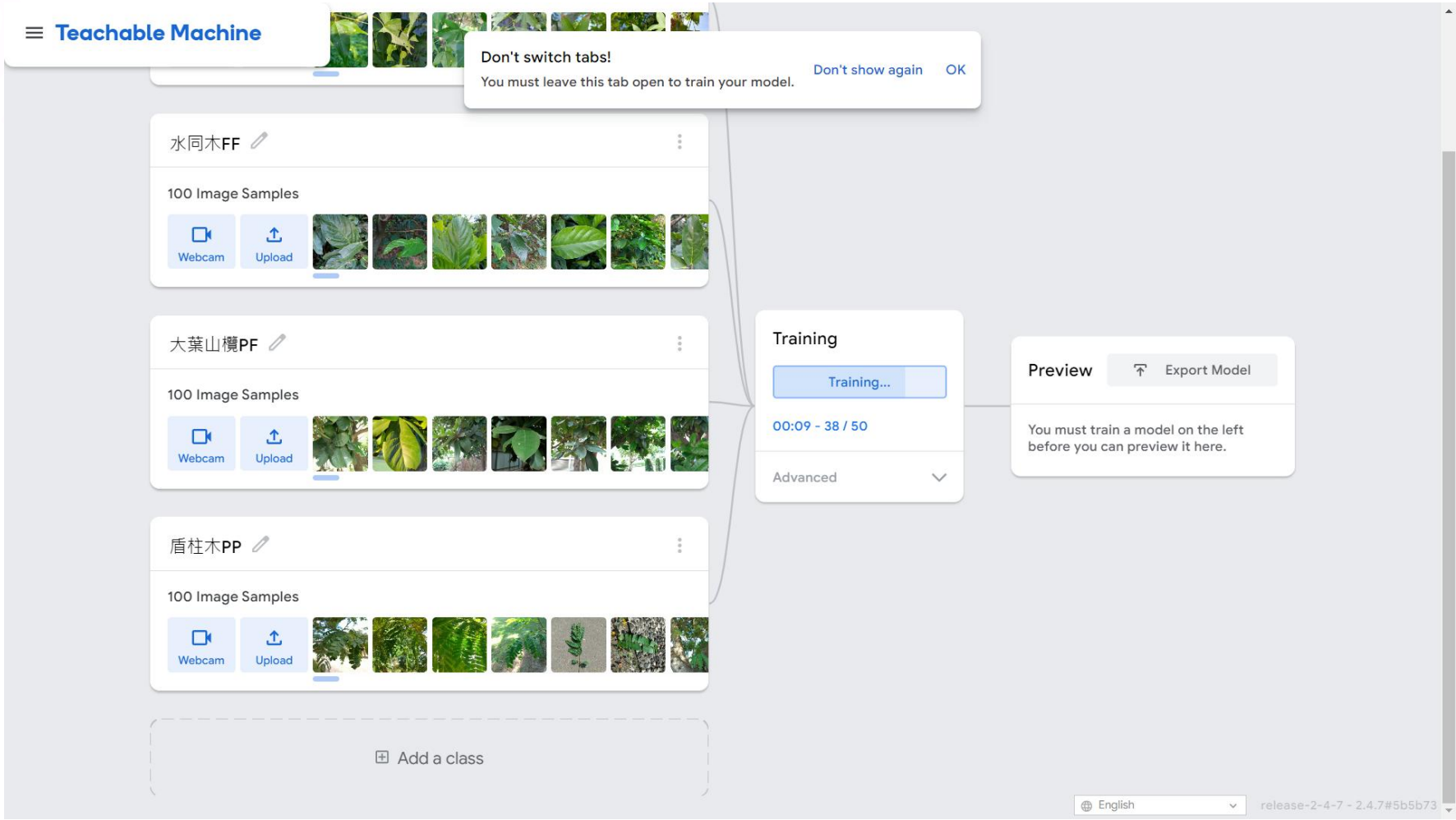
c. 訓練模型



Modeling ft. Teachable Machine

1. 建立模型

   c. 訓練模型

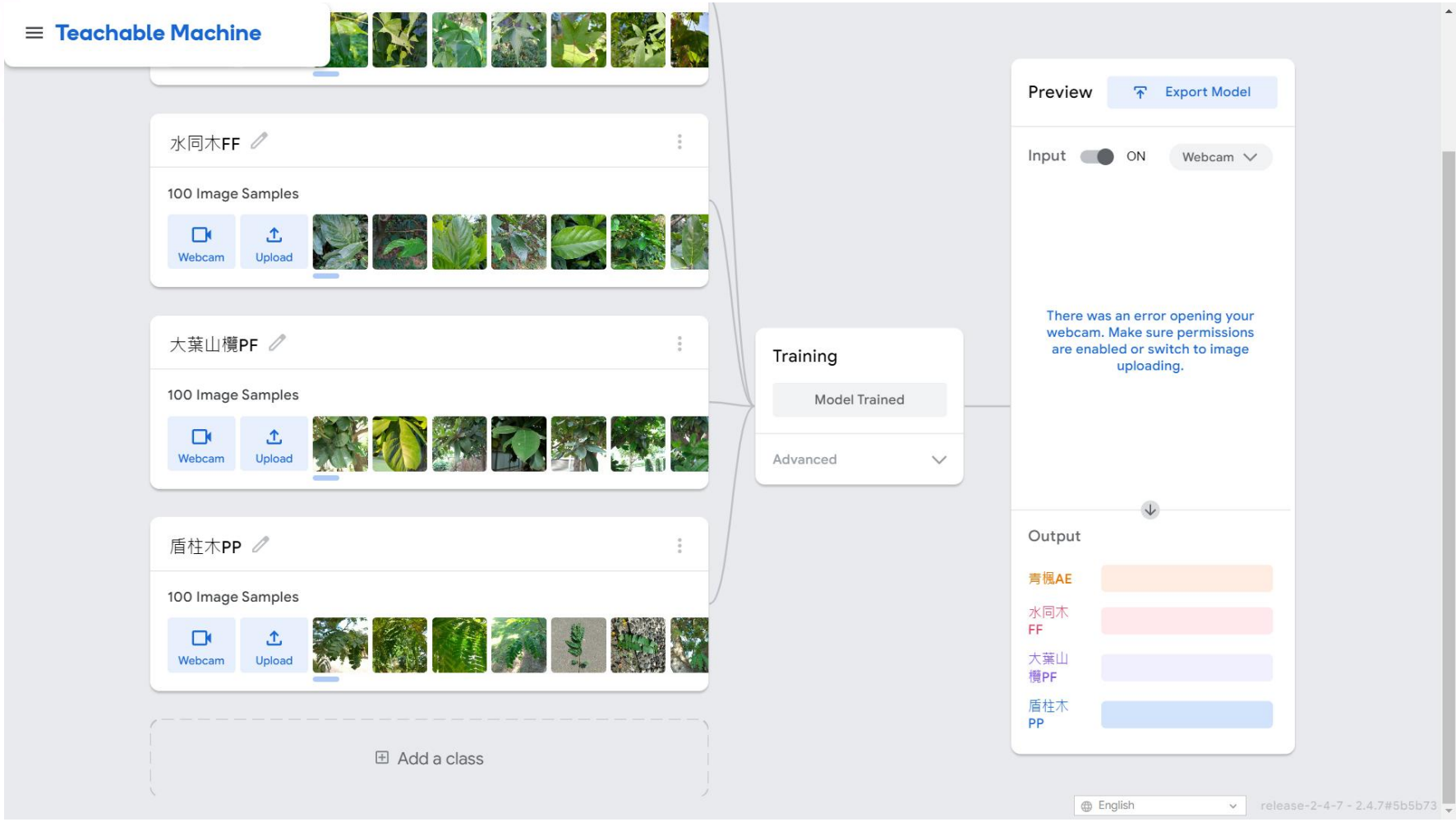


© 2026 Enos Chou

Modeling ft. Teachable Machine

1. 建立模型

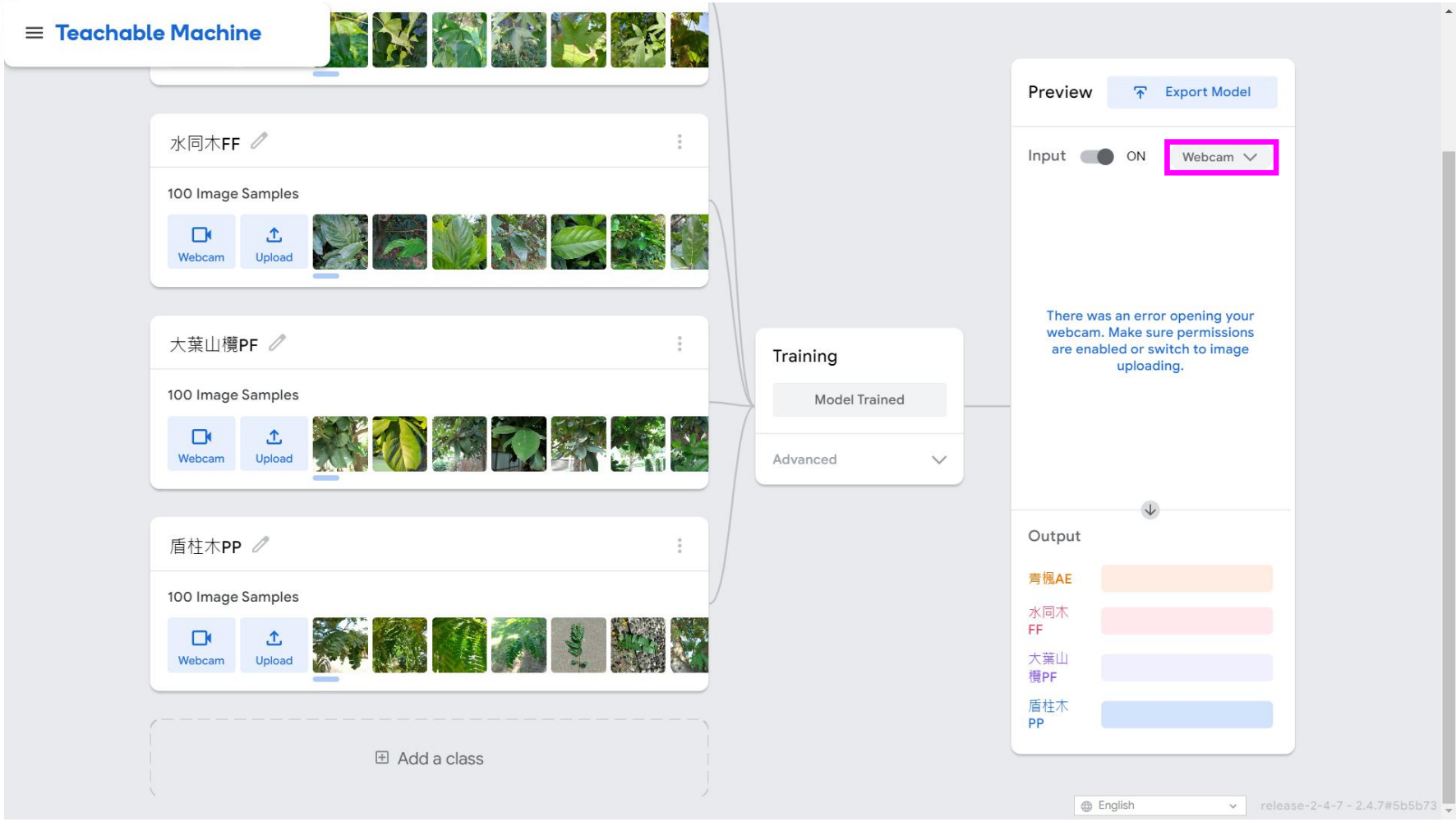
   c. 訓練模型



Modeling ft. Teachable Machine

1. 建立模型

   d. 測試模型

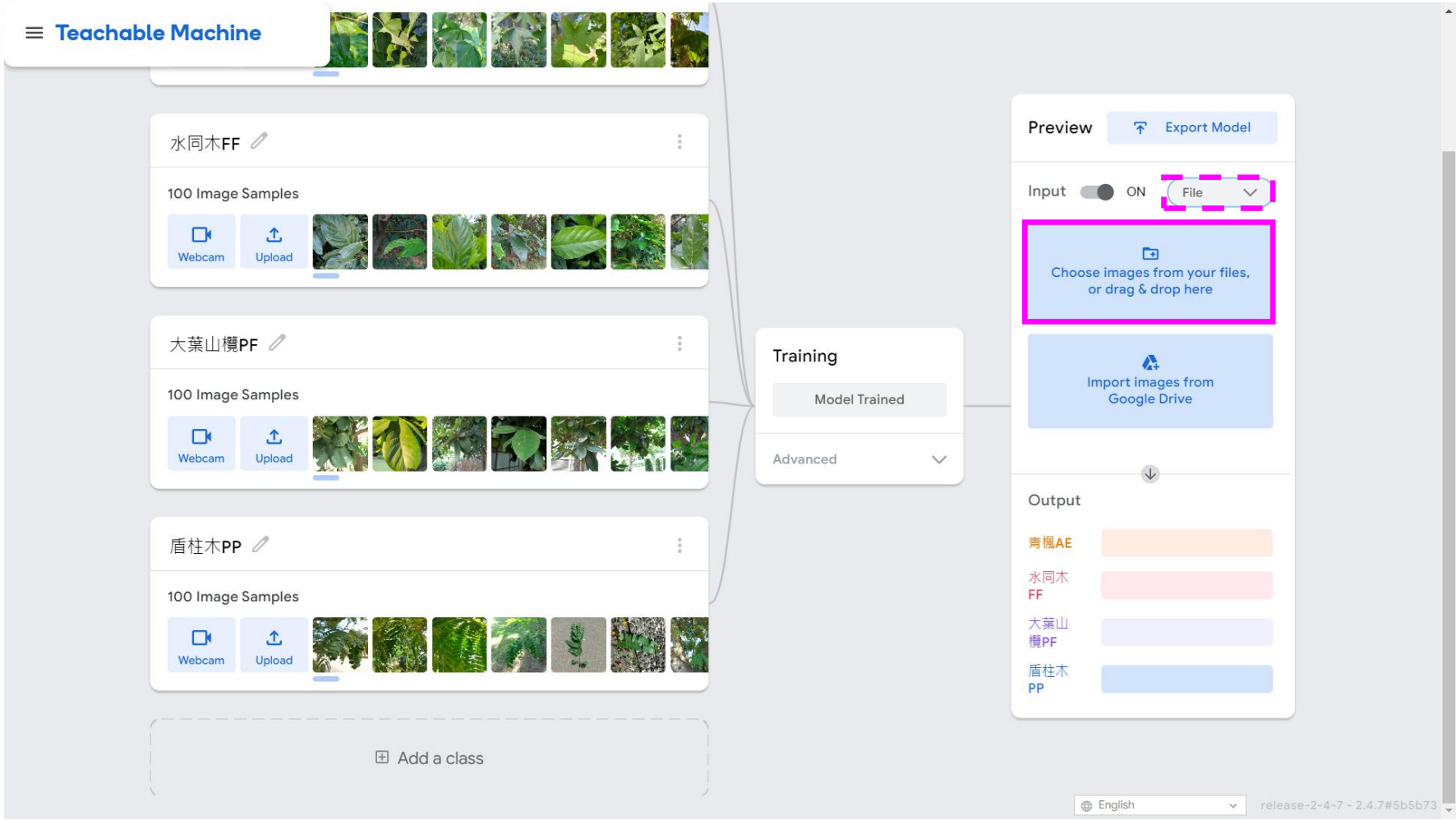


© 2026 Enos Chou

Modeling ft. Teachable Machine

1. 建立模型

   d. 測試模型

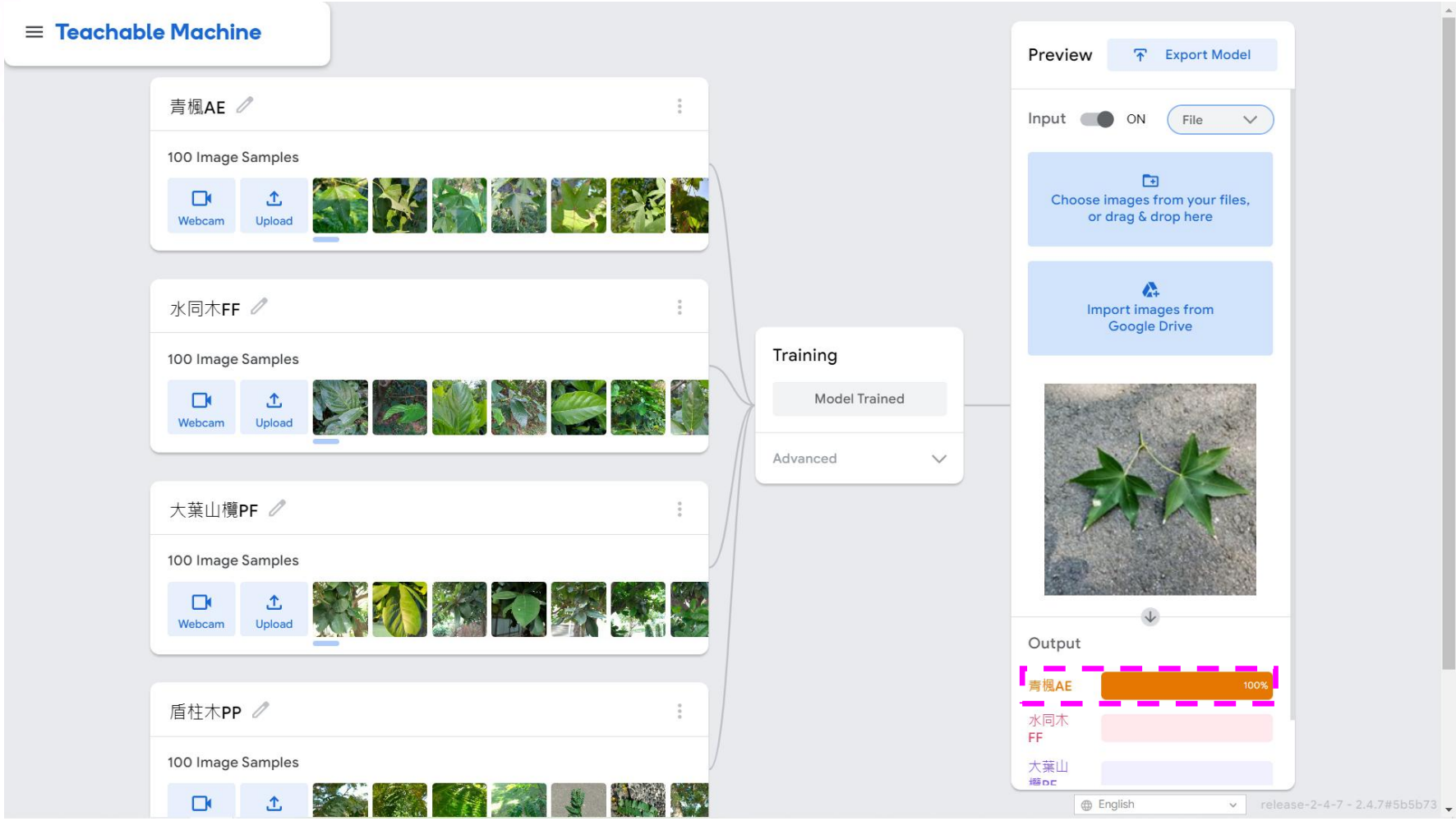


Modeling ft. Teachable Machine

1. 建立模型

   d. 測試模型

      ae.jpg



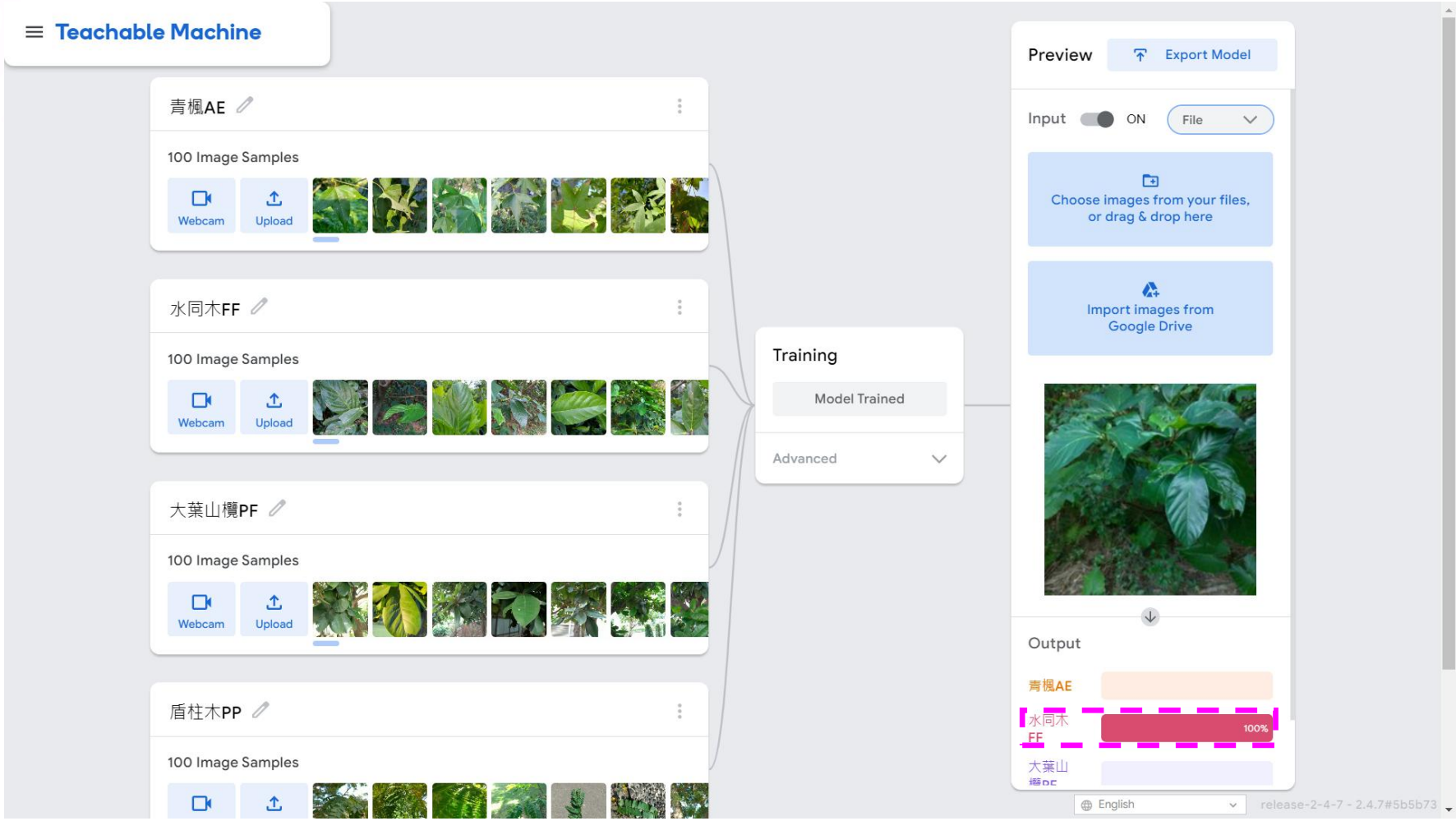
© 2026 Enos Chou

Modeling ft. Teachable Machine

1. 建立模型

   d. 測試模型

      ff.jpg



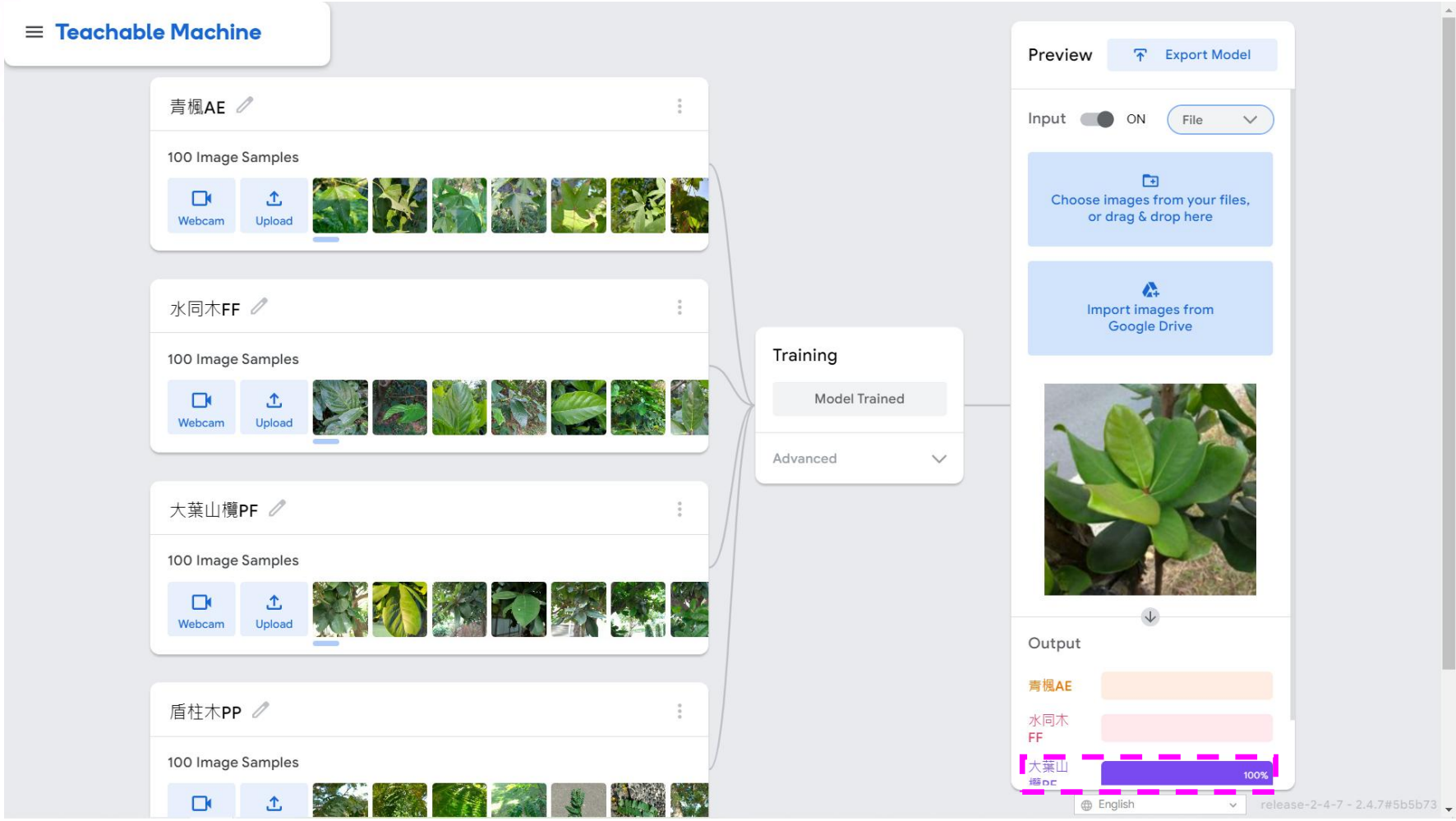
© 2026 Enos Chou

Modeling ft. Teachable Machine

1. 建立模型

   d. 測試模型

   pf.jpg



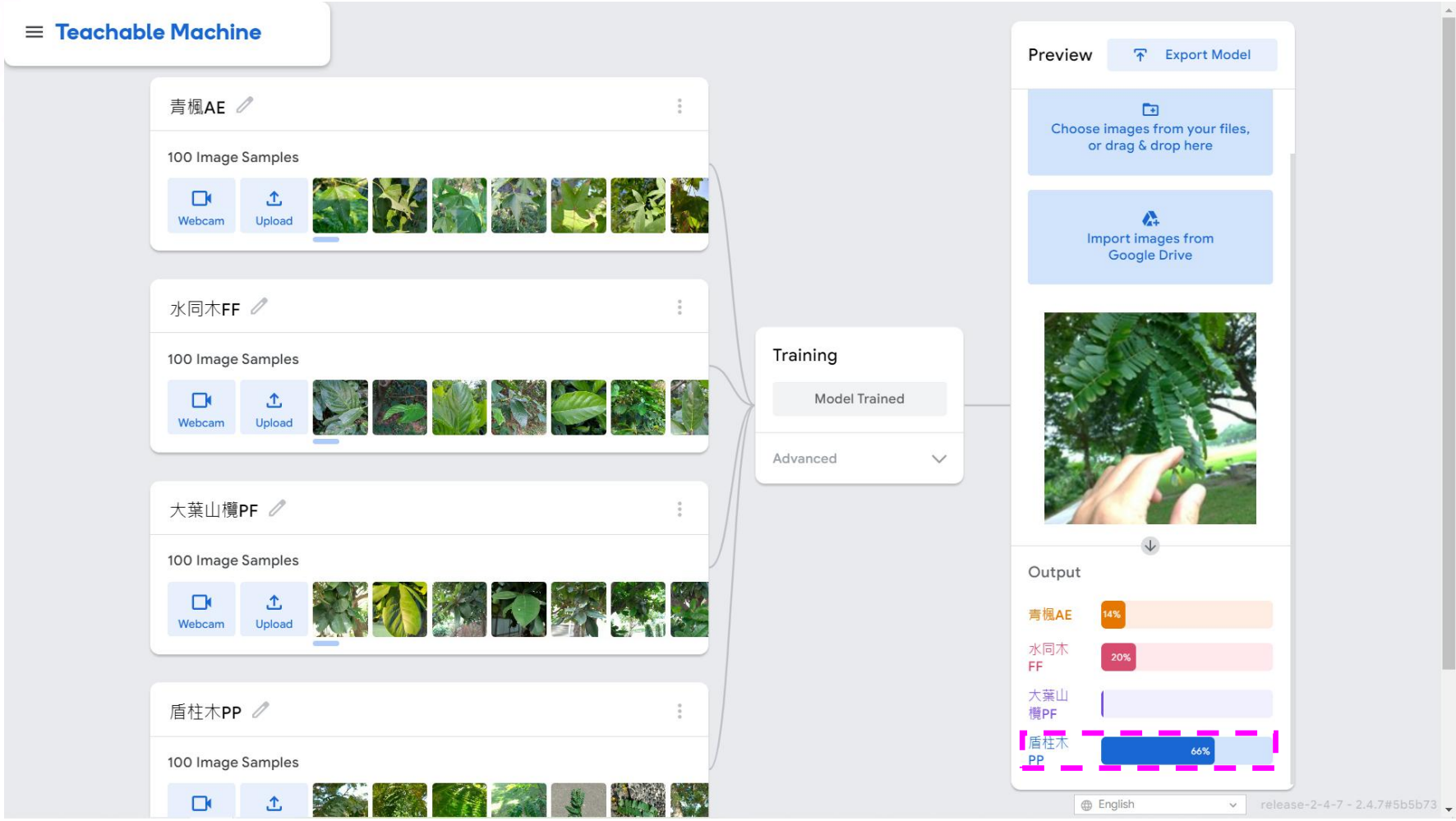
© 2026 Enos Chou

Modeling ft. Teachable Machine

1. 建立模型

   d. 測試模型

   pp.jpg

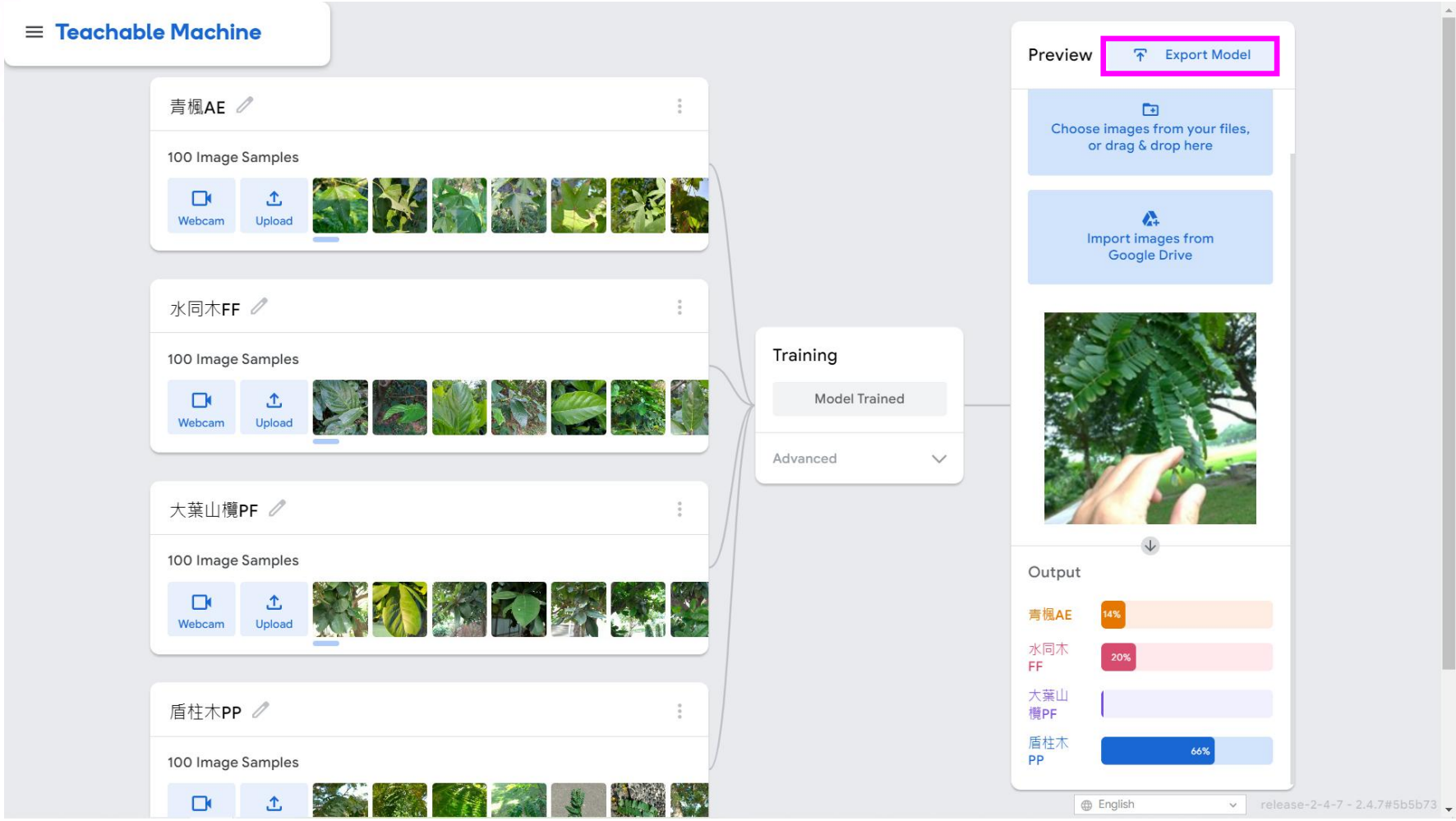


© 2026 Enos Chou

Modeling ft. Teachable Machine

1. 建立模型

   e. 匯出模型



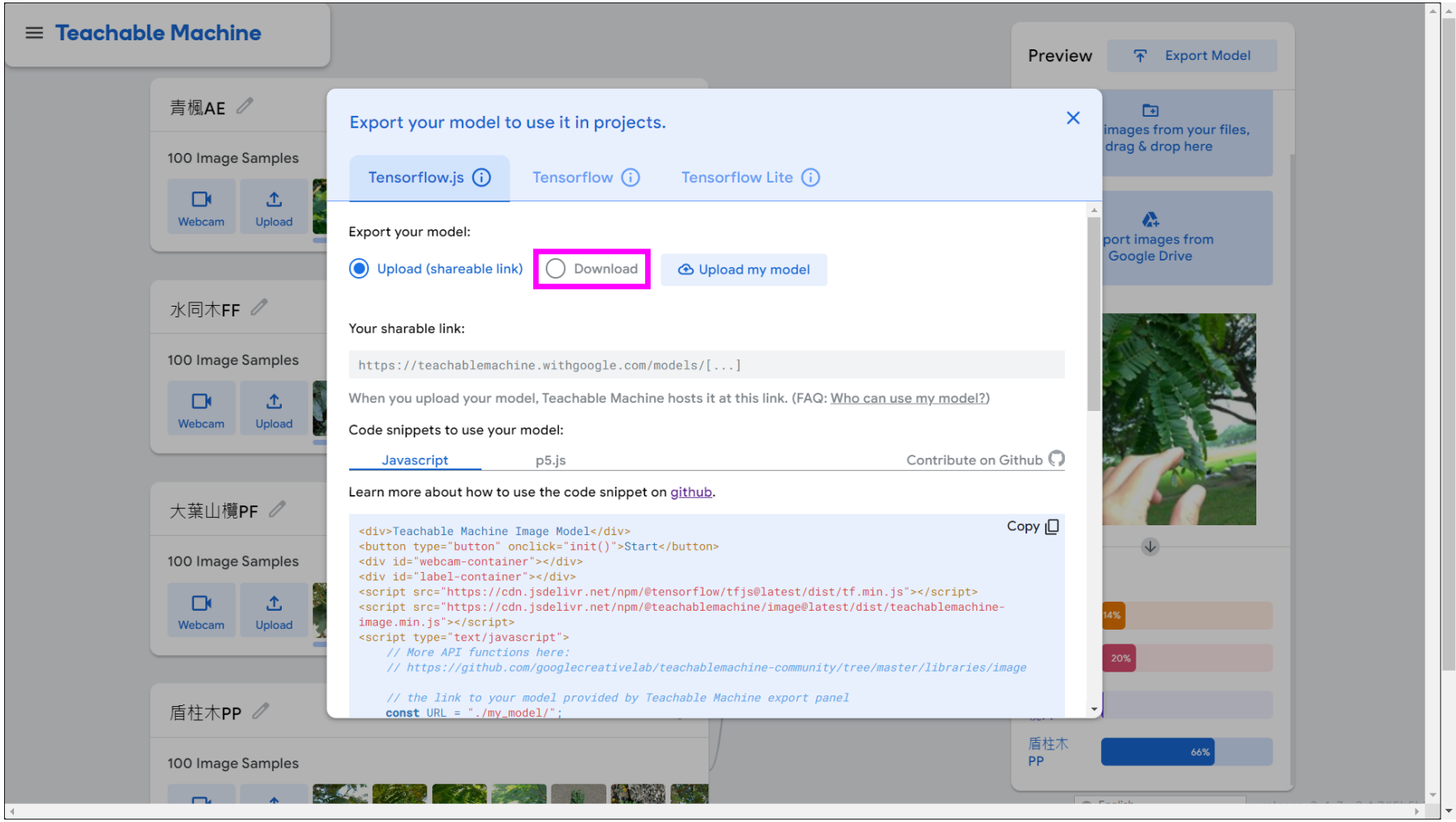
© 2026 Enos Chou

Modeling ft. Teachable Machine

1. 建立模型

e. 匯出模型

TensorFlow.js

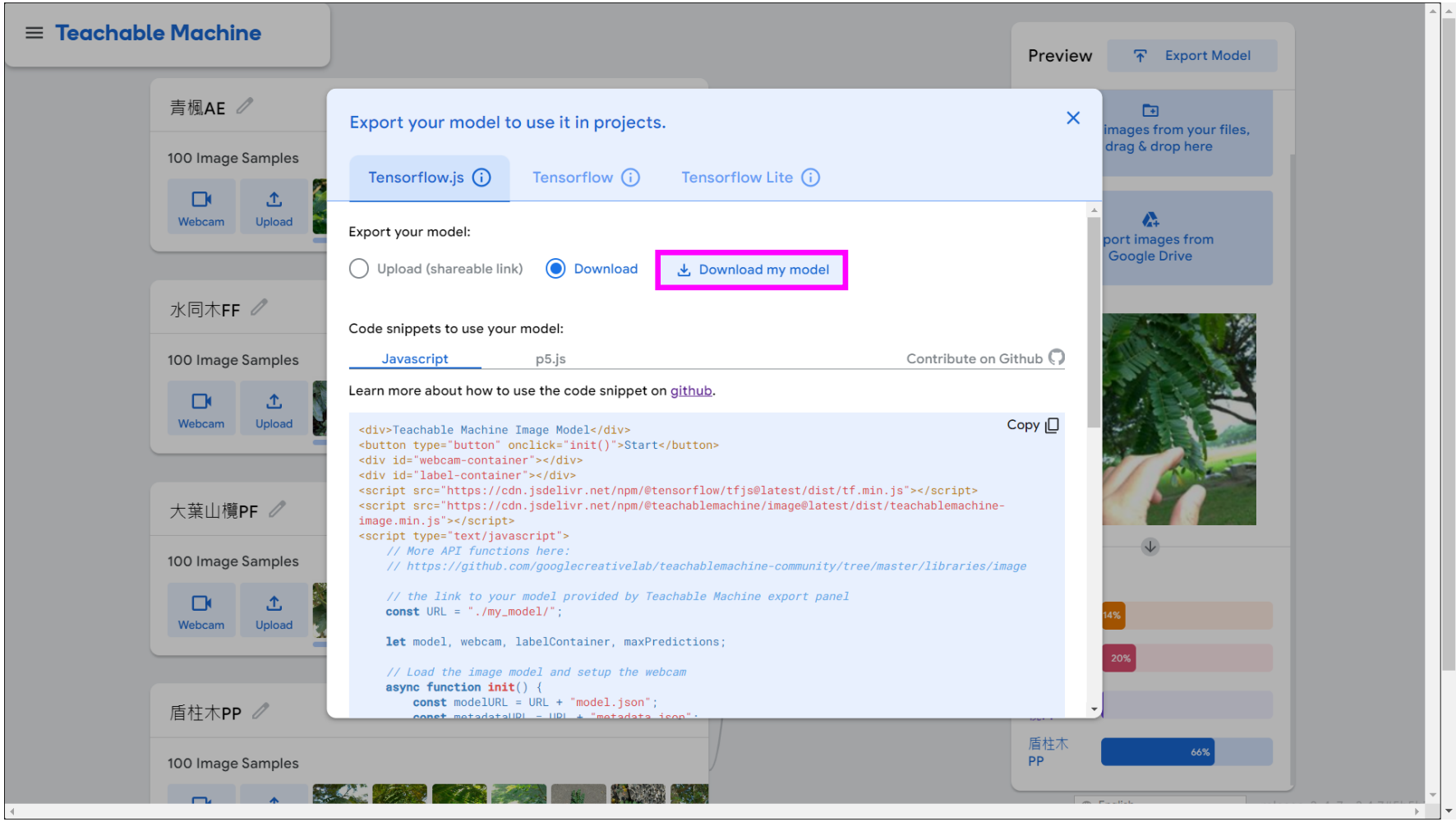


Modeling ft. Teachable Machine

1. 建立模型

   e. 匯出模型

      TensorFlow.js (JavaScript)

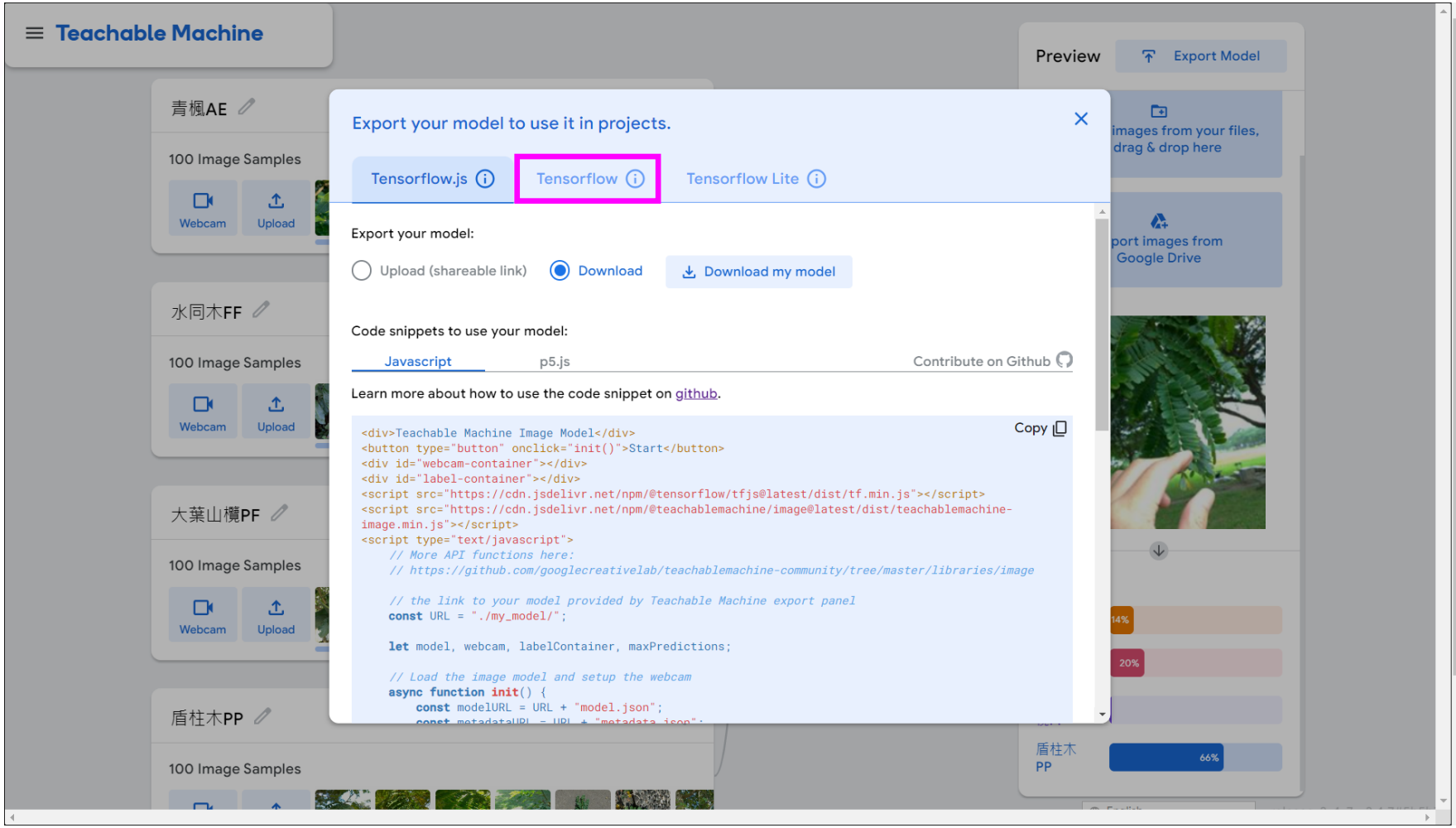


Modeling ft. Teachable Machine

1. 建立模型

   e. 匯出模型

      TensorFlow

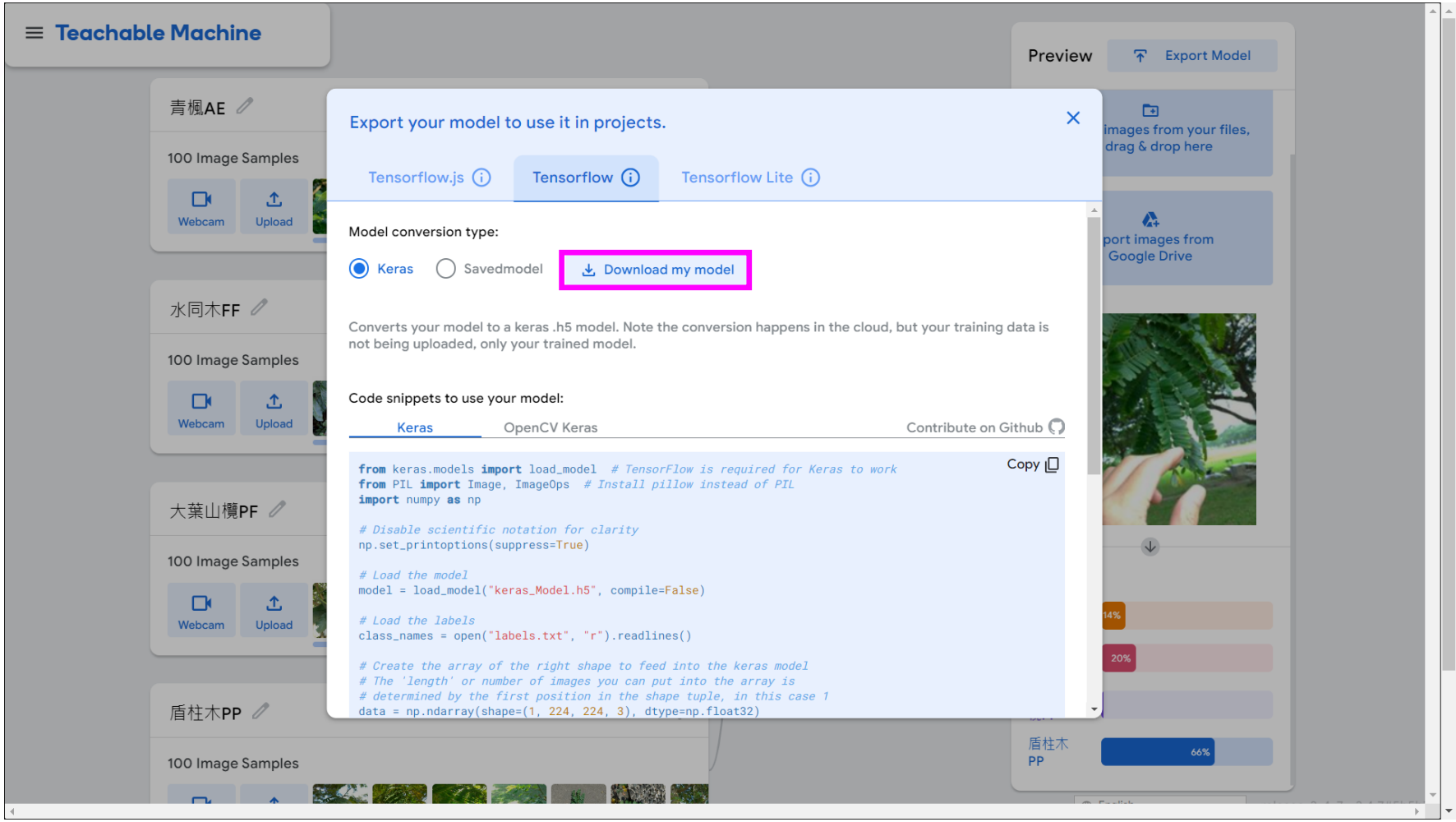


Modeling ft. Teachable Machine

1. 建立模型

e. 匯出模型

TensorFlow (Keras)

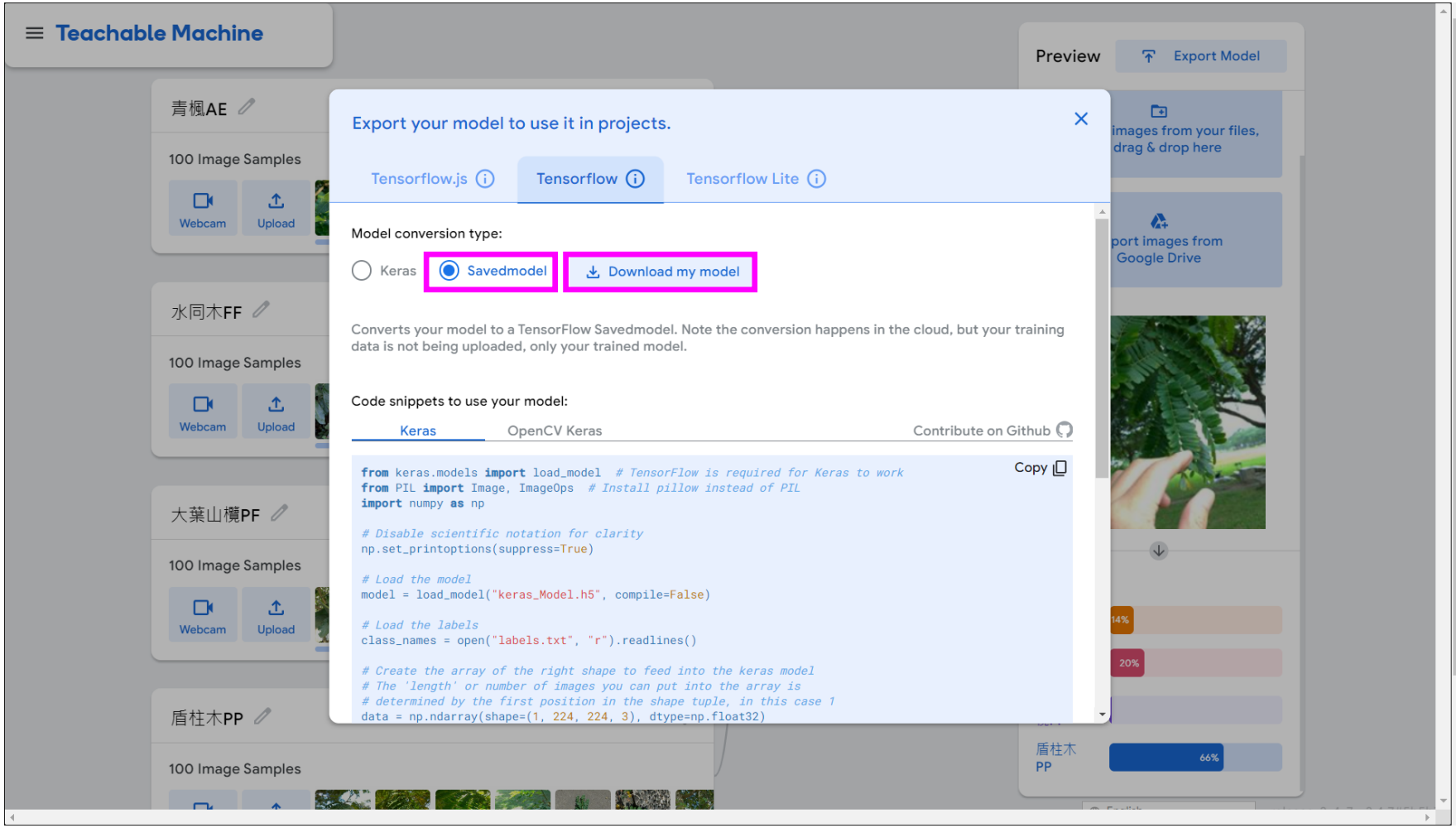


Modeling ft. Teachable Machine

1. 建立模型

e. 匯出模型

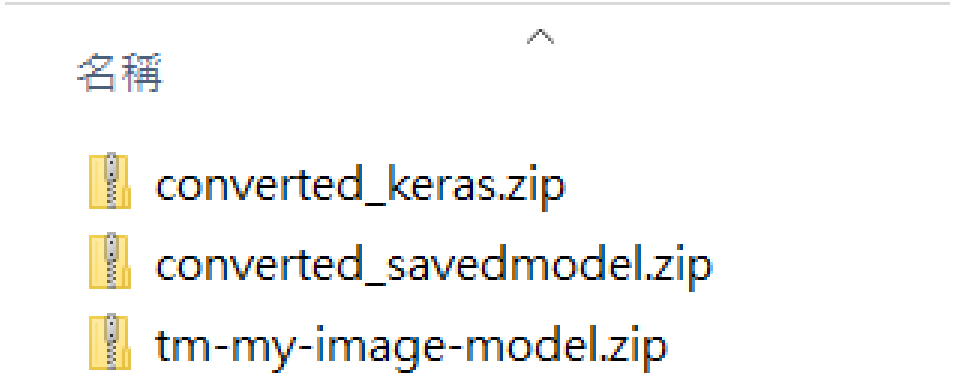
TensorFlow
(Savedmodel)



Modeling ft. Teachable Machine

1. 建立模型

e. 匯出模型



© 2026 Enos Chou

Application ft. Teachable Machine

2. 整合應用

a. Inference

① load model to classify

```
from tensorflow.keras.models import load_model
from PIL import ImageOps
import numpy as np
```

```
model = load_model('YOUR_HDF5_FILE')

def classify(image):
    image = ImageOps.fit(image, (224, 224))
    image = (np.expand_dims(image, axis=0).astype(np.float32) / 127.5) - 1
    prediction = model.predict(image)
    return labels(np.argmax(prediction))
```


© 2026 Enos Chou

Application ft. Teachable Machine

2. 整合應用

a. Inference

② render result

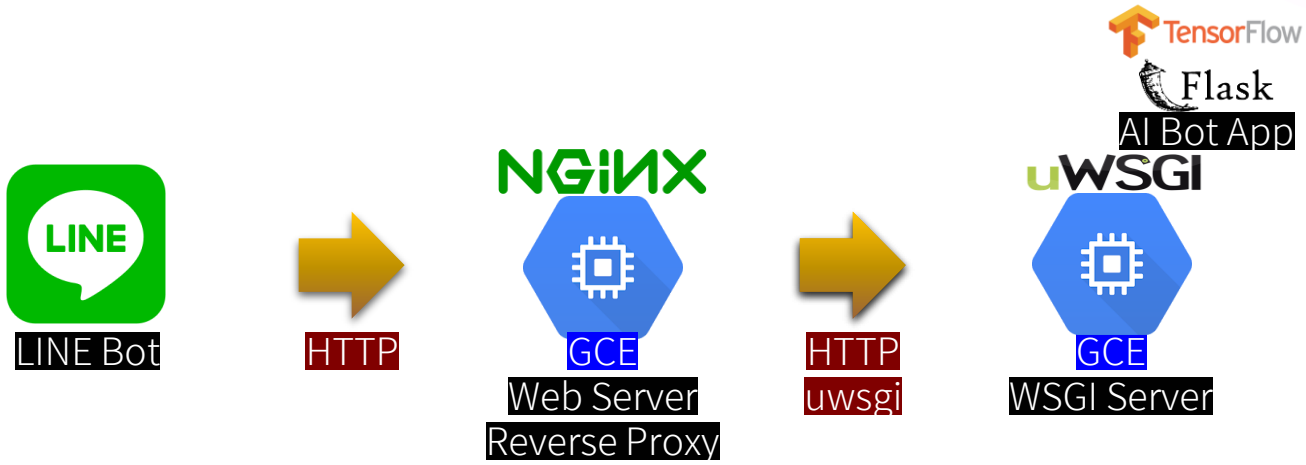
```
labels_file = 'YOUR_LABELS_FILE'

def labels(i):
    with open(labels_file, 'r', encoding='utf-8') as f:
        row = f.readline()
        while row:
            row = row.strip('\n').split()
            if len(row) > 1 and int(row[0]) == i:
                return row[1]
            row = f.readline()
    return 'unknown'
```


© 2026 Enos Chou

Application ft. Teachable Machine

2. 整合應用

b. to LINE Bot



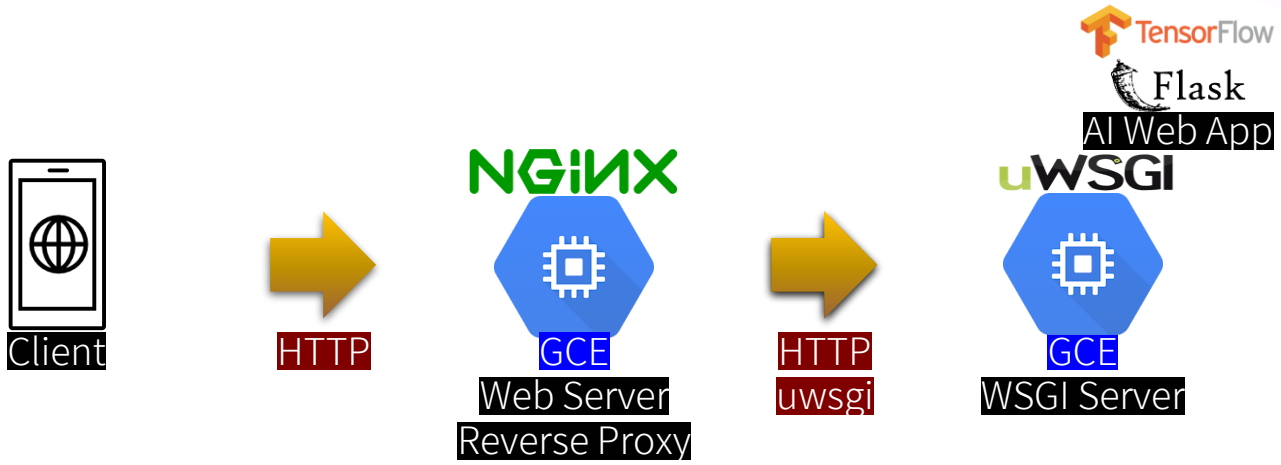
© 2026 Enos Chou

Your Turn - TM Web

運用 Teachable Machine 建立的模型，以 Flask 整合成為 AI 網頁應用 (45 mins)

請選擇圖片檔案: 選擇檔案 未選擇任何檔案 預測

標籤: 青楓AE/ 信心: 0.999



請選擇圖片檔案: 選擇檔案 pf.jpg

標籤: 大葉山欖PF/ 信心: 1.000

© 2026 Enos Chou

Web & API 整合應用

ft. Flask

© 2026 Enos Chou

Web

HTML

目的

- 規範網頁格式以呈現網頁內容

規格

- HTML5

HTTP

目的

① 規範網際網路傳輸協定以傳輸網頁
② 規範網際網路傳輸協定以傳輸資料

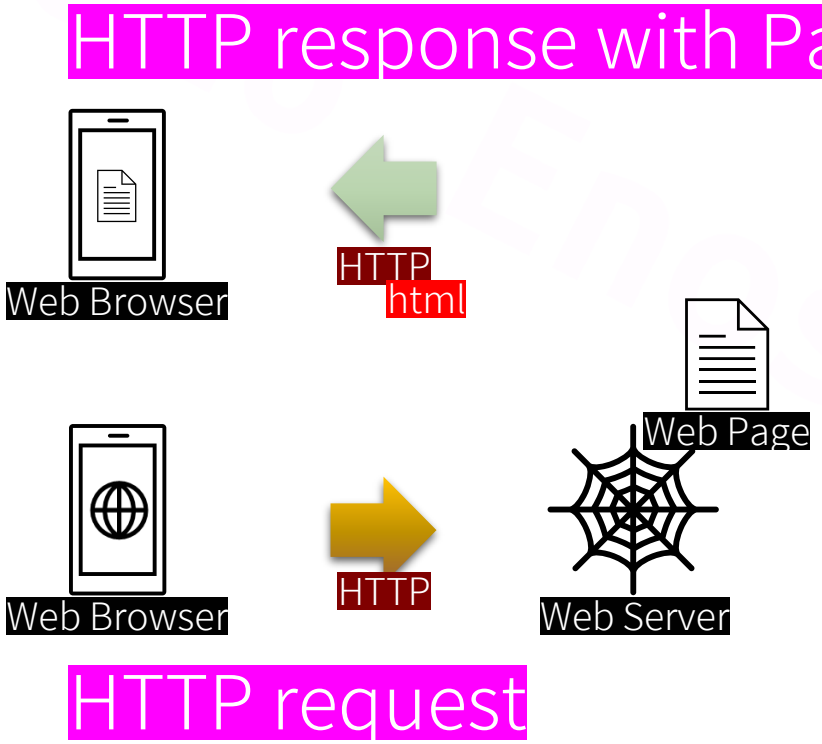
規格

- RFC 2616, RFC 7540

© 2026 Enos Chou

Web

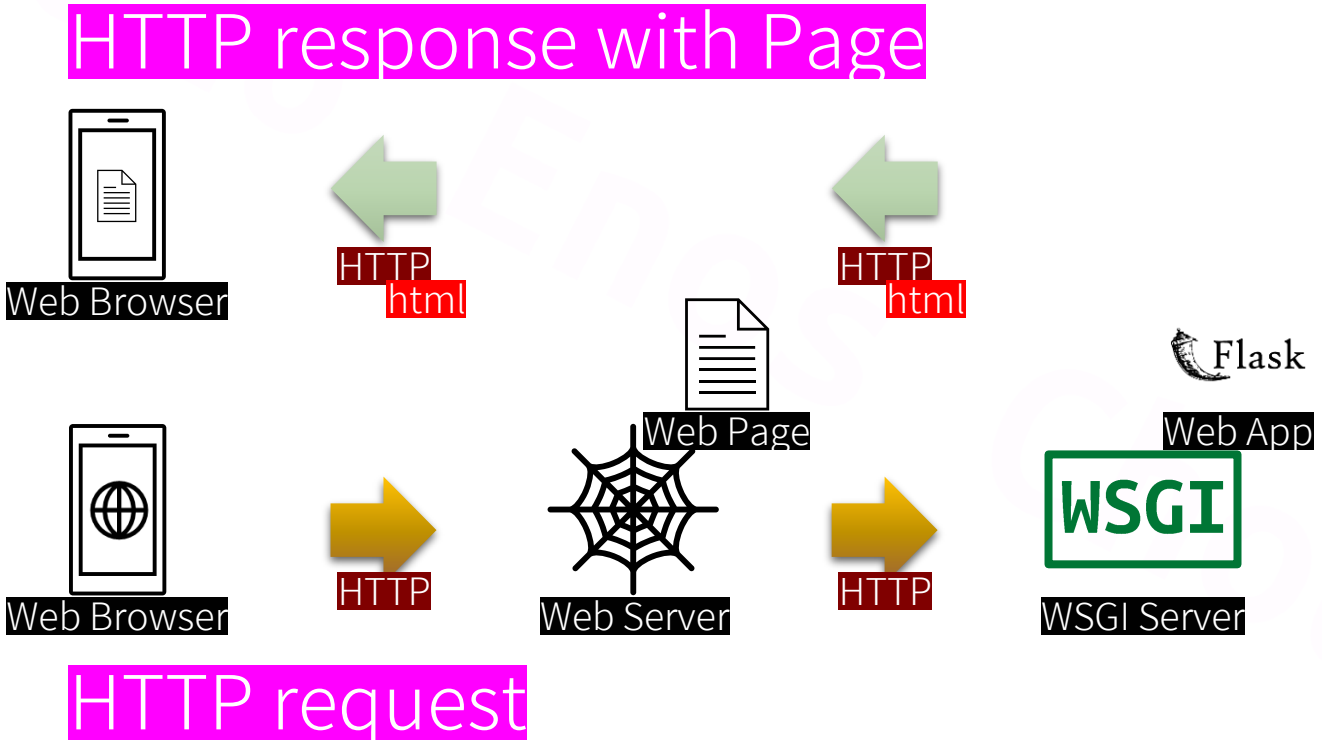
Web Concept



© 2026 Enos Chou

Web

Web Concept



Web ft. Flask

Web in Flask Server

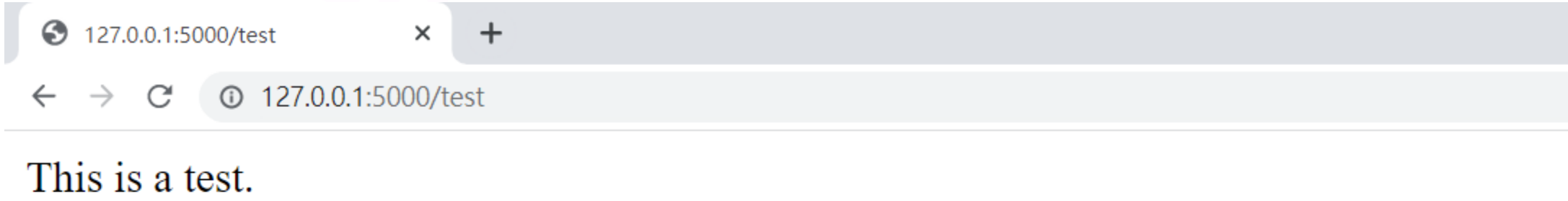
```python
from flask import Flask

app = Flask(__name__)

@app.route('/test')
def test():
    return "This is a test."

app.run()
```

```
* Serving Flask app "__main__" (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
```

© 2026 Enos Chou

Web

Web browsed by Client



Web ft. Flask

Web in Flask Server

```
from flask import Flask

app = Flask(__name__)

@app.route("/test")
def test():
    return "This is a test."

app.run()
```

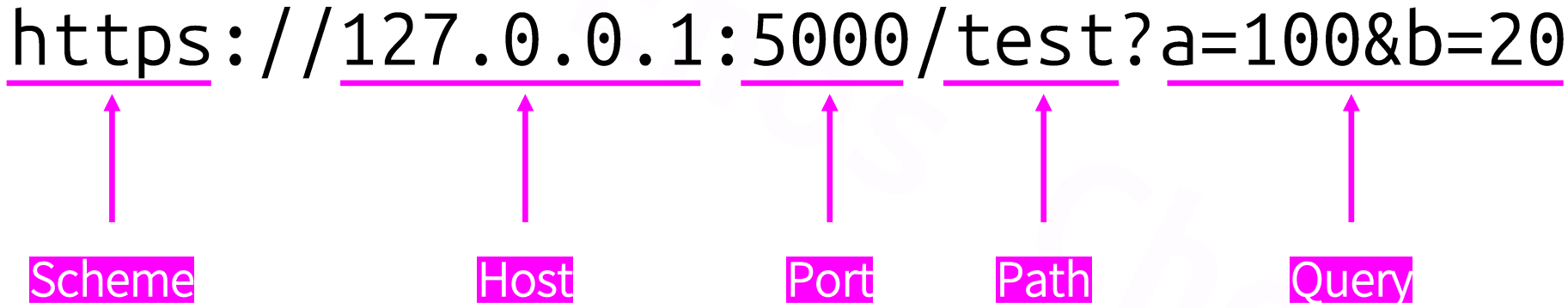
URL Path

© 2026 Enos Chou

Web

URL

```
@app.route("/test")
```



© 2026 Enos Chou

Web ft. Flask

Web in Flask Server

```
from flask import Flask

app = Flask(__name__)

@app.route("/test")
def test():
    return "This is a test."

app.run()
```

HTML Page

© 2026 Enos Chou

Web

HTML 標準語法

```
<html>
    <head>
        <title>HTML Test</title>
    </head>

    <body>
        <font color="red">How are you?</font>
        <br>
        <a href="https://palletsprojects.com/p/flask/">Flask</a>
    </body>
</html>
```

© 2026 Enos Chou

Web

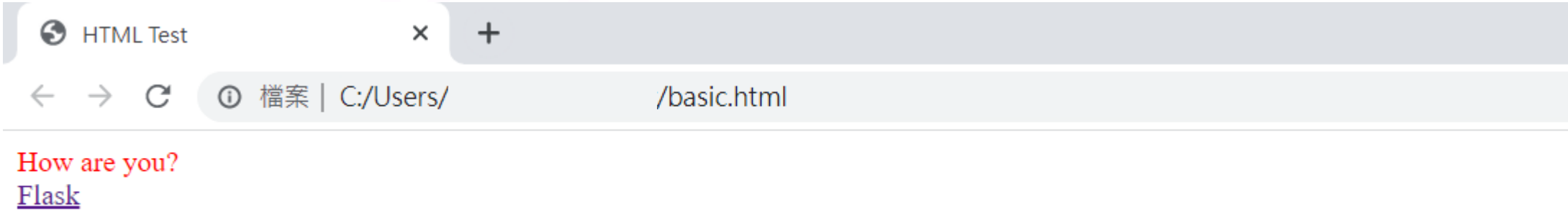
HTML 標準語法

```
<html>
    <head>
        <title>HTML Test</title>
    </head>
    <body>
        <font color="red">How are you?</font>
        <br>
        <a href="https://palletsprojects.com/p/flask/">Flask</a>
    </body>
</html>
```

© 2026 Enos Chou

Web

HTML 標準語法



Web ft. Flask

Web in Flask Server

```
from flask import Flask

app = Flask(__name__)

@app.route("/test")
def test():
    return "This is a test."

app.run()
```

Launch the Web Server

© 2026 Enos Chou

Web ft. Flask

Web in Flask Server

主要參數 in app.run()

- `host` - 預設 `'127.0.0.1'`；對外服務時設為 `'0.0.0.0'`
- `port` - 預設 `5000`

© 2026 Enos Chou

Web ft. Flask for Input

1. Web page for input
2. Web page to Flask
3. Flask to respond

© 2026 Enos Chou

Web ft. Flask for Input

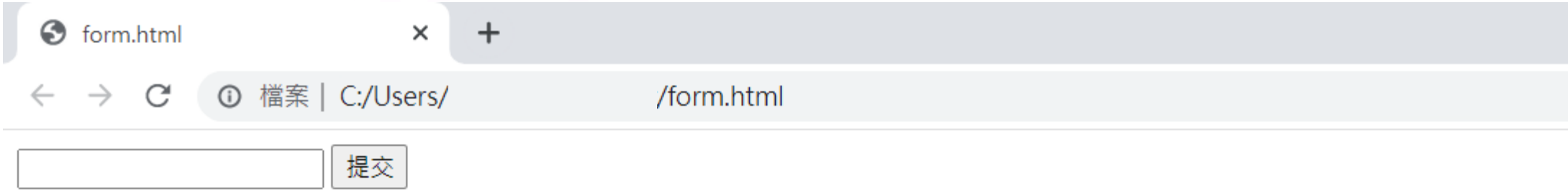
1. Web page for input

```
<form>
    <input type="text">
    <input type="submit">
</form>
```

© 2026 Enos Chou

Web ft. Flask for Input

1. Web page for input



Web ft. Flask for Input

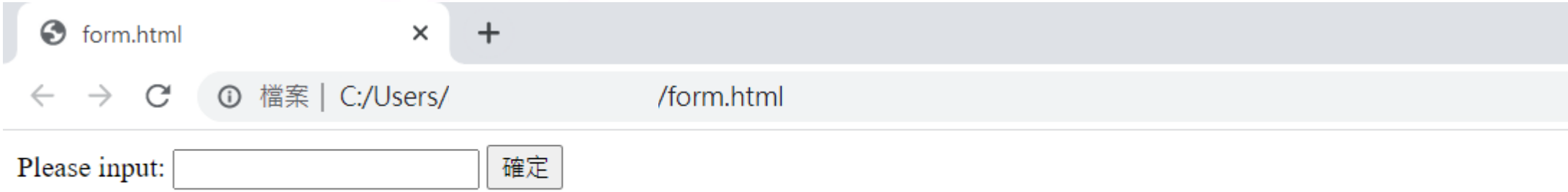
1. Web page for input

```
<form>
    Please input: <input type="text">
    <input type="submit" value="確定">
</form>
```

© 2026 Enos Chou

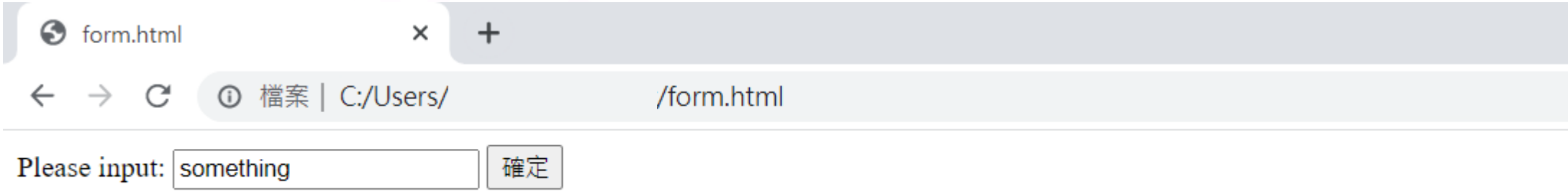
Web ft. Flask for Input

1. Web page for input



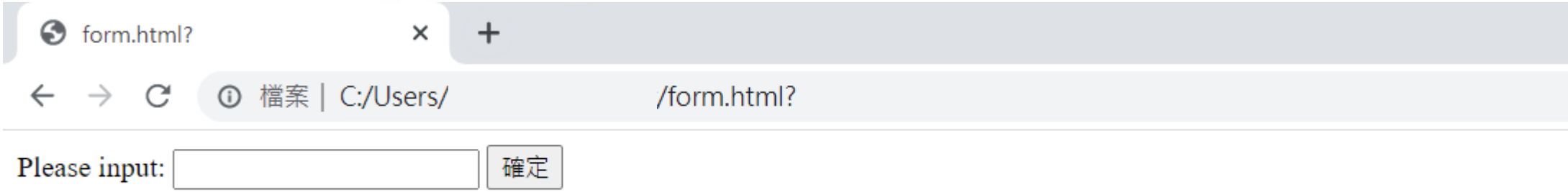
Web ft. Flask for Input

1. Web page for input



Web ft. Flask for Input

1. Web page for input



Web ft. Flask for Input

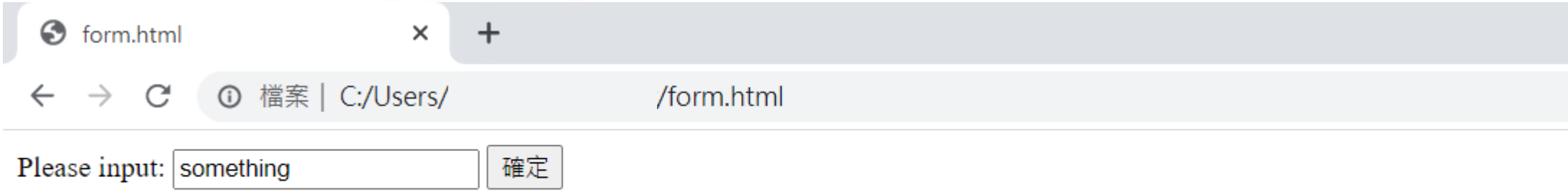
1. Web page for input

```
<form action="topic">
    Please input: <input type="text" name="topic">
    <input type="submit" value="確定">
</form>
```

© 2026 Enos Chou

Web ft. Flask for Input

1. Web page for input



Web ft. Flask for Input

1. Web page for input

© 2026 Enos Chou

Web ft. Flask for Input

2. Web page to Flask

```python
from flask import Flask

app = Flask(__name__)

@app.route("/inp")
def inp():
    page = '''
    <form action="reply">
        Please input: <input type="text" name="topic">
        <input type="submit" value="確定">
    </form>
    '''
    return page

app.run()
```

© 2026 Enos Chou

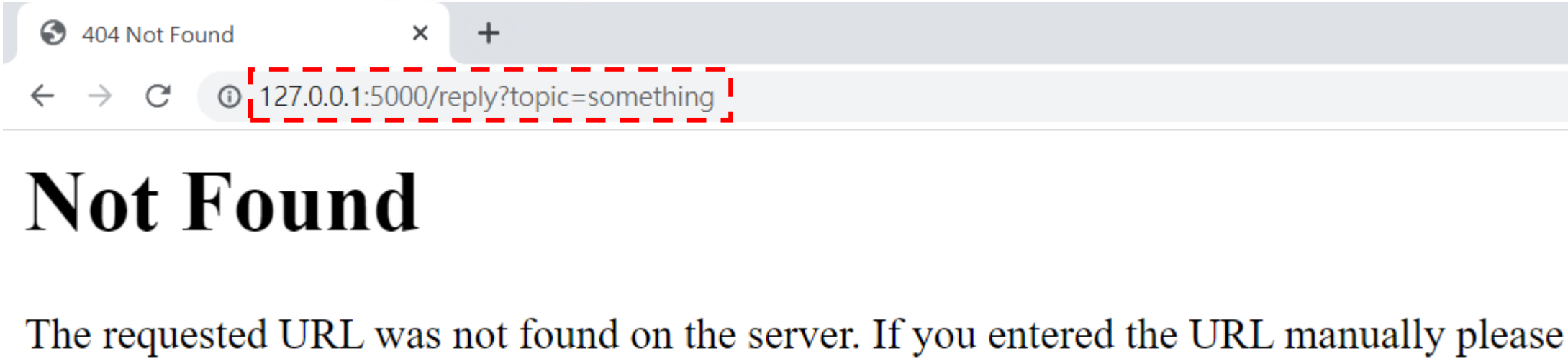
Web ft. Flask for Input

2. Web page to Flask

© 2026 Enos Chou

Web ft. Flask for Input

2. Web page to Flask

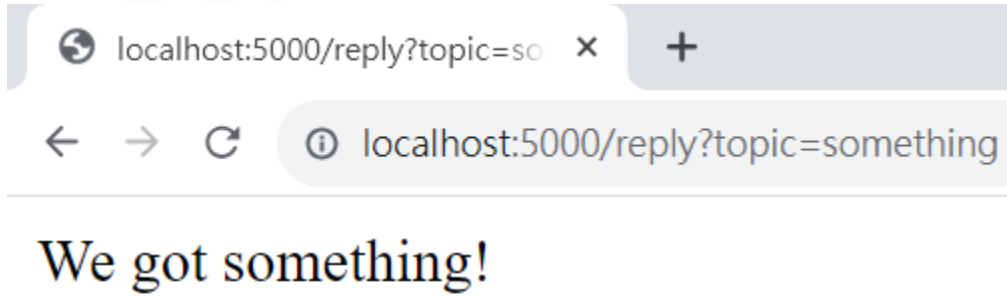


Web ft. Flask for Input

3. Flask to respond

```python
from flask import Flask, request

app = Flask(__name__)

@app.route("/inp")
def inp():
    page = '''
    <form action="reply">
        Please input: <input type="text" name="topic">
        <input type="submit" value="確定">
    </form>
    '''
    return page

@app.route("/reply")
def reply():
    a = request.args.get('topic')
    return f'We got {a}!'

app.run()
```

http://127.0.0.1:5000/reply?topic=something

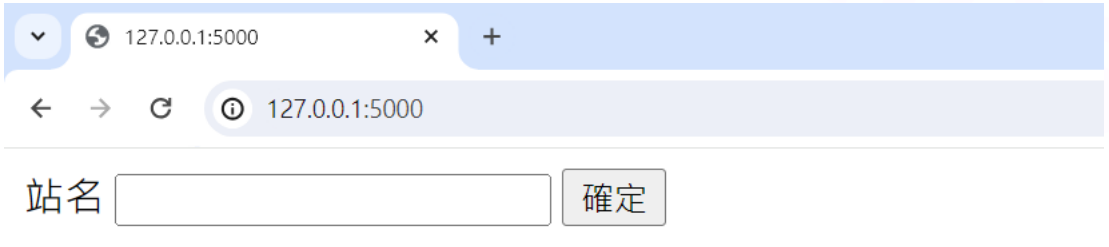


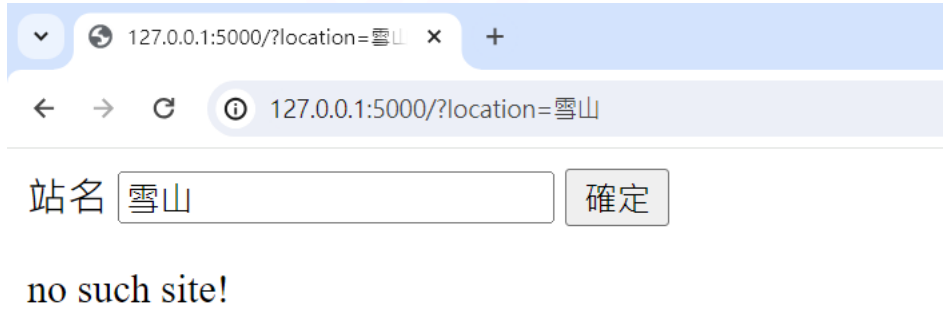
© 2026 Enos Chou

Your Turn - Weather Web

運用 WeaG 開發即時天氣資訊網頁服務 (20 mins)

Output





© 2026 Enos Chou

Your Turn - Weather Web

運用 WeaG 開發即時天氣資訊網頁服務 (20 mins)

Hint

<input> 中的 attribute value 可以設定顯示值

© 2026 Enos Chou

Web Template

Decouple Page from Function

```
1  from flask import Flask, request
2
3  app = Flask(__name__)
4
5  @app.route("/inp")
6  def inp():
7      page = '''
8      <form action="reply">
9          Please input: <input type="text" name="topic">
10         <input type="submit" value="確定">
11     </form>
12     '''
13     return page
14
15 @app.route("/reply")
16 def reply():
17     a = request.args.get('topic')
18     return f'We got {a}!'
19
20 app.run()
```

© 2026 Enos Chou

Web Template

1. Save page as HTML file
2. Return page via render_template()
3. Return variable page

© 2026 Enos Chou

Web Template

1. Save page as HTML file

```
<form action="topic">
    Please input: <input type="text" name="topic">
    <input type="submit" value="確定">
</form>
```

儲存為 inp.html

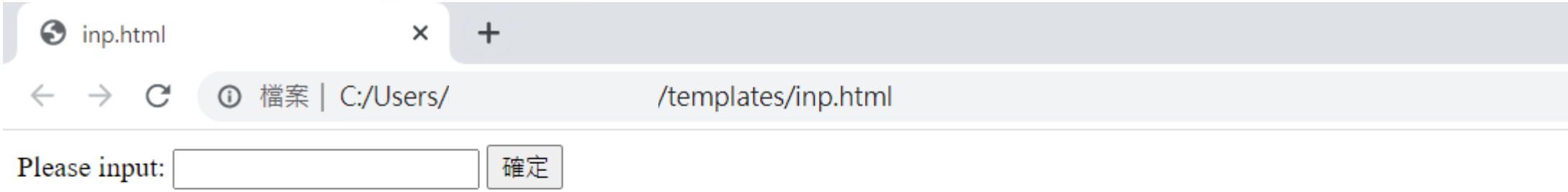
Note

須儲存於 templates 子目錄

© 2026 Enos Chou

Web Template

1. Save page as HTML file



Web Template

2. Return page via render_template()

```
from flask import render_template

@app.route('/inp')
def inp():
    return render_template('inp.html')
```

© 2026 Enos Chou

Web Template

2. Return page via render_template()

```python
from flask import Flask, request, render_template

app = Flask(__name__)

@app.route("/inp")
def inp():
    return render_template('inp.html')

@app.route("/reply")
def reply():
    a = request.args.get('topic')
    return f'We got {a}!'

app.run()
```

© 2026 Enos Chou

Web Template

3. Return variable page

```
@app.route('/reply')
def reply():
    a = request.args.get('topic')
    return f'We got {a}!'
```

```
@app.route('/reply')
def reply():
    a = request.args.get('topic')
    return render_template('reply.html', a=a)
```

© 2026 Enos Chou

Web Template

3. Return variable page

```
We got {{a}}!
```
儲存為 reply.html

Note

{{}} 內容代表變數

© 2026 Enos Chou

Web Template

3. Return variable page

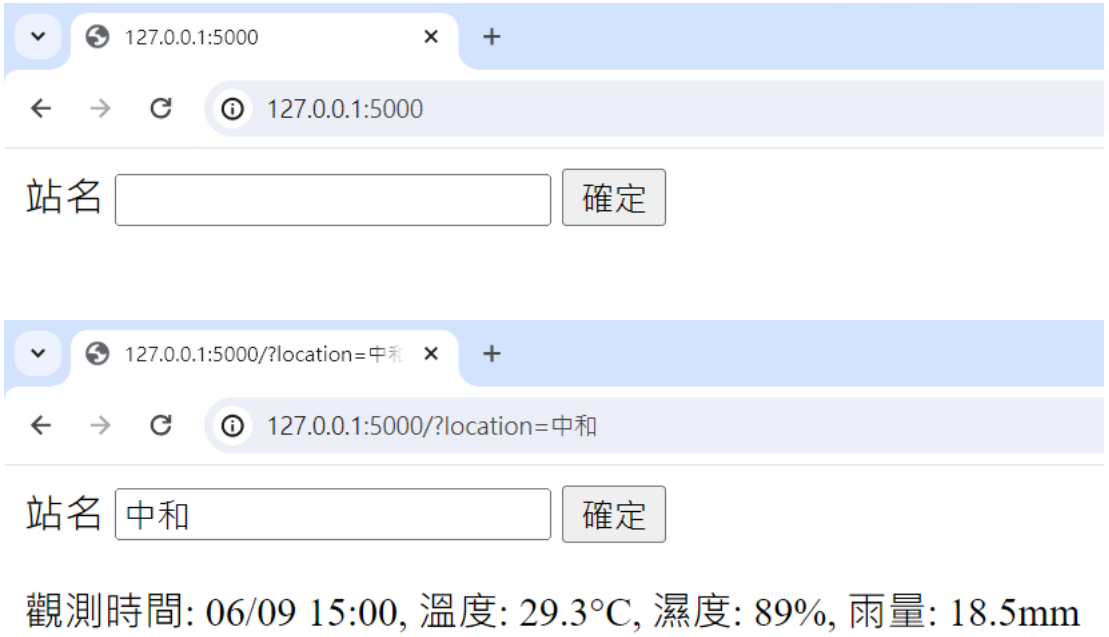
```python
from flask import Flask, request, render_template

app = Flask(__name__)

@app.route("/inp")
def inp():
    return render_template('inp.html')

@app.route("/reply")
def reply():
    a = request.args.get('topic')
    return render_template('reply.html', a=a)

app.run()
```

© 2026 Enos Chou

Your Turn - Weather Template

運用 WeaG 以 render_template() 開發即時天氣資訊網頁服務 (5 mins)

Output



Web Template Variables

1. Basic Template

```
<form>
    站名 <input type="text" name="site" value={{ site or "" }}>
    <input type="submit" value="確定">
</form>

{{ (info or "") | safe }}
```

© 2026 Enos Chou

Web Template Variables

2. Variables Template

```
<form>
    站名 <input type="text" name="site" value={{ site or "" }}>
    <input type="submit" value="確定">
</form>
```

© 2026 Enos Chou

Web Template Variables

2. Variables Template

```
{% if info %}
    {% if info.O is not none %}
    時間: {{ info.O }} <br>
    {% endif %}
    {% if info.T is not none %}
    溫度: {{ info.T }}度 <br>
    {% endif %}
    {% if info.H is not none %}
    濕度: {{ info.H * 100 }}%<br>
    {% endif %}
    {% if info.R is not none %}
    雨量: {{ info.R }}mm
    {% endif %}
{% elif info is mapping %}
    no such site!
{% endif %}
```

© 2026 Enos Chou

Web Error Response

Flask to return Error

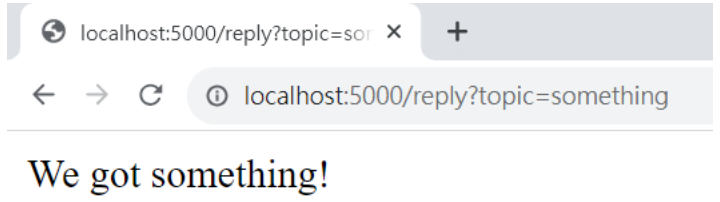
```
import flask

① flask.abort(status_code)
   flask.abort(400)

② return str, status_code
   return 'some error', 400

③ return flask.Response(str, status_code)
   return flask.Response('some error', 400)
```

© 2026 Enos Chou

Web Methods

GET

```python
from flask import Flask, request

app = Flask(__name__)

@app.route("/inp")
def inp():
    page = '''
    <form action="reply">
        Please input: <input type="text" name="topic">
        <input type="submit" value="確定">
    </form>
    '''
    return page

@app.route("/reply")
def reply():
    a = request.args.get('topic')
    return f'We got {a}!'

app.run()
```



Web Methods

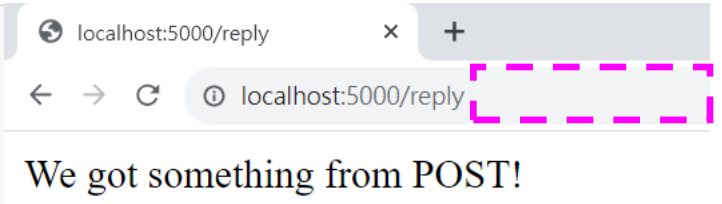
POST

```
1  from flask import Flask, request
2
3  app = Flask(__name__)
4
5  @app.route("/inp")
6  def inp():
7      page = '''
8      <form action="reply" method="post">
9          Please input: <input type="text" name="topic">
10         <input type="submit" value="確定">
11     </form>
12     '''
13     return page
14
15 @app.route("/reply", methods=['GET', 'POST'])
16 def reply():
17     a = request.args.get('topic') or request.form.get('topic') + ' from POST'
18     return f'We got {a}!'
19
20 app.run()
```

1. Add form method POST in page

2. Add methods POST handling in @app.route

3. Retrieve POST data by request.form



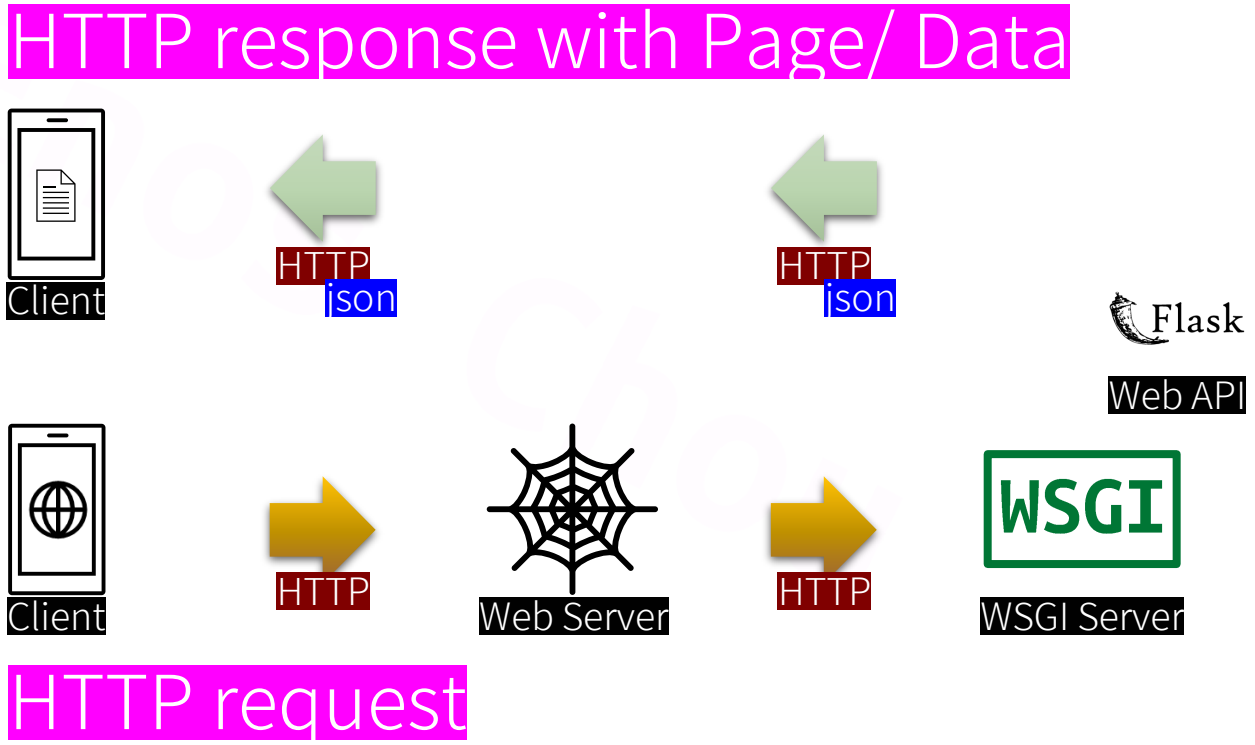
API

Web API Concept

- 讓雙邊透過網路溝通的媒介

  雙邊： Client,
  Server

  溝通： HTTP request,
  HTTP response

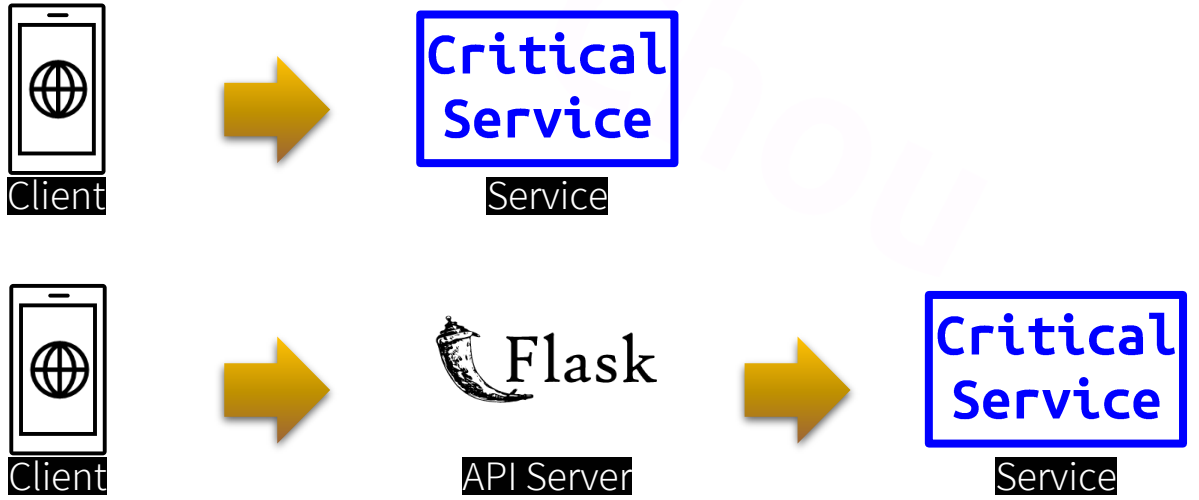


© 2026 Enos Chou

API

Web API Purpose

1. 中央控制、獨特的應用服務，用以開通服務、計費…等
2. 聊天機器人服務，由 API 負責 LLM 的直接聯繫 (而非 Client)，避免密碼外流以及不正常使用導致費用過高的風險
3. DB 存取服務，由 API 負責 DB 的實際存取 (而非 Client)，避免密碼外流以及入侵風險

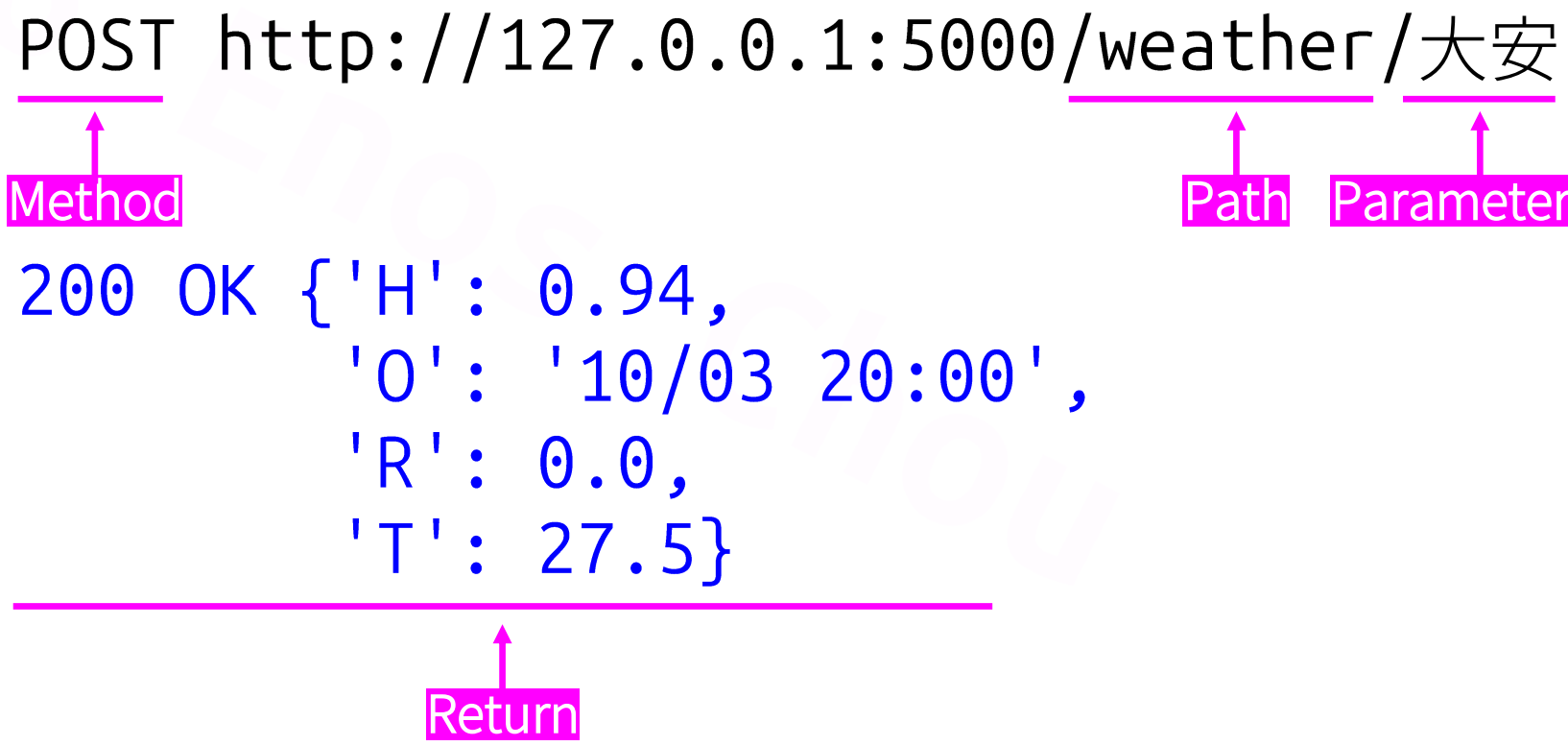


© 2026 Enos Chou

API

Web API Design

a. Method
b. Path
c. Parameters
d. Return



© 2026 Enos Chou

API

Major Methods in RESTful API

DELETE	delete the data
GET	get the data
PATCH	update the data partially
POST	execute the process or create the data
PUT	update the data entirely

© 2026 Enos Chou

API

Method & Path in Web API

- `GET  /getUser/123`
- `POST /createUser/`
- `POST /deleteUser/123`
- `POST /updateUser/123`

Method & Path in RESTful API

- `GET    /user/123`
- `POST   /user`
- `DELETE /user/123`
- `PATCH  /user/123`

© 2026 Enos Chou

API

Parameters Design

- in URL Path
- in Query
- in Header
- Form in Body
- JSON in Body

© 2026 Enos Chou

API

Return Design

- JSON in Body

© 2026 Enos Chou

API ft. Flask

Basic API

parameters through query

Client

```
import requests

url = 'http://localhost:5000/weather'
params = {'site': '大安'}
r = requests.post(url, params=params)

print(r.url)
print(r.status_code)
print(r.json())
```

Server

```
from flask import Flask, request
from wea import WeaG

app = Flask(__name__)
w = WeaG()

@app.route('/weather', methods=['POST'])
def weather():
    r = None
    if request.args:
        site = request.args.get('site')
        r = w.grab(site)
    return r or {'reason': 'no such site'}

app.run()
```

© 2026 Enos Chou

API ft. Flask

Basic API

parameters through path

Client

```
import requests

url = 'http://localhost:5000/weather/大安'
r = requests.post(url)

print(r.url)
print(r.status_code)
print(r.json())
```

Server

```
from flask import Flask, request
from wea import WeaG

app = Flask(__name__)
w = WeaG()

@app.route('/weather/<site>', methods=['POST'])
def weather(site):
    r = w.grab(site)
    return r or {'reason': 'no such site'}

app.run()
```

© 2026 Enos Chou

API ft. Flask

Basic API

parameters through header

Server

```
from flask import Flask, request, abort
from wea import WeaG

app = Flask(__name__)
w = WeaG()

@app.route("/weather/<site>",
           methods=['POST'])
def weather(site):
    token = request.headers.get('Authorization')
    if token != 'Bearer JWT_TOKEN':
        abort(401)
    r = w.grab(site)
    return r or {'reason': 'no such site'}

app.run()
```

Client

```
import requests

url = 'http://localhost:5000/weather/大安'
headers = {'Authorization': 'Bearer JWT_TOKEN'}
r = requests.post(url, headers=headers)

print(r.url)
print(r.status_code)
```

© 2026 Enos Chou

API ft. Flask

Basic API

parameters through multipart/form-data

Client

```
import requests

url = 'http://localhost:5000/weather'
#files = {'site': (None, '大安')}
#r = requests.post(url, files=files)
data = {'site': '大安'}
r = requests.post(url, data=data)

print(r.status_code)
print(r.json())
```

Server

```
from flask import Flask, request
from wea import WeaG

app = Flask(__name__)
w = WeaG()

@app.route("/weather", methods=['POST'])
def weather():
    r = None
    if request.form:
        site = request.form.get('site')
        r = w.grab(site)
    return r or {'reason': 'no such site'}

app.run()
```

API ft. Flask

Basic API

parameters through json in body

Client

```
import requests

url = 'http://localhost:5000/weather'
_json = {'site': '大安'}
r = requests.post(url, json=_json)

print(r.url)
print(r.status_code)
print(r.json())
```

Server

```
from flask import Flask, request
from wea import WeaG

app = Flask(__name__)
w = WeaG()

@app.route("/weather", methods=['POST'])
def weather():
    r = None
    if request.json:
        site = request.json.get('site')
        r = w.grab(site)
    return r or {'reason': 'no such site'}

app.run()
```

© 2026 Enos Chou

API ft. Flask

Upload API

through multipart/form-data

Client

```
import requests

url = 'http://localhost:5000/upload'
files = {'file': ('bp.jpg',
                  open('bp.jpg', 'rb'))}
r = requests.post(url, files=files)

print(r.url)
print(r.status_code)
print(r.json())
```

Server

```
from flask import Flask, request
from werkzeug.utils import secure_filename

app = Flask(__name__)
@app.route("/upload", methods=['POST'])
def upload():
    if request.files and 'file' in request.files:
        file = request.files['file']
        if file and file.filename:
            name = secure_filename(file.filename)
            file.save(name)
            return {'filename': name}
    return {'reason': 'error'}, 400

app.run()
```

© 2026 Enos Chou

API ft. Flask

Upload API

through json in body

Client

```
import base64
import requests

url = 'http://localhost:5000/upload'
with open('bp.jpg', 'rb') as f:
    data = base64.b64encode(f.read()).decode('utf-8')
_json = {'image_base64': data}
r = requests.post(url, json=_json)

print(r.url)
print(r.status_code)
print(r.json())
```

Server

```
from flask import Flask, request

app = Flask(__name__)
@app.route("/upload", methods=['POST'])
def upload():
    if request.is_json:
        j = request.get_json()
        if b64 := j.get('image_base64'):
            image = base64.b64decode(b64)
            ...
            return {'success': True}
    return {'reason': 'error'}, 400

app.run()
```

© 2026 Enos Chou

API ft. Flask

Upload File & Data API

through multipart/form-data

Client

```
import requests

url = 'http://localhost:5000/upload'
files = {'file': ('bp.jpg',
                  open('bp.jpg', 'rb'))}
data = {'userid': 'test1'}
r = requests.post(url, files=files, data=data)

print(r.url)
print(r.status_code)
print(r.json())
```

Server

```
from flask import Flask, request
from werkzeug.utils import secure_filename

app = Flask(__name__)

@app.route("/upload", methods=['POST'])
def upload():
    r = {}
    if ((files := request.files) \
        and (file := files.get('file')):
        if filename := file.filename:
            filename = secure_filename(filename)
            file.save(filename)
            r['filename'] = filename
    if ((form := request.form) \
        and (userid := form.get('userid')):
        r['userid'] = userid

    return r or {'reason': 'error'}, 400

app.run()
```

© 2026 Enos Chou

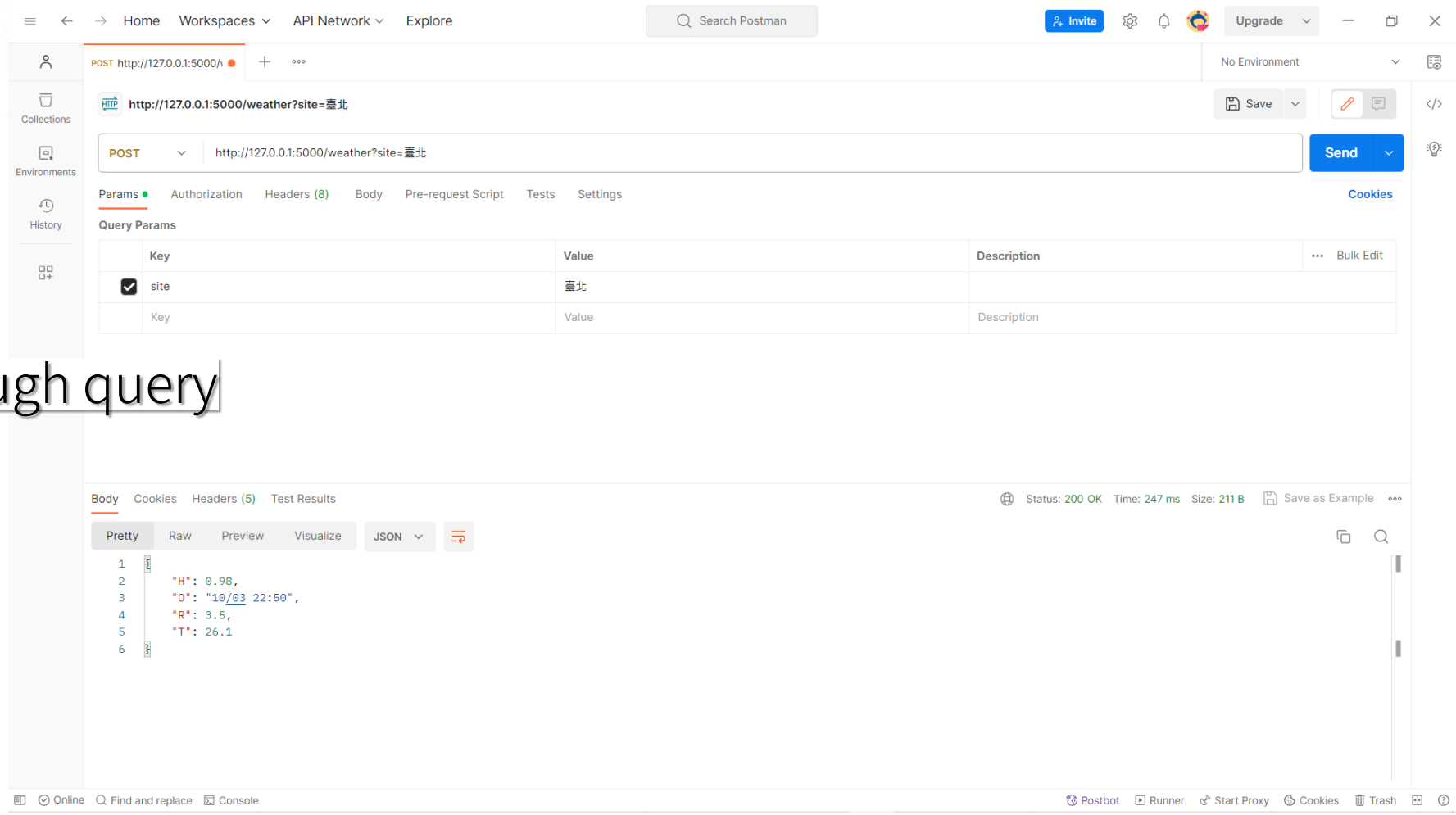
API Client ft. Postman

1. Download & Install Postman
2. Explore HTTP ft. Postman
    a. HTTP Request vs Response
    b. HTTP Header vs Body

© 2026 Enos Chou

API Client ft. Postman

Basic API

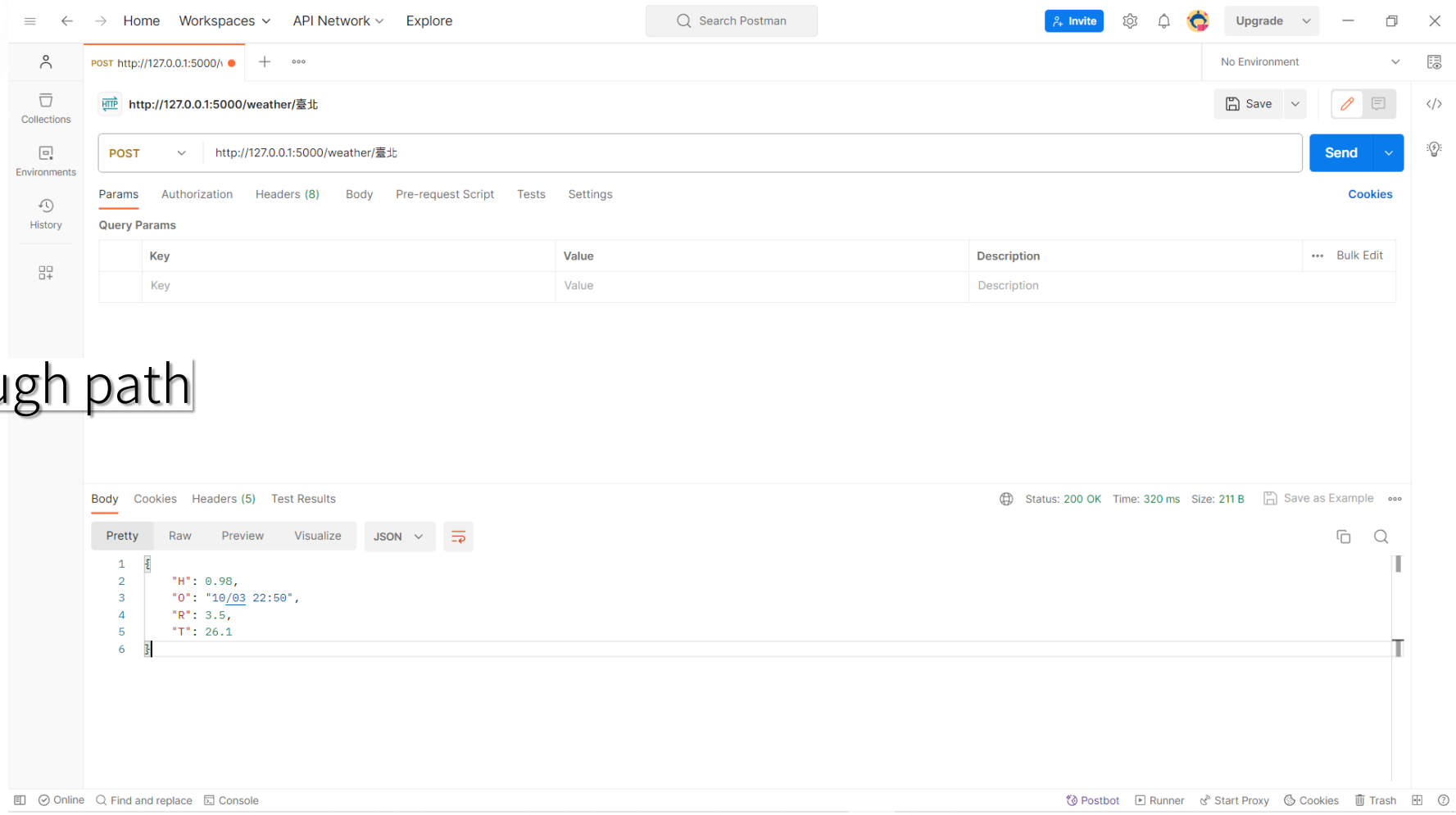


parameters through query

API Client ft. Postman

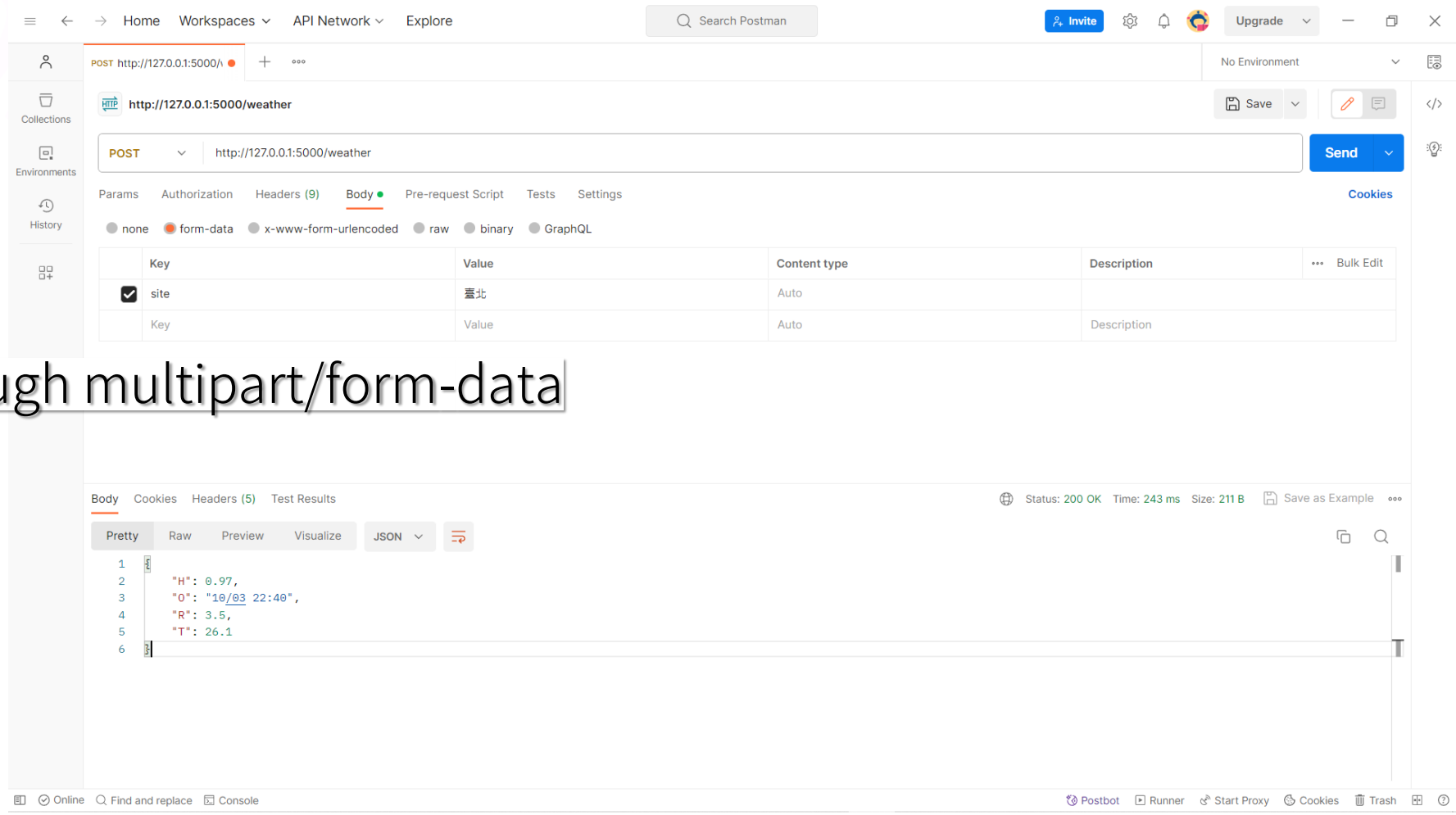
Basic API

parameters through path



API Client ft. Postman

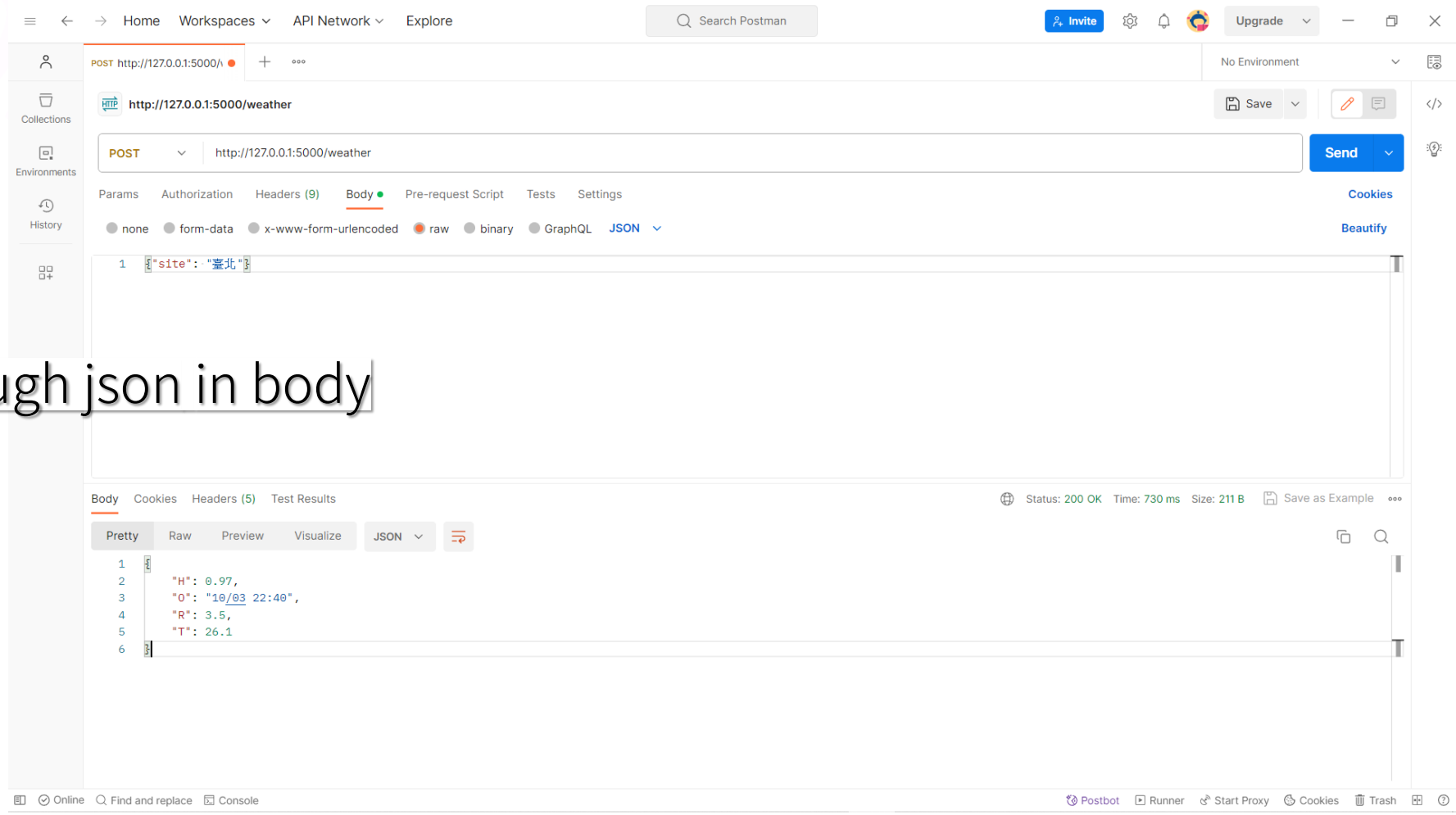
Basic API



parameters through multipart/form-data

API Client ft. Postman

Basic API



parameters through json in body

API Delivery ft. Swagger

目的

運用語法 OpenAPI 描述 API ，以自動產出 API 使用介面

適用範圍

- API 交付
- API 測試

OpenAPI 語法

- YAML
- JSON

© 2026 Enos Chou

API Delivery ft. Swagger

Glance of Open Data @ CWA

API Delivery ft. Swagger in YAML

Basic API

parameters through query

General SPEC

```yaml
openapi: 3.1.0

info:
  title: Weather APIs
  description: Weather APIs for integration and test

servers:
  - url: http://127.0.0.1:5000
    description: Weather APIs test environment
```

API Request SPEC

```yaml
paths:
  /weather:
    post:
      summary: current weather in query
      description: current weather in query

      parameters:
        - name: site
          in: query
          description: 指定測站名稱
          required: true
          schema:
            type: string
            description: 測站名稱
            example: 臺北
```

API Response SPEC

```yaml
responses:
  "200":
    description: observed weather elements
    content:
      application/json:
        schema:
          type: object
          properties:
            O:
              type: string
              description: 觀測時間
            T:
              type: float
              description: 溫度
            H:
              type: float
              description: 濕度
            R:
              type: float
              description: 雨量
          example: {"O": "10/06 10:00",
                    "H": 0.93,
                    "R": 0.0,
                    "T": 28.0}
```

© 2026 Enos Chou

API Delivery ft. Swagger

1. 下載 Swagger UI
   a. 進入 https://github.com/swagger-api/
   b. swagger-ui > 向下捲 > download the latest release
   c. 取得 swagger-ui zip 檔
2. 準備 Swagger 規格檔
   a. 解壓縮 swagger-ui zip 檔
   b. 將待測試之 swagger 規格檔置入解壓縮後的 dist 目錄
3. 啟動 HTTP Server 進行測試
   a. 於 dist 目錄執行 python -m http.server
   b. 以瀏覽器開啟 http://localhost:8000 進行測試

© 2026 Enos Chou

CORS Glance

CORS Issue

Server response

Code	Details
Undocumented	**Failed to fetch.** **Possible Reasons:** • CORS • Network Failure • URL scheme must be "http" or "https" for CORS request.

CORS Glance

CORS Issue

a. 針對瀏覽器
b. 依據瀏覽器的 "Same Origin Policy" 規範，網頁只能存取同源的資源
   - same protocol, same domain, same port
c. 若瀏覽器欲獲取非同源資源，瀏覽器會自動發起跨域 (CORS) 請求，若 server 適切回應即可正確存取資源
d. 限制 JavaScript 的行為為主

© 2026 Enos Chou

CORS Glance

CORS Solution ft. Flask

- Package
  ```
  pip install flask-cors
  ```

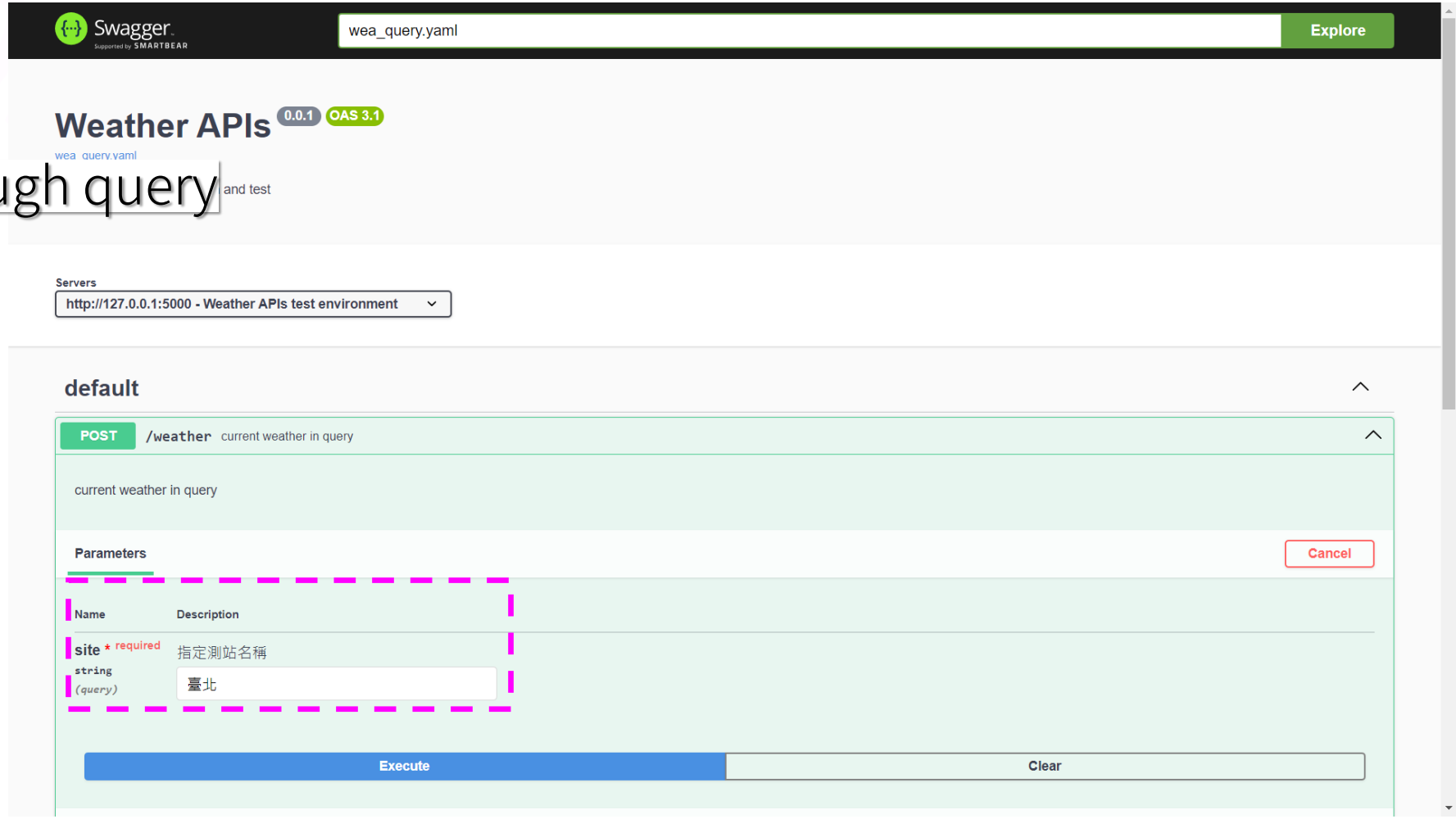
- Code
  ```
  from flask_cors import CORS
  CORS(app)
  ```

© 2026 Enos Chou

API Delivery ft. Swagger in YAML

Basic API

parameters through query



API Delivery ft. Swagger in YAML

Basic API

parameters through query

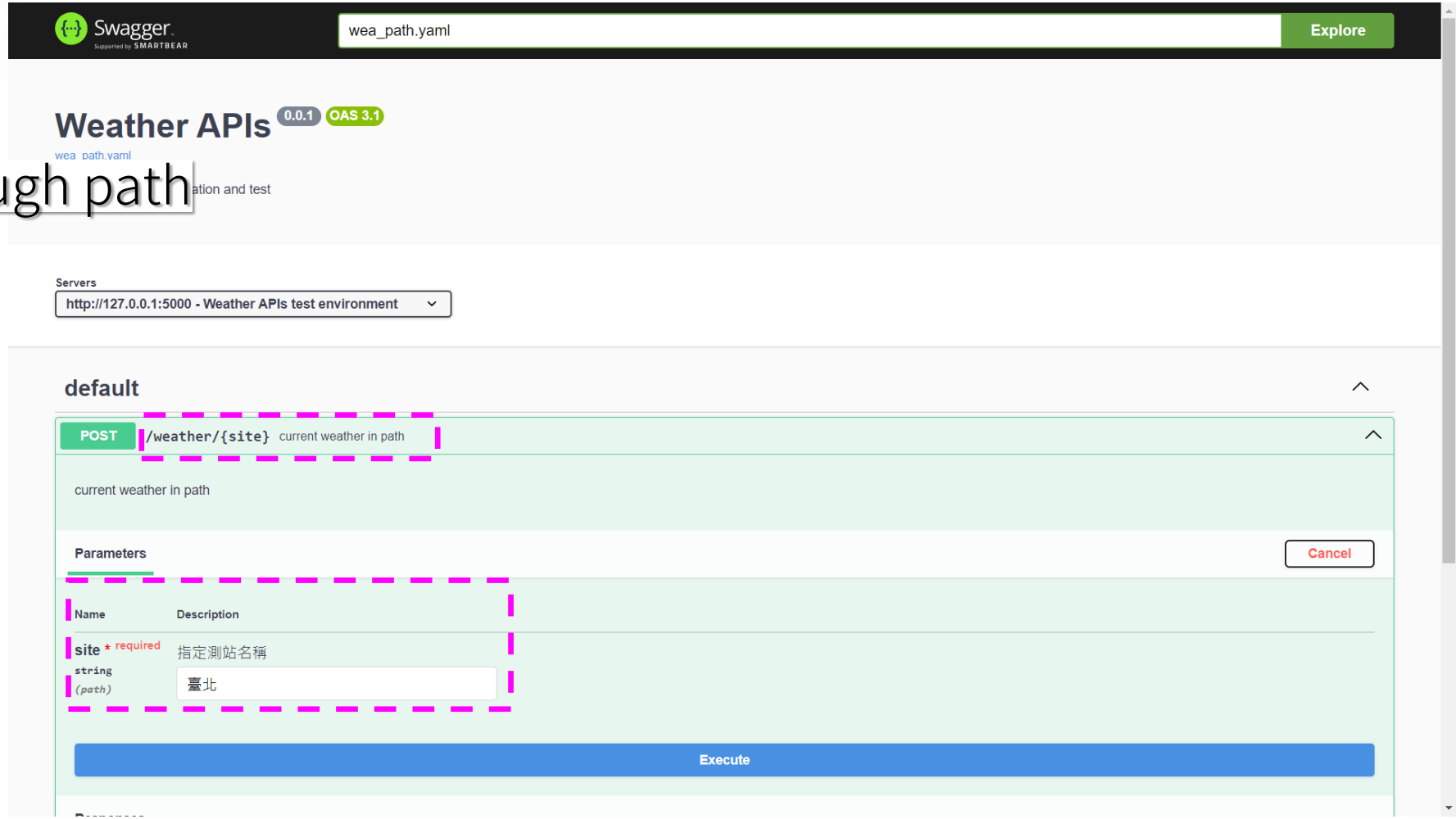
```yaml
openapi: 3.1.0

info:
  title: Weather APIs
  description: Weather APIs for integration and test

servers:
  - url: http://127.0.0.1:5000
    description: Weather APIs test environment

paths:
  /weather:
    post:
      summary: current weather in query
      description: current weather in query

      parameters:
        - name: site
          in: query
          description: 指定測站名稱
          required: true
          schema:
            type: string
            description: 測站名稱
            example: 臺北

      responses:
        "200":
          description: observed weather elements
          content:
            application/json:
              schema:
                type: object
                properties:
                  O:
                    type: string
                    description: 觀測時間
                  T:
                    type: float
                    description: 溫度
                  H:
                    type: float
                    description: 濕度
                  R:
                    type: float
                    description: 雨量
                example: {"O": "10/06 10:00",
                          "H": 0.93,
                          "R": 0.0,
                          "T": 28.0}
```

© 2026 Enos Chou

API Delivery ft. Swagger in YAML

Basic API

parameters through path



API Delivery ft. Swagger in YAML

Basic API

parameters through path

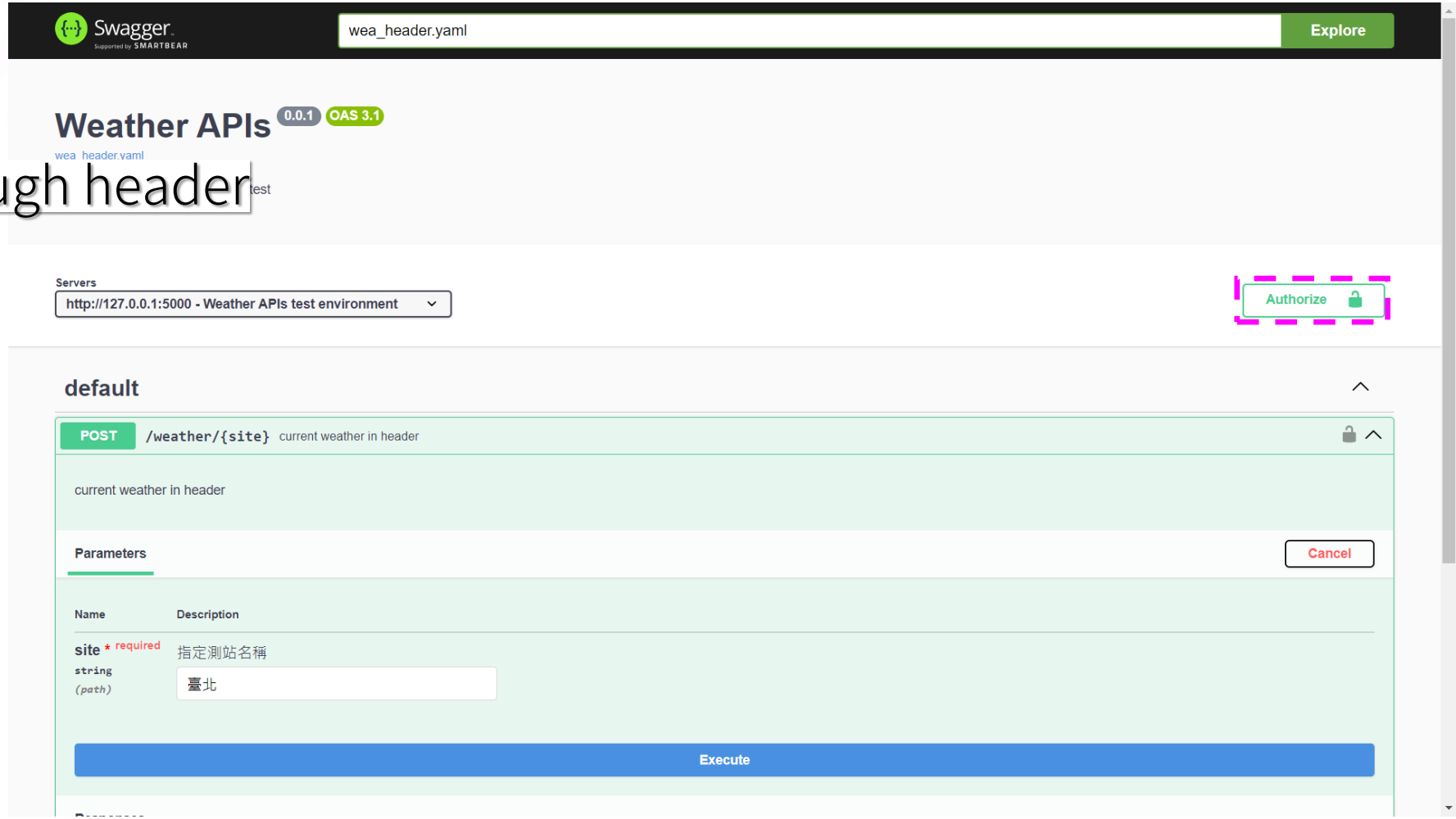
```yaml
openapi: 3.1.0

info:
  title: Weather APIs
  description: Weather APIs for integration and test
  version: 0.0.1

servers:
  - url: http://127.0.0.1:5000
    description: Weather APIs test environment

paths:
  /weather/{site}:
    post:
      summary: current weather in path
      description: current weather in path

      parameters:
        - name: site
          in: path
          description: 指定測站名稱
          required: true
          schema:
            type: string
            description: 測站名稱
            example: 臺北
```

© 2026 Enos Chou

API Delivery ft. Swagger in YAML

Basic API

parameters through header



API Delivery ft. Swagger in YAML

Basic API

parameters through header

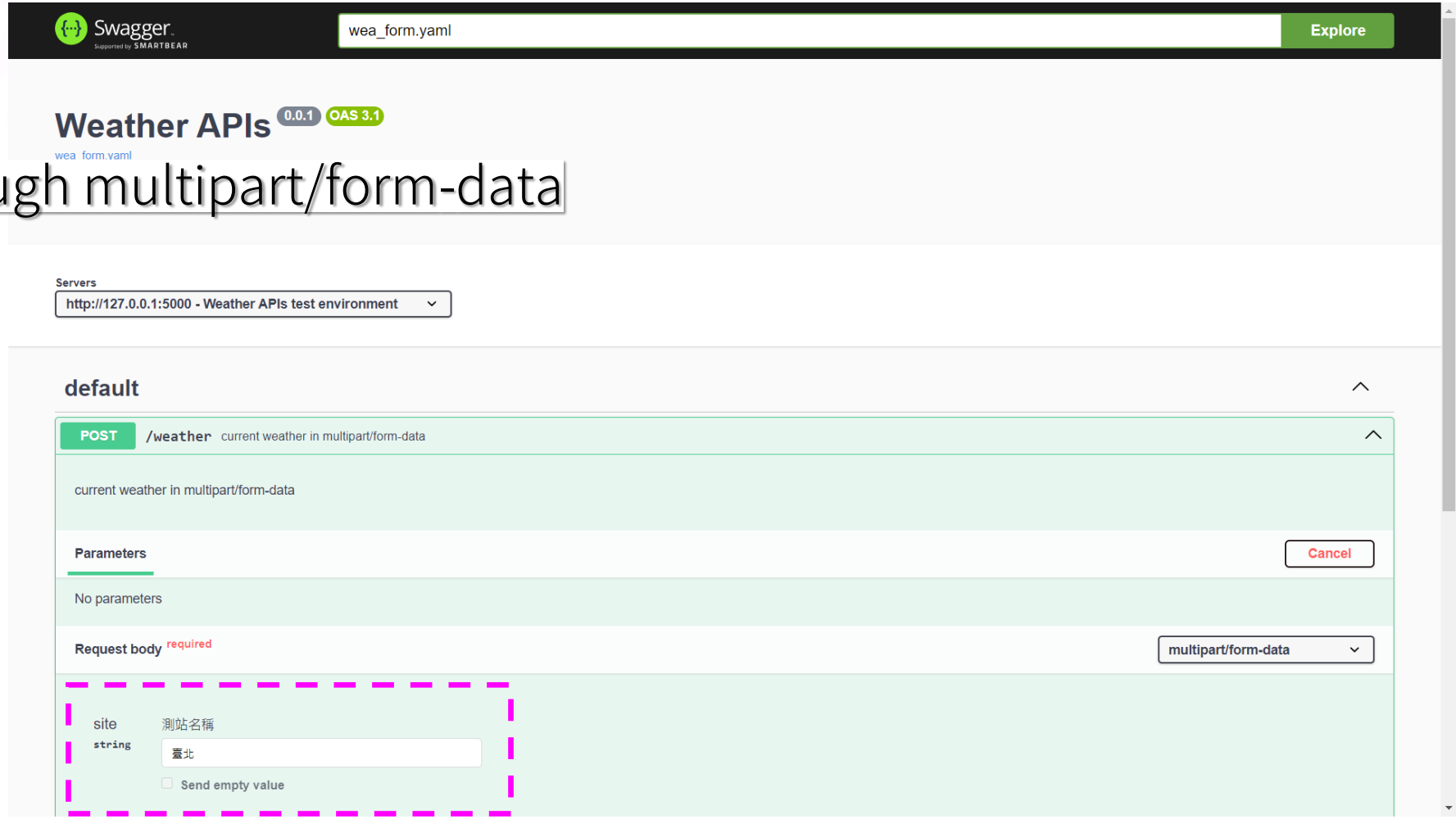
```yaml
openapi: 3.1.0

info:
  title: Weather APIs
  description: Weather APIs for integration and test
  version: 0.0.1

servers:
  - url: http://127.0.0.1:5000
    description: Weather APIs test environment

security:
  - bearerAuth: []
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT

paths:
  /weather/{site}:
    post:
      summary: current weather in header
      description: current weather in header
```

© 2026 Enos Chou

API Delivery ft. Swagger in YAML

Basic API

parameters through multipart/form-data



API Delivery ft. Swagger in YAML

Basic API

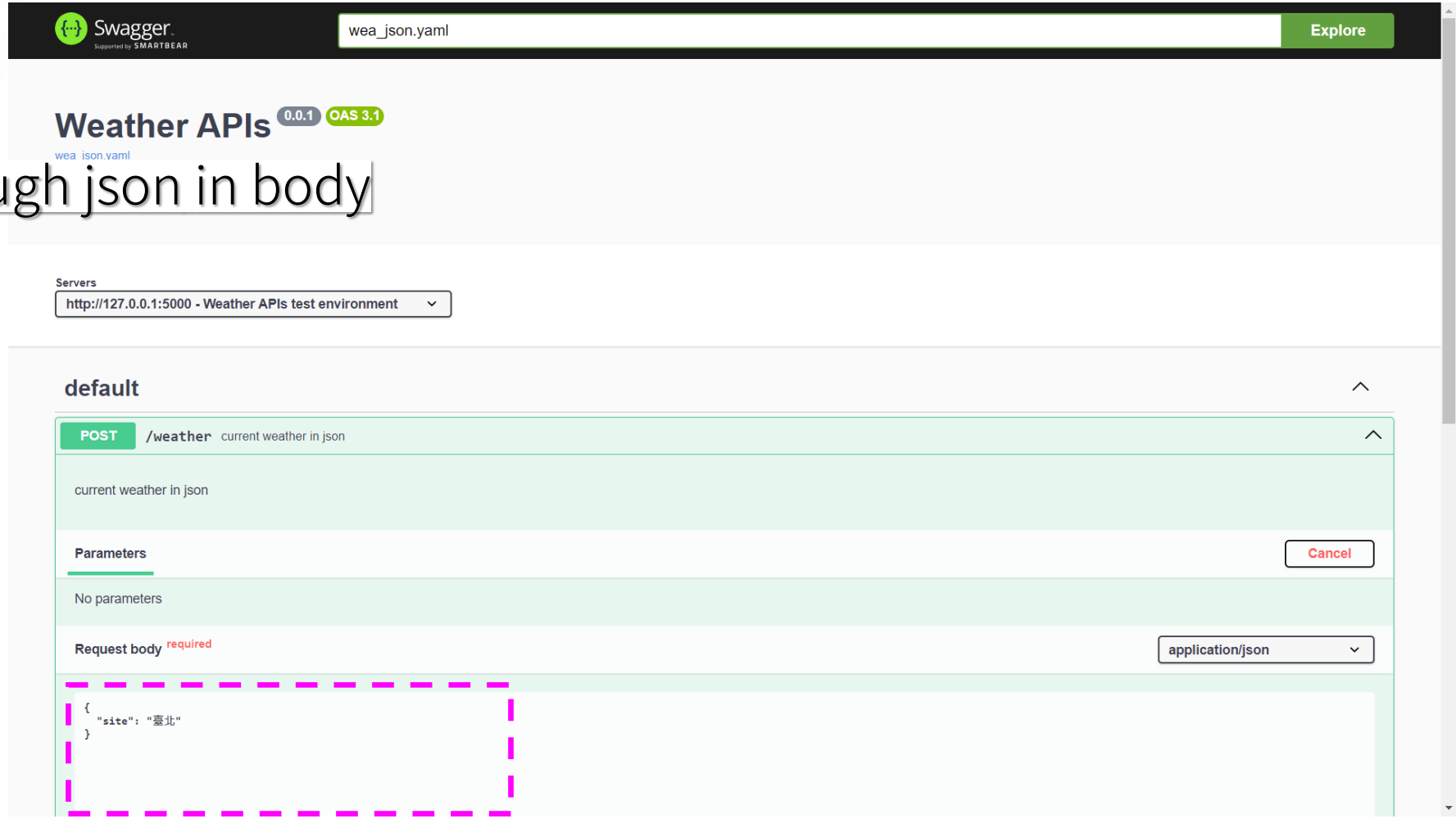
parameters through multipart/form-data

```yaml
info:
  title: Weather APIs
  description: Weather APIs for integration and test
  version: 0.0.1

servers:
  - url: http://127.0.0.1:5000
    description: Weather APIs test environment

paths:
  /weather:
    post:
      summary: current weather in multipart/form-data
      description: current weather in multipart/form-data

      requestBody:
        required: true
        content:
          multipart/form-data:
            schema:
              type: object
              properties:
                site:
                  type: string
                  description: 測站名稱
                  example: 臺北
```

API Delivery ft. Swagger in YAML

Basic API

parameters through json in body



API Delivery ft. Swagger in YAML

Basic API

parameters through json in body

```yaml
info:
  title: Weather APIs
  description: Weather APIs for integration and test
  version: 0.0.1

servers:
  - url: http://127.0.0.1:5000
    description: Weather APIs test environment

paths:
  /weather:
    post:
      summary: current weather in json
      description: current weather in json

      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              properties:
                site:
                  type: string
                  description: 測站名稱
                  example: 臺北
```

© 2026 Enos Chou

整合工具提示

© 2026 Enos Chou

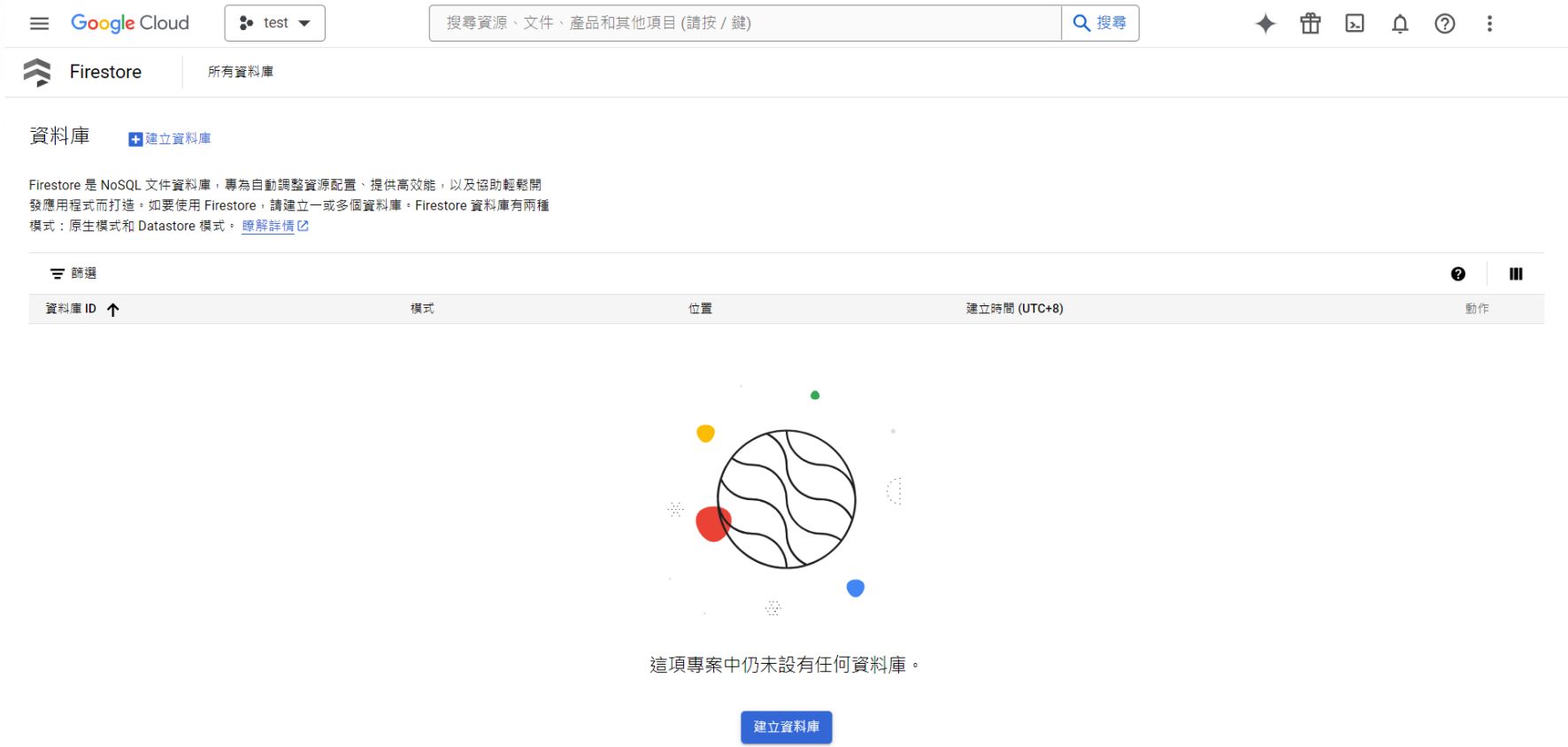
工具列表

目的	產品、服務或套件	產品、服務或套件
Text Storage	Google Cloud Firestore	MongoDB Atlas
Blob Storage	Google Cloud Storage	Azure Blob Storage
Audio Recording	PyAudio	
Audio/ Video Conversion	FFmpeg ft. Pydub	
Audio Play	PyGame	playsound3
Chat	Azure OpenAI ft. LangChain	
Text to Speech	Google Cloud Text-to-Speech API Azure OpenAI gpt-4o-mini-tts	Google Cloud Text-to-Speech API ft. Gemini
Reverse Proxy Tunneling	ngrok	Cloudflare Tunnel Client
Ports in Use	Ports in Use on Windows	Ports in Use on macOS or Linux

© 2026 Enos Chou

Google Cloud Firestore

啟用 Firestore



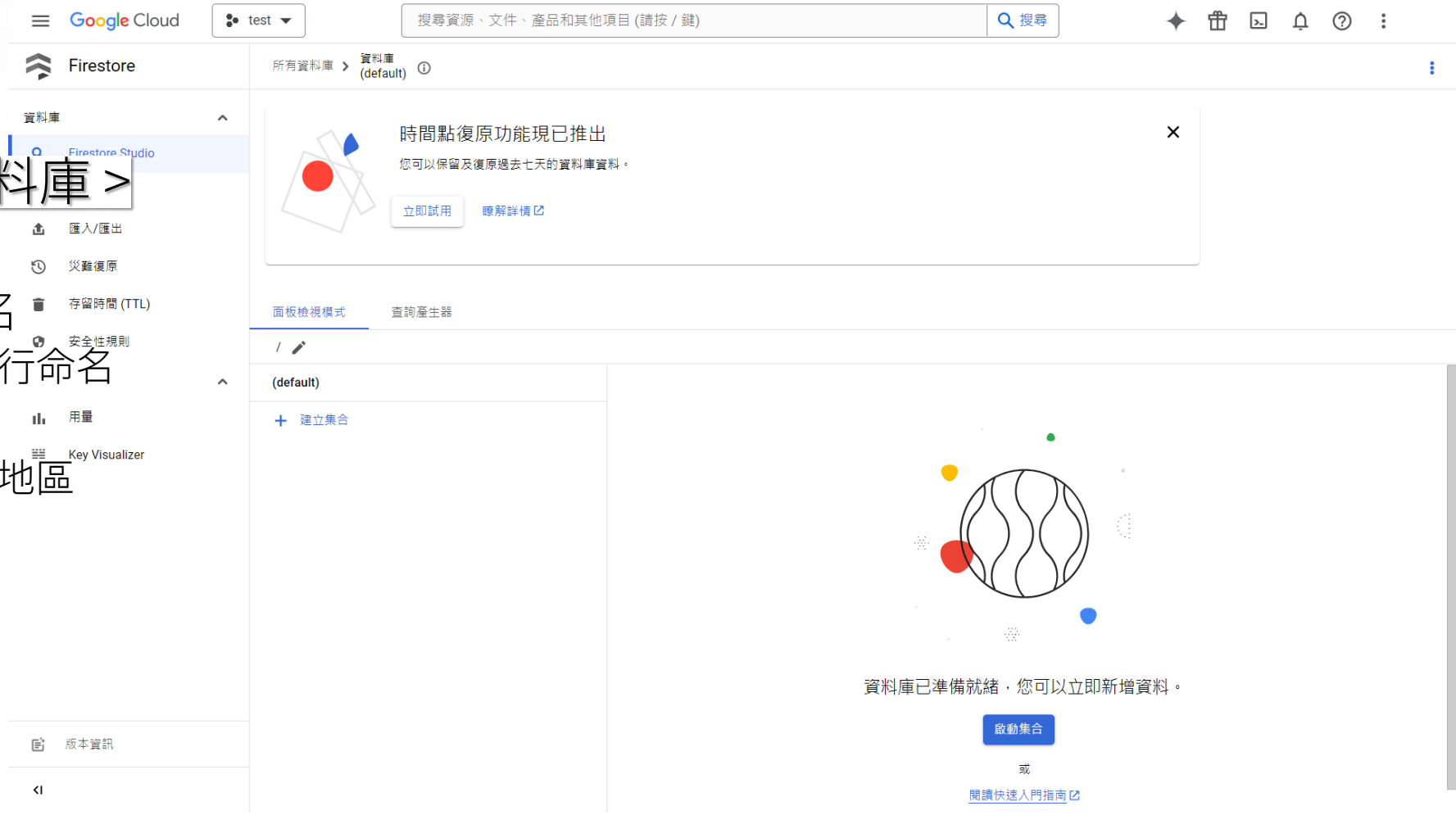
Google Cloud Firestore

建立資料庫

建立 Firestore 資料庫 >

① 為資料庫命名
資料庫 ID：自行命名

② 位置類型
Region：選擇地區



Google Cloud Firestore

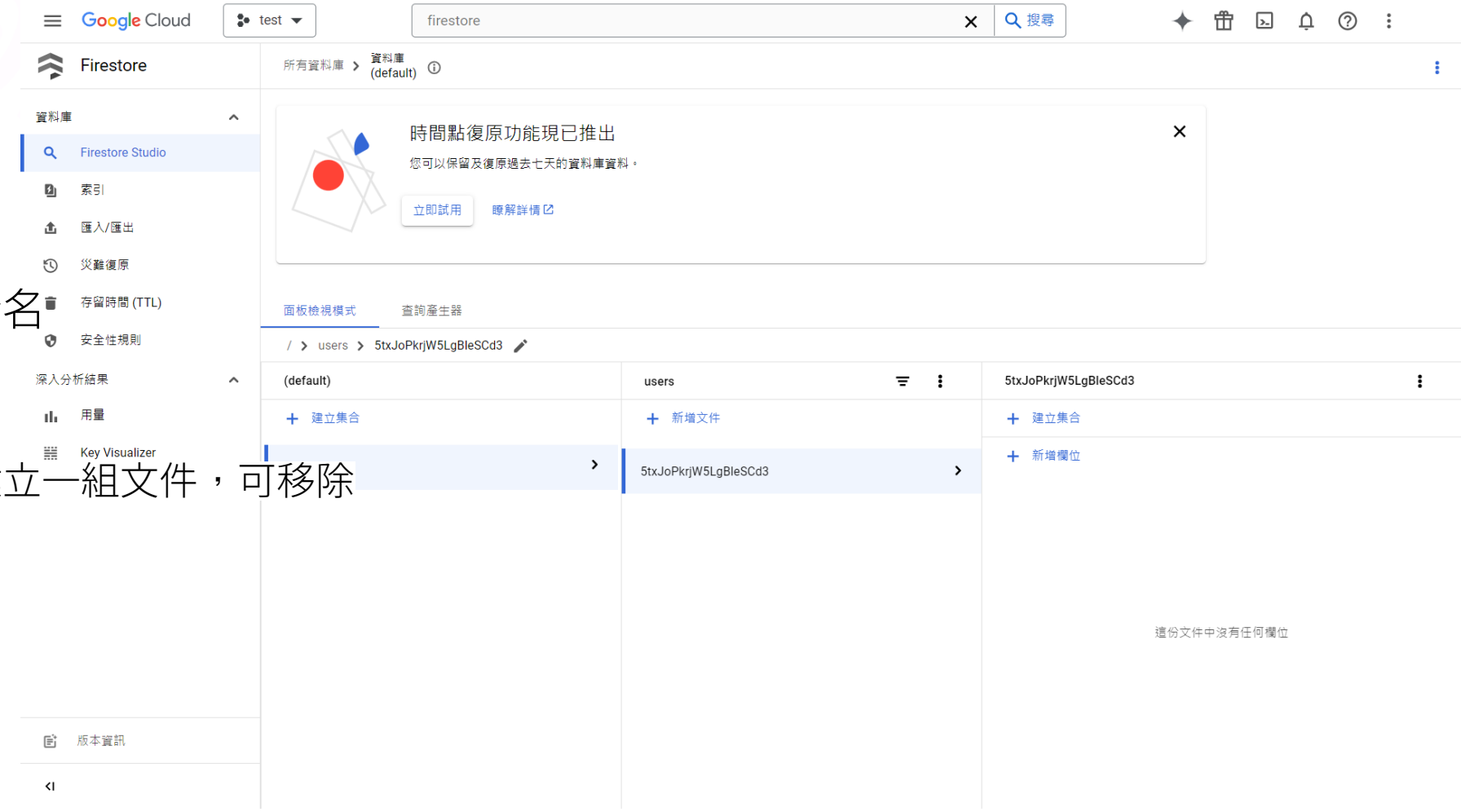
建立集合

建立集合 >

集合 ID：自行命名

Note

此時會自動建立一組文件，可移除



Google Cloud Firestore

權限

服務帳戶角色

`Datastore: Cloud Datastore 使用者`

© 2026 Enos Chou

Google Cloud Firestore

用戶端開發

安裝套件

```
pip install google-cloud-firestore
```

© 2026 Enos Chou

Google Cloud Firestore

用戶端開發

基礎語法

連結 DB
建立 Collection
新增資料
更新資料
更新數值
取得資料
搜尋資料
刪除資料

© 2026 Enos Chou

Google Cloud Firestore

用戶端開發

資料設計

```
user = {
    'user_id': 'U12345678901234567890123456789012',
    'display_name': 'John',
    'count': 0
}
```

© 2026 Enos Chou

Google Cloud Firestore

用戶端開發

初始化 Collection

```
from google.cloud import firestore

YOUR_SERVICE = service_account_key_file_name_str
YOUR_DATABASE = database_name_str  # None 為預設 DB
YOUR_COLLECTION = collection_name_str
```

© 2026 Enos Chou

Google Cloud Firestore

用戶端開發

初始化 Collection

```
try:
    firestore_client = firestore.Client.from_service_account_json(
        YOUR_SERVICE,
        database=YOUR_DATABASE
    )
    collection = firestore_client.collection(YOUR_COLLECTION)
except Exception as e:
    print(e)
```

© 2026 Enos Chou

Google Cloud Firestore

用戶端開發

新增與更新資料

```
# via add & update, have to take care of the existence of user_id

from time import strftime

user.update({'follow': strftime('%Y/%m/%d-%H:%M:%S')})

if collection.document(user.user_id).get().exists:  # 判斷資料是否存在
    collection.document(user_id).update(user)  # 更新
else:
    collection.add(document_data=user, document_id=user.user_id)  # 新增
```

© 2026 Enos Chou

Google Cloud Firestore

用戶端開發

新增與更新資料

```
# via set, regardless of whether user_id exists

from time import strftime

collection.document(user.user_id).set(
    {'follow': strftime('%Y/%m/%d-%H:%M:%S')},
    merge=True
)  # set 資料
```

© 2026 Enos Chou

Google Cloud Firestore

用戶端開發

更新數值

```
from google.cloud import firestore

user_id = 'some user id'
u = collection.document(user_id)

if u.get().exists:
    u.update({'count': firestore.Increment(1)})  # 更新數值
```

© 2026 Enos Chou

Google Cloud Firestore

用戶端開發

取得資料

```
user_id = 'some user id'
u = collection.document(user_id)

# if user_id not exists, u.get().to_dict() would return None
if u.get().exists:
    print(u.get().to_dict().get('count'))  # 取得資料
    print(u.get(['count']).to_dict() # 取得資料
```

© 2026 Enos Chou

Google Cloud Firestore

用戶端開發

搜尋資料 (取得所有欄位)

```
sometime = '20250901'

# find all data where a specific filed is greater than a specific value
# the result us will be a list even if empty
query = collection.where(filter=firestore.FieldFilter('follow', '>', sometime))
us = [u.to_dict() for u in query.stream()]

# find all data where a specific field is equal to a specific value
query = collection.where(filter=firestore.FieldFilter('follow', '==', sometime))
us = [u.to_dict() for u in query.stream()]
```

© 2026 Enos Chou

Google Cloud Firestore

用戶端開發

搜尋資料 (取得部分欄位)

```
sometime = '20250901'

# find all data where a specific filed is greater than a specific value
query = collection.where(filter=firestore.FieldFilter('follow', '>', sometime))
query = query.select(['display_name', 'follow'])
us = [u.to_dict() for u in query.stream()
```

© 2026 Enos Chou

Google Cloud Firestore

用戶端開發

刪除資料

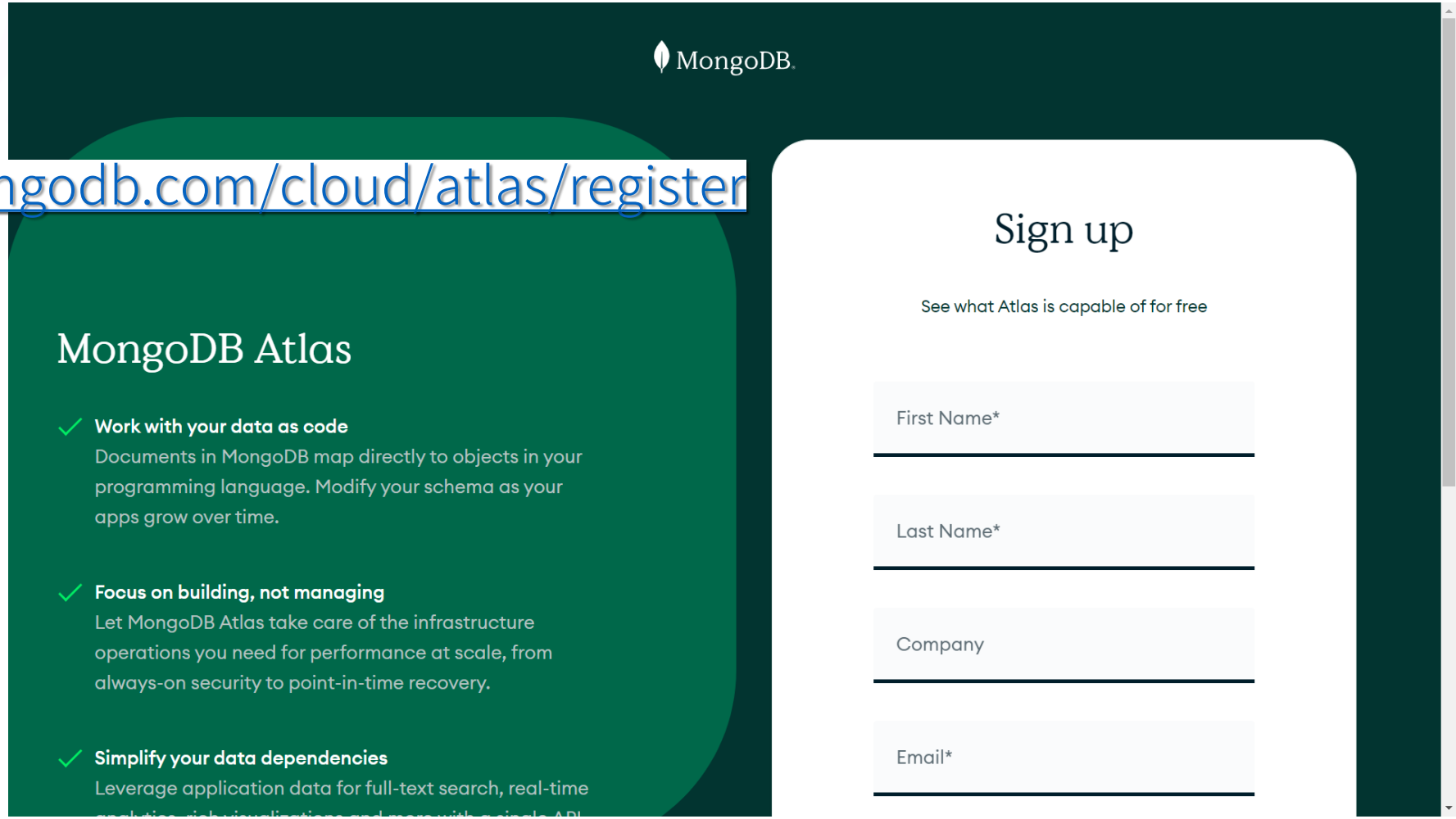
```
user_id = 'some user id'
u = collection.document(user_id)

if u.get().exists:
    u.update({'count': firestore.DELETE_FIELD})  # 刪除資料中欄位
    u.delete()  # 刪除資料
```

© 2026 Enos Chou

MongoDB Atlas

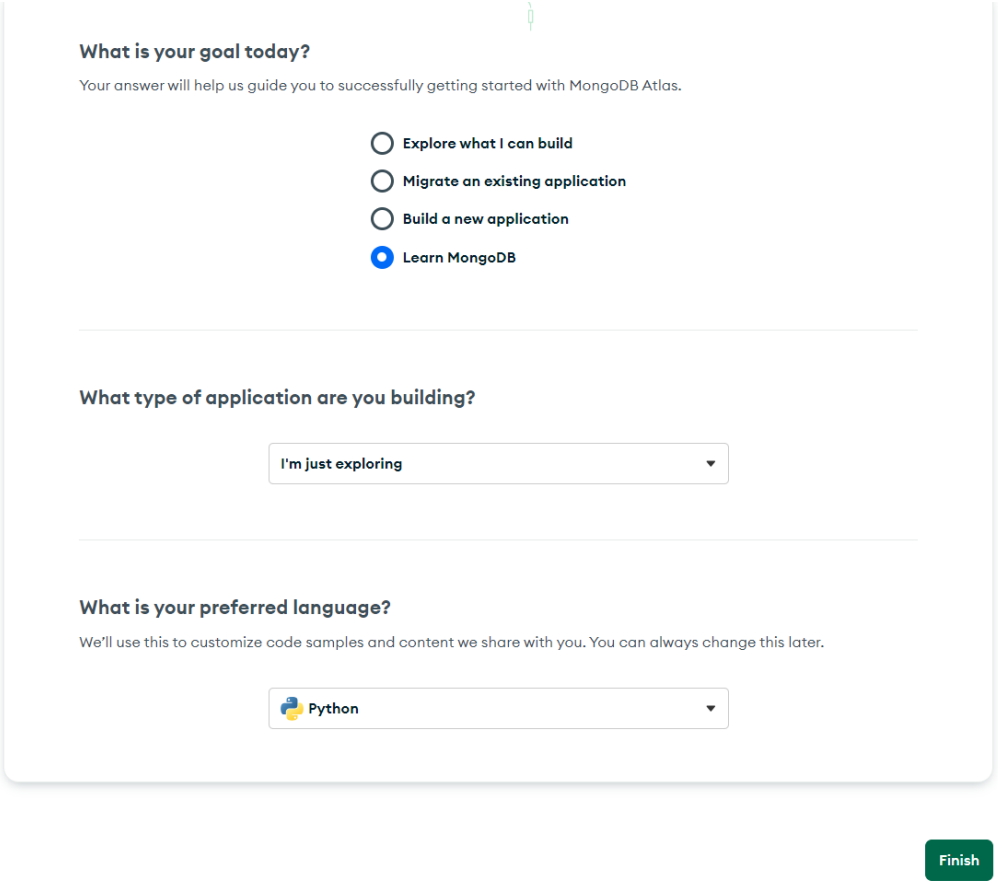
完成註冊

https://www.mongodb.com/cloud/atlas/register



MongoDB Atlas

完成歡迎頁

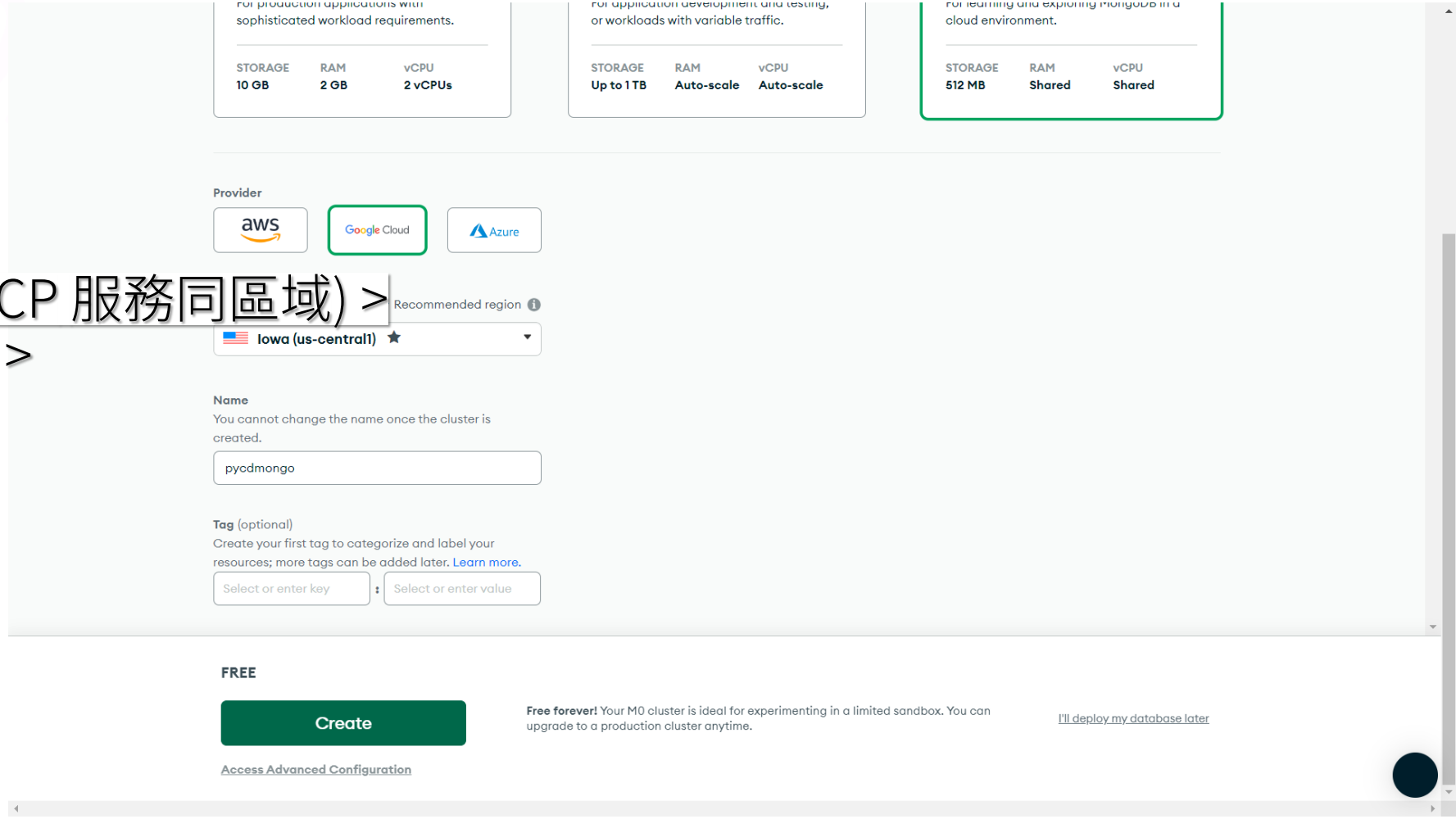


© 2026 Enos Chou

MongoDB Atlas

部署 DB

M0 (FERR) >
GCP >
Region (與其他 GCP 服務同區域) >
Name (自行命名) >
Create

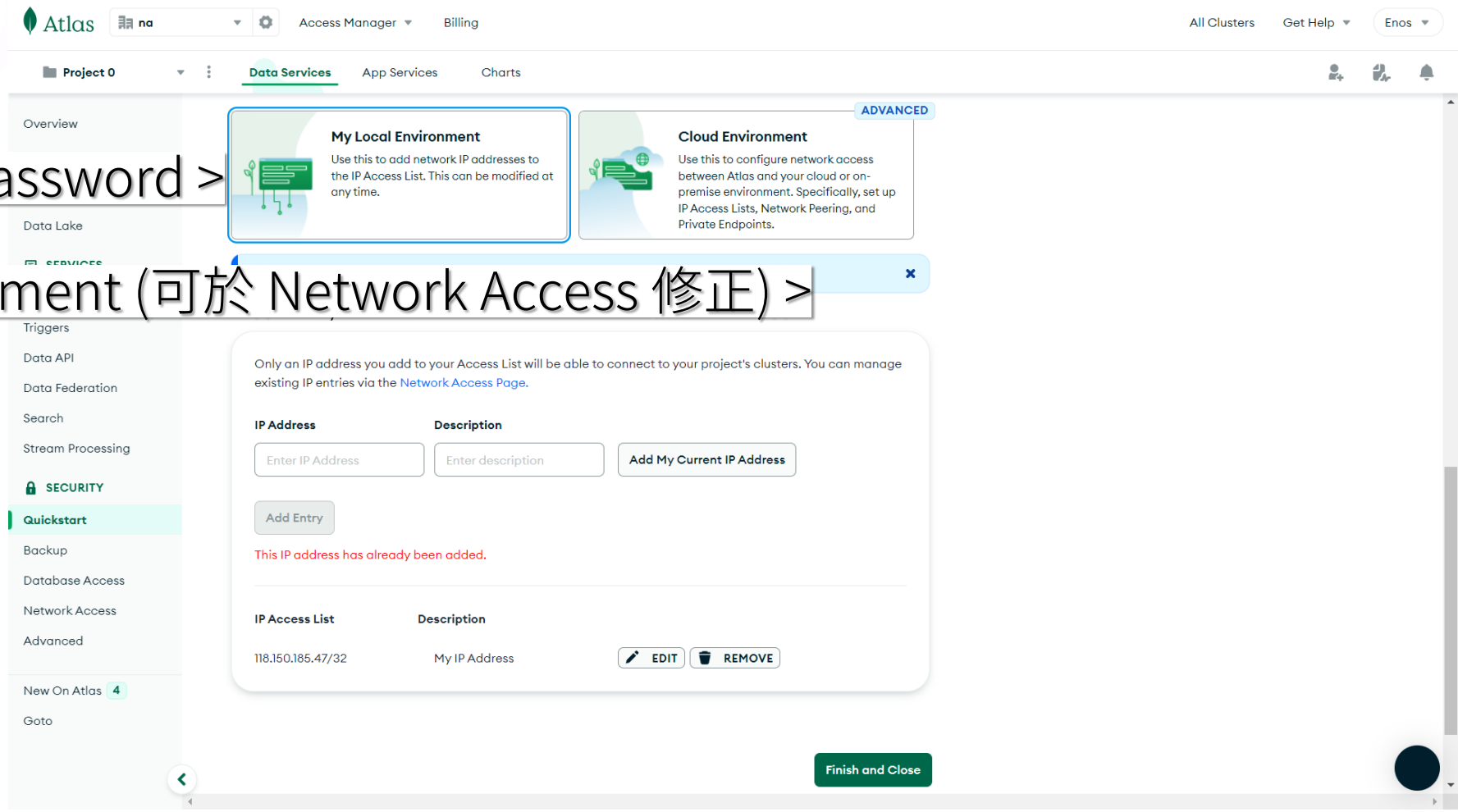


© 2026 Enos Chou

MongoDB Atlas

部署 DB

Username and Password >
Create >
My Local Environment (可於 Network Access 修正) >
Finish and Close



MongoDB Atlas

用戶端開發

安裝套件

```
pip install pymongo
```

© 2026 Enos Chou

MongoDB Atlas

用戶端開發

基礎語法

連結 DB
建立 Collection
新增資料
更新資料
更新數值
取得資料
搜尋資料
刪除資料

© 2026 Enos Chou

MongoDB Atlas

用戶端開發

資料設計

```
user = {
    'user_id': 'U12345678901234567890123456789012',
    'display_name': 'John',
    'count': 0
}
```

© 2026 Enos Chou

MongoDB Atlas

用戶端開發

初始化 Collection

```
from pymongo.mongo_client import MongoClient

YOUR_URI = 'mongodb+srv://...t7gco2o.mongodb.net/?retryWrites=true&w=majority'
YOUR_DATABASE = database_name_str
YOUR_COLLECTION = collection_name_str
```

© 2026 Enos Chou

MongoDB Atlas

用戶端開發

初始化 Collection

```
try:
    mongo_client = MongoClient(YOUR_URI)
    mongo_client.admin.command('ping')  # 確認連線狀態
    db = mongo_client[YOUR_DATABASE]
    collection = db[YOUR_COLLECTION]
except Exception as e:
    print(e)
```

© 2026 Enos Chou

MongoDB Atlas

用戶端開發

新增與更新資料

```
from time import strftime

user.update({'follow': strftime('%Y/%m/%d-%H:%M:%S')})

if collection.find_one({'_id': user['user_id']}):  # 判斷資料是否存在
    collection.update_one({'_id': user['user_id']},
                          {'$set': user})  # 更新
else:
    collection.insert_one(user | {'_id': user['user_id']})  # 新增
```

© 2026 Enos Chou

MongoDB Atlas

用戶端開發

更新數值

```
user_id = 'some user id'

if collection.find_one({'_id': user_id}):  # 判斷資料是否存在
    collection.update_one({'_id': user_id},
                          {'$inc': {'count': 1}})  # 更新數值
```

© 2026 Enos Chou

MongoDB Atlas

用戶端開發

取得資料

```
user_id = 'some user id'

# if user_id not exists, would return None
u = collection.find_one({'_id': user_id})  # 取得一筆資料
if u:
    print(u.get('count'))
```

© 2026 Enos Chou

MongoDB Atlas

用戶端開發

搜尋資料 (取得所有欄位)

```
sometime = '20250901'

# find all data where a specific filed is greater than a specific value
# the result us will be a list even if empty
us = list(collection.find({'follow': {'$gt', sometime}}))

# find all data where a specific field is equal to a specific value
us = list(collection.find({'follow': sometime}))
```

© 2026 Enos Chou

MongoDB Atlas

用戶端開發

搜尋資料 (取得部分欄位)

```
sometime = '20250901'

# find all data where a specific filed is greater than a specific value
us = list(collection.find({'follow': {'$gt', sometime}}),
                          {'display_name': 1, 'follow': 1})  # 1: 保留
                                                             # 0: 忽略
```

© 2026 Enos Chou

MongoDB Atlas

用戶端開發

刪除資料

```
user_id = 'some user id'

if collection.find_one({'_id': user_id}):
    collection.update_one({'_id': user_id},
                          {'$unset': {'count': ''}})  # 刪除資料中欄位

r = collection.delete_one({'_id': user_id})  # 刪除資料
print('delete done' if r.delete_count else 'nothing')
```

© 2026 Enos Chou

Google Cloud Storage

建立值區

建立 bucket

Cloud Storage >
建立 bucket >
全域不重複的永久名稱：*YOUR_BUCKET* > 繼續 >
選取資料的儲存位置：Region：*任選一區* > 繼續 >
選取資料儲存方式：設定預設級別：Standard > 繼續 >
選取如何控制物件的存取權：精細 > 建立 >
系統會禁止公開存取 > 確認

Google Cloud Storage

權限

服務帳戶角色

`Cloud Storage: Storage 物件使用者`

IAM 與管理 > 服務帳戶 >
+ CREATE SERVICE ACCOUNT >
服務帳號詳細資料：服務帳戶名稱：*自取*/ 服務帳號說明：*自取*> 建立並繼續 >
將專案存取權授予這個服務帳戶：請選擇角色：Storage 物件使用者 > 完成

Google Cloud Storage

用戶端開發

安裝套件

```
pip install google-cloud-storage
```

© 2026 Enos Chou

Google Cloud Storage

用戶端開發

基礎語法

初始化 Bucket
上傳 Blob
限制 Blob 公開下載時間
下載 Blob
刪除 Blob

© 2026 Enos Chou

Google Cloud Storage

用戶端開發

初始化 Bucket

```
from google.cloud import storage

YOUR_SERVICE = service_account_key_file_name_str
YOUR_BUECKET = bucket_name_str

try:
    storage_client = storage.Client.from_service_account_json(YOUR_SERVICE)
    bucket = storage_client.bucket(YOUR_BUCKET)
except Exception as e:
    print(e)
```

© 2026 Enos Chou

Google Cloud Storage

用戶端開發

上傳 Blob

```
# upload from file

YOUR_FILE = image_name_str
YOUR_BLOB = YOUR_FILE

blob = bucket.blob(YOUR_BLOB)
blob.upload_from_filename(YOUR_FILE, content_type='image/jpeg')
```

© 2026 Enos Chou

Google Cloud Storage

用戶端開發

上傳 Blob

```
# upload from bytes

YOUR_BLOB = image_name_str

content = ...  # received from network or others
blob = bucket.blob(YOUR_BLOB)
blob.upload_from_string(content, content_type='image/jpeg')
```

© 2026 Enos Chou

Google Cloud Storage

用戶端開發

限制 Blob 公開下載時間

```
from time import time


...
# url generated this way can be released for further use
# with better security
url = blob.generate_signed_url(int(time())+180)  # 限制公開下載 3 分鐘
...
```

© 2026 Enos Chou

Google Cloud Storage

用戶端開發

下載 Blob

```
# download into file

YOUR_BLOB = image_name_str
YOUR_FILE = YOUR_BLOB

blob = bucket.blob(YOUR_BLOB)
if blob.exists():
    blob.download_to_filename(YOUR_FILE)
```

© 2026 Enos Chou

Google Cloud Storage

用戶端開發

下載 Blob

```
# download into memory and then take action

from io import BytesIO
from PIL import Image  # pip install pillow

blob = bucket.blob(YOUR_BLOB)
if blob.exists():
    bs = blob.download_as_bytes()  # 下載直接寫入記憶體
    Image.open(BytesIO(bs)).show()  # 直接從記憶體開啟
```

© 2026 Enos Chou

Google Cloud Storage

用戶端開發

刪除 Blob

```
YOUR_BLOB = image_name_str

blob = bucket.blob(YOUR_BLOB)

if blob.exists():
    blob.delete()
```

© 2026 Enos Chou

Azure Blob Storage

用戶端開發

安裝套件

```
pip install azure-storage-blob
```

© 2026 Enos Chou

Azure Blob Storage

用戶端開發

基礎語法

初始化 Container
上傳 Blob
限制 Blob 公開下載時間
下載 Blob
刪除 Blob

© 2026 Enos Chou

Azure Blob Storage

用戶端開發

初始化 Container

```
from azure.storage.blob import BlobServiceClient

YOUR_BLOB_CONNSTR = 'YOUR_BLOB_CONNSTR'   # 儲存體帳戶連接字串

try:
    blob_service_client = BlobServiceClient.from_connection_string(
        YOUR_BLOB_CONNSTR
    )
except Exception as e:
    print(e)
```

© 2026 Enos Chou

Azure Blob Storage

用戶端開發

初始化 Container

```
YOUR_BLOB_CONTAINER = 'YOUR_BLOB_CONTAINER'   # 容器名稱

container_client = blob_service_client.get_container_client(
    YOUR_BLOB_CONTAINER
)
```

© 2026 Enos Chou

Azure Blob Storage

用戶端開發

上傳 Blob

```
# upload from file

YOUR_FILE = image_name_str  # 欲上傳的檔案名稱
YOUR_BLOB = YOUR_FILE  # 上傳至容器的 Blob 名稱

with open(YOUR_FILE, 'rb') as f:
    data = f.read()
container_client.upload_blob(name=YOUR_BLOB, data=data,
                             content_type='image/jpeg')
```

© 2026 Enos Chou

Azure Blob Storage

用戶端開發

上傳 Blob

```
# upload from file with metadata

YOUR_FILE = image_name_str
YOUR_BLOB = YOUR_FILE

with open(YOUR_FILE, 'rb') as f:
    data = f.read()
container_client.upload_blob(name=YOUR_BLOB, data=data,
                             metadata={'author': 'Enos'})  # 自行定義
                                                           # 中繼資料
```

© 2026 Enos Chou

Azure Blob Storage

用戶端開發

上傳 Blob

```
# upload from bytes

YOUR_BLOB = image_name_str

data = ...  # received from network or others

container_client.upload_blob(name=YOUR_BLOB, data=data,
                             content_type='image/jpeg')
```

© 2026 Enos Chou

Azure Blob Storage

用戶端開發

限制 Blob 公開下載時間

```
from datetime import datetime, timedelta, timezone

from azure.storage.blob import generate_blob_sas, BlobSasPermissions

YOUR_ACCOUNT = 'YOUR_ACCOUNT'   # 儲存體帳戶名稱
YOUR_ACCOUNT_KEY ='YOUR_ACCOUNT_KEY'   # 儲存體帳戶金鑰
```

© 2026 Enos Chou

Azure Blob Storage

用戶端開發

限制 Blob 公開下載時間

```
sas_token = generate_blob_sas(
    account_name=YOUR_ACCOUNT,
    container_name=YOUR_BLOB_CONTAINER,
    blob_name=YOUR_BLOB,
    account_key=YOUR_ACCOUNT_KEY,
    permission=BlobSasPermissions(read=True),
    # 限制公開下載 1 分鐘
    expiry=datetime.now(timezone.utc) + timedelta(minutes=1)
)
```

© 2026 Enos Chou

Azure Blob Storage

用戶端開發

限制 Blob 公開下載時間

```
url = f'https://{YOUR_ACCOUNT}.blob.core.windows.net/' \
      f'{YOUR_BLOB_CONTAINER}/{YOUR_BLOB}?{sas_token}'
...
```

© 2026 Enos Chou

Azure Blob Storage

用戶端開發

下載 Blob

```
# download into file

YOUR_FILE = YOUR_BLOB

try:
    data = container_client.download_blob(blob=YOUR_BLOB)
    with open(YOUR_FILE, 'wb') as f:
        f.write(data.readall())
except Exception as e:
    print(e)
```

© 2026 Enos Chou

Azure Blob Storage

用戶端開發

下載 Blob

```
# download into memory and then take action

YOUR_FILE = YOUR_BLOB

from io import BytesIO

from PIL import Image  # pip install pillow
```

© 2026 Enos Chou

Azure Blob Storage

用戶端開發

下載 Blob

```
# download into memory and then take action

try:
    data = container_client.download_blob(blob=YOUR_BLOB)
    bs = BytesIO()
    data.readinto(bs)  # 下載直接寫入記憶體
    Image.open(bs).show()  # 直接從記憶體開啟
except Exception as e:
    print(e)
```

© 2026 Enos Chou

Azure Blob Storage

用戶端開發

刪除 Blob

```
# delete blob but may be recovered in default 7 days

YOUR_BLOB = 'YOUR_BLOB'

try:
    container_client.delete_blob(blob=YOUR_BLOB)
except Exception as e:
    print(e)
```

© 2026 Enos Chou

Azure Blob Storage

用戶端開發

刪除 Blob

```
# delete blob forever

YOUR_BLOB = 'YOUR_BLOB'

try:
    container_client.delete_blob(blob=YOUR_BLOB,
                                 delete_snapshots='include')
except Exception as e:
    print(e)
```

© 2026 Enos Chou

PyAudio

用戶端開發

參考資料

https://people.csail.mit.edu/hubert/pyaudio/

© 2026 Enos Chou

FFmpeg ft. Pydub

FFmpeg 安裝

a. Windows

① https://ffmpeg.org
② Download
③ Windows
④ Windows EXE Files: Windows builds from gyan.dev
⑤ ffmpeg-release-full.7z
⑥ 解壓縮至 Program Files 中的 FFmpeg
⑦ 將 C:\Program Files\FFmpeg\bin 加入系統環境變數 Path

© 2026 Enos Chou

FFmpeg ft. Pydub

FFmpeg 安裝

b. Ubuntu

```
sudo apt install ffmpeg
```

© 2026 Enos Chou

FFmpeg ft. Pydub

用戶端開發

安裝套件

```
pip install pydub
```

© 2026 Enos Chou

FFmpeg ft. Pydub

用戶端開發

WAV to MP3

```
from pydub import AudioSegment

audio = AudioSegment.from_wav('YOUR_WAV')
audio.export('YOUR_MP3', format='mp3')
```

© 2026 Enos Chou

FFmpeg ft. Pydub

用戶端開發

MP3 to WAV

```
from pydub import AudioSegment

audio = AudioSegment.from_mp3('YOUR_MP3')
audio.export('YOUR_WAV', format='wav')
```

© 2026 Enos Chou

PyGame

用戶端開發

安裝套件

```
pip install pygame
```

© 2026 Enos Chou

PyGame

用戶端開發

初始化

```
import pygame

YOUR_AUDIO = audio_file_name_str

pygame.mixer.init()
```

© 2026 Enos Chou

PyGame

用戶端開發

播放 Audio

```
try:
    pygame.mixer.music.load(YOUR_AUDIO)
    pygame.mixer.music.play()
    while pygame.mixer.music.get_busy():
        sleep(0.1)
finally:
    pygame.mixer.quit()
```

© 2026 Enos Chou

playsound3

用戶端開發

安裝套件

```
pip install playsound3
```

© 2026 Enos Chou

playsound3

用戶端開發

播放 Audio

```
from playsound3 import playsound

YOUR_AUDIO = audio_file_name_str

playsound(YOUR_AUDIO)  # .mp3 and .wav are both OK
```

© 2026 Enos Chou

Azure OpenAI ft. LangChain

用戶端開發

安裝套件

```
pip install langchain-openai
```

© 2026 Enos Chou

Azure OpenAI ft. LangChain

用戶端開發

初始化

```
from langchain_openai import ChatOpenAI

YOUR_KEY = azure_ai_key_str
YOUR_ENDPOINT = endpoint_name_str  # ex: https://xxx.openai.azure.com/
YOUR_MODEL = deployment_name_str  # ex: gpt-5.2-chat

llm = ChatOpenAI(api_key=YOUR_KEY,
                 base_url=urljoin(YOUR_ENDPOINT, '/openai/v1/'),
                 model=YOUR_MODEL)
```

© 2026 Enos Chou

Azure OpenAI ft. LangChain

用戶端開發

準備聊天上下文

```
YOUR_INSTRUCTIONS = system_instructions_str
YOUR_CONTENT = chat_text_content_str

messages = [
    {"role": "system", "content": YOUR_INSTRUCTIONS},
    {"role": "user", "content": [{"type": "text", "text": "早安，您好嗎"}]},
    {"role": "assistant", "content": [{"type": "text", "text": "我很好，想吃飯嗎"}]},
    {"role": "user", "content": [{"type": "text", "text": YOUR_CONTENT}]}
]
```

© 2026 Enos Chou

Azure OpenAI ft. LangChain

用戶端開發

聊天

```
response = llm.invoke(messages)

print(response.content)
```

© 2026 Enos Chou

Google Cloud Text-to-Speech API

準備環境

啟用 API

Cloud Text-to-Speech API

© 2026 Enos Chou

Google Cloud Text-to-Speech API

用戶端開發

安裝套件

```
pip install google-cloud-texttospeech
```

© 2026 Enos Chou

Google Cloud Text-to-Speech API

用戶端開發

初始化

```
from google.cloud import texttospeech

YOUR_SERVICE = service_account_key_file_name_str

tts_client = texttospeech.TextToSpeechClient.from_service_account_json(
    YOUR_SERVICE
)
```

© 2026 Enos Chou

Google Cloud Text-to-Speech API

用戶端開發

設定

```
YOUR_INPUT = text_input_str

synthesis_input = texttospeech.SynthesisInput(text=YOUR_INPUT)
voice = texttospeech.VoiceSelectionParams(
    language_code='cmn-tw',  # 指定語言
    ssml_gender=texttospeech.SsmlVoiceGender.FEMALE  # 指定聲優
)
audio_config = texttospeech.AudioConfig(
    audio_encoding=texttospeech.AudioEncoding.MP3  # 指定輸出格式
)
```

© 2026 Enos Chou

Google Cloud Text-to-Speech API

用戶端開發

文字轉語音

```
response = tts_client.synthesize_speech(
    input=synthesis_input,
    voice=voice,
    audio_config=audio_config
)
```

© 2026 Enos Chou

Google Cloud Text-to-Speech API

用戶端開發

儲存為音檔

```
YOUR_AUDIO = audio_file_name_str  # 指定輸出檔案 ex: tts.mp3

with open(YOUR_AUDIO, 'wb') as f:
    f.write(response.audio_content)
```

© 2026 Enos Chou

Google Cloud Text-to-Speech API ft. Gemini

準備環境

啟用 API

```
Cloud Text-to-Speech API
Gemini API or Generative Language API
```

© 2026 Enos Chou

Google Cloud Text-to-Speech API ft. Gemini

權限

服務帳戶角色

`Vertex AI: Agent Platform 使用者`

© 2026 Enos Chou

Google Cloud Text-to-Speech API ft. Gemini

用戶端開發

參考資料

https://cloud.google.com/text-to-speech/docs/gemini-tts

© 2026 Enos Chou

Google Cloud Text-to-Speech API ft. Gemini

用戶端開發

安裝套件

```
pip install google-cloud-texttospeech
```

© 2026 Enos Chou

Google Cloud Text-to-Speech API ft. Gemini

用戶端開發

初始化

```
from google.cloud import texttospeech

YOUR_SERVICE = service_account_key_file_name_str

tts_client = texttospeech.TextToSpeechClient.from_service_account_json(
    YOUR_SERVICE
)
```

© 2026 Enos Chou

Google Cloud Text-to-Speech API ft. Gemini

用戶端開發

設定

```
YOUR_INPUT = text_input_str  # 轉語音內容 ex: 今天都在下雨，很煩
YOUR_PROMPT = text_prompt_str  # 指定提示詞 ex: 傷心的語調

synthesis_input = texttospeech.SynthesisInput(
    text=YOUR_INPUT,
    prompt=YOUR_PROMPT
)
```

© 2026 Enos Chou

Google Cloud Text-to-Speech API ft. Gemini

用戶端開發

設定

```
YOUR_MODEL = model_name_str  # 指定模型 ex: gemini-3.1-flash-tts-preview
YOUR_VOICE = voice_name_str  # 指定聲優 ex: Sulafat

voice = texttospeech.VoiceSelectionParams(
    model=YOUR_MODEL,
    language_code='cmn-tw',  # 指定語言
    name=YOUR_VOICE
)
```

© 2026 Enos Chou

Google Cloud Text-to-Speech API ft. Gemini

用戶端開發

設定

```
audio_config = texttospeech.AudioConfig(
    audio_encoding=texttospeech.AudioEncoding.MP3   # 指定輸出格式
)
```

© 2026 Enos Chou

Google Cloud Text-to-Speech API ft. Gemini

用戶端開發

文字轉語音

```
response = tts_client.synthesize_speech(
    input=synthesis_input,
    voice=voice,
    audio_config=audio_config
)
```

© 2026 Enos Chou

Google Cloud Text-to-Speech API ft. Gemini

用戶端開發

儲存為音檔

```
YOUR_AUDIO = audio_file_name_str  # 指定輸出檔案 ex: tts.mp3

with open(YOUR_AUDIO, 'wb') as f:
    f.write(response.audio_content)
```

© 2026 Enos Chou

Azure OpenAI gpt-4o-mini-tts

部署

Azure AI Foundry 部署

`gpt-4o-mini-tts`

© 2026 Enos Chou

Azure OpenAI gpt-4o-mini-tts

用戶端開發

參考資料

https://platform.openai.com/docs/guides/text-to-speech

© 2026 Enos Chou

Azure OpenAI gpt-4o-mini-tts

用戶端開發

安裝套件

```
pip install openai
```

© 2026 Enos Chou

Azure OpenAI gpt-4o-mini-tts

用戶端開發

初始化

```
from openai import AzureOpenAI

YOUR_KEY = azure_ai_key_str  # Azure AI Foundry > 資源管理 > 金鑰與端點
YOUR_ENDPOINT = endpoint_name_str  # 同上,
                                   # ex: https://xxx.openai.azure.com/

model = AzureOpenAI(api_key=YOUR_KEY,
                    azure_endpoint=YOUR_ENDPOINT,
                    api_version='2025-04-01-preview')
```

© 2026 Enos Chou

Azure OpenAI gpt-4o-mini-tts

用戶端開發

文字轉語音

```
YOUR_MODEL = 'gpt-4o-mini-tts'
YOUR_INPUT = text_input_str  # 轉語音內容 ex: 今天都在下雨，很煩
YOUR_INSTRUCTIONS = system_instructions_str  # 指定提示詞 ex: 傷心的語調
YOUR_VOICE = voice_name_str  # 指定聲優 ex: nova
YOUR_FORMAT = response_audio_format_str  # 指定輸出格式 ex: mp3, default
```

© 2026 Enos Chou

Azure OpenAI gpt-4o-mini-tts

用戶端開發

文字轉語音

```
r = model.audio.speech.create(
    model=YOUR_MODEL,
    input=YOUR_INPUT,
    instructions=YOUR_INSTRUCTIONS,
    voice=YOUR_VOICE,
    response_format=YOUR_FORMAT
)
```

© 2026 Enos Chou

Azure OpenAI gpt-4o-mini-tts

用戶端開發

儲存為音檔

```
YOUR_AUDIO = audio_file_name_str  # 指定輸出檔案 ex: tts.mp3

r.write_to_file(YOUR_AUDIO)
```

© 2026 Enos Chou

ngrok

1. 安裝

① https://ngrok.com
② Sign up
③ Name/ Email/ Password/ I accept the terms of service and privacy policy > Sign up
④ Verify Email > 進入 email 信箱，按連結
⑤ Multi-factor Authentication > Skip > Got it
⑥ Survey > ... > Continue
⑦ Welcome > Choose OS > Download > ...
⑧ 記錄 Authtoken

© 2026 Enos Chou

ngrok

1. 安裝

⑨ 解壓縮或安裝下載之 ngrok 套件
⑩ 於 ngrok 路徑開啟 命令提示字元 或 終端機 (Terminal)
① `ngrok config add-authtoken` *YOUR_AUTHTOKEN*

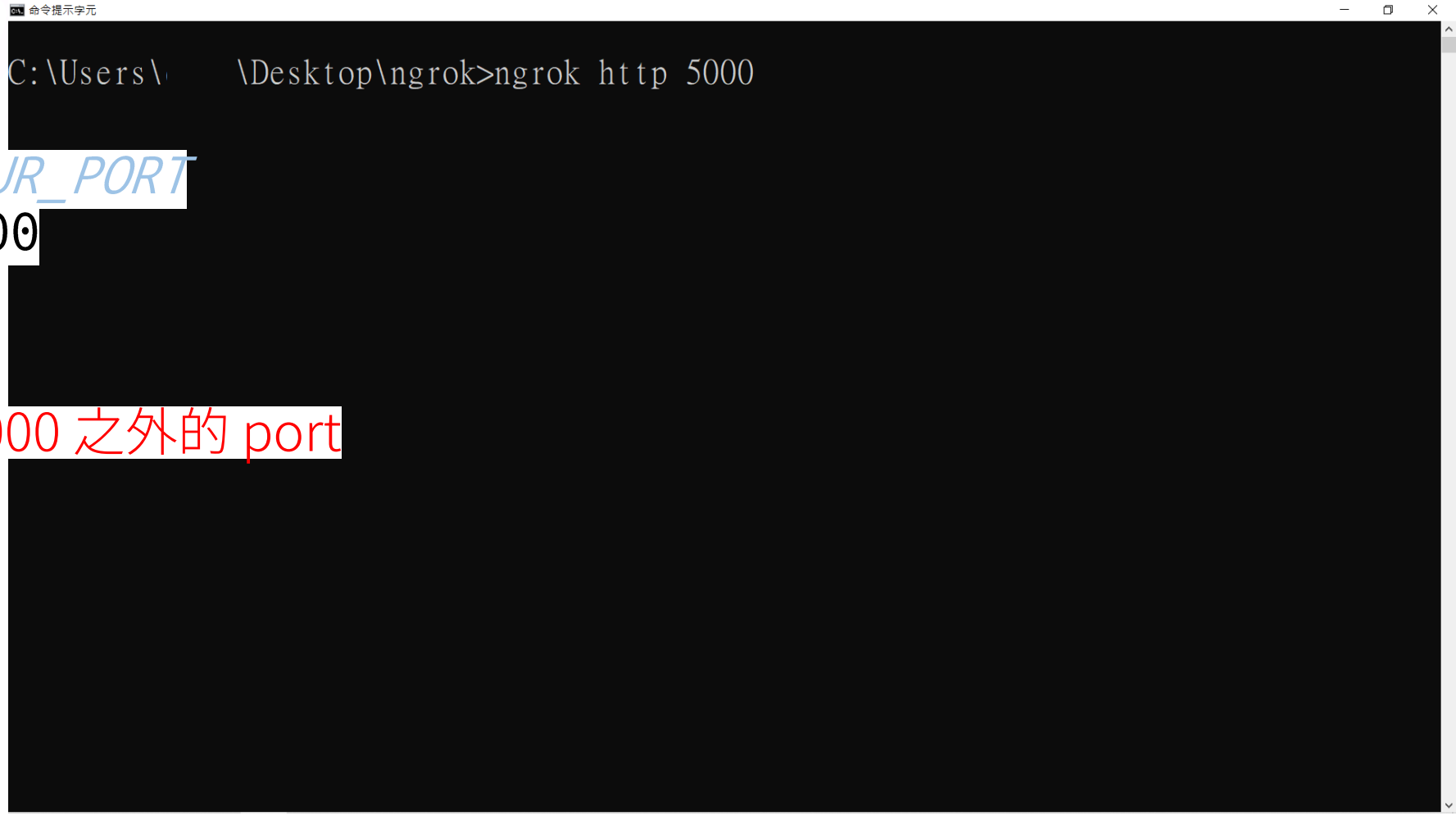
© 2026 Enos Chou

ngrok

2. 建立 tunnel

```
ngrok http YOUR_PORT
ngrok http 5000
```

Note
macOS 須使用 5000 之外的 port



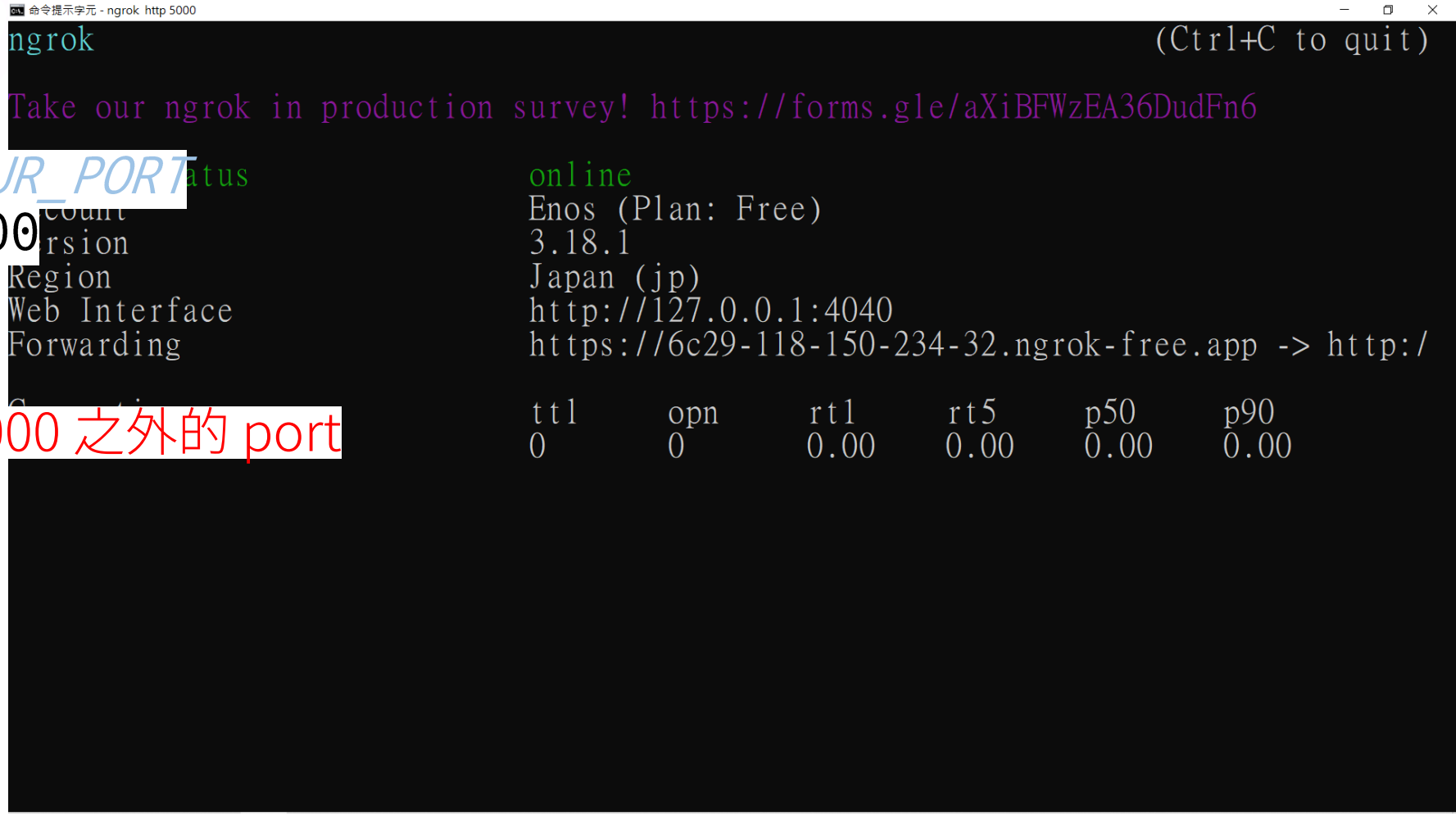
ngrok

2. 建立 tunnel

```
ngrok http YOUR_PORT
ngrok http 5000
```

Note

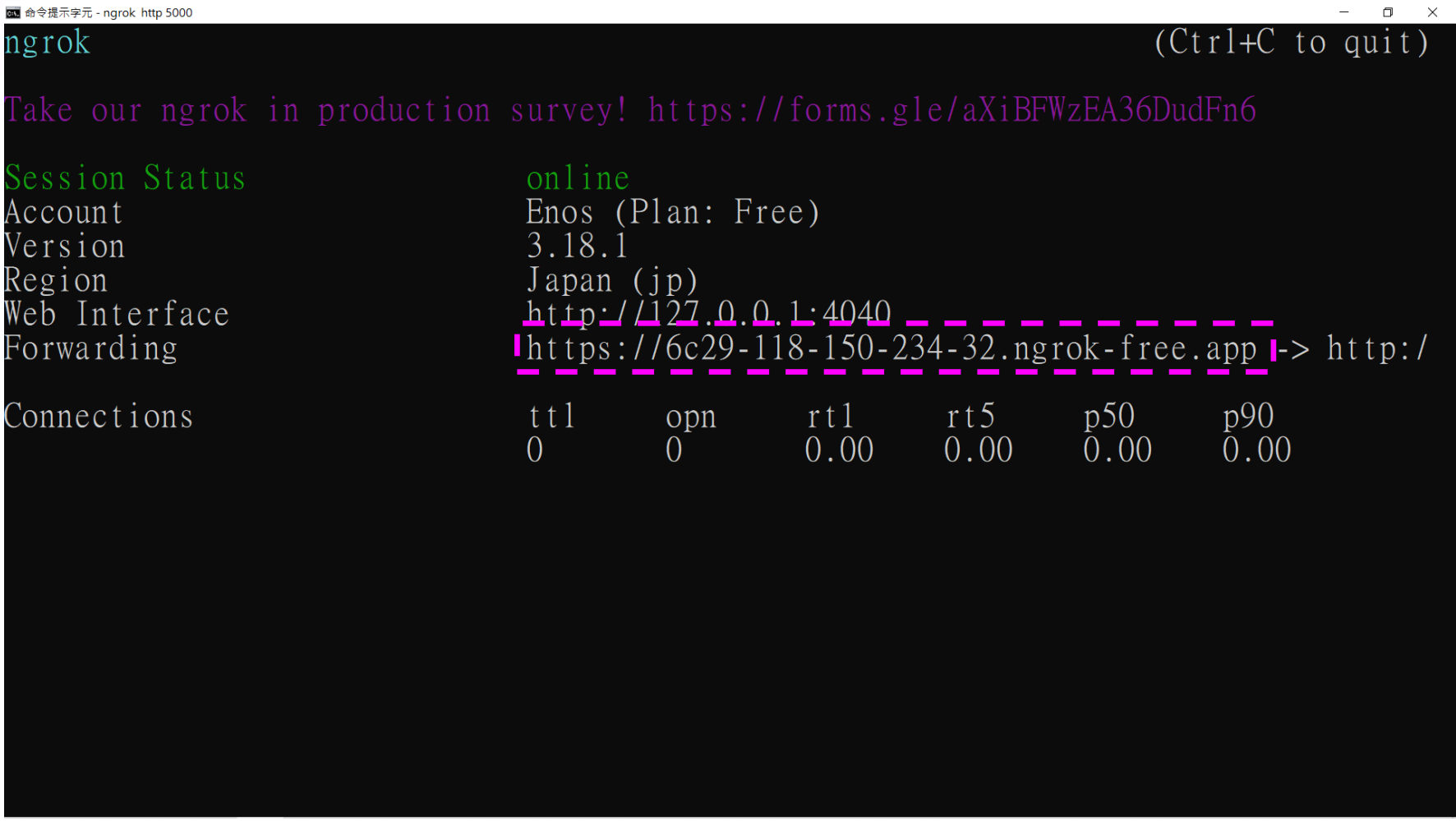
macOS 須使用 5000 之外的 port



ngrok

2. 建立 tunnel

記錄轉址位址



© 2026 Enos Chou

Cloudflare Tunnel Client

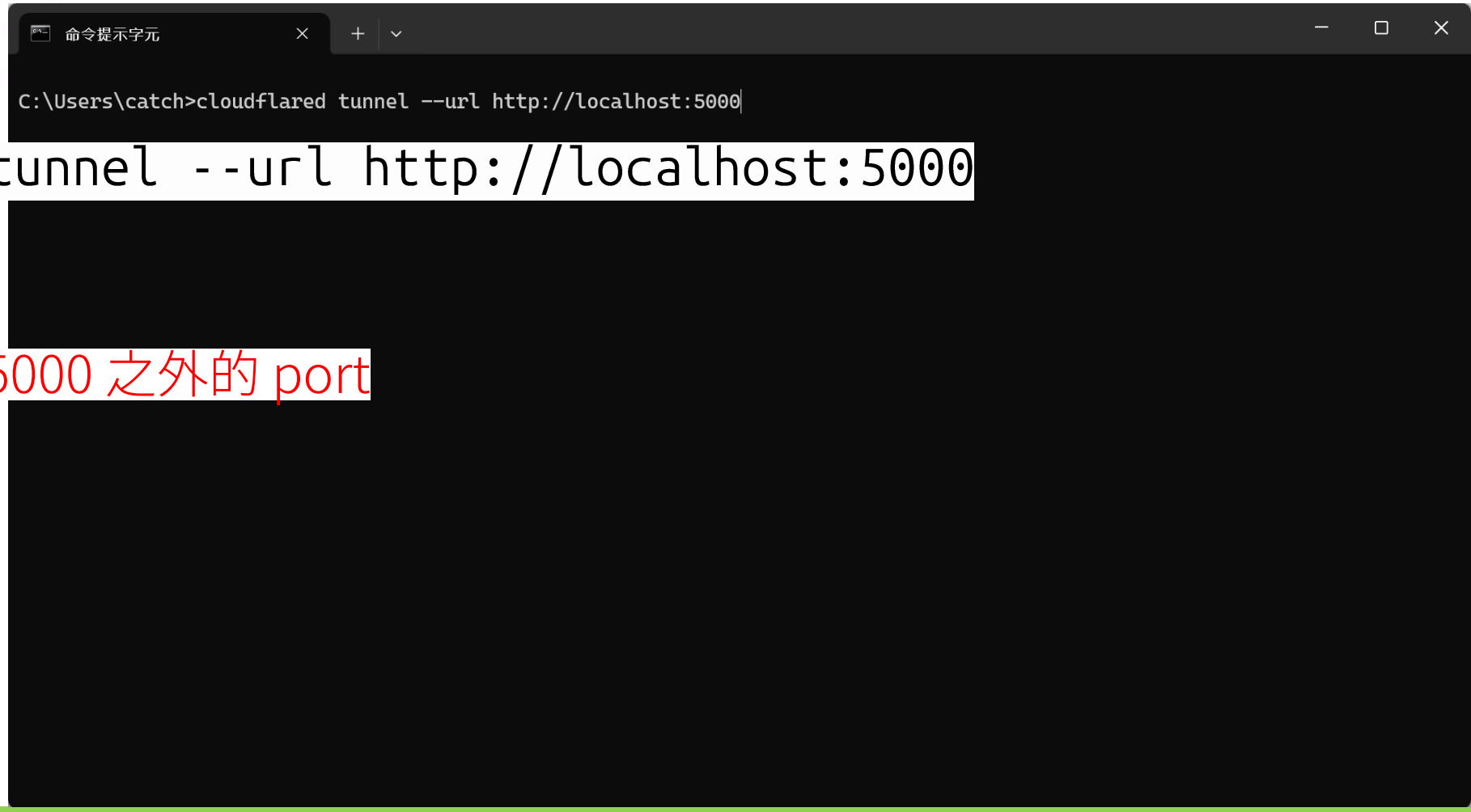
1. 安裝

① https://github.com/cloudflare/cloudflared/releases
② 於 Assets 下載對應的 OS 版本
Windows
`cloudflared-windows-amd64.exe or cloudflared-windows-amd64.msi`
macOS
`cloudflared-darwin-amd64.tgz or cloudflared-darwin-arm64.tgz`
③ 點擊安裝

© 2026 Enos Chou

Cloudflare Tunnel Client

2. 建立 tunnel




```
cloudflared tunnel --url http://localhost:5000
```

Note
macOS 須使用 5000 之外的 port

© 2026 Enos Chou

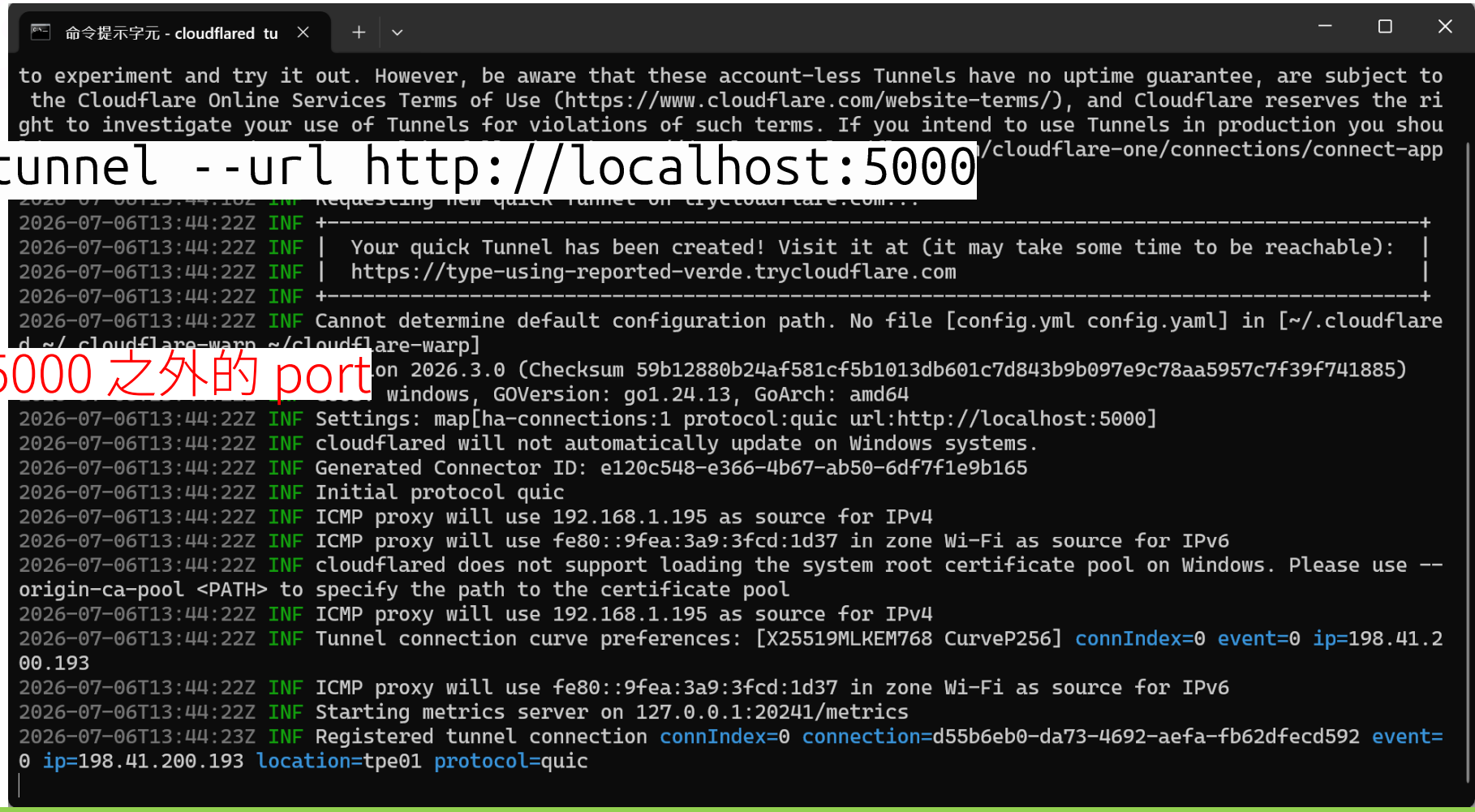
Cloudflare Tunnel Client

2. 建立 tunnel

```
cloudflared tunnel --url http://localhost:5000
```

Note

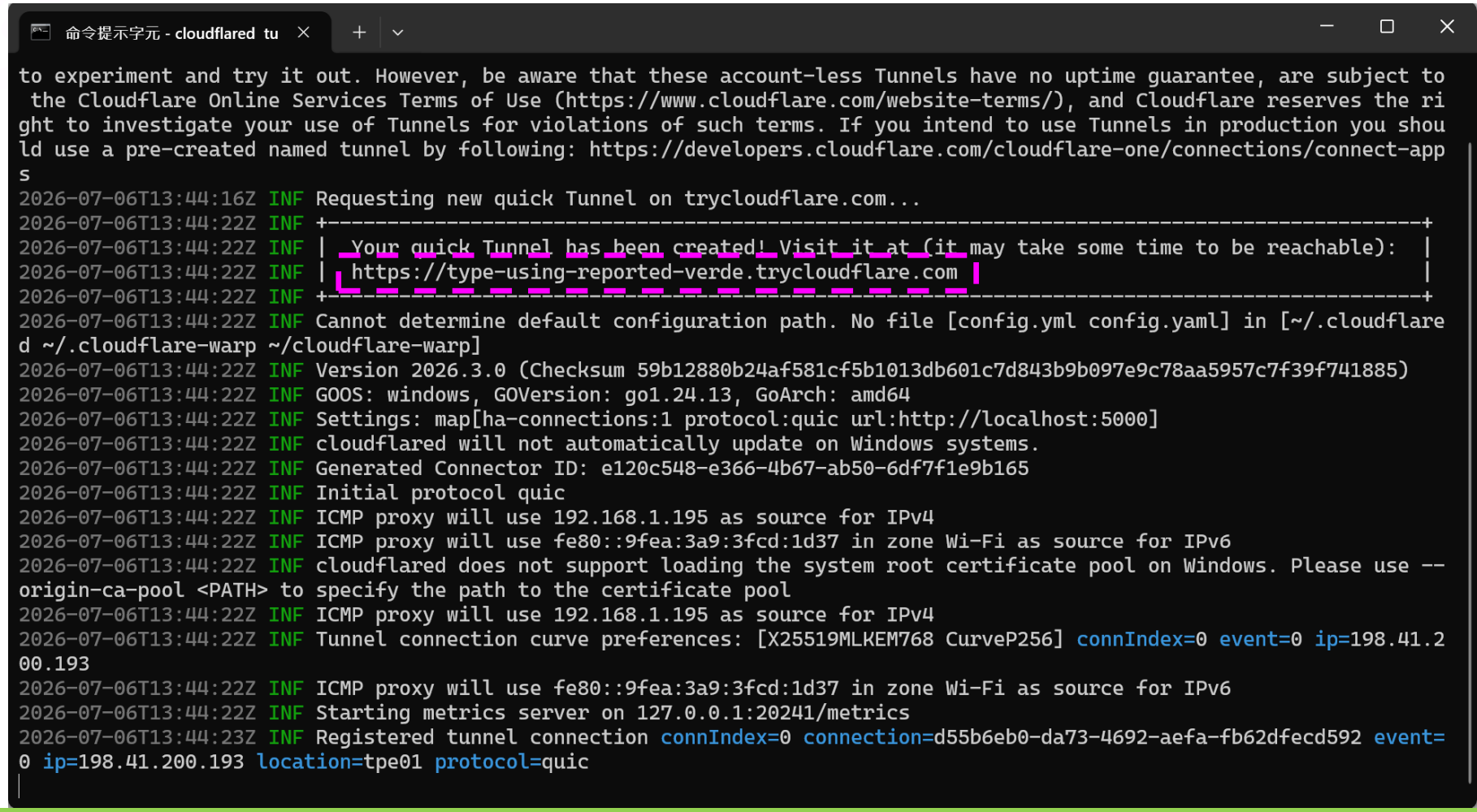
macOS 須使用 5000 之外的 port



Cloudflare Tunnel Client

2. 建立 tunnel

記錄轉址位址



© 2026 Enos Chou

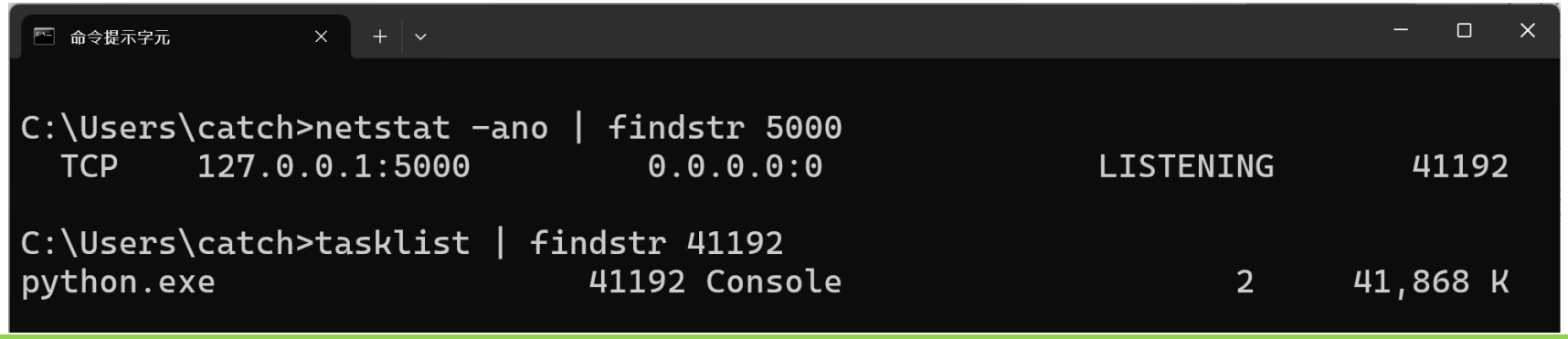
Ports in Use on Windows

1. 找出特定 port 被佔用的 PID

```
netstat -ano | findstr 5000
```

2. 找出特定 PID 代表的程式

```
tasklist | findstr 41192
```



© 2026 Enos Chou

Ports in Use on macOS or Linux

1. 找出特定 port 被佔用的 PID

```
lsof -i :5000
lsof | grep :5000

ps -ef | grep :5000
ps aux | grep :5000
```

© 2026 Enos Chou

The End

© 2026 Enos Chou